

Cálculo Numérico

Roteiro de Estudos

Bacharelado em Ciência da Computação

Material de Apoio às Aulas

2026

Prefácio

Este roteiro de estudos foi desenvolvido para acompanhar o curso de Cálculo Numérico do Bacharelado em Ciência da Computação, seguindo uma abordagem baseada em metodologias ativas de aprendizagem.

O material está estruturado para proporcionar uma compreensão profunda dos fundamentos teóricos dos métodos numéricos, sempre conectando a teoria com aplicações práticas na computação. Cada capítulo apresenta:

- Fundamentação teórica rigorosa com demonstrações de teoremas.
- Análise de convergência e complexidade computacional.
- Exemplos de aplicações reais com referências completas.
- Implementações em Python com código comentado.
- Conexões com problemas atuais da ciência da computação.
- Exercícios graduados para consolidação do aprendizado.

Como usar este material:

Este roteiro foi projetado para estudo ativo. Recomenda-se:

1. Ler o material *antes* das aulas presenciais.
2. Executar todos os códigos apresentados, modificando parâmetros.
3. Tentar demonstrar os teoremas antes de ler as demonstrações.
4. Explorar as referências complementares indicadas
5. Resolver os exercícios propostos.
6. Participar ativamente dos grupos de discussão.

Convenções utilizadas:

- Teoremas e definições aparecem destacados em caixas numeradas.
- Aplicações computacionais são apresentadas em caixas verdes.
- Notas importantes aparecem em caixas laranjas.
- Alertas sobre erros comuns aparecem em caixas vermelhas.
- Código fonte está em fonte monoespaçada com destaque de sintaxe.
- Referências bibliográficas aparecem como [?] ou em URLs.

Erros e sugestões. Este material está em permanente aperfeiçoamento. Erros, sugestões e contribuições são muito bem-vindos e podem ser comunicados diretamente à professora responsável.

Bons estudos!

Angela Leite Moreno
Alfenas, 2026

Sumário

Prefácio	iii
1 Introdução	1
1.1 O que é Cálculo Numérico?	1
1.1.1 Técnicas de Solução: Comparação	2
1.1.2 Da Modelagem à Solução	2
1.1.3 Exemplo Motivador: Integração Numérica	4
1.1.4 Aplicação: Inteligência Artificial e <i>Deep Learning</i>	5
1.1.5 Breve Histórico	6
1.2 Representação de Números em Computadores	7
1.3 Motivação: Por que a Representação Importa?	8
1.3.1 Dependência dos Resultados e o Problema do Edifício	9
1.4 Tipos de Erros	9
1.5 Sistemas de Numeração e Bases	10
1.5.1 Forma Geral	10
1.5.2 Sistema Decimal (Base 10)	11
1.5.3 Sistema Binário (Base 2)	11
1.5.4 Sistema Hexadecimal (Base 16)	11
1.6 Conversão entre Bases	12
1.6.1 Decimal para Binária	12
1.6.2 Binária para Decimal	13
1.6.3 Hexadecimal para Binária	13
1.7 Aritmética de Ponto Flutuante	14
1.7.1 Representação	14
1.7.2 Bits, Bytes e Palavras	14
1.7.3 Sistema $F(\beta, t, e_{\min}, e_{\max})$	15
1.8 Erros na Representação dos Números	16
1.8.1 <i>Overflow</i> e <i>Underflow</i>	16
1.8.2 Mantissa Finita	16
1.9 Truncamento e Arredondamento	17

1.10	Conceitos de Erro Numérico	18
1.10.1	Número Aproximado	18
1.10.2	Erro Absoluto e Erro Relativo	18
1.10.3	Erro de Arredondamento em Ponto Flutuante e Precisão de Máquina	19
1.10.4	Propagação de Erros	20
1.11	Desastres Causados por Erros Numéricos	21
1.11.1	Missil Patriot (1991) — 28 mortes	21
1.11.2	Foguete Ariane 5 (1996) — US\$ 370 milhões	22
1.11.3	Índice de Vancouver (1982) — Queda fictícia de bolsa	23
1.11.4	Therac-25 (1985–1987) – 6 mortes por radiação	23
1.11.5	Bug FDIV do Pentium (1994) — US\$ 475 milhões	24
1.12	Condicionamento e Estabilidade Numérica	25
1.12.1	Condicionamento de Problemas	25
1.12.2	Estabilidade de Algoritmos	26
1.13	Complexidade Computacional de Algoritmos Numéricos	27
1.13.1	Notação Big- $\mathcal{O}$	27
1.13.2	Análise de Algoritmos Numéricos	28
1.14	Ferramentas Computacionais	29
1.15	Exercícios	30
1.16	Referências e Leituras Complementares	33
2	Zeros de Funções Reais	35
2.1	Introdução	35
2.1.1	Critérios de Parada	36
2.2	Aplicações Motivadoras	36
2.3	Teoremas Fundamentais	37
2.3.1	Teorema Fundamental da Álgebra	37
2.3.2	Teorema de Abel-Ruffini	38
2.3.3	Localização de Raízes	38
2.3.4	Funções Transcendentes	39
2.4	Fase I: Localização Gráfica e Tabela de Sinais	39
2.5	Método da Bissecção	40
2.5.1	Algoritmo	40
2.5.2	Análise de Convergência	40
2.5.3	Implementação	41
2.6	Método da Posição Falsa	42
2.6.1	Motivação e Fórmula	42
2.6.2	Convergência e Comparação com a Bissecção	43

2.6.3	Implementação	43
2.7	Método do Ponto Fixo	44
2.7.1	Formulação	44
2.7.2	Convergência	45
2.7.3	Diagramas de Teia de Aranha	45
2.7.4	Newton como Ponto Fixo de Ordem Dois	47
2.7.5	Implementação	47
2.8	Método de Newton-Raphson	48
2.8.1	Derivação Geométrica	48
2.8.2	Algoritmo	49
2.8.3	Análise de Convergência	49
2.8.4	Raízes Múltiplas e Método Modificado	50
2.8.5	Aplicação a Funções Transcendentes	52
2.8.6	Implementação	52
2.9	Método da Secante	53
2.9.1	Motivação e Fórmula	53
2.9.2	Análise de Convergência	53
2.9.3	Implementação e Comparação	54
2.10	Otimização Unidimensional	55
2.11	Sistemas de Equações Transcendentes	55
2.12	Métodos Específicos para Polinômios	56
2.12.1	Algoritmo de Horner	56
2.12.2	Deflação de Polinômios (Briot-Ruffini)	57
2.12.3	Fórmula de Bhaskara Numericamente Estável	58
2.13	Equação de Kepler — Projeto Integrador	58
2.14	Exercícios	60
2.15	Referências e Leituras Complementares	63
3	Sistemas de Equações Lineares: Métodos Diretos	65
3.1	Introdução e Motivação	65
3.2	Existência e Unicidade da Solução	67
3.3	Operações Elementares e Equivalência de Sistemas	67
3.4	Método de Eliminação de Gauss	68
3.4.1	Ideia e Algoritmo	68
3.4.2	Análise de Complexidade	70
3.4.3	Implementação	70
3.5	Estratégias de Pivoteamento	71
3.5.1	O Problema do Pivô Nulo ou Pequeno	71

3.5.2	Pivoteamento Parcial	72
3.5.3	Pivoteamento Completo	72
3.6	Fatoração LU	73
3.6.1	Motivação	73
3.6.2	Existência	73
3.6.3	Algoritmo de Doolittle	74
3.6.4	Fatoração PLU com Pivoteamento	75
3.7	Matrizes com Estrutura Especial	77
3.7.1	Matrizes Simétricas Positivas Definidas — Fatoração de Cholesky	77
3.7.2	Matrizes Tridiagonais — Algoritmo de Thomas	79
3.8	Análise de Erros e Condicionamento	81
3.8.1	Número de Condição de uma Matriz	81
3.9	Comparação e Escolha do Método	82
3.10	Exercícios	83
3.11	Referências e Leituras Complementares	85
4	Sistemas de Equações Lineares: Métodos Iterativos	87
4.1	Motivação e Aplicações	87
4.2	Conceitos Fundamentais	90
4.2.1	Ideia Geral	90
4.2.2	Decomposição da Matriz	90
4.2.3	Critério de Parada	91
4.3	Método de Gauss-Jacobi	92
4.3.1	Derivação do Método	92
4.3.2	Formulação Componente a Componente	93
4.3.3	Algoritmo	93
4.3.4	Implementação	98
4.4	Método de Gauss-Seidel	101
4.4.1	Motivação e Derivação	101
4.4.2	Formulação Componente a Componente	102
4.4.3	Implementação	104
4.5	Análise de Convergência	106
4.5.1	Teorema Fundamental	106
4.5.2	Critérios Práticos de Convergência	107
4.6	Método SOR (<i>Successive Over-Relaxation</i>)	111
4.6.1	Motivação e Derivação	111
4.6.2	ω Ótimo	112
4.7	Método do Gradiente Conjugado	116

4.7.1	Motivação: Minimização de Função Energia	116
4.7.2	Algoritmo	117
4.7.3	Exemplo Resolvido Passo a Passo	117
4.7.4	Implementação	118
4.8	Pré-Condicionamento	120
4.8.1	Motivação	120
4.8.2	Pré-condicionadores Clássicos	120
4.8.3	Gradiente Conjugado Pré-Condicionado (PCG)	120
4.9	Comparação e Escolha do Método	121
4.10	Exercícios	122
4.11	Referências e Leituras Complementares	125
5	Interpolação Numérica	127
5.1	Introdução e Motivação	127
5.1.1	Por que usar polinômios?	128
5.2	Existência e Unicidade do Interpolador Polinomial	130
5.2.1	Visão matricial (para entender a estrutura)	130
5.3	Interpolação de Lagrange	131
5.3.1	Ideia construtiva	131
5.3.2	Exemplo resolvido passo a passo	132
5.3.3	Custo computacional e limitação	133
5.3.4	Implementação	133
5.4	Diferenças Divididas e Interpolação de Newton	135
5.4.1	Motivação: adição incremental de pontos	135
5.4.2	Diferenças Divididas	136
5.4.3	Fórmula de Newton	136
5.4.4	Tabela de diferenças divididas	137
5.4.5	Exemplo resolvido passo a passo	137
5.4.6	Implementação	138
5.5	Erro de Interpolação	141
5.5.1	Fórmula do erro	141
5.5.2	Exemplo: análise do erro com função conhecida	142
5.6	Fenômeno de Runge e Nós de Chebyshev	143
5.6.1	O problema com pontos equiespaçados	143
5.6.2	Nós de Chebyshev	144
5.6.3	Exemplo comparativo	144
5.6.4	Implementação	145
5.7	Splines Cúbicas	147

5.7.1	Motivação: por que polinômios por partes?	147
5.7.2	Definição e contagem de graus de liberdade	148
5.7.3	Condições de fronteira	149
5.7.4	Construção: o sistema tridiagonal	149
5.7.5	Exemplo resolvido passo a passo	149
5.7.6	Implementação	150
5.8	Interpolação Inversa	154
5.8.1	O problema	154
5.8.2	Dois métodos de solução	154
5.8.3	Exemplo resolvido	155
5.8.4	Implementação	156
5.9	Quadro Comparativo dos Métodos	158
5.10	Lista de Exercícios de Revisão do Capítulo	159
5.11	Referências e Leituras Complementares	161
6	Ajuste de Curvas — Método dos Mínimos Quadrados	163
6.1	Introdução e Motivação	163
6.1.1	Diagrama de dispersão: o ponto de partida	164
6.2	Método dos Mínimos Quadrados	166
6.2.1	Por que minimizar a soma dos quadrados?	166
6.2.2	Coeficiente de determinação r^2	166
6.2.3	Análise de resíduos	167
6.3	Regressão Linear Simples	168
6.3.1	Derivação das equações normais	168
6.3.2	Fórmulas explícitas	168
6.3.3	Exemplo resolvido passo a passo	169
6.3.4	Implementação	170
6.4	Regressão Polinomial	172
6.4.1	Equações normais de grau m	172
6.4.2	Princípio da Parcimônia	172
6.4.3	Exemplo resolvido passo a passo	173
6.4.4	Implementação	174
6.5	Regressão Múltipla	176
6.5.1	Modelo e formulação matricial	176
6.5.2	Escala das variáveis	177
6.5.3	Exemplo resolvido passo a passo	177
6.5.4	Implementação	178
6.6	Linearização de Modelos Não-Lineares	180

6.6.1	Estratégia geral	180
6.6.2	Modelo exponencial: $y = ae^{bx}$	180
6.6.3	Modelo logarítmico: $y = a + b \ln x$	181
6.6.4	Modelos com modelo Hiperbólico: $y = 1/(a_0 + a_1x)$	181
6.6.5	Exemplos resolvidos passo a passo	181
6.6.6	Implementação	183
6.7	Roteiro de Ajuste de Curvas	186
6.8	Quadro Comparativo dos Modelos	186
6.9	Lista de Exercícios de Revisão do Capítulo	187
6.10	Referências e Leituras Complementares	189
A	Python para Cálculo Numérico	191
A.1	Configuração do Ambiente	191
A.1.1	Instalação Recomendada	191
A.1.2	Jupyter Notebook e Google Colab	192
A.1.3	Cabeçalho Padrão dos Scripts	192
A.2	NumPy — Arrays e Álgebra Linear	192
A.2.1	Criação de Arrays	193
A.2.2	Operações Vetorizadas	193
A.2.3	Álgebra Linear com <code>numpy.linalg</code>	194
A.2.4	Fatiamento (<i>Slicing</i>) e Indexação	194
A.3	SciPy — Métodos Numéricos Prontos	195
A.3.1	Zeros de Funções: <code>scipy.optimize</code>	195
A.3.2	Sistemas Lineares: <code>scipy.linalg</code>	196
A.3.3	Integração Numérica: <code>scipy.integrate</code>	196
A.3.4	Interpolação: <code>scipy.interpolate</code>	197
A.4	Matplotlib — Visualização de Resultados	198
A.4.1	Gráficos de Funções e Convergência	198
A.4.2	Visualização de Erros e Convergência	199
A.5	Boas Práticas e Armadilhas Comuns	200
A.5.1	Armadilhas de Ponto Flutuante	200
A.5.2	Cancelamento Catastrófico em Python	201
A.5.3	Acurácia Mista Float32 / Float64	201
A.5.4	Medição de Tempo e Perfil de Desempenho	202
A.6	Receitas Práticas para os Capítulos	203
A.6.1	Capítulo 1 — Erros e Ponto Flutuante	203
A.6.2	Capítulo 2 — Zeros de Funções	204
A.6.3	Capítulo 3 — Sistemas Lineares	204

A.7	Exercícios de Programação	206
B	Complexidade Computacional e Análise de Algoritmos	207
B.1	Notação Assintótica	207
B.1.1	As Três Notações Fundamentais	207
B.1.2	Propriedades da Notação $\mathcal{O}$	208
B.2	Análise de Algoritmos Numéricos Fundamentais	209
B.2.1	Operações Elementares: Flops	209
B.2.2	Avaliação de Polinômios: Horner vs. Naïve	209
B.2.3	Métodos Iterativos para Zeros de Funções	210
B.3	Complexidade de Operações Matriciais	211
B.3.1	Eliminação de Gauss: Contagem Exata	211
B.3.2	Fatoração de Cholesky: Custo pela Metade	212
B.3.3	Hierarquia de Custos Matriciais	212
B.4	Modelos de Custo: além dos Flops	212
B.4.1	Hierarquia de Memória	212
B.4.2	Paralelismo e GPUs	213
B.5	Tabelas de Referência Rápida	214
B.5.1	Classes de Complexidade em Cálculo Numérico	214
B.5.2	Ordens de Erro dos Métodos Numéricos	214
B.5.3	Custos Exatos dos Algoritmos do Livro	215
B.6	Exercícios	215

Lista de Figuras

1.1	As três famílias de abordagens para resolução de problemas matemáticos.	1
1.2	<i>Pipeline</i> básico de resolução numérica.	3
1.3	Etapas do processo de modelagem e simulação numérica.	3
1.4	Etapas do processo de modelagem e simulação numérica com erros	4
1.5	Aproximação de $\int_a^b f(x) dx$ por somas de trapézios. Esquerda: 1 trapézio ($A \approx A_{\text{trap}}$). Centro: 2 trapézios ($A \approx A_1 + A_2$). Direita: 4 trapézios — quanto mais subintervalos, menor o erro.	4
1.6	Rede neural com duas camadas ocultas. A seta inferior representa a propagação dos gradientes no <i>backpropagation</i> , na direção na qual o desaparecimento ou explosão de gradiente se manifesta.	6
1.8	Estrutura dos formatos IEEE 754. S = sinal (0 = positivo, 1 = negativo). Float32 é usado em GPU e aplicações que priorizam velocidade; Float64 é o padrão em cálculo científico.	15
1.9	Acúmulo linear do erro de representação de 0,1 no sistema Patriot ao longo de mais de 100 horas de operação contínua. A cada décimo de segundo, $9,5 \times 10^{-8}$ s de erro se acumula.	21
1.10	Explosão do Ariane 5 em seu voo inaugural (4 de junho de 1996), 37 segundos após o lançamento. A falha foi causada por <i>overflow</i> na conversão Float64 $\rightarrow$ Int16. Fonte: ESA/CNES/ARIANESPACE.	22
1.11	Processador Intel Pentium (1993). O erro na tabela de divisão da FPU causou um <i>recall</i> de US\$ 475 milhões — o maior da história da indústria de semicondutores até então. Fonte: Wikimedia Commons.	24
1.12	Crescimento do custo computacional para as quatro complexidades mais relevantes em Cálculo Numérico. Note como $\mathcal{O}(n^3)$ torna-se rapidamente proibitivo.	28
2.1	Gráfico de $f(x) = x^3 - 9x + 3$ com as três raízes reais identificadas. A tabela de sinais confirma uma mudança de sinal em cada sub-intervalo $[-3, -2]$, $[0, 1]$ e $[2, 3]$	40
2.2	Primeira iteração da bisseção para $f(x) = x^3 - x - 2$ em $[1, 2]$. Como $f(c_1) < 0$, a raiz está em $[c_1, b] = [1, 5, 2]$	41

2.3	Comparação entre o candidato da bisseção (c_{bis}) e o da posição falsa (x_{PF}) para $f(x) = x^3 - 9x + 3$ com $a = 0,2$ e $b = 2,0$. Como $ f(a) \ll f(b) $, a posição falsa está muito mais próxima da raiz real $\tilde{x}$	43
2.4	Diagramas de teia de aranha para as quatro funções de iteração de $x^2 + x - 6 = 0$ em torno de $\tilde{x}_2 = 2$. Setas verdes indicam aproximação ao ponto fixo (convergência); setas vermelhas indicam afastamento (divergência). Em φ_1 e φ_2 , a derivada é positiva: os iterados avançam sempre pelo mesmo lado de $\tilde{x}$, formando uma escada convergente para φ_1 ($\varphi'_1 = 0,8$) e divergente para φ_2 ($\varphi'_2 = 6$). Em φ_3 e φ_4 , a derivada é negativa: os iterados alternam de lado, formando uma espiral convergente para φ_3 ($ \varphi'_3 = 0,25$) e divergente para φ_4 ($ \varphi'_4 = 4$).	47
2.5	Iteração de Newton para $f(x) = x^3 - x - 2$	49
3.1	Pipeline de resolução de $Ax = b$ pela fatoração LU. A matriz A é decomposta em L (triangular inferior unitária) e U (triangular superior). Dois sistemas triangulares são resolvidos sequencialmente.	66
3.2	Processo de triangularização pela eliminação de Gauss para o sistema do example 3.1. Cada seta representa uma etapa de eliminação que zera todos os elementos abaixo do pivô de uma coluna.	67
3.3	Pivoteamento parcial na etapa 2: o elemento de maior módulo na coluna 2 (abaixo do pivô atual) é -3 , portanto as linhas 2 e 3 são permutadas antes de prosseguir com a eliminação.	72
3.4	Pivoteamento parcial busca apenas na coluna atual; pivoteamento completo busca em toda a submatriz não processada (região delimitada).	73
3.5	Estrutura da fatoração $A = LU$. L armazena os multiplicadores da eliminação gaussiana na sua parte inferior; U é a matriz triangular superior resultante. Ambos residem na mesma estrutura de memória de A	73
3.6	Fatoração PLU: a matriz de permutação P registra as trocas de linha do pivoteamento parcial, de modo que $PA = LU$	75
3.7	Estrutura da fatoração de Cholesky $A = LL^T$. Como A é SPD, basta armazenar L — a metade superior é redundante. O custo é $n^3/3$ flops, metade do LU geral.	77
3.8	Estrutura de uma matriz tridiagonal 6×6 : apenas $3n - 2$ elementos são não-nulos, possibilitando armazenamento e resolução em $O(n)$ em vez de $O(n^2)$ ou $O(n^3)$	79
3.9	Amplificação de erros de acordo com o número de condição $\kappa(A)$. Uma perturbação de 1% em b pode resultar em até $\kappa\%$ de erro em x . Matrizes com $\kappa \approx 1$ são bem-condicionadas; $\kappa \gg 1$ indica mal-condicionamento. . .	81

3.10	Fluxograma de escolha do método direto. A estrutura da matriz A e o padrão de uso (um único b vs. múltiplos b) determinam a estratégia ótima.	83
B.1	Custo em flops dos principais algoritmos matriciais em função de n . Cholesky custa exatamente metade de Gauss/LU para matrizes simétricas definidas positivas. Strassen é assintoticamente superior a LU, mas o ganho prático só aparece para $n \gtrsim 1\,000$.	212

Lista de Algoritmos

1	Método da Bissecção	40
2	Método de Newton-Raphson	49
3	Método da Secante	53
4	Eliminação de Gauss com Substituição Reversa	68
5	Gauss com Pivoteamento Parcial	72
6	Fatoração LU — Método de Doolittle	74
7	Fatoração de Cholesky	77
8	Método de Gauss-Jacobi	93
9	Método de Gauss-Seidel	102
10	Gradiente Conjugado (para A SPD)	117
11	PCG	121

Capítulo 1

Introdução

*“The purpose of computing is insight,
not numbers.”*

— Richard Hamming

1.1 O que é Cálculo Numérico?

Diante de um problema matemático, três grandes famílias de abordagens estão disponíveis:

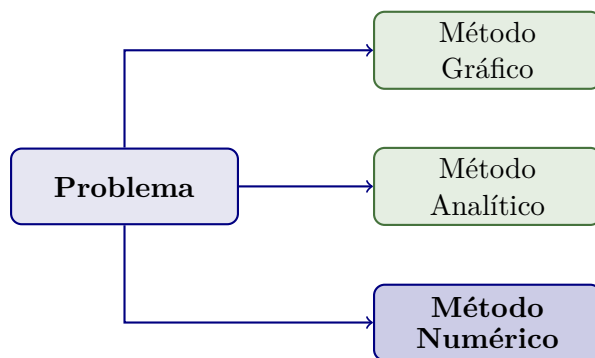


Figura 1.1: As três famílias de abordagens para resolução de problemas matemáticos.

O Cálculo Numérico faz parte da família dos métodos computacionais.

Definição 1.1 (Cálculo Numérico). O Cálculo Numérico é o conjunto de ferramentas e métodos utilizados para obter a solução de problemas matemáticos de forma **aproximada**, por meio de operações aritméticas computacionais. Aplica-se sobretudo a problemas sem solução exata em forma fechada.

Compreende três atividades complementares:

- *análise* dos processos que resolvem o problema por operações aritméticas;
- *desenvolvimento* das sequências de operações que levam às respostas numéricas desejadas;
- *uso do computador* para obtenção efetiva das respostas.

1.1.1 Técnicas de Solução: Comparação

As três abordagens têm características distintas, resumidas na [table 1.1](#).

A **abordagem teórica** usa informações analíticas para obter uma expressão explícita da solução, mais geral, mas restrita a geometrias simples e problemas lineares.

A **abordagem experimental** usa equipamentos de medição para simular processos físicos, mais realista, mas com custo de equipamento, problemas de escala e dificuldade de medição.

A **abordagem numérica** usa ferramentas computacionais para simular numericamente, sem restrição de linearidade ou geometria, permitindo evolução temporal, mas sujeita a erros de truncamento e arredondamento, e exige a prescrição correta das condições de contorno.

Tabela 1.1: Comparação entre as técnicas de solução de problemas matemáticos.

Técnica	Vantagens	Desvantagens
Teórica	Fórmula fechada; resultado geral; mais elegante	Restrita a geometrias simples e problemas lineares
Experimental	Mais realista; captura fenômenos complexos	Equipamento específico; problemas de escala; custo operacional; dificuldade de medição
Numérica	Sem restrição de linearidade ou geometria; permite evolução temporal	Erros de truncamento e arredondamento; custos operacionais; prescrição das condições de contorno

1.1.2 Da Modelagem à Solução

A resolução de um problema real por métodos numéricos percorre dois estágios distintos:

Modelagem: obtenção de um modelo matemático que descreve o comportamento do fenômeno.

Resolução: aplicação de métodos numéricos ao modelo.

Ambas as fases são passíveis de erros.



Figura 1.2: *Pipeline* básica de resolução numérica.

Ao resolver um problema numericamente, o profissional precisa:

1. **Decidir** se existe um método numérico adequado para o problema;
2. **Escolher o método** mais apropriado, avaliando suas vantagens e limitações;
3. **Avaliar a qualidade** da solução obtida – compreendendo exatamente o que o computador está fazendo.

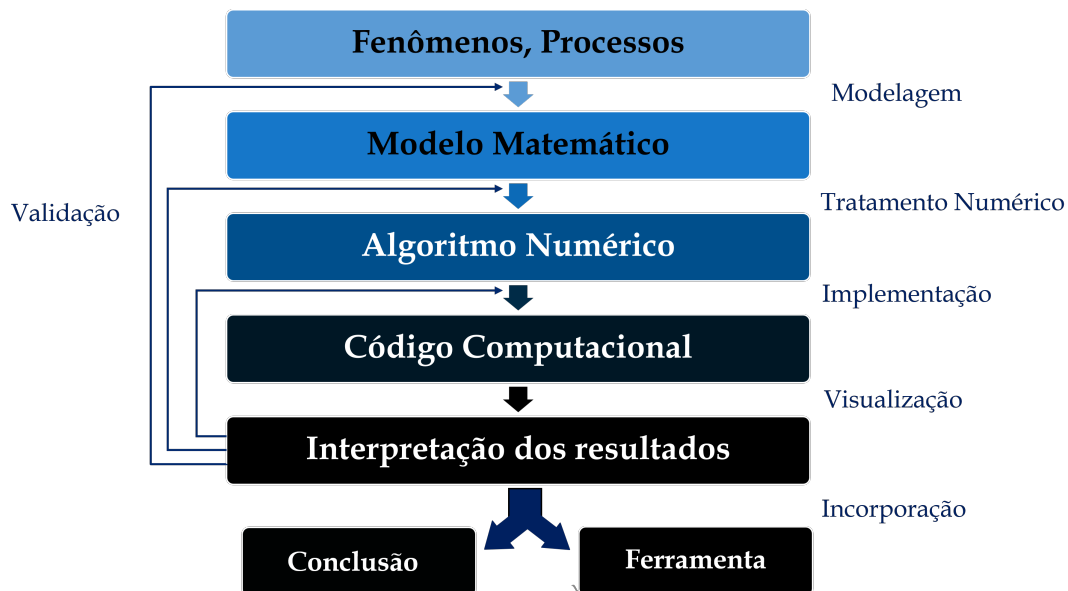


Figura 1.3: Etapas do processo de modelagem e simulação numérica.

Não é raro que os resultados finais estejam distantes do esperado, ainda que todas as fases de resolução tenham sido realizadas corretamente. Os resultados dependem também:

- (a) da **precisão dos dados de entrada**;
- (b) da **forma como** esses dados são representados no computador;
- (c) das **operações numéricas** efetuadas.

O processo completo de uma solução numérica envolve mais etapas, e cada uma pode introduzir erros:

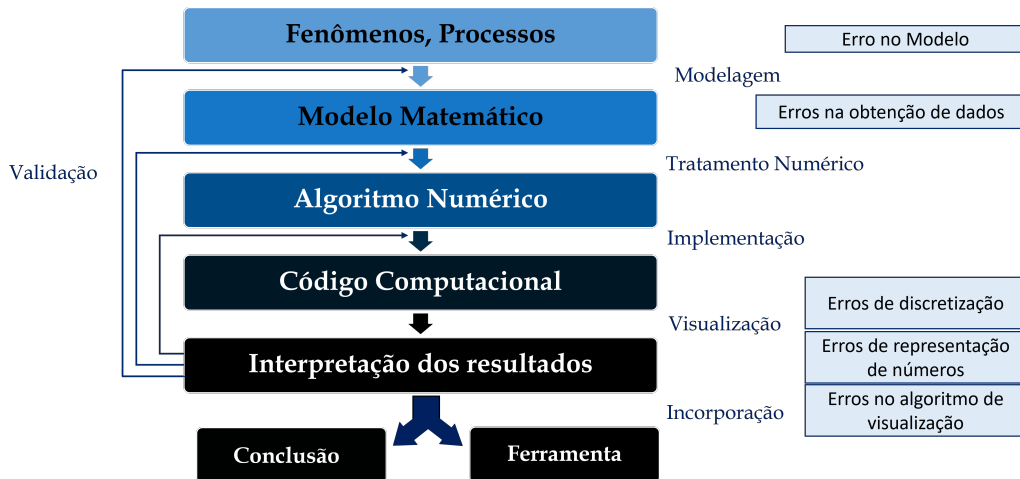


Figura 1.4: Etapas do processo de modelagem e simulação numérica com erros

1.1.3 Exemplo Motivador: Integração Numérica

Exemplo 1.1 (Por que métodos numéricos em integração?). Calcular a área sob uma curva $f(x)$ no intervalo $[a, b]$ corresponde ao problema:

$$\text{Área} = \int_a^b f(x) dx.$$

A abordagem numérica mais direta substitui a área por uma soma de *trapézios*. As três figuras abaixo mostram como o erro diminui progressivamente ao aumentar o número de subdivisões.

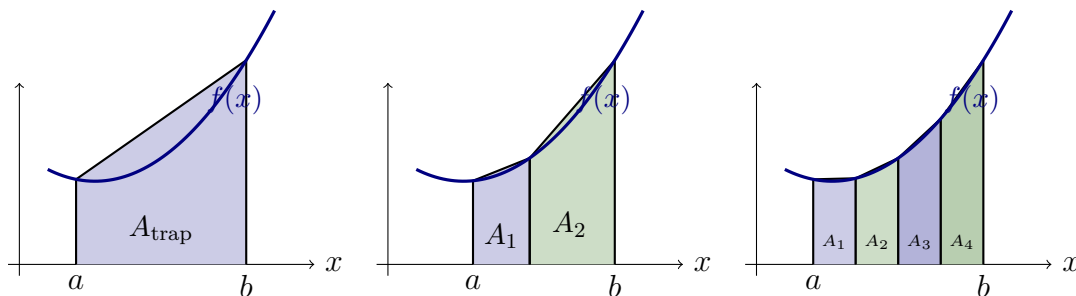


Figura 1.5: Aproximação de $\int_a^b f(x) dx$ por somas de trapézios. **Esquerda:** 1 trapézio ($A \approx A_{\text{trap}}$). **Centro:** 2 trapézios ($A \approx A_1 + A_2$). **Direita:** 4 trapézios — quanto mais subintervalos, menor o erro.

Um exemplo paradigmático em que a abordagem numérica é *indispensável* é a integral

$$\int_0^1 e^{-x^2} dx,$$

fundamental em estatística e teoria de probabilidade, mas que *não possui solução em forma fechada*. Somente métodos numéricos permitem calculá-la com precisão arbitrária.

O modelo computacional completo para a integração envolve três etapas:

1. **Implementação:** usar uma linguagem computacional (Python, MATLAB, Scilab, R, C, FORTRAN, ...) para implementar o modelo numérico;
2. **Verificação:** construir, sempre que possível, problemas com solução analítica conhecida e verificar a acurácia da aproximação;
3. **Aplicação:** obter soluções numéricas de problemas de interesse prático.

1.1.4 Aplicação: Inteligência Artificial e *Deep Learning*

O Cálculo Numérico é o alicerce invisível dos sistemas de Inteligência Artificial modernos. Cada vez que um modelo de linguagem gera texto, uma rede neural classifica uma imagem ou um sistema de recomendação sugere um filme, centenas de bilhões de operações de ponto flutuante são executadas — e a qualidade numérica dessas operações determina diretamente a qualidade do resultado.

Aplicação Computacional

Inteligência Artificial e *Deep Learning* [4]:

Redes neurais profundas resolvem sistemas de equações não-lineares com milhões (ou bilhões) de parâmetros durante o treinamento. O algoritmo de *backpropagation* utiliza diretamente três pilares do Cálculo Numérico:

- **Diferenciação numérica:** cálculo de gradientes por regra da cadeia (diferenciação automática).
- **Métodos iterativos de otimização:** descida de gradiente estocástico (SGD), Adam, RMSProp.
- **Álgebra linear numérica:** operações matriciais massivas, como produto de matrizes densas ($\mathcal{O}(n^3)$ ou $\mathcal{O}(n^{2,37})$ via Strassen), decomposições e normas.

Aplicação Computacional

Explosão e Desaparecimento de Gradiente [?]:

Durante o treinamento de redes profundas, os gradientes propagados pelo *backpropagation* podem sofrer dois fenômenos numéricos extremos:

- **Explosão de gradiente** (*overflow*): gradientes crescem exponencialmente através das camadas — os pesos assumem valores `inf` ou `NaN` e o treinamento diverge.
- **Desaparecimento de gradiente** (*underflow*): gradientes decaem para zero — as camadas iniciais param de aprender, pois recebem sinal de atualização numericamente nulo.

Soluções práticas incluem *gradient clipping* (limitação explícita da norma do gradiente), normalização de lote (*BatchNorm*), inicialização cuidadosa de pesos (He, Xavier) e arquiteturas com conexões residuais (ResNet) ou memória seletiva (LSTM, GRU).

Conexão direta com este capítulo: a mesma limitação de representação em ponto flutuante que fez o Míssil Patriot falhar ([section 1.11](#)) também causa o desaparecimento de gradiente — números muito pequenos são arredondados para zero pela mantissa finita.

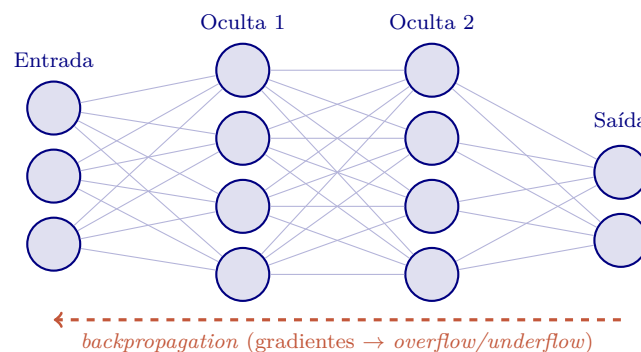


Figura 1.6: Rede neural com duas camadas ocultas. A seta inferior representa a propagação dos gradientes no *backpropagation*, na direção na qual o desaparecimento ou explosão de gradiente se manifesta.

1.1.5 Breve Histórico

O Cálculo Numérico não é disciplina recente — tem raízes que remontam à antiguidade. A [table 1.2](#) apresenta os principais marcos.

Tabela 1.2: Linha do tempo do Cálculo Numérico.

Período	Desenvolvimento
~1650 a.C.	Papiro de Rhind: método da falsa posição (Egito antigo)
~300 a.C.	Método babilônico para raízes quadradas (precursor de Newton)
Séc. IX	Al-Khwarizmi: métodos algébricos sistemáticos
Séc. XVII	Newton: método iterativo para raízes de equações
1947	Von Neumann aplica métodos numéricos ao ENIAC
1950s–60s	Algoritmos matriciais estáveis; bibliotecas LINPACK e EISPACK
1990s–2000s	NumPy, LAPACK, Eigen — bibliotecas modernas
2010s	Integração com Machine Learning e computação em GPU/TPU

Nota Importante

O Método Egípcio da Falsa Posição:

O Papiro de Rhind (também chamado Papiro Ahmes, datado de ~1650 a.C., atualmente no British Museum) contém o mais antigo registro conhecido de métodos numéricos sistemáticos. O escriba Ahmes descreve o método da “falsa posição” (*regula falsi*) para resolver equações do tipo:

$$x + \frac{x}{7} = 19. \quad (1.1)$$

Procedimento:

1. *Tentar* um valor inicial (“falso”), digamos $x_0 = 7$.
2. Calcular o resultado: $f(7) = 7 + 7/7 = 8$.
3. Como queremos 19, e obtivemos 8, o fator de correção é $19/8$.
4. Solução: $x = 7 \times (19/8) = 16.625$.

Este método é precursor da interpolação linear e dos métodos de busca de raízes modernos!

Referência: Robins, G., & Shute, C. (1987). *The Rhind Mathematical Papyrus*. British Museum Publications.

1.2 Representação de Números em Computadores

1.3 Motivação: Por que a Representação Importa?

Antes de estudar qualquer método numérico, é preciso entender como os números são **representados** e **armazenados** em sistemas computacionais e de que forma essa representação introduz erros nos cálculos. Os dois exemplos a seguir ilustram o problema.

Exemplo 1.2 (Área de uma circunferência). Calcule $A = \pi r^2$ para $r = 100$ metros.

$$A = \pi \cdot 100^2 = 10\,000\pi.$$

Como π é irracional, sua representação em qualquer máquina é sempre *aproximada*. Dependendo do número de dígitos utilizados, obtemos resultados distintos:

Valor de π usado	Resultado
3,14	31 400
3,1416	31 416
3,141692654	31 416,92654

Exemplo 1.3 (Somatório com ponto flutuante). Calcule $S = \sum_{i=1}^{3000} x_i$ para $x_i = 0,5$ e para $x_i = 0,11$.

	$x_i = 0,5$	$x_i = 0,11$
Calculadora	15 000	3 300
Computador	15 000	3 299,99691

O computador não obteve o resultado exato para $x_i = 0,11$.

Nota Importante

As diferenças observadas nos exemplos acima podem ocorrer em razão de:

1. A **base** utilizada para armazenar os números;
2. A **forma como os números são armazenados** (ponto flutuante);
3. Os **erros cometidos nas operações aritméticas**.

Para compreender essas causas, estudaremos a seguir os sistemas de numeração, a aritmética de ponto flutuante e os conceitos de erro numérico.

1.3.1 Dependência dos Resultados e o Problema do Edifício

Não é raro que os resultados finais estejam distantes do esperado mesmo quando todas as fases de resolução foram executadas corretamente. Os resultados dependem:

- (a) da **precisão dos dados de entrada**;
- (b) da **forma como** esses dados são representados no computador;
- (c) das **operações numéricas** efetuadas.

Para ilustrar, considere o seguinte problema concreto: você está no topo de um edifício de altura desconhecida. Com uma bola de metal e um cronômetro, como determinar a altura? Pela equação cinemática:

$$y = y_0 + v_0 t + \frac{1}{2} g t^2,$$

em que y_0 é a posição inicial, $v_0 = 0$ a velocidade inicial (solta em repouso), t o tempo de queda e $g \approx 9,8 \text{ m/s}^2$ a aceleração gravitacional.

Se a bola leva $t = 2,0 \text{ s}$, obtemos $h = \frac{1}{2} \times 9,8 \times 4 = 19,6 \text{ m}$.

Essa resposta é confiável? A pergunta fundamental do Cálculo Numérico.

Identificamos três fontes de erro neste problema:

- **Erros de modelagem:** o modelo ignora resistência do ar, velocidade do vento e a forma do objeto. Simplificações do modelo matemático introduzem erros sistemáticos.
- **Erros nos dados de entrada:** a leitura do cronômetro tem precisão limitada. Para $t = 2,3 \text{ s}$, a altura calculada é $h \approx 25,92 \text{ m}$, mas o valor lido do cronômetro depende dos reflexos humanos.
- **Erros de representação:** a representação binária de g e de t no computador pode introduzir erros de arredondamento que se propagam pelo cálculo.

1.4 Tipos de Erros

Em qualquer solução numérica, erros podem surgir em diferentes etapas. É útil classificá-los em três categorias:

Definição 1.2 (Erros de Modelagem). Erros obtidos pelo uso de dados experimentais incorretos ou pela simplificação excessiva do modelo matemático em relação ao fenômeno real. São associados à *matemática do modelo*, não ao método numérico em si.

Definição 1.3 (Erros de Truncamento). Erros provenientes da substituição de um processo matemático *exato* (como uma série infinita ou uma derivada) por uma *aproximação finita*. Por exemplo, truncar a série de Taylor de e^x em n termos introduz um erro de truncamento proporcional ao $(n + 1)$ -ésimo termo.

Definição 1.4 (Erros de Arredondamento). Erros causados pela representação de um número real com um número **finito de algarismos significativos**. Decorrem da imperfeição intrínseca da aritmética de ponto flutuante — um número como 0,1 não tem representação exata em binário.

Nota Importante

A distinção entre erros de truncamento e erros de arredondamento é fundamental para análise de métodos numéricos. Em geral:

- Erros de **truncamento** são controlados escolhendo métodos de ordem mais alta (mais termos, passo menor);
- Erros de **arredondamento** são controlados usando precisão maior (Float64 em vez de Float32, por exemplo).
- Existe um *trade-off*: reduzir o passo para diminuir o truncamento aumenta o número de operações e, portanto, o acúmulo de erros de arredondamento.

1.5 Sistemas de Numeração e Bases

1.5.1 Forma Geral

Normalmente utilizamos o sistema **decimal** para representar números. Nesse sistema, a posição de cada dígito indica a potência de 10 que ele representa. Por exemplo:

$$649 = 6 \cdot 10^2 + 4 \cdot 10^1 + 9 \cdot 10^0 = 6 \text{ centenas} + 4 \text{ dezenas} + 9 \text{ unidades.}$$

Em geral, qualquer número pode ser decomposto em potências de sua **base** β :

Definição 1.5 (Representação posicional). Um número N na base β é escrito como

$$(N)_\beta = (d_n d_{n-1} \cdots d_1 d_0, d_{-1} d_{-2} \cdots d_{-m})_\beta$$

em que cada dígito d_j satisfaz $0 \leq d_j < \beta$ e o valor decimal correspondente é

$$(N)_{10} = \sum_{j=-m}^n d_j \beta^j.$$

1.5.2 Sistema Decimal (Base 10)

Todos os múltiplos e submúltiplos são potências de 10. Exemplos:

$$\begin{aligned}(1834)_{10} &= 1 \cdot 10^3 + 8 \cdot 10^2 + 3 \cdot 10^1 + 4 \cdot 10^0, \\ (37,194)_{10} &= 3 \cdot 10^1 + 7 \cdot 10^0 + 1 \cdot 10^{-1} + 9 \cdot 10^{-2} + 4 \cdot 10^{-3}, \\ (6,032 \times 10^{23})_{10} &= (\text{notação científica}).\end{aligned}$$

1.5.3 Sistema Binário (Base 2)

Definição 1.6 (Sistema binário). Utiliza apenas os símbolos **0** e **1**, denominados *bits* (*binary digits*). Todo múltiplo e submúltiplo é potência de 2.

O sistema binário moderno foi formalizado por Leibniz no século XVII. Em 1854, Boole criou a álgebra que leva seu nome, e em 1937 Shannon mostrou como implementá-la em circuitos elétricos: a base de toda a computação digital moderna.

Exemplo 1.4 (Binário para decimal).

$$\begin{aligned}(101101)_2 &= 2^5 + 2^3 + 2^2 + 2^0 = 32 + 8 + 4 + 1 = (45)_{10}, \\ (100,101)_2 &= 2^2 + 2^{-1} + 2^{-3} = 4 + 0,5 + 0,125 = (4,625)_{10}.\end{aligned}$$

1.5.4 Sistema Hexadecimal (Base 16)

Definição 1.7 (Sistema hexadecimal). Usa 16 símbolos: 0–9 e A–F, em que A = 10, B = 11, ..., F = 15.

O sistema hexadecimal é amplamente utilizado em computação por sua compactação: um único dígito hex representa exatamente 4 bits, tornando-o muito mais legível que a sequência binária equivalente.

Exemplo 1.5 (Hexadecimal).

$$\begin{aligned}(AF83)_{16} &= 10 \cdot 16^3 + 15 \cdot 16^2 + 8 \cdot 16 + 3, \\ (1B3,E2)_{16} &= 1 \cdot 16^2 + 11 \cdot 16 + 3 + 14 \cdot 16^{-1} + 2 \cdot 16^{-2}, \\ (0,D4C)_{16} &= 13 \cdot 16^{-1} + 4 \cdot 16^{-2} + 12 \cdot 16^{-3}.\end{aligned}$$

1.6 Conversão entre Bases

1.6.1 Decimal para Binária

- **Parte inteira:** divisões sucessivas por 2; os restos, lidos de baixo para cima, formam a representação binária. O mecanismo pode ser visto a seguir:

Parte 1 — inteiro: $71_{10} \rightarrow$ binário

Divisões sucessivas por 2 · leia os restos de baixo para cima

Divisão	Quociente	Resto	bit
$71 \div 2$	35	1	b_0
$35 \div 2$	17	1	b_1
$17 \div 2$	8	1	b_2
$8 \div 2$	4	0	b_3
$4 \div 2$	2	0	b_4
$2 \div 2$	1	0	b_5
$1 \div 2$	0	1	b_6

leia de baixo
↑

$$(71)_{10} = (1000111)_2$$

Verificação — cada bit $\times$ sua potência de 2

1	0	0	0	1	1	1
2^6	2^5	2^4	2^3	2^2	2^1	2^0
64	0	0	0	4	2	1

$$64 + 0 + 0 + 0 + 4 + 2 + 1 = 71 \checkmark$$

- **Parte fracionária:** multiplicações sucessivas por 2; a parte inteira de cada produto é o próximo bit. O mecanismo pode ser visto a seguir:

Fração: $0,2_{10} \rightarrow$ binário

Multiplicações sucessivas por 2 · leia as partes inteiras de cima para baixo

Multiplicação	Bit
$0,2 \times 2 = 0,4$	0
$0,4 \times 2 = 0,8$	0
$0,8 \times 2 = 1,6$	1
$0,6 \times 2 = 1,2$	1
$0,2 \times 2 = 0,4 \quad \circlearrowleft$	0...

padrão
“0011”
repete ∞

$$0,2_{10} = 0,0011_2 = 0,00110011 \dots_2$$

Exemplo 1.6 (Conversão decimal para binário).

$$\begin{aligned}(71)_{10} &= (1000111)_2, \\ (23480)_{10} &= (101101110111000)_2, \\ (167,1875)_{10} &= (10100111,0011)_2, \\ (0,2)_{10} &= (0,00110011\dots)_2 \quad (\text{representação infinita!}).\end{aligned}$$

Cuidado

Um número sem representação binária finita é *truncado ou arredondado* ao ser armazenado, gerando erros que podem parecer inexplicáveis. O número $0,1_{10}$, por exemplo, não tem representação exata em binário — e isso foi a causa direta do desastre do Míssil Patriot (veja [section 1.11](#)).

1.6.2 Binária para Decimal

Basta aplicar a [theorem 1.5](#): multiplica-se cada bit pela potência de 2 correspondente à sua posição.

Nota Importante

Todo número *inteiro* decimal pode ser representado exatamente em binário. O mesmo **não** é verdade para números fracionários. Analise os casos $(0,1)_{10}$ e $(0,11)_{10}$: nenhum dos dois possui representação binária finita.

1.6.3 Hexadecimal para Binária

Como $16 = 2^4$, cada dígito hexadecimal corresponde exatamente a **4 bits**. A conversão é feita agrupando os bits de 4 em 4 a partir da vírgula. Não há perda de dígitos significativos nessa conversão.

Exemplo 1.7 (Conversão hex–binário).

$$\begin{aligned}(A7C)_{16} &= (\underbrace{1010}_A \underbrace{0111}_7 \underbrace{1100}_C)_2, \\ (3D,F)_{16} &= (\underbrace{0011}_3 \underbrace{1101}_D, \underbrace{1111}_F)_2.\end{aligned}$$

A conversão inversa agrupa os bits em quartetos:

$$(1000\ 1010\ 0110\ 1110)_2 = (8A6E)_{16}.$$

Aplicação Computacional

Por que hexadecimal em computação?

Endereços de memória, códigos de cor em CSS (#FF5733), chaves criptográficas e outras estruturas computacionais são frequentemente exibidas em hexadecimal porque um byte (8 bits) é representado por exatamente 2 dígitos hex — muito mais legível que 8 dígitos binários.

$$0xFF = 1111\ 1111_2 = 255_{10}$$

1.7 Aritmética de Ponto Flutuante

1.7.1 Representação

A representação de números reais mais utilizada em máquinas é a de **ponto flutuante**. Um número nesse sistema tem a forma:

Definição 1.8 (Número em ponto flutuante).

$$\pm 0,d_1d_2\cdots d_t \times \beta^e$$

em que:

- β é a *base* (tipicamente 2, 10 ou 16);
- $d_1d_2\cdots d_t$ é a *mantissa*, com $1 \leq d_1 \leq \beta - 1$ (número normalizado, $d_1 \neq 0$);
- t é o número de dígitos da mantissa (precisão);
- e é o *expoente* inteiro, com $e_{\min} \leq e \leq e_{\max}$.

1.7.2 Bits, Bytes e Palavras

- Um dígito binário é denominado **bit** (*binary digit*).
- Um grupo de 8 bits corresponde a **1 byte**.
- A representação de um número no computador ocupa um número fixo de bits chamado *palavra de computador*.
- Quanto maior a palavra, mais veloz e mais preciso o computador.
- Computadores modernos operam com palavras de **64 bits**.

A estrutura de uma palavra de ponto flutuante no padrão **IEEE 754** é dividida em três campos: sinal (S), expoente e mantissa.

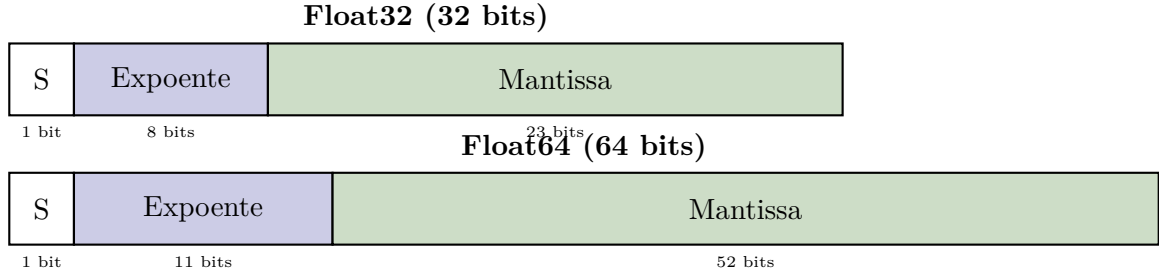


Figura 1.8: Estrutura dos formatos IEEE 754. S = sinal (0 = positivo, 1 = negativo). Float32 é usado em GPU e aplicações que priorizam velocidade; Float64 é o padrão em cálculo científico.

Tabela 1.3: Formatos IEEE 754 mais utilizados.

Tipo	Bits totais	Sinal	Expoente	Mantissa	Precisão
Float32 (float)	32	1	8	23	≈ 7 dígitos decimais
Float64 (double)	64	1	11	52	≈ 15 dígitos decimais

Nota Importante

Python utiliza Float64 (*double precision*) como padrão para variáveis do tipo `float`. Com NumPy é possível controlar a precisão explicitamente usando `np.float32` ou `np.float64`.

1.7.3 Sistema $F(\beta, t, e_{\min}, e_{\max})$

Definição 1.9 (Sistema de ponto flutuante). Uma aritmética de ponto flutuante F é completamente caracterizada por quatro inteiros:

$$F(\beta, t, e_{\min}, e_{\max}), \quad F \subset \mathbb{R}.$$

O número total de elementos de F (incluindo o zero) é:

$$2(\beta - 1) \beta^{t-1} (e_{\max} - e_{\min} + 1) + 1.$$

Exemplo 1.8 ($F(2, 2, -1, 2)$). Os números representáveis (normalizados, $d_1 \neq 0$) têm a forma $\pm 0, d_1 d_2 \times 2^e$, com $d_1 = 1$, $d_2 \in \{0, 1\}$ e $e \in \{-1, 0, 1, 2\}$. O total de elementos é:

$$2 \cdot 1 \cdot 2^1 \cdot (2 - (-1) + 1) + 1 = 2 \cdot 2 \cdot 4 + 1 = \mathbf{17}.$$

1.8 Erros na Representação dos Números

O conjunto dos reais é **infinito**; a representação em ponto flutuante é **finita**. Essa limitação tem duas origens:

1.8.1 *Overflow e Underflow*

Definição 1.10 (*Overflow*). Ocorre quando o resultado de uma operação produz um expoente **superior** a $e_{\max}$. No padrão IEEE 754, o sistema retorna **inf** (infinito).

Definição 1.11 (*Underflow*). Ocorre quando o resultado produz um expoente **inferior** a $e_{\min}$. O sistema pode retornar zero ou um número desnormalizado.

Exemplo 1.9. Overflow e underflow em $F(10, 2, -5, 5)$:

1. $x = 875$ e $y = 3172$. Calcule $x \times y$.
Resultado exato: $2\,776\,500 = 0,27765 \times 10^7$. Aqui temos um expoente: $7 > e_{\max} = 5$: **overflow**.
2. $x = 0,064$ e $y = 7312$. Calcule $x \div y$.
Resultado exato: $\approx 8,75 \times 10^{-6}$. Aqui temos um expoente: $-6 < e_{\min} = -5$: **underflow**.

```

1 import numpy as np
2
3 max_f64 = np.finfo(np.float64).max
4 min_f64 = np.finfo(np.float64).tiny
5
6 print(f"Maximo Float64 : {max_f64:.4e}")      # ~1.7977e+308
7 print(f"Minimo Float64 : {min_f64:.4e}")      # ~2.2251e-308
8
9 print(f"Overflow   : {max_f64 * 2}")          # inf
10 print(f"Underflow  : {min_f64 / 1e100}")      # 0.0

```

Listing 1.1: Demonstração de *overflow* e *underflow* em Python

1.8.2 Mantissa Finita

Mesmo que o expoente esteja dentro do intervalo permitido, a mantissa representa apenas um número finito de dígitos. Números que exigem infinitos dígitos são *truncados* ou *arredondados*.

Exemplo 1.10 (Mantissa finita — base 10). Numa máquina com 6 dígitos na mantissa, o número 0,11 é armazenado exatamente:

$$(0,11)_{10} = 0,110000 \times 10^0.$$

Já o número π exigiria uma mantissa infinita: $\pi = 0,314159265 \dots \times 10^1$, sendo armazenado de forma aproximada.

Exemplo 1.11 (Mantissa finita — base 2).

$$(0,6)_{10} = (0,1001100110011\dots)_2 \Rightarrow \text{representação infinita!}$$

$$(0,7)_{10} = (0,1011001100110\dots)_2 \Rightarrow \text{representação infinita!}$$

Ao armazenar esses valores com mantissa finita, introduz-se erro de representação, que se acumula em operações repetidas (como no [example 1.3](#)).

1.9 Truncamento e Arredondamento

Ao representar um número com mantissa de t dígitos, os dígitos além da posição t devem ser descartados. Há duas estratégias:

Definição 1.12 (Truncamento). Desprezam-se todos os dígitos a partir da posição $(t+1)$, sem modificar d_t :

$$0, d_1 d_2 \dots d_t \underbrace{d_{t+1} d_{t+2} \dots}_{\text{desprezados}}.$$

Definição 1.13 (Arredondamento). • Se $d_{t+1} < 5$: despreza-se tudo a partir de d_{t+1} (mantém d_t).

- Se $d_{t+1} \geq 5$: soma-se 1 a d_t e despreza-se o restante.

Exemplo 1.12 (Arredondamento $\times$ truncamento em $F(10, 3, -4, 4)$). Representar cada número no formato $\pm 0, d_1 d_2 d_3 \times 10^e$.

Número	Forma normalizada	Truncamento	Arredondamento
1,25	$0,125 \times 10^1$	$0,125 \times 10^1$	$0,125 \times 10^1$
10,053	$0,10053 \times 10^2$	$0,100 \times 10^2$	$0,101 \times 10^2$
-238,15	$-0,23815 \times 10^3$	$-0,238 \times 10^3$	$-0,238 \times 10^3$
2,71828...	$0,271828 \dots \times 10^1$	$0,271 \times 10^1$	$0,272 \times 10^1$
0,000007	$0,700 \times 10^{-5}$	$0,700 \times 10^{-5}$	$0,700 \times 10^{-5}$
718235,82	$0,71823582 \times 10^6$	$0,718 \times 10^6$	$0,718 \times 10^6$

1.10 Conceitos de Erro Numérico

Nota Importante

Nenhum resultado obtido por cálculos eletrônicos ou métodos numéricos tem valor se não houver **conhecimento e controle** sobre os possíveis erros envolvidos. A análise de erros é uma **etapa fundamental** em qualquer solução numérica.

1.10.1 Número Aproximado

Definição 1.14 (Aproximação). Um número $\tilde{x}$ é uma *aproximação* do valor exato x se a diferença entre eles é pequena.

- **Aproximação por falta:** $\tilde{x} < x$.
- **Aproximação por excesso:** $\tilde{x} > x$.

Exemplo 1.13. $\sqrt{2}$ é irracional e só pode ser aproximado:

$$\underbrace{1,41}_{\text{por falta}} < \sqrt{2} < \underbrace{1,42}_{\text{por excesso}}.$$

1.10.2 Erro Absoluto e Erro Relativo

Definição 1.15 (Erro absoluto).

$$EA_x = |x - \tilde{x}|.$$

Mede o **tamanho** do desvio em unidades do problema.

Definição 1.16 (Erro relativo).

$$ER_x = \frac{|x - \tilde{x}|}{|x|}, \quad x \neq 0.$$

Mede o erro de forma **proporcional** ao valor exato, permitindo comparar aproximações de grandezas diferentes.

O erro absoluto informa o *tamanho* do desvio; o erro relativo informa a *qualidade* da aproximação — sendo mais informativo quando se comparam grandezas de ordens de magnitude distintas.

Exemplo 1.14 (Por que o erro relativo é mais informativo?). Sejam $x = 100$, $\tilde{x} = 100,1$, $y = 0,0006$ e $\tilde{y} = 0,0004$.

$$\begin{aligned} EA_x &= |100 - 100,1| = 0,1, & ER_x &= \frac{0,1}{100} = 0,1\%, \\ EA_y &= |0,0006 - 0,0004| = 0,0002, & ER_y &= \frac{0,0002}{0,0006} \approx 33,3\%. \end{aligned}$$

Os erros absolutos são da mesma ordem, mas o erro relativo revela que a aproximação de y é **330 vezes pior** do que a de x .

1.10.3 Erro de Arredondamento em Ponto Flutuante e Precisão de Máquina

Teorema 1.17 (Erro de arredondamento). Seja $x \in \mathbb{R}$ representável em um sistema de ponto flutuante. Existe $\tilde{x}$ (representação de x) tal que:

$$\frac{|x - \tilde{x}|}{|x|} \leq \varepsilon_m, \quad \varepsilon_m = \beta^{1-t}.$$

A constante ε_m é chamada *precisão de máquina* ou *épsilon de máquina*.

Para Float64 ($\beta = 2$, $t = 53$): $\varepsilon_m = 2^{-52} \approx 2,22 \times 10^{-16}$ — cerca de 15 dígitos decimais corretos.

```

1 import numpy as np
2
3 eps = np.finfo(np.float64).eps
4 print(f"Epsilon de maquina (NumPy) : {eps}") #
   2.220446049250313e-16
5
6 # Calculo experimental: menor eps tal que 1.0 + eps > 1.0
7 eps_exp = 1.0
8 while 1.0 + eps_exp / 2.0 > 1.0:
9     eps_exp /= 2.0
10 print(f"Epsilon experimental      : {eps_exp}")
11
12 # Comparacao Float32 vs Float64
13 print(f"Float32 epsilon: {np.finfo(np.float32).eps:.2e}") #
   ~1.19e-07
14 print(f"Float64 epsilon: {np.finfo(np.float64).eps:.2e}") #
   ~2.22e-16

```

Listing 1.2: Precisão de máquina em Python

1.10.4 Propagação de Erros

Teorema 1.18 (Propagação em operações básicas). Sejam x e y com erros relativos ε_x e ε_y . Então:

$$\text{Adição/Subtração: } \frac{|E(x \pm y)|}{|x \pm y|} \approx \frac{|x| \varepsilon_x + |y| \varepsilon_y}{|x \pm y|}, \quad (1.2)$$

$$\text{Multiplicação: } \frac{|E(x \cdot y)|}{|x \cdot y|} \approx \varepsilon_x + \varepsilon_y, \quad (1.3)$$

$$\text{Divisão: } \frac{|E(x/y)|}{|x/y|} \approx \varepsilon_x + \varepsilon_y. \quad (1.4)$$

Cuidado

Cancelamento catastrófico:

Ocorre na subtração de números próximos. Se $x \approx y$, então $|x - y|$ é pequeno, mas o denominador da fórmula de adição/subtração no [theorem 1.18](#) também é pequeno, podendo ampliar enormemente o erro relativo:

$$\frac{|x| \varepsilon_x + |y| \varepsilon_y}{|x - y|} \gg \varepsilon_x, \varepsilon_y \quad \text{quando } |x - y| \ll |x|, |y|.$$

Este é um dos erros mais insidiosos da computação numérica, pois não gera mensagem de erro, apenas produz um resultado impreciso.

1.11 Desastres Causados por Erros Numéricos

A importância do cálculo numérico correto vai além da teoria. Os exemplos a seguir documentam consequências concretas e dramáticas de falhas na representação numérica.

1.11.1 Míssil Patriot (1991) — 28 mortes

Cuidado

Em 25 de fevereiro de 1991, durante a Guerra do Golfo, um míssil Scud atingiu um quartel em Dhahran (Arábia Saudita), matando 28 soldados. O sistema antimíssil Patriot falhou por **erro de arredondamento acumulado**.

Causa: O sistema rastreava o tempo em décimos de segundo, usando um inteiro de 24 bits convertido para segundos pela multiplicação por 0,1. O problema: 0,1 não tem representação exata em binário:

$$0,1_{10} = (0,0001100110011 \dots)_2 \quad (\text{dízima periódica}).$$

Com 24 bits: $0,1 \approx 0,09999999403953552$. Erro por décimo de segundo: $\approx 9,5 \times 10^{-8} \text{ s}$.

Após mais de 100 horas de operação contínua ($\approx 360\,000$ décimos de segundo):

$$\text{Erro acumulado} = 360\,000 \times 9,5 \times 10^{-8} \approx 0,34 \text{ s}.$$

Um Scud viaja a $\approx 1\,676 \text{ m/s}$; em 0,34 s percorre $\approx 570 \text{ m}$ — suficiente para sair da janela de busca do radar.

Referência: gao1992.

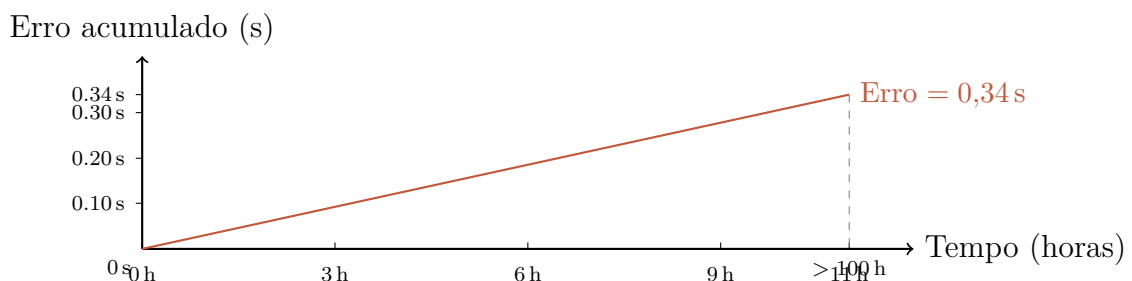


Figura 1.9: Acúmulo linear do erro de representação de 0,1 no sistema Patriot ao longo de mais de 100 horas de operação contínua. A cada décimo de segundo, $9,5 \times 10^{-8} \text{ s}$ de erro se acumula.

1.11.2 Foguete Ariane 5 (1996) — US\$ 370 milhões

Cuidado

Em 4 de junho de 1996, o Ariane 5 explodiu 36,7 segundos após o lançamento. A missão custou US\$ 370 milhões e 10 anos de desenvolvimento.

Causa: O software de controle inercial (herdado diretamente do Ariane 4, sem adaptação) converteu a velocidade horizontal de Float64 para Int16. O Ariane 5 tinha velocidade horizontal maior do que o Ariane 4 e o valor excedeu o máximo representável por um inteiro de 16 bits ($2^{15} - 1 = 32\,767$), causando **overflow**.

A exceção gerada não foi tratada: o sistema inercial primário desligou; o sistema de backup (com o mesmo software) também falhou imediatamente; o foguete recebeu dados de trajetória errôneos e se autodestruuiu.

Lição: o software do Ariane 4 foi validado para as condições desse foguete – mas não foi re-validado para as condições do Ariane 5, que era mais veloz.

Referência: lions1996.



Figura 1.10: Explosão do Ariane 5 em seu voo inaugural (4 de junho de 1996), 37 segundos após o lançamento. A falha foi causada por *overflow* na conversão Float64 → Int16. Fonte: ESA/CNES/ARIANESPACE.

1.11.3 Índice de Vancouver (1982) — Queda fictícia de bolsa

Cuidado

O Vancouver Stock Exchange lançou um índice em janeiro de 1982, começando em 1000,000 pontos. Em 22 meses, o índice chegou a 524,811 pontos — apesar de não ter havido nenhum crash real na bolsa.

Causa: Após cada recálculo do índice (milhares por dia), o resultado era **truncado** (não arredondado) para 3 casas decimais. Cada operação perdia em média 0,0005 ponto; com $\sim 3\,000$ transações diárias:

$$\text{Erro diário} \approx 3\,000 \times 0,0005 = 1,5 \text{ ponto.}$$

Em ~ 480 dias úteis: perda acumulada de ≈ 720 pontos.

Após a descoberta, o índice foi recalculado corretamente e voltou a $\approx 1\,098,892$ pontos — o valor correto.

Referência: quinn2006.

1.11.4 Therac-25 (1985–1987) – 6 mortes por radiação

Cuidado

Entre 1985 e 1987, o acelerador linear de radioterapia Therac-25 administrou overdoses massivas de radiação em pelo menos 6 pacientes, causando mortes e mutilações permanentes [?].

Causas (múltiplas, incluindo *overflow*):

- ***Overflow* em contador de 8 bits:** após 256 incrementos, o contador retornava a zero. Nessa condição, o sistema entrava em modo de alta energia *sem* ativar a proteção adequada.
- Condições de corrida (*race conditions*) no software de controle.
- Remoção dos interlocks mecânicos de segurança por confiança excessiva no software.

Doses administradas: até **100 vezes** a dose prescrita (20 000 rad em vez de 200 rad).

Lição: remoção de salvaguardas físicas exige validação formal rigorosa do software substituto. Contadores com *overflow* são armadilhas clássicas em sistemas de segurança crítica.

1.11.5 Bug FDIV do Pentium (1994) — US\$ 475 milhões

Cuidado

Em 1994, foi descoberto um erro na Unidade de Ponto Flutuante (FPU) do processador Intel Pentium [?]. O bug afetava divisões de ponto flutuante em casos específicos.

Causa técnica: cinco entradas incorretas na tabela de *lookup* usada pelo algoritmo de divisão SRT (Sweeney-Robertson-Tocher). Em divisões específicas, o resultado era impreciso a partir do quinto dígito significativo. Exemplo clássico:

$$\frac{4\,195\,835}{3\,145\,727} = 1,333820449136241 \quad (\text{correto})$$

O Pentium retornava 1,3337**39**068902037 — erro no 5º dígito.

Intel estimou que o usuário típico encontraria o erro uma vez a cada 27 000 anos. Matemáticos e cientistas em aplicações críticas discordaram veementemente. O *recall* voluntário custou US\$ 475 milhões.

Lição: erros de baixa probabilidade em hardware amplamente distribuído tornam-se estatisticamente certos em escala. Precisão importa mesmo em aplicações “convencionais”.

```
>>> 4195835 / 3145727
1.3337390689020376 (ERRADO - Pentium)

Correto: 1.333820449136241002
      ^^^
      Erro aqui!
```

Figura 1.11: Processador Intel Pentium (1993). O erro na tabela de divisão da FPU causou um *recall* de US\$ 475 milhões — o maior da história da indústria de semicondutores até então. Fonte: Wikimedia Commons.

Nota Importante**Lições comuns dos cinco desastres:**

1. **Erros pequenos se acumulam:** $9,5 \times 10^{-8} \text{ s} \times 360\,000 = 0,34 \text{ s} = 28 \text{ mortes}$.
2. **Truncamento $\neq$ arredondamento:** o VSE perdeu ~ 476 pontos por truncamento sistemático em vez de arredondamento.
3. **Conversões de tipo são perigosas:** $\text{Float64} \rightarrow \text{Int16} = \text{US\$ } 370 \text{ milhões}$.
4. **Software deve ser re-validado para novas condições de uso:** o Ariane 4 não foi testado com velocidades do Ariane 5.
5. **Overflow em contadores causa falhas silenciosas:** o Therac-25 não emitia mensagem de erro — apenas administrava a dose errada.
6. **Precisão importa mesmo em aplicações de uso geral:** um erro de $0,006\%$ no Pentium gerou crise de US\$ 475 milhões.

1.12 Condicionamento e Estabilidade Numérica

A qualidade de uma solução numérica depende não apenas do algoritmo, mas também da sensibilidade do próprio problema matemático às perturbações nos dados. Essas duas propriedades — condicionamento do problema e estabilidade do algoritmo — são conceitualmente distintas e igualmente importantes.

1.12.1 Condicionamento de Problemas

Definição 1.19 (Problema bem-condicionado e mal-condicionado). Um problema é *bem-condicionado* se pequenas perturbações nos dados de entrada produzem pequenas mudanças na solução. Caso contrário, é *mal-condicionado*.

Definição 1.20 (Número de condição). Para um problema $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, o número de condição mede a amplificação máxima do erro relativo:

$$\kappa = \lim_{\delta \rightarrow 0} \sup_{\|\Delta x\| \leq \delta} \frac{\|f(x + \Delta x) - f(x)\| / \|f(x)\|}{\|\Delta x\| / \|x\|}.$$

Para sistemas lineares $Ax = b$:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|.$$

Um número de condição próximo de 1 indica problema bem-condicionado; $\kappa \gg 1$ indica mal-condicionamento.

Exemplo 1.15 (Número de condição — sistemas lineares). Seja $A = \begin{pmatrix} 1 & 1 \\ 1 & 1,0001 \end{pmatrix}$. Perturbando o vetor b em δb , o erro relativo na solução pode ser amplificado por até $\kappa(A) \approx 40\,000$. Um erro de entrada de 0,01% pode gerar erro de saída de 400%.

Aplicação Computacional

Visão Computacional — Reconstrução 3D [?]:

Sistemas de câmeras múltiplas resolvem sistemas lineares para triangulação de pontos 3D. Matrizes mal-condicionadas ($\kappa \gg 1$) ocorrem tipicamente quando:

- Câmeras estão quase colineares (ângulo de baseline muito pequeno);
- Pontos de cena estão muito distantes do par de câmeras;
- Resolução da imagem é insuficiente para localização precisa dos pontos correspondentes.

Regularização de Tikhonov (equivalente a *ridge regression* em aprendizado de máquina) penaliza soluções de grande norma, melhorando efetivamente o condicionamento numérico do sistema.

1.12.2 Estabilidade de Algoritmos

Definição 1.21 (Algoritmo numericamente estável). Um algoritmo é *numericamente estável* se produz uma solução exata para um problema próximo ao original. Formalmente: se o algoritmo calcula $\tilde{y}$ para a entrada x , então existe $\tilde{x}$ próximo de x tal que $\tilde{y} = f(\tilde{x})$ exatamente.

Estabilidade e condicionamento são conceitos complementares: um algoritmo estável aplicado a um problema mal-condicionado ainda pode produzir resultados de baixa qualidade — porque o problema em si amplifica os erros.

Exemplo 1.16 (Soma de números — estabilidade prática). Considere somar n números positivos de magnitudes muito diferentes. Três estratégias de complexidade idêntica ($\mathcal{O}(n)$) produzem resultados numericamente distintos:

1. **Ordem crescente:** soma do menor ao maior — minimiza o cancelamento catastrófico acumulado.
2. **Ordem aleatória:** pode acumular erros dependendo dos dados.
3. **Soma *pairwise*:** dividir e conquistar. Comportamento similar à soma crescente, $\mathcal{O}(\log n)$ níveis.

```

1 import numpy as np
2
3 def soma_crescente(arr):
4     return float(np.sum(np.sort(arr)))          # crescente
5
6 def soma_pairwise(arr):
7     if len(arr) == 1: return arr[0]
8     mid = len(arr) // 2
9     return soma_pairwise(arr[:mid]) + soma_pairwise(arr[mid:])
10
11 n = 10_000
12 arr = np.array([1.0] * n + [1e16])           # 1 gigante + muitos
13                                             pequenos
14 print(f"Soma direta : {float(np.sum(arr)):.6e}")    # pode
15                                             perder os 1s
16 print(f"Soma crescente: {soma_crescente(arr):.6e}")  # mais
17                                             estavel
18 print(f"Soma pairwise : {soma_pairwise(arr):.6e}")  # idem
19 print(f"Valor esperado: {1e16 + n:.6e}")

```

Listing 1.3: Comparação de estabilidade na soma de números com magnitudes muito distintas

1.13 Complexidade Computacional de Algoritmos Numéricos

Além da precisão, um algoritmo numérico deve ser avaliado pelo seu *custo computacional*: quantas operações de ponto flutuante (*flops*) são necessárias e como esse custo escala com o tamanho do problema.

1.13.1 Notação Big- $\mathcal{O}$

Definição 1.22 (Notação $\mathcal{O}$). Dizemos que $f(n) = \mathcal{O}(g(n))$ se existem constantes $c > 0$ e n_0 tais que

$$|f(n)| \leq c \cdot g(n), \quad \forall n \geq n_0.$$

A notação descreve o comportamento assintótico — como o custo cresce quando $n \rightarrow \infty$, independentemente de constantes multiplicativas.

Tabela 1.4: Classes de complexidade mais comuns em algoritmos numéricos.

Complexidade	Nome	Exemplo em CN
$\mathcal{O}(1)$	Constante	Acesso a elemento de <i>array</i>
$\mathcal{O}(\log n)$	Logarítmica	Busca binária em tabela
$\mathcal{O}(n)$	Linear	Avaliação de polinômio (Horner)
$\mathcal{O}(n \log n)$	Log-linear	FFT (<i>Fast Fourier Transform</i>)
$\mathcal{O}(n^2)$	Quadrática	Produto matriz-vetor; eliminação de Gauss (banda)
$\mathcal{O}(n^3)$	Cúbica	Eliminação de Gauss geral; decomposição LU
$\mathcal{O}(n^{2.37})$	Strassen	Produto matriz-matriz (algoritmo de Strassen)
$\mathcal{O}(2^n)$	Exponencial	Subconjuntos; alguns problemas NP-difíceis

1.13.2 Análise de Algoritmos Numéricos

Algoritmos numéricos são avaliados por quatro critérios simultâneos:

1. **Complexidade de tempo:** número de operações de ponto flutuante (*flops*).
2. **Complexidade de espaço:** memória necessária.
3. **Precisão:** erro máximo da aproximação.
4. **Estabilidade:** robustez a erros de arredondamento.

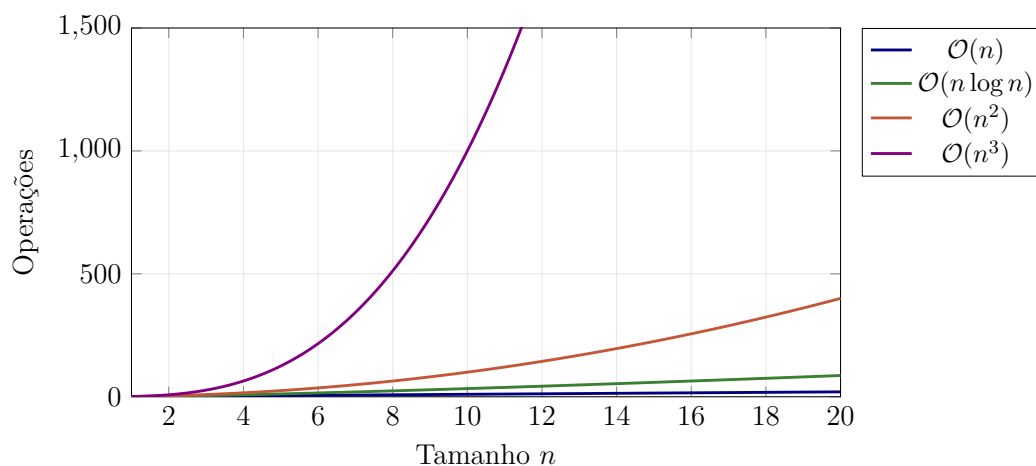


Figura 1.12: Crescimento do custo computacional para as quatro complexidades mais relevantes em Cálculo Numérico. Note como $\mathcal{O}(n^3)$ torna-se rapidamente proibitivo.

Aplicação Computacional**Operações Matriciais em Ciência de Dados [3]:**

Em aprendizado de máquina e ciência de dados, operações matriciais dominam o custo computacional:

Operação	Complexidade	Onde aparece
Produto matriz-vetor	$\mathcal{O}(n^2)$	Inferência em camada densa
Produto matriz-matriz	$\mathcal{O}(n^3)$ / $\mathcal{O}(n^{2,37})$	Backpropagation, atenção
Decomposição LU	$\mathcal{O}(n^3)$	Sistemas lineares, regressão
SVD	$\mathcal{O}(mn^2)$	Recomendação, PCA, compressão
FFT	$\mathcal{O}(n \log n)$	Processamento de sinais

Para matrizes $10\,000 \times 10\,000$, $\mathcal{O}(n^3)$ implica $\sim 10^{12}$ operações — inviável em CPU sem paralelização. GPUs modernas realizam $\sim 10^{13}$ *flops*/segundo, tornando possível treinar modelos de linguagem com bilhões de parâmetros.

1.14 Ferramentas Computacionais

A resolução prática de problemas numéricos exige uma linguagem de programação e bibliotecas adequadas.

Linguagens de programação mais usadas:

- **Python** — dominante em ciência de dados e ensino;
- **MATLAB** / **Scilab** / **R** — computação científica e estatística;
- **C**, **C++**, **FORTRAN** — alto desempenho em sistemas críticos.

Ecossistema Python para cálculo numérico:

- **NumPy** — arrays multidimensionais e álgebra linear;
- **SciPy** — integração numérica, otimização, EDOs e mais;
- **SymPy** — computação simbólica (derivadas, integrais exatas);
- **Matplotlib** — visualização de dados e funções;
- **JAX** — diferenciação automática e aceleração em GPU/TPU.

```
1 import numpy as np
2
3 # Criar sistema linear aleatorio (5x5)
4 A = np.random.rand(5, 5)
5 b = np.random.rand(5)
6
7 # Resolucao numerica por eliminacao Gaussiana
8 x = np.linalg.solve(A, b)
9
10 # Verificacao do residuo: ||Ax - b||
11 residuo = np.linalg.norm(A @ x - b)
12 print(f"Residuo: {residuo:.2e}") # deve ser da ordem de 1e-15
13
14 # Python usa float64 por padrao
15 print(f"Tipo de dados: {x.dtype}") # float64
16 print(f"Epsilon de maquina: {np.finfo(np.float64).eps:.2e}")
```

Listing 1.4: Exemplo de uso do NumPy: resolução de sistema linear $Ax = b$

Nota Importante

Python usa `float64` (IEEE 754, precisão dupla) por padrão para variáveis do tipo `float`. Isso garante $\varepsilon_m \approx 2,22 \times 10^{-16}$, ou seja, cerca de 15 dígitos decimais corretos na maior parte dos cálculos.

1.15 Exercícios

Exercício 1.1. No sistema $F(10, 3, -4, 4)$, represente cada número abaixo usando **truncamento** e **arredondamento**. Calcule os erros absoluto e relativo em cada caso.

- (a) 1,25
- (b) 10,053
- (c) 2,71828... (e de Euler)
- (d) 718 235,82

Exercício 1.2. Uma calculadora armazena apenas **4 dígitos** na mantissa com arredondamento. Um agricultor registra $x = 17\,534$ árvores e $y = 21\,178$ frutos por árvore.

- (a) Como esses valores são armazenados na máquina?
- (b) Calcule EA_x , ER_x , EA_y e ER_y .
- (c) Qual dos dois valores sofre maior erro relativo? Justifique.

Exercício 1.3. No sistema $F(10, 2, -5, 5)$, com formato $\pm 0, d_1 d_2 \times 10^e$:

- (a) $x = 4,32$ e $y = 0,064$. Calcule $x + y$ e determine o erro.
- (b) $x = 372$ e $y = 371$. Calcule $x - y$ e discuta o cancelamento catastrófico.
- (c) $x = 691$ e $y = 2,71$. Calcule $x + y$.

Exercício 1.4. Escreva um programa Python que calcule experimentalmente a precisão de máquina para `float32` e `float64`. Compare com os valores teóricos 2^{-23} e 2^{-52} .

Exercício 1.5. A equação $e^x = 3x$ não tem solução em forma fechada.

- (a) Como você encontraria a raiz numericamente?
- (b) Quais informações sobre o erro da solução aproximada seriam relevantes reportar?

Exercício 1.6. Explique, com exemplos concretos, a diferença entre **erro de truncamento** e **erro de arredondamento**. Em que situações cada um é dominante?

Exercício 1.7. No problema do edifício, suponha que o cronômetro tem precisão de $\pm 0,05$ s e que $g = 9,80 \text{ m/s}^2$ é conhecido com erro absoluto de $0,01 \text{ m/s}^2$. Para $t = 2,0$ s:

- (a) Calcule a altura h e o erro absoluto EA_h usando o teorema de propagação de erros ([theorem 1.18](#)).
- (b) Qual fonte de incerteza (em g ou em t) contribui mais para o erro em h ?

Exercício 1.8 (Desafio). A função $f(x) = \sqrt{x+1} - \sqrt{x}$ para x grande sofre de cancelamento catastrófico.

- (a) Calcule $f(10^{10})$ diretamente em Python. O que ocorre?
- (b) Reescreva $f(x)$ de forma matematicamente equivalente e numericamente estável (dica: racionalizar).
- (c) Implemente e compare as duas versões. Para quais valores de x a diferença se torna significativa?

Exercício 1.9 (Número de condição). Demonstre analiticamente que:

- (a) $\kappa(I) = 1$ para a matriz identidade (melhor caso possível).
- (b) $\kappa(A) = \infty$ para qualquer matriz singular A .
- (c) Para a matriz de Hilbert $H_{ij} = 1/(i + j - 1)$, calcule $\kappa(H)$ para $n = 3, 5, 8$ usando Python (`np.linalg.cond`) e comente sobre o condicionamento.

Exercício 1.10 (Fórmula de Binet e propagação de erros). A sequência de Fibonacci $F_n = F_{n-1} + F_{n-2}$, com $F_0 = 0$ e $F_1 = 1$, tem solução fechada (fórmula de Binet):

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right].$$

- (a) Implemente as versões recursiva, iterativa e via fórmula de Binet.
- (b) Compare os resultados para $n = 50, 75, 100$. O que acontece com a fórmula de Binet para n grande? Justifique em termos de propagação de erros.
- (c) O menor termo $\left(\frac{1 - \sqrt{5}}{2} \right)^n \rightarrow 0$ rapidamente. Como isso afeta a precisão? Estime o erro relativo introduzido ao ignorar esse termo.

Exercício 1.11 (*Forward error* $\times$ *backward error*). Considere resolver $f(x) = x^2 - 2 = 0$ com a aproximação $\tilde{x} = 1,41421$.

- (a) Calcule o *forward error*: $|\tilde{x} - \sqrt{2}|$.
- (b) Calcule o *backward error*: $|f(\tilde{x})| = |(\tilde{x})^2 - 2|$.
- (c) Explique por que backward error ≈ 0 não implica forward error ≈ 0 em problemas mal-condicionados.
- (d) Qual dos dois critérios é mais fácil de calcular na prática? Qual é mais informativo sobre a qualidade da solução?

Exercício 1.12 (Projeto — calculadora de precisão arbitrária). Implemente uma calculadora usando a biblioteca `decimal` do Python com 50 dígitos de precisão. Compare os resultados das seguintes operações “problemáticas” usando `float64` $\times$ precisão arbitrária:

- (a) $1/3 + 1/3 + 1/3 - 1$.
- (b) $\sum_{i=1}^{10000} 0,1 - 1000$.
- (c) $\sqrt{2^2} - 2$.

Explique cada discrepância em termos dos conceitos deste capítulo.

1.16 Referências e Leituras Complementares

As referências citadas neste capítulo e leituras adicionais recomendadas estão reunidas na bibliografia ao final do livro. Para facilitar a consulta, os itens mais relevantes para este capítulo são:

Livros-texto fundamentais:

- **Cálculo Numérico brasileiro:** franco2007; ruggiero1997; sperandio2003.
- **Referências internacionais clássicas:** burden2015; higham2002 — referência definitiva sobre erros numéricos e estabilidade.

Artigos e relatórios técnicos:

- goldberg1991: leitura obrigatória sobre aritmética de ponto flutuante.
- gao1992: análise oficial da falha do Patriot.
- lions1996: relatório da comissão de inquérito do Ariane 5.
- leveson1993: análise do Therac-25.
- pascanu2013: gradiente em redes recorrentes.

Leituras adicionais recomendadas:

- trefethen1997: álgebra linear numérica com ênfase em estabilidade.
- golub2013: enciclopédia de computações matriciais.
- goodfellow2016: *Deep Learning* — conexões com CN em redes neurais.
- hartley2003: visão computacional e condicionamento de sistemas.
- Arnold, D.N. (2000). *The Patriot Missile Failure* e outros desastres numéricos: <https://www-users.cse.umn.edu/~arnold/disasters/>
- Padrão IEEE 754 (2019): <https://ieeexplore.ieee.org/document/8766229>
- Documentação NumPy: <https://numpy.org/doc/stable/>

Capítulo 2

Zeros de Funções Reais

“God made the integers, all else is the work of man.”

— Leopold Kronecker

2.1 Introdução

Determinar os zeros de uma função real é um dos problemas mais recorrentes na matemática computacional. Dado $f : D \subset \mathbb{R} \rightarrow \mathbb{R}$, chamamos de **zero** ou **raiz** de f qualquer valor $\tilde{x} \in D$ tal que $f(\tilde{x}) = 0$. Geometricamente, as raízes reais correspondem às abscissas dos pontos nos quais gráfico de $y = f(x)$ intercepta o eixo Ox .

A resolução do problema $f(x) = 0$ divide-se em dois estágios complementares. O primeiro é a **localização**: identificar um intervalo $[a, b]$ que contenha uma única raiz. O segundo é o **refinamento**: a partir de uma ou duas aproximações iniciais em $[a, b]$, melhorá-las sucessivamente até atingir uma precisão ε prefixada.

Nota Importante

Em problemas de Ciência e Engenharia, a equação $f(\tilde{x}) = 0$ raramente tem solução analítica. Funções polinomiais de grau $n \geq 5$ não possuem fórmula geral (Teorema de Abel-Ruffini); equações transcendentais quase nunca a têm. Métodos numéricos iterativos são, portanto, a ferramenta padrão para este problema.

2.1.1 Critérios de Parada

Ao longo das iterações, são utilizados três critérios práticos para decidir quando parar:

1. **Resíduo:** $|f(x_k)| < \varepsilon$.
2. **Erro absoluto:** $|x_k - x_{k-1}| < \varepsilon$.
3. **Erro relativo:** $|x_k - x_{k-1}|/|x_k| < \varepsilon$.

Cuidado

Os critérios 1 e 2 nem sempre são equivalentes. Em funções quase horizontais, $|f(x_k)|$ pode ser pequeno mesmo quando x_k está longe da raiz. Em funções muito íngremes, x_k pode estar próximo da raiz enquanto $|f(x_k)|$ ainda é grande. Recomenda-se combinar pelo menos dois critérios e definir um número máximo de iterações para garantir término.

2.2 Aplicações Motivadoras

Aplicação Computacional

Computação Gráfica (Curvas de Bézier) [?]:

Curvas de Bézier cúbicas são a base de toda computação gráfica vetorial:

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \quad t \in [0, 1].$$

Encontrar a interseção de duas curvas $\mathbf{B}_1(s) = \mathbf{B}_2(t)$ leva, após eliminação de uma variável, a um polinômio de grau até 9. Aplicações: fontes TrueType/PostScript, Adobe Illustrator, Blender, AutoCAD.

Aplicação Computacional

Teoria de Controle (Análise de Estabilidade) [?]:

A estabilidade do sistema linear $d\mathbf{x}/dt = \mathbf{A}\mathbf{x}$ é determinada pelos autovalores de $\mathbf{A}$, raízes do polinômio característico $p(\lambda) = \det(\lambda\mathbf{I} - \mathbf{A}) = 0$. O sistema é estável se, e somente se, $\text{Re}(\lambda_i) < 0$ para todo i .

Aplicação Computacional**Física de Semicondutores (Nível de Fermi) [?]:**

A concentração de portadores satisfaz $n = N_c F_{1/2}(\eta)$, onde $F_{1/2}(\eta)$ é a integral de Fermi-Dirac de ordem $\frac{1}{2}$. Dado n , determinar o nível de Fermi η requer resolver $n - N_c F_{1/2}(\eta) = 0$ — equação transcendente sem solução analítica, resolvida por Newton-Raphson em simuladores TCAD de transistores, células solares e LEDs.

Aplicação Computacional**Engenharia Química (Equações de Estado) [?]:**

A equação de van der Waals para gases reais, $(P + a/V^2)(V - b) = RT$, conduz a um polinômio cúbico em V para dados P e T . Modelos mais complexos (Peng-Robinson, Soave-Redlich-Kwong) geram equações transcendentais resolvidas milhares de vezes por segundo em simuladores de processos.

Aplicação Computacional**Criptografia de Curvas Elípticas (ECC) [?, ?]:**

A criptografia moderna (Bitcoin, TLS/SSL, assinaturas digitais) baseia-se em operações sobre curvas elípticas:

$$y^2 = x^3 + ax + b \pmod{p}.$$

A curva `secp256k1` usada no Bitcoin é $y^2 = x^3 + 7 \pmod{2^{256} - 2^{32} - 977}$. Operações de grupo requerem encontrar raízes de polinômios modulares e resolver equações envolvendo funções elípticas — problemas tratados por métodos numéricos adaptados a corpos finitos.

2.3 Teoremas Fundamentais

2.3.1 Teorema Fundamental da Álgebra

Teorema 2.1 (Fundamental da Álgebra — Gauss, 1799). Todo polinômio $p(x)$ de grau $n \geq 1$ com coeficientes complexos possui pelo menos uma raiz complexa. Equivalentemente,

$$p(x) = a_n(x - r_1)(x - r_2) \cdots (x - r_n), \quad r_i \in \mathbb{C}.$$

Esboço. Passo 1. Para $|z|$ grande, $|p(z)|/|z|^n \geq |a_n| - \sum_{i < n} |a_i|/|z|^{n-i} \rightarrow |a_n| > 0$, logo $|p(z)| \rightarrow \infty$.

Passo 2. A função contínua $|p|$ atinge seu mínimo global em algum $z_0 \in \mathbb{C}$ (compacto suficientemente grande).

Passo 3. Se $p(z_0) \neq 0$, escrevendo a expansão local $p(z_0 + h) = p(z_0) + c_m h^m + O(h^{m+1})$ e escolhendo a fase de h tal que $c_m h^m$ aponta contra $p(z_0)$, obtém-se $|p(z_0 + h)| < |p(z_0)|$ para $|h|$ pequeno — contradição com a minimalidade. Logo $p(z_0) = 0$. $\square$

Corolário 2.2 (Raízes complexas conjugadas). Um polinômio $p \in \mathbb{R}[x]$ fatora sobre $\mathbb{R}$ como

$$p(x) = a_n \prod_{i=1}^k (x - r_i) \prod_{j=1}^m (x^2 + b_j x + c_j), \quad b_j^2 - 4c_j < 0.$$

Demonstração. Se $z = a + bi$ ($b \neq 0$) é raiz de $p \in \mathbb{R}[x]$, tome o conjugado de $p(z) = 0$: como os coeficientes são reais, $\overline{p(z)} = p(\bar{z}) = 0$, logo $\bar{z}$ também é raiz. O fator $(x - z)(x - \bar{z}) = x^2 - 2ax + (a^2 + b^2) \in \mathbb{R}[x]$. $\square$

2.3.2 Teorema de Abel-Ruffini

Teorema 2.3 (Abel-Ruffini, 1824). Não existe fórmula geral em radicais para as raízes de equações polinomiais de grau $n \geq 5$.

Nota Importante

Este teorema não afirma que equações de grau 5 não possuem solução: todas as possuem pelo [theorem 2.1](#). Afirma que não há *fórmula universal* análoga a Bhaskara (grau 2) ou à fórmula de Cardano (grau 3). Métodos numéricos são, portanto, indispensáveis para $n \geq 5$.

2.3.3 Localização de Raízes

Teorema 2.4 (Teorema do Valor Intermediário — Bolzano). Se $f : [a, b] \rightarrow \mathbb{R}$ é contínua e $f(a) \cdot f(b) < 0$, então existe pelo menos um ponto $\tilde{x} \in (a, b)$ tal que $f(\tilde{x}) = 0$. Se adicionalmente f' existir e preservar o sinal em $[a, b]$, a raiz é única.

Demonstração. Suponha $f(a) < 0 < f(b)$. Seja $S = \{x \in [a, b] : f(x) \leq 0\}$. S é não vazio (pois $a \in S$) e limitado superiormente; pelo axioma do supremo, $\tilde{x} = \sup S$ existe em $[a, b]$.

$f(\tilde{x}) \leq 0$: Existe $(x_n) \subset S$ com $x_n \rightarrow \tilde{x}$. Por continuidade, $f(\tilde{x}) = \lim f(x_n) \leq 0$.

$f(\tilde{x}) \geq 0$: Como $f(b) > 0$, temos $\tilde{x} < b$. Existem $y_n > \tilde{x}$ com $y_n \rightarrow \tilde{x}$ e $y_n \notin S$, logo $f(y_n) > 0$. Por continuidade, $f(\tilde{x}) = \lim f(y_n) \geq 0$.

Portanto $f(\tilde{x}) = 0$. $\square$

Teorema 2.5 (Cota de Cauchy para Raízes). Todas as raízes de $p(x) = a_n x^n + \dots + a_0$ satisfazem

$$|x| \leq 1 + \frac{M}{|a_n|}, \quad M = \max_{0 \leq i < n} |a_i|.$$

Demonstração. Se $|r| \leq 1$, a desigualdade é imediata. Se $|r| > 1$, de $p(\tilde{x}) = 0$ extraímos $a_n r^n = -(a_{n-1} r^{n-1} + \dots + a_0)$. Tomando módulo e majorando:

$$|a_n| |r|^n \leq M(|r|^{n-1} + \dots + 1) < M \frac{|r|^n}{|r| - 1},$$

logo $|a_n|(|r| - 1) < M$ e $|r| < 1 + M/|a_n|$. □

Exemplo 2.1 (Cota de Cauchy — aplicação). Para $p(x) = x^3 - 6x^2 + 11x - 6$:

$$M = \max\{6, 11, 6\} = 11,$$

logo, $|x| \leq 1 + 11 = 12$. As raízes verdadeiras 1, 2, 3 estão bem dentro do intervalo $[-12, 12]$ garantido pela cota.

2.3.4 Funções Transcendentes

Definição 2.6 (Função Transcendente). Uma função f é *transcendente* se não pode ser expressa como solução de equação polinomial com coeficientes racionais. Exemplos: exponenciais (e^x), logarítmicas ($\ln x$), trigonométricas ($\sin x$, $\cos x$), hiperbólicas e funções especiais (Bessel, Gamma, Erf).

Funções transcendentais usuais são infinitamente diferenciáveis em seus domínios. A *boa notícia*: os métodos desenvolvidos nas próximas seções aplicam-se **sem modificações conceituais** tanto a equações polinomiais quanto a equações transcendentais.

2.4 Fase I: Localização Gráfica e Tabela de Sinais

A localização de raízes pode ser feita por dois métodos complementares.

Método gráfico: Esboçar o gráfico de $f(x)$ e identificar as abscissas dos cruzamentos com o eixo Ox . Alternativamente, reescrever $f(x) = 0$ como $g(x) = h(x)$ e localizar visualmente as interseções de $y = g(x)$ e $y = h(x)$.

Tabela de sinais: Avaliar f em pontos de uma grade uniforme e registrar as mudanças de sinal — cada mudança indica ao menos uma raiz naquele sub-intervalo, pelo [theorem 2.4](#).

Exemplo 2.2 (Localização por tabela de sinais). $f(x) = x^3 - 9x + 3$:

x	-3	-2	0	1	2	3
$f(x)$	-	+	+	-	-	+

Mudanças de sinal em $[-3, -2]$, $[0, 1]$ e $[2, 3]$: pelo menos uma raiz em cada sub-intervalo.

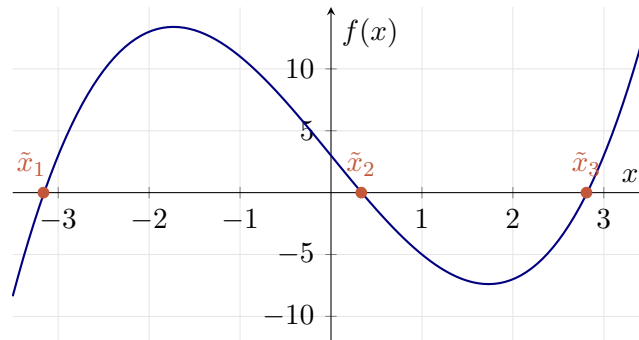


Figura 2.1: Gráfico de $f(x) = x^3 - 9x + 3$ com as três raízes reais identificadas. A tabela de sinais confirma uma mudança de sinal em cada sub-intervalo $[-3, -2]$, $[0, 1]$ e $[2, 3]$.

2.5 Método da Bisseção

2.5.1 Algoritmo

O método da bisseção é o mais simples e robusto, baseando-se diretamente no [theorem 2.4](#).

Algoritmo 1 Método da Bisseção

Entrada: f contínua; $[a, b]$ com $f(a)f(b) < 0$; tolerância $\varepsilon > 0$

Saída: Aproximação da raiz com erro $< \varepsilon$

```

1: se  $f(a) \cdot f(b) \geq 0$  então
2:   retorne erro: "sinais iguais"
3: end se
4: enquanto  $|b - a| > \varepsilon$  faça
5:    $c \leftarrow (a + b)/2$ 
6:   se  $|f(c)| < \varepsilon$  então retorne  $c$ 
7:   end se
8:   se  $f(a) \cdot f(c) < 0$  então
9:      $b \leftarrow c$ 
10:  senão
11:     $a \leftarrow c$ 
12:  end se
13: end enquanto
14: retorne  $(a + b)/2$ 
```

2.5.2 Análise de Convergência

Teorema 2.7 (Convergência Linear da Bisseção). Seja $\tilde{x}$ a raiz exata em $[a_0, b_0]$ e $x^{(k)} = (a_k + b_k)/2$. Então:

$$|x^{(k)} - r| \leq \frac{b_0 - a_0}{2^{k+1}}.$$

O número de iterações para garantir erro $< \varepsilon$ é $k \geq \log_2((b_0 - a_0)/\varepsilon) - 1$.

Demonstração. A cada iteração o intervalo é dividido ao meio:

$$b_k - a_k = \frac{b_0 - a_0}{2^k}.$$

Como $r \in [a_k, b_k]$ e $x^{(k)}$ é o ponto médio:

$$|x^{(k)} - r| \leq \frac{b_k - a_k}{2} = \frac{b_0 - a_0}{2^{k+1}}.$$

□

Exemplo 2.3 (Número de iterações). Para $[0, 1]$ com $\varepsilon = 10^{-10}$: $k \geq \log_2(10^{10}) - 1 \approx 32$ iterações.

A figura abaixo ilustra a primeira iteração da bissecção em $f(x) = x^3 - x - 2$:

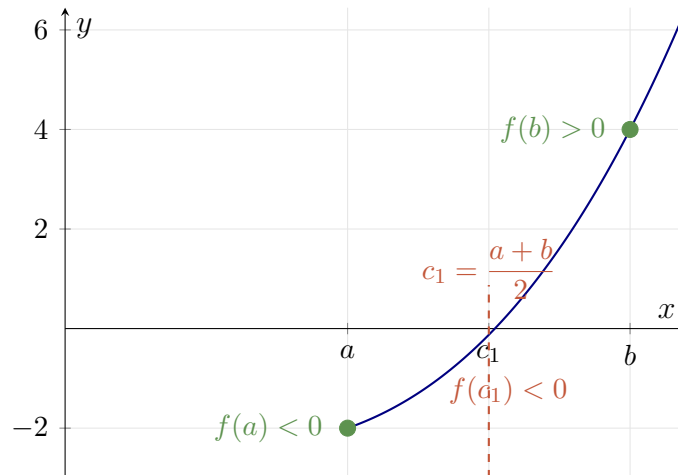


Figura 2.2: Primeira iteração da bissecção para $f(x) = x^3 - x - 2$ em $[1, 2]$. Como $f(c_1) < 0$, a raiz está em $[c_1, b] = [1,5, 2]$.

2.5.3 Implementação

```

1 def bissecao(f, a, b, tol=1e-10, max_iter=100):
2     """
3     Encontra raiz de f em [a,b] usando bissecao.
4     Retorna: (raiz, n_iteracoes, historico)
5     """
6     fa, fb = f(a), f(b)
7     if fa * fb > 0:
8         raise ValueError("f(a) e f(b) devem ter sinais opostos.")
9     historico = []

```

```

10     for k in range(max_iter):
11         c = (a + b) / 2
12         fc = f(c)
13         historico.append(c)
14         if abs(fc) < tol or abs(b - a) < tol:
15             return c, k+1, historico
16         if fa * fc < 0:
17             b, fb = c, fc
18         else:
19             a, fa = c, fc
20     return (a + b) / 2, max_iter, historico
21
22 # Exemplo: x^3 - x - 2 = 0 em [1, 2]
23 f = lambda x: x**3 - x - 2
24 raiz, iters, _ = bissecao(f, 1, 2, tol=1e-10)
25 print(f"Raiz: {raiz:.15f}    Iteracoes: {iters}")

```

Listing 2.1: Método da bisseção

2.6 Método da Posição Falsa

2.6.1 Motivação e Fórmula

A bisseção escolhe sempre o ponto médio de $[a, b]$, ignorando os valores de f nos extremos. Se $|f(a)| \ll |f(b)|$, a raiz está muito mais próxima de a . O **Método da Posição Falsa** (*Regula Falsi*) corrige isso usando a interseção da reta secante pelos pontos $(a, f(a))$ e $(b, f(b))$ com o eixo Ox :

$$x = \frac{a f(b) - b f(a)}{f(b) - f(a)}.$$

A iteração procede como na bisseção: calcula-se x , verifica-se o sinal de $f(x)$, e um dos extremos é substituído por x mantendo $f(a)f(b) < 0$.

Nota Importante

A bisseção é um caso especial da posição falsa quando $f(a) = -f(b)$: nesse caso a fórmula reduz-se a $(a + b)/2$.

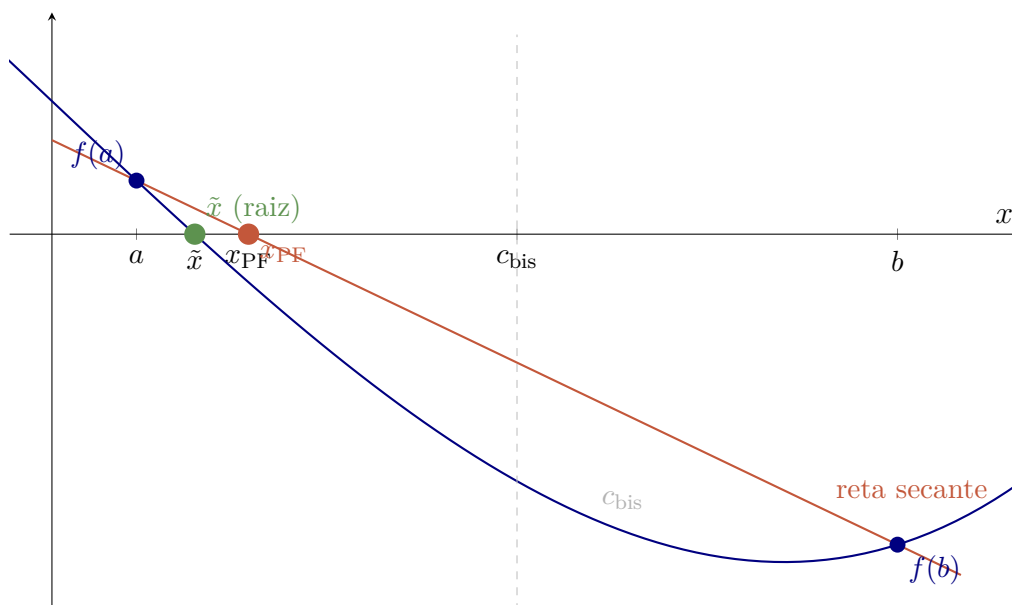


Figura 2.3: Comparação entre o candidato da bisseção (c_{bis}) e o da posição falsa (x_{PF}) para $f(x) = x^3 - 9x + 3$ com $a = 0,2$ e $b = 2,0$. Como $|f(a)| \ll |f(b)|$, a posição falsa está muito mais próxima da raiz real $\tilde{x}$.

2.6.2 Convergência e Comparação com a Bisseção

A posição falsa sempre converge, em geral mais rapidamente que a bisseção. Um dos extremos pode porém ficar *fixo* por muitas iterações quando a função tem curvatura acentuada, tornando a convergência lenta nesses casos.

	Bisseção	Posição Falsa
Candidato	$\frac{a+b}{2}$	$\frac{af(b) - bf(a)}{f(b) - f(a)}$
Convergência	Garantida	Garantida
Velocidade	Linear	Superlinear*
Exige f'	Não	Não

*Em geral mais rápida; pode ser lenta com extremo fixo.

2.6.3 Implementação

```

1 def posicao_falsa(f, a, b, eps1=1e-8, eps2=1e-8, max_iter=200):
2     """
3     Encontra raiz de f em [a,b] pela posição falsa.
4     eps1: tolerância no comprimento do intervalo
5     eps2: tolerância no resíduo

```

```

6      """
7      fa, fb = f(a), f(b)
8      if fa * fb > 0:
9          raise ValueError("f(a) e f(b) devem ter sinais opostos.")
10     historico = []
11     sinal_a = fa
12     for k in range(1, max_iter + 1):
13         x = (a * fb - b * fa) / (fb - fa)
14         fx = f(x)
15         historico.append(x)
16         if abs(fx) < eps2 or abs(b - a) < eps1:
17             return x, k, historico
18         if sinal_a * fx > 0:
19             a, fa = x, fx
20         else:
21             b, fb = x, fx
22     return x, max_iter, historico
23
24 # Exemplo: x*log10(x) - 1 em [2, 3]
25 import math
26 f = lambda x: x * math.log10(x) - 1
27 raiz, iters, _ = posicao_falsa(f, 2, 3)
28 print(f"Raiz: {raiz:.10f}    Iteracoes: {iters}")

```

Listing 2.2: Método da posição falsa (*Regula Falsi*)

2.7 Método do Ponto Fixo

2.7.1 Formulação

Resolver $f(x) = 0$ é equivalente a encontrar um *ponto fixo* $x^* = g(x^*)$ de uma função g obtida reescrevendo a equação original. A iteração simples é $x^{(k+1)} = g(x^{(k)})$.

Há infinitas formas de obter g ; a escolha determina se o método converge ou diverge, e a que velocidade.

Exemplo 2.4 (Reformulações de $e^x - 3x = 0$). Três reescritas possíveis:

$$g_1(x) = e^x/3 \quad g_2(x) = \ln(3x) \quad g_3(x) = x - (e^x - 3x)/(e^x - 3) \quad (\text{Newton})$$

Apenas g com $|g'(x^*)| < 1$ garante convergência local.

2.7.2 Convergência

Teorema 2.8 (Mapa de Contração — Banach). Se $g : [a, b] \rightarrow [a, b]$ é diferenciável com $|g'(x)| \leq L < 1$ para todo $x \in [a, b]$, então:

1. O ponto fixo x^* é **único** em $[a, b]$.
2. Para qualquer $x^{(0)} \in [a, b]$, a iteração converge para x^* .
3. $|x^{(k)} - x^*| \leq \frac{L^k}{1 - L} |x^{(1)} - x^{(0)}|$.

Demonstração. Pelo Teorema do Valor Médio:

$$|x^{(k+1)} - x^*| = |g(x^{(k)}) - g(x^*)| = |g'(\tilde{x}_k)| |x^{(k)} - x^*| \leq L |x^{(k)} - x^*|.$$

Iterando: $|x^{(k)} - x^*| \leq L^k |x^{(0)} - x^*| \rightarrow 0$. Para obter a estimativa do item 3:

$$|x^{(k)} - x^*| \leq \sum_{j=k}^{\infty} |x^{(j+1)} - x^{(j)}| \leq L^k |x^{(1)} - x^{(0)}| \sum_{j=0}^{\infty} L^j = \frac{L^k}{1 - L} |x^{(1)} - x^{(0)}|. \quad \square$$

Nota Importante

Critério prático: antes de usar g , calcule $|g'(x)|$ na vizinhança da raiz estimada. Se $|g'| > 1$ em toda a vizinhança, troque de formulação. Se $g'(x^*) = 0$, a convergência sobe para ordem 2 (quadrática) — esse é exatamente o caso do método de Newton!

2.7.3 Diagramas de Teia de Aranha

A **teia de aranha** é uma ferramenta gráfica para visualizar a convergência ou divergência do método de ponto fixo. Partindo de $x^{(0)}$ na bissetriz $y = x$, alternamos setas verticais (subindo até a curva $y = g(x)$) com setas horizontais (indo até a bissetriz), em que cada horizontal indica o valor do próximo iterado. Setas vermelhas indicam afastamento; setas verdes indicam aproximação ao ponto fixo $\tilde{x}$.

A equação $x^2 + x - 6 = 0$ tem raízes $\tilde{x}_1 = -3$ e $\tilde{x}_2 = 2$. O método do ponto fixo reescreve a equação na forma $x = \varphi(x)$ e itera $x^{(k+1)} = \varphi(x^{(k)})$. Pelo teorema do ponto fixo de Banach, a iteração converge se $|\varphi'(x^*)| < 1$ numa vizinhança da raiz. Além da magnitude, o *signal* de $\varphi'(x^*)$ determina o padrão geométrico da sequência de iterados:

- $\varphi'(x^*) \in (0, 1)$: **escada convergente** na qual os iterados se aproximam de x^* monotonicamente pelo mesmo lado;
- $\varphi'(x^*) > 1$: **escada divergente** na qual o afastamento é monótono;

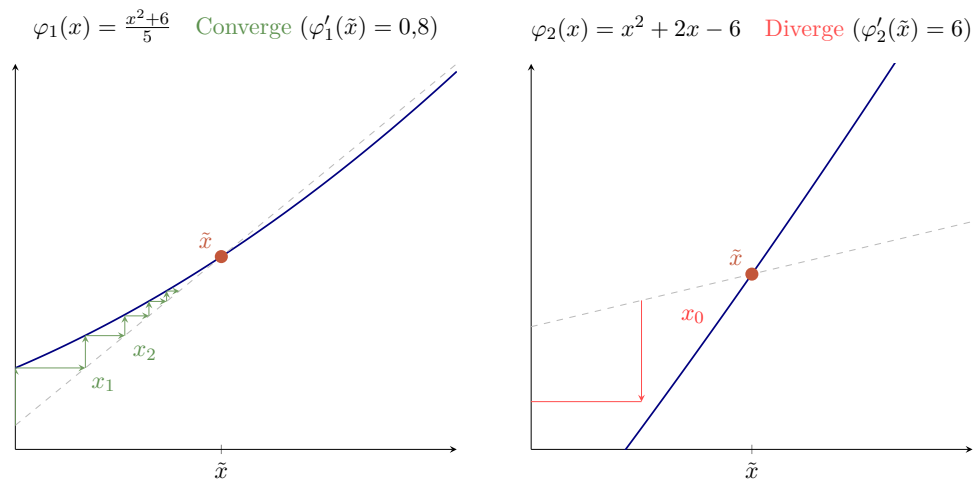
- $\varphi'(x^*) \in (-1, 0)$: **espiral convergente** na qual os iterados alternam de lado e se aproximam;
- $\varphi'(x^*) < -1$: **espiral divergente** na qual alternância com afastamento crescente.

Para ilustrar os quatro comportamentos, construímos as funções da Tabela 2.1, todas verificadas em $\tilde{x}_2 = 2$.

Tabela 2.1: Funções de iteração para $x^2 + x - 6 = 0$ em $\tilde{x}_2 = 2$, cobrindo os quatro padrões geométricos possíveis.

	$\varphi(x)$	Origem	$\varphi'(x)$	$ \varphi'(2) $	Padrão
φ_1	$\frac{x^2 + 6}{5}$	$5x = x^2 + 6$	$\frac{2x}{5}$	0,8	Escada convergente
φ_2	$x^2 + 2x - 6$	$x = x^2 + x - 6 + x$	$2x + 2$	6	Escada divergente
φ_3	$\sqrt{6 - x}$	$x^2 + x = 6 \Rightarrow x^2 = 6 - x$	$\frac{-1}{2\sqrt{6 - x}}$	0,25	Espiral convergente
φ_4	$6 - x^2$	$x + x^2 = 6 \Rightarrow x = 6 - x^2$	$-2x$	4	Espiral divergente

Os diagramas de teia de aranha da Figura 2.4 tornam visível a diferença entre esses comportamentos: as setas percorrem verticalmente a curva $y = \varphi(x)$ (passo $x^{(k)} \mapsto \varphi(x^{(k)})$) e horizontalmente a bissetriz $y = x$ (leitura do próximo iterado). Quando as setas se afastam do ponto fixo $\tilde{x}$, a função diverge; quando se enrolam em torno dele, converge. O sentido de rotação — ou a ausência dele — revela o sinal de $\varphi'(\tilde{x})$.



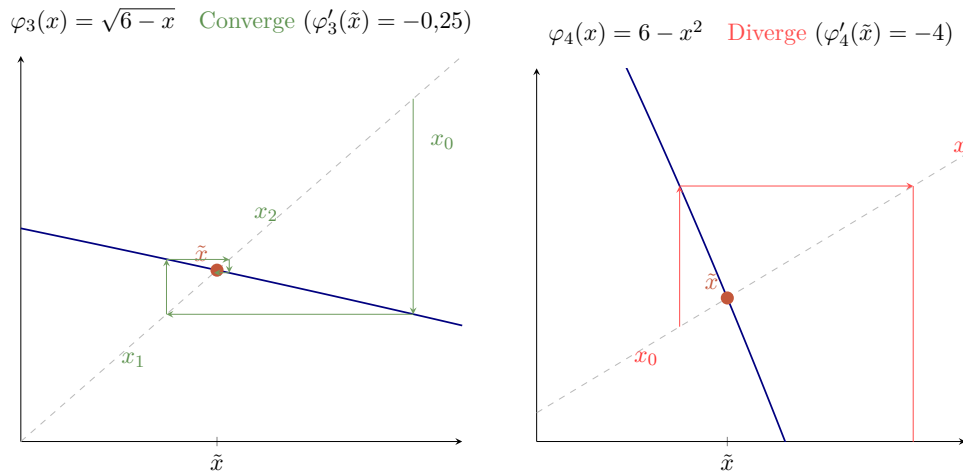


Figura 2.4: Diagramas de teia de aranha para as quatro funções de iteração de $x^2+x-6=0$ em torno de $\tilde{x}_2=2$. **Setas verdes** indicam aproximação ao ponto fixo (convergência); **setas vermelhas** indicam afastamento (divergência). Em φ_1 e φ_2 , a derivada é positiva: os iterados avançam sempre pelo mesmo lado de $\tilde{x}$, formando uma **escada** convergente para φ_1 ($\varphi'_1=0,8$) e divergente para φ_2 ($\varphi'_2=6$). Em φ_3 e φ_4 , a derivada é negativa: os iterados alternam de lado, formando uma **espiral** convergente para φ_3 ($|\varphi'_3|=0,25$) e divergente para φ_4 ($|\varphi'_4|=4$).

2.7.4 Newton como Ponto Fixo de Ordem Dois

Teorema 2.9 (Newton $\equiv$ Ponto Fixo com $g'(x^*)=0$). O método de Newton para $f(x)=0$ é iteração de ponto fixo com $g(x)=x-f(x)/f'(x)$. Calculando:

$$g'(x) = \frac{f(x)f''(x)}{[f'(x)]^2} \Rightarrow g'(x^*) = 0.$$

Logo Newton converge automaticamente com ordem quadrática.

Demonstração. Por regra do quociente:

$$g'(x) = 1 - \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}.$$

Em x^* : $f(x^*)=0$, logo $g'(x^*)=0$. □

2.7.5 Implementação

```

1 def ponto_fixo(g, x0, tol=1e-10, max_iter=100):
2     """
3     Encontra ponto fixo de g por iteracao x_{k+1} = g(x_k).
4     Retorna: (raiz, n_iteracoes, historico)
5     """
6     x = x0

```

```

7     historico = [x]
8     for k in range(max_iter):
9         x_new = g(x)
10        historico.append(x_new)
11        if abs(x_new - x) < tol:
12            return x_new, k+1, historico
13
14        x = x_new
15    print("Aviso: convergencia nao atingida.")
16    return x, max_iter, historico
17
18 # sqrt(2) via g(x) = (x + 2/x)/2 -- convergencia quadratica
19 g_sqrt2 = lambda x: 0.5 * (x + 2.0/x)
20 raiz, iters, hist = ponto_fixo(g_sqrt2, 1.0)
21 print(f"sqrt(2) = {raiz:.15f} ({iters} iteracoes)")
22
23 # Verificar convergencia quadratica
24 import math
25 for i, xi in enumerate(hist[:6]):
26     print(f"    x[{i}] = {xi:.10f}, erro = {abs(xi -
27         math.sqrt(2)):.2e}")

```

Listing 2.3: Método do ponto fixo

2.8 Método de Newton-Raphson

2.8.1 Derivação Geométrica

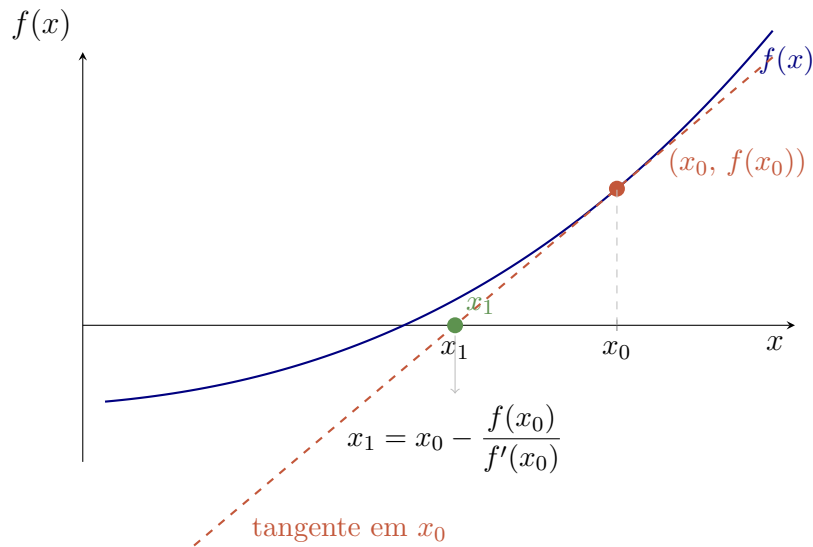
O método de Newton aproxima f localmente pela reta tangente em x_k :

$$L(x) = f(x_k) + f'(x_k)(x - x_k).$$

A raiz de $L(x) = 0$ fornece a próxima aproximação:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

A cada iteração, o número de dígitos corretos **dobra** (convergência quadrática) — tornando Newton o mais eficiente entre os métodos clássicos.

Figura 2.5: Iteração de Newton para $f(x) = x^3 - x - 2$.

2.8.2 Algoritmo

Algoritmo 2 Método de Newton-Raphson

Entrada: f, f' ; aproximação inicial x_0 ; tolerância ε

Saída: Aproximação da raiz

- 1: $k \leftarrow 0$
 - 2: **repita**
 - 3: $f_k \leftarrow f(x_k); \quad df_k \leftarrow f'(x_k)$
 - 4: **se** $|df_k| < 10^{-14}$ **então retorne** erro: “derivada nula”
 - 5: **end se**
 - 6: $x_{k+1} \leftarrow x_k - f_k/df_k$
 - 7: $k \leftarrow k + 1$
 - 8: **até** $|x_k - x_{k-1}| < \varepsilon$ **ou** $|f_k| < \varepsilon$ **ou** $k > k_{\max}$
 - 9: **retorne** x_k
-

2.8.3 Análise de Convergência

Teorema 2.10 (Convergência Quadrática de Newton). Seja $f \in C^2$, $\tilde{x}$ raiz simples ($f(\tilde{x}) = 0$, $f'(\tilde{x}) \neq 0$). Com x_0 suficientemente próximo de $\tilde{x}$:

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^2} = \frac{|f''(\tilde{x})|}{2|f'(\tilde{x})|}, \quad e_k = x_k - r.$$

Demonstração. Expandindo $f(\tilde{x}) = 0$ em Taylor em torno de x_k com resto de Lagrange:

$$0 = f(x_k) + f'(x_k)(r - x_k) + \frac{f''(\tilde{x}_k)}{2}(r - x_k)^2, \quad \tilde{x}_k \text{ entre } x_k \text{ e } r.$$

Como $e_k = x_k - r$ (logo $r - x_k = -e_k$) e $f(\tilde{x}) = 0$:

$$0 = f(x_k) - f'(x_k)e_k + \frac{f''(\tilde{x}_k)}{2}e_k^2.$$

Dividindo por $f'(x_k) \neq 0$:

$$\frac{f(x_k)}{f'(x_k)} = e_k - \frac{f''(\tilde{x}_k)}{2f'(x_k)}e_k^2.$$

O erro na iteração seguinte é:

$$\begin{aligned} e_{k+1} &= x_{k+1} - r = \left(x_k - \frac{f(x_k)}{f'(x_k)} \right) - r \\ &= e_k - \frac{f(x_k)}{f'(x_k)} = e_k - \left[e_k - \frac{f''(\tilde{x}_k)}{2f'(x_k)}e_k^2 \right] = \frac{f''(\tilde{x}_k)}{2f'(x_k)}e_k^2. \end{aligned}$$

Quando $k \rightarrow \infty$: $\tilde{x}_k \rightarrow r$ e $f'(x_k) \rightarrow f'(\tilde{x})$, logo $|e_{k+1}|/|e_k|^2 \rightarrow |f''(\tilde{x})|/(2|f'(\tilde{x})|)$. □

Nota Importante

Convergência quadrática significa que o número de dígitos corretos dobra a cada iteração. Se $e_0 \approx 10^{-2}$ e $M \approx 1$:

$$e_1 \approx 10^{-4}, \quad e_2 \approx 10^{-8}, \quad e_3 \approx 10^{-16} \quad (\text{precisão de máquina!}).$$

2.8.4 Raízes Múltiplas e Método Modificado

Teorema 2.11 (Newton para Raízes Múltiplas). Se $\tilde{x}$ é raiz de multiplicidade $m > 1$ de f , o método de Newton padrão converge *linearmente* com fator $(m - 1)/m$. O método modificado

$$x_{k+1} = x_k - m \frac{f(x_k)}{f'(x_k)}$$

restaura a convergência quadrática.

Demonstração. Parte 1 — convergência linear. Escrevendo $f(x) = (x - r)^m h(x)$ com $h(\tilde{x}) \neq 0$:

$$f'(x) = m(x - r)^{m-1}h(x) + (x - r)^m h'(x).$$

A função de iteração $g(x) = x - f(x)/f'(x)$ satisfaz:

$$g(x) = x - \frac{(x-r)^m h(x)}{m(x-r)^{m-1}h(x) + (x-r)^m h'(x)} = x - \frac{x-r}{m + (x-r)h'(x)/h(x)}.$$

Derivando e avaliando em $\tilde{x}$ (onde $(x-r) \rightarrow 0$):

$$g'(\tilde{x}) = 1 - \frac{1}{m} = \frac{m-1}{m} \in (0, 1).$$

Logo a convergência é linear com fator $(m-1)/m$.

Parte 2 — método modificado restaura ordem quadrática. Defina $\mu(x) = f(x)/f'(x)$. Das expressões acima, $\mu(x) = (x-r)/[m + (x-r)h'(x)/h(x)]$, portanto $\mu'(\tilde{x}) = 1/m$. A função de iteração modificada é $g_m(x) = x - m\mu(x)$. Derivando:

$$g'_m(\tilde{x}) = 1 - m\mu'(\tilde{x}) = 1 - m \cdot \frac{1}{m} = 0.$$

Como $g'_m(\tilde{x}) = 0$, pelo [theorem 2.8](#) a convergência é quadrática. □

Exemplo 2.5 (Raiz dupla: $f(x) = (x-2)^2$). Newton padrão com $x_0 = 1$: $x_{k+1} = (x_k + 2)/2$, gerando $1 \rightarrow 1,5 \rightarrow 1,75 \rightarrow 1,875 \rightarrow \dots$ (convergência lenta). Newton modificado ($m = 2$): $x_{k+1} = x_k - (x_k - 2) = 2$ — convergência em **1 iteração**.

Cuidado

Newton pode falhar ou convergir lentamente em vários casos:

1. $f'(x_k) = 0$: falha por divisão por zero.
2. Raízes múltiplas: degrada para convergência linear.
3. x_0 distante da raiz: possível divergência ou convergência para raiz errada.
4. Pontos de inflexão: convergência lenta.
5. Certas funções: oscilação indefinida (ex.: $f(x) = x^{1/3}$).

Estratégia robusta: usar bisseção para obter x_0 próximo da raiz, então alternar para Newton.

2.8.5 Aplicação a Funções Transcendentes

Exemplo 2.6 ($e^x = 3x$ — Newton). $f(x) = e^x - 3x$, $f'(x) = e^x - 3$. Há raízes em $(0,5, 1)$ e em $(1, 2)$. Com $x_0 = 0,7$:

$$x_1 \approx 0,6127, \quad x_2 \approx 0,6191, \quad x_3 \approx 0,6191 \quad \checkmark$$

Exemplo 2.7 ($\cos(x) = x$ — Constante de Dottie). $f(x) = \cos(x) - x$, $f'(x) = -\sin(x) - 1$. Com $x_0 = 0,5$:

$$x_1 \approx 0,7552, \quad x_2 \approx 0,7391, \quad x_3 \approx 0,7391.$$

Raiz: $x^* \approx 0,7390851332$ (constante de Dottie).

2.8.6 Implementação

```

1 def newton(f, df, x0, tol=1e-10, max_iter=100):
2     """
3     Encontra raiz de f usando Newton-Raphson.
4     df: derivada de f. Retorna: (raiz, n_iter, historico)
5     """
6     x = x0
7     historico = [x]
8     for k in range(max_iter):
9         fx = f(x)
10        dfx = df(x)
11        if abs(dfx) < 1e-14:
12            raise ValueError(f"Derivada nula em x = {x:.6f}")
13        x_new = x - fx / dfx
14        historico.append(x_new)
15        if abs(x_new - x) < tol or abs(fx) < tol:
16            return x_new, k+1, historico
17        x = x_new
18    return x, max_iter, historico
19
20 # f(x) = x^3 - x - 2
21 f = lambda x: x**3 - x - 2
22 df = lambda x: 3*x**2 - 1
23 raiz, iters, _ = newton(f, df, x0=1.5)
24 print(f"Raiz: {raiz:.15f} Iteracoes: {iters}")

```

Listing 2.4: Método de Newton-Raphson

2.9 Método da Secante

2.9.1 Motivação e Fórmula

Quando f' é difícil de calcular, custosa ou inexistente, aproximamos a derivada por diferença finita:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituindo em Newton:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} = \frac{x_{k-1}f(x_k) - x_kf(x_{k-1})}{f(x_k) - f(x_{k-1})}.$$

São necessárias **duas** aproximações iniciais x_0 e x_1 . Geometricamente, x_{k+1} é a abscissa do zero da reta secante pelos pontos $(x_{k-1}, f(x_{k-1}))$ e $(x_k, f(x_k))$.

Algoritmo 3 Método da Secante

Entrada: f ; aproximações iniciais x_0, x_1 ; tolerância ε

Saída: Aproximação da raiz

```

1:  $k \leftarrow 1$ 
2: repita
3:    $f_k \leftarrow f(x_k); f_{k-1} \leftarrow f(x_{k-1})$ 
4:   se  $|f_k - f_{k-1}| < 10^{-14}$  então retorne erro: “denominador nulo”
5:   end se
6:    $x_{k+1} \leftarrow x_k - f_k (x_k - x_{k-1}) / (f_k - f_{k-1})$ 
7:    $k \leftarrow k + 1$ 
8: até  $|x_k - x_{k-1}| < \varepsilon$  ou  $|f_k| < \varepsilon$ 
9: retorne  $x_k$ 
```

2.9.2 Análise de Convergência

Teorema 2.12 (Convergência da Secante). Sob as hipóteses do método de Newton, a secante converge com ordem $\phi = (1 + \sqrt{5})/2 \approx 1,618$ (razão áurea):

$$|e_{k+1}| \approx C |e_k|^\phi.$$

Esboço. Expandindo $f(x_j) = f'(\tilde{x})e_j + \frac{1}{2}f''(\tilde{x})e_j^2 + O(e_j^3)$ para $j = k-1, k$ e substituindo na fórmula da secante (o termo $f'(\tilde{x})$ cancela no quociente), obtém-se:

$$e_{k+1} \approx \frac{f''(\tilde{x})}{2f'(\tilde{x})} e_k e_{k-1} =: C e_k e_{k-1}.$$

Assumindo $|e_{k+1}| \approx K|e_k|^\phi$, a equação de balanço de expoentes produz $\phi = 1 + 1/\phi$, cuja solução positiva é $\phi = (1 + \sqrt{5})/2$. $\square$

Embora Newton seja de ordem 2, a secante usa apenas *uma* avaliação de f por iteração (Newton usa uma de f e uma de f'). Na prática, a secante é frequentemente preferível quando f' é computacionalmente custosa.

Exemplo 2.8 (Secante para $x^2 + x - 6$, $\tilde{x} = 2$, $x_0 = 1,5$, $x_1 = 1,8$).

$$x_2 \approx 2,165, \quad x_3 \approx 1,996.$$

Convergência rápida sem cálculo de derivada.

2.9.3 Implementação e Comparação

```

1 def secante(f, x0, x1, tol=1e-10, max_iter=100):
2     """
3     Encontra raiz de f pelo m todo da secante.
4     Requer duas aproximacoes iniciais x0, x1.
5     """
6     historico = [x0, x1]
7     for k in range(max_iter):
8         fx0, fx1 = f(x0), f(x1)
9         if abs(fx1 - fx0) < 1e-14:
10             raise ValueError("Denominador muito pequeno")
11         x_new = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
12         historico.append(x_new)
13         if abs(x_new - x1) < tol or abs(f(x_new)) < tol:
14             return x_new, k+1, historico
15         x0, x1 = x1, x_new
16     return x1, max_iter, historico
17
18 # Comparacao dos tres metodos em f(x) = x^3 - x - 2
19 f = lambda x: x**3 - x - 2
20 df = lambda x: 3*x**2 - 1
21
22 r_b, i_b, _ = bissecao(f, 1, 2)
23 r_s, i_s, _ = secante(f, 1, 2)
24 r_n, i_n, _ = newton(f, df, 1.5)
25 print(f"Bissecao: {i_b:3d} iters | Secante: {i_s:3d} iters |
      Newton: {i_n:3d} iters")

```

Listing 2.5: Método da secante e comparação com bisseção e Newton

O [table 2.2](#) resume as características dos cinco métodos estudados.

Tabela 2.2: Comparação dos métodos de localização de zeros.

Método	Ordem	Aval./iter.	Exige f'	Início	Robustez
Bisseção	1	1	Não	$[a, b]$ com Δ sinal	Alta
Posição Falsa	≤ 2	1	Não	$[a, b]$ com Δ sinal	Alta
Ponto Fixo	1 (linear)	1	Não	Depende de g	Moderada
Newton	2	1+1	Sim	x_0 próximo à raiz	Moderada
Secante	$\approx 1,618$	1	Não	x_0, x_1 próximos	Moderada

2.10 Otimização Unidimensional

Encontrar extremos de $f(x)$ reduz-se a resolver $f'(x) = 0$. Aplicamos Newton à função $g = f'$:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Após encontrar o ponto crítico, verificamos o sinal de f'' para determinar se é mínimo ou máximo.

Exemplo 2.9 (Minimizar $f(x) = x^2 - 2 \ln x$, $x > 0$). $f'(x) = 2x - 2/x = 0 \Rightarrow x = 1$. Newton aplicado a f' : $x_{k+1} = x_k - (x_k^3 - 1)/(x_k^2 + 1)$. Com $x_0 = 2$: $2 \rightarrow 0,6 \rightarrow 1,176 \rightarrow 1,008 \rightarrow 1,000$. Como $f''(x) = 2 + 2/x^2 > 0$ para todo $x > 0$, o ponto é mínimo global.

2.11 Sistemas de Equações Transcendentes

Teorema 2.13 (Newton para Sistemas). Para resolver $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ com $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - J_{\mathbf{F}}(\mathbf{x}^{(k)})^{-1} \mathbf{F}(\mathbf{x}^{(k)}), \quad J_{\mathbf{F}} = \begin{bmatrix} \partial f_1 / \partial x_1 & \cdots & \partial f_1 / \partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_n / \partial x_1 & \cdots & \partial f_n / \partial x_n \end{bmatrix}.$$

Na prática, resolve-se o sistema linear $J \Delta \mathbf{x} = -\mathbf{F}$ a cada iteração (custo $O(n^3)$).

Exemplo 2.10 (Sistema 2×2 : $x^2 + y^2 = 4$, $e^x + y = 1$).

$$\mathbf{F}(x, y) = \begin{bmatrix} x^2 + y^2 - 4 \\ e^x + y - 1 \end{bmatrix}, \quad J = \begin{bmatrix} 2x & 2y \\ e^x & 1 \end{bmatrix}.$$

```

1 from scipy.optimize import fsolve
2 import numpy as np
3
4 def sistema(v):
5     x, y = v
6     return [x**2 + y**2 - 4, np.exp(x) + y - 1]
```

```

7
8 def jacobiana(v):
9     x, y = v
10    return [[2*x, 2*y], [np.exp(x), 1]]
11
12 sol1 = fsolve(sistema, [1, 1], fprime=jacobiana)
13 sol2 = fsolve(sistema, [-2, 0], fprime=jacobiana)
14 print(f"Solucao 1: x={sol1[0]:.6f}, y={sol1[1]:.6f}")
15 print(f"Solucao 2: x={sol2[0]:.6f}, y={sol2[1]:.6f}")

```

2.12 Métodos Específicos para Polinômios

2.12.1 Algoritmo de Horner

Para polinômios, a avaliação de $p(x_0)$ pode ser feita de forma ótima reescrevendo $p(x) = (\cdots((a_n x + a_{n-1})x + a_{n-2})x + \cdots + a_1)x + a_0$ e computando da direita para a esquerda.

Teorema 2.14 (Algoritmo de Horner). A recorrência

$$b_n = a_n, \quad b_i = a_i + x_0 b_{i+1} \quad (i = n-1, \dots, 0)$$

calcula $p(x_0) = b_0$ com exatamente n multiplicações e n adições (custo ótimo).

Demonstração. Por indução finita, a afirmação é $b_i = \sum_{j=i}^n a_j x_0^{j-i}$.

Base ($i = n$): $b_n = a_n = a_n x_0^0$. ✓

Passo: Supondo válido para b_{i+1} :

$$b_i = a_i + x_0 b_{i+1} = a_i + x_0 \sum_{j=i+1}^n a_j x_0^{j-i-1} = \sum_{j=i}^n a_j x_0^{j-i}. \checkmark$$

Para $i = 0$: $b_0 = \sum_{j=0}^n a_j x_0^j = p(x_0)$. □

Horner também fornece $p'(x_0)$ com custo adicional mínimo: de $p(x) = (x - x_0)q(x) + b_0$ com $q(x) = b_n x^{n-1} + \cdots + b_1$, tem-se $p'(x_0) = q(x_0)$; basta aplicar Horner a q em x_0 .

```

1 def horner_com_derivada(coef, x0):
2     """coef = [a_n, ..., a_0]. Retorna (p(x0), p'(x0))."""
3     p = coef[0]
4     dp = 0.0
5     for c in coef[1:]:

```

```

6         dp = dp * x0 + p    # acumula p' (Horner duplo)
7         p  = p  * x0 + c    # acumula p
8     return p, dp
9
10 def newton_horner(coef, x0, tol=1e-12, max_iter=50):
11     """Newton com avaliacao eficiente por Horner."""
12     x = x0
13     for _ in range(max_iter):
14         px, dpx = horner_com_derivada(coef, x)
15         if abs(dpx) < 1e-15:
16             break
17         dx = px / dpx
18         x -= dx
19         if abs(dx) < tol:
20             break
21     return x
22
23 # p(x) = x^3 - 6x^2 + 11x - 6 (raizes 1, 2, 3)
24 coef = [1, -6, 11, -6]
25 print(f"x0=1.5 -> {newton_horner(coef, 1.5):.10f}") # -> 1.0
26 print(f"x0=2.5 -> {newton_horner(coef, 2.5):.10f}") # -> 3.0

```

Listing 2.6: Horner com $p(x_0)$ e $p'(x_0)$ simultâneos; Newton+Horner

2.12.2 Deflação de Polinômios (Briot-Ruffini)

Conhecida uma raiz $\tilde{x}$ de $p(x)$, fatoramos $p(x) = (x - r)q(x)$. O algoritmo de Briot-Ruffini encontra os coeficientes de $q(x)$ por divisão sintética usando a mesma recorrência de Horner.

Cuidado

Deflação acumula erros numéricos. Se $\tilde{x}$ é aproximado, o polinômio deflacionado $q(x)$ terá coeficientes com erros que se propagam para as raízes seguintes. A estratégia correta é usar deflação apenas para obter aproximações iniciais e depois refinar cada raiz com Newton no polinômio *original* $p(x)$.

2.12.3 Fórmula de Bhaskara Numericamente Estável

A fórmula clássica $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ sofre cancelamento catastrófico quando $b^2 \gg 4ac$.

Teorema 2.15 (Bhaskara Estável). Para $ax^2 + bx + c = 0$, calcular de forma estável:

$$r_1 = \frac{-b - \operatorname{sgn}(b)\sqrt{\Delta}}{2a}, \quad r_2 = \frac{c}{a r_1}, \quad \Delta = b^2 - 4ac.$$

O sinal em r_1 evita a subtração catastrófica; a segunda raiz usa a relação de Viète $r_1 r_2 = c/a$. Para $b = -10^6$ e $c = 1$, a fórmula clássica com 6 dígitos perde completamente $r_2 \approx 10^{-6}$, enquanto a fórmula estável a recupera com plena precisão.

```

1 import numpy as np
2
3 def raizes_quadratica(a, b, c):
4     """Resolve ax^2 + bx + c = 0 de forma numericamente
5         estavel."""
6     delta = b**2 - 4*a*c
7     if delta < 0:
8         r = -b / (2*a); i = np.sqrt(-delta) / (2*a)
9         return complex(r, i), complex(r, -i)
10    s = np.sqrt(delta)
11    r1 = (-b - np.sign(b)*s) / (2*a) # evita cancelamento
12    r2 = c / (a * r1)                # relacao de Viete
13    return r1, r2
14
15 # Teste com equacao problematica: x^2 - 1e6*x + 1 = 0
16 r1, r2 = raizes_quadratica(1, -1e6, 1)
17 print(f"r1 = {r1:.10e}    r2 = {r2:.10e}")
18 print(f"r1*r2 = {r1*r2:.10f}    (deve ser 1.0)")

```

Listing 2.7: Bhaskara numericamente estável

2.13 Equação de Kepler — Projeto Integrador

A equação de Kepler da mecânica celeste,

$$E - e \sin E = M,$$

relaciona a anomalia excêntrica E (incógnita), a excentricidade e ($0 < e < 1$) e a anomalia média M (dado). É uma equação transcendente clássica que serve como projeto integrador dos métodos deste capítulo.

Definindo $f(E) = E - e \sin E - M$ e $f'(E) = 1 - e \cos E > 0$, Newton converge em poucas iterações para qualquer $e < 0,9$:

$$E_{k+1} = E_k - \frac{E_k - e \sin E_k - M}{1 - e \cos E_k}.$$

```

1  import numpy as np
2
3  def kepler_newton(M, e, tol=1e-12, max_iter=50):
4      """Resolve E - e*sin(E) = M para E (anomalia excentrica)."""
5      E = M      # chute inicial: anomalia media
6      for k in range(max_iter):
7          f = E - e * np.sin(E) - M
8          df = 1 - e * np.cos(E)
9          dE = f / df
10         E -= dE
11         if abs(dE) < tol:
12             return E, k+1
13     return E, max_iter
14
15 # Orbita da Terra: e = 0.0167
16 e = 0.0167
17 M_vals = np.linspace(0, 2*np.pi, 500)
18 E_vals = np.array([kepler_newton(M, e)[0] for M in M_vals])
19
20 residuo = np.max(np.abs(E_vals - e*np.sin(E_vals) - M_vals))
21 print(f"Residuo maximo: {residuo:.2e}")    # deve ser < 1e-11
22
23 # Numero de iteracoes para M = pi/4
24 _, n = kepler_newton(np.pi/4, e)
25 print(f"Iteracoes para M=pi/4: {n}")      # tipicamente 3-4

```

Listing 2.8: Resolução da equação de Kepler por Newton [?]

Para a órbita da Terra ($e = 0,0167$) e $M = \pi/4$, o método converge em apenas 3 a 4 iterações com $\varepsilon = 10^{-10}$.

2.14 Exercícios

Exercício 2.1 (Localização e teorema de Bolzano). Para cada função, verifique se o Teorema de Bolzano garante a existência de raiz no intervalo indicado e determine se a raiz é única (use f').

(a) $f(x) = x^3 - 9x + 3$, $[0, 1]$

(b) $f(x) = e^x - 3x$, $[0, 1]$

(c) $f(x) = \ln x - x + 2$, $[1, 4]$

(d) $f(x) = \cos x - x$, $[0, \pi/2]$

Exercício 2.2 (Cota de Cauchy). Para cada polinômio, calcule a Cota de Cauchy, determine o intervalo $[-R, R]$ que contém todas as raízes reais e, por tabela de sinais, localize sub-intervalos de comprimento 1 contendo cada raiz.

(a) $p(x) = x^3 - 6x^2 + 11x - 6$

(b) $p(x) = x^4 - 10x^3 + 35x^2 - 50x + 24$

(c) $p(x) = 2x^3 - 5x^2 - x + 6$

Exercício 2.3 (Método da bisseção — iterações manuais). Aplique a bisseção a $f(x) = x^3 - x - 2$ em $[1, 2]$ com $\varepsilon = 10^{-2}$.

(a) Execute as iterações em tabela com colunas k , a_k , b_k , c_k , $f(c_k)$, $b_k - a_k$.

(b) Use a fórmula teórica para prever o número de iterações e compare com o resultado experimental.

(c) Qual é o erro máximo garantido após 10 iterações?

Exercício 2.4 (Bisseção — diferentes funções). Para cada equação, isole uma raiz em $[a, b]$ pelo Teorema de Bolzano e realize 5 iterações da bisseção, organizando em tabela.

(a) $e^x = 3x$

(b) $\ln x = x - 2$

(c) $\sin x = x/2$ (raiz positiva não nula)

Exercício 2.5 (Método da posição falsa). Aplique a posição falsa a $f(x) = x \log_{10}(x) - 1$ em $[2, 3]$.

- (a) Calcule as duas primeiras iterações manualmente mostrando a fórmula da secante em cada passo.
- (b) Identifique qual extremo permanece fixo. Explique geometricamente por quê isso ocorre.
- (c) Compare com a bisseção no mesmo intervalo: qual chute foi mais próximo da raiz?

Exercício 2.6 (Funções de iteração — $x^2 + x - 6 = 0$). Para $\tilde{x}_2 = 2$, calcule $|\varphi'(\tilde{x}_2)|$ para cada função abaixo e decida sobre convergência. Para as convergentes, execute 5 iterações a partir de $x_0 = 2,5$ em tabela com colunas $k, x_k, |x_k - x_{k-1}|, |f(x_k)|$.

$$\varphi_1(x) = 6 - x^2, \quad \varphi_2(x) = \sqrt{6 - x}, \quad \varphi_3(x) = 6/x - 1, \quad \varphi_4(x) = 6/(x + 1).$$

Exercício 2.7 (Newton — iterações e comparação). Para cada equação, aplique Newton com o x_0 indicado até $|f(x_k)| < 10^{-8}$, em tabela com colunas $k, x_k, f(x_k), f'(x_k), |x_k - x_{k-1}|$. Compare com bisseção.

- (a) $f(x) = x^3 - x - 2, \quad x_0 = 2$
- (b) $f(x) = e^x - 3x, \quad x_0 = 0,5$
- (c) $f(x) = \ln x - \cos x, \quad x_0 = 1,3$
- (d) $f(x) = x^3 - 9x + 3, \quad x_0 = 0,5$ (raiz em $[0, 1]$)

Exercício 2.8 (Raízes múltiplas). Considere $f(x) = (x - 1)^3$.

- (a) Aplique Newton padrão a partir de $x_0 = 0$, execute 10 iterações e estime a razão $|e_{k+1}|/|e_k|$. Confirme convergência linear com fator $2/3$.
- (b) Aplique o método modificado com $m = 3$ e observe a convergência quadrática.
- (c) Para $f(x) = (x - 2)^2(x + 1)$, encontre as raízes por Newton padrão e modificado a partir de $x_0 = 1,5$ e $x_0 = -0,5$.

Exercício 2.9 (Método da secante). Aplique a secante a $f(x) = x^3 - x - 2$ com $x_0 = 1$ e $x_1 = 2$ até $|f(x_k)| < 10^{-6}$. Organize em tabela com colunas $k, x_k, f(x_k), |x_k - x_{k-1}|$. Compare com Newton ($x_0 = 2$) e bisseção ($[1, 2]$): número de iterações e avaliações de f .

Exercício 2.10 (Algoritmo de Horner). Avalie cada polinômio em x_0 usando Horner, mostrando a tabela de recorrência. Calcule também $p'(x_0)$ via Horner duplo.

- (a) $p(x) = x^4 - 3x^3 + 2x^2 - x + 5, \quad x_0 = 2$

(b) $p(x) = 2x^5 - x^4 + 3x^3 - 2x^2 + x - 7$, $x_0 = -1$

Exercício 2.11 (Newton + Horner — deflação). Para $p(x) = x^4 - 10x^3 + 35x^2 - 50x + 24$ (raízes 1, 2, 3, 4):

- (a) Parta de $x_0 = 0,5$ e encontre $\tilde{x}_1 = 1$ via Newton+Horner.
- (b) Deflacione por Briot-Ruffini para obter $q(x)$ de grau 3.
- (c) Repita para as demais raízes, aplicando Newton+Horner a q, r, s sucessivamente.
- (d) Refine cada raiz no polinômio *original* $p(x)$ e compare a precisão antes e depois do refinamento.

Exercício 2.12 (Comparação completa). Resolva $f(x) = e^x - 3x = 0$ na raiz em $[0, 1]$ com $\varepsilon = 10^{-8}$, aplicando bisseção, posição falsa, ponto fixo ($\varphi(x) = e^x/3$), Newton e secante. Para cada método, registre o número de iterações e de avaliações de f . Construa uma tabela comparativa.

Exercício 2.13 (Bhaskara estável). Para $f(x) = x^2 - 10^6x + 1$, calcule as raízes pela fórmula clássica com 6 dígitos significativos e compare com a fórmula estável.

- (a) Mostre onde ocorre o cancelamento catastrófico na fórmula clássica.
- (b) Verifique que a fórmula estável recupera $r_2 \approx 10^{-6}$ com plena precisão usando a relação de Viète.

Exercício 2.14 (Equações transcendentais — Newton). Resolva com Newton ($\varepsilon = 10^{-10}$) e compare com bisseção:

- (a) $\sin(x) = x/2$ (primeira raiz positiva não nula)
- (b) $x \ln(x) = 1$
- (c) $\tan(x) = x$ (primeira raiz positiva)

Exercício 2.15 (Sistema não-linear). Resolva o sistema $\begin{cases} x^2 - y^2 = 1 \\ xy = 2 \end{cases}$ via Newton multi-variável. Encontre todas as soluções reais.

Exercício 2.16 (Projeto — Equação de Kepler). Para $e = 0,0167$ (órbita da Terra):

- (a) Para $M = \pi/4$, resolva $E - e \sin E = M$ por Newton, registrando todas as iterações em tabela.

- (b) Implemente uma função que resolve para qualquer $M \in [0, 2\pi]$ e plote E versus M .
- (c) Compare Newton com bisseção em termos de iterações e avaliações de f para $M = \pi/2$.

Exercício 2.17 (Projeto — localizador robusto de raízes). Crie uma função `encontrar_raizes(f, a, b, tol, metodo)` que:

- (a) Divide $[a, b]$ em $N = 100$ subintervalos e detecta mudanças de sinal.
- (b) Em cada subintervalo com mudança de sinal, aplica o método escolhido (`'bissecão'`, `'pos_falsa'`, `'newton'`, `'secante'`).
- (c) Retorna todas as raízes reais encontradas.

Teste com: $p(x) = x^4 - 10x^3 + 35x^2 - 50x + 24$; $f(x) = \sin(5x) - x/2$ em $[-\pi, \pi]$.

Exercício 2.18 (Desafio — casos problemáticos de Newton). (a) Para $f(x) = x^{1/3}$, aplique Newton a partir de $x_0 = 0,1$. O que acontece? Explique geometricamente.

- (b) Construa um exemplo em que Newton oscila indefinidamente para $f(x) = x^3 - 2x + 2$.
- (c) Para $f(x) = \arctan(x)$, determine analiticamente os valores de x_0 a partir dos quais Newton converge.

2.15 Referências e Leituras Complementares

As referências citadas neste capítulo e leituras adicionais estão reunidas na bibliografia ao final do livro. Os itens mais relevantes para este capítulo são:

Livros-texto:

- franco2007; ruggiero1997; sperandio2003 — referências brasileiras com exemplos numéricos detalhados.
- burden2015 — cap. 2: soluções de equações em uma variável.
- press2007 — cap. 9: root finding and nonlinear sets of equations.
- kelley2003 — Newton para equações não-lineares.

Artigos e referências especializadas:

- muller1956 — método de Muller (generalização da secante para três pontos).

- danby1988 — equação de Kepler e mecânica celeste.
- farin2002 — curvas de Bézier e CAGD.
- koblitz1987,hankerson2004 — criptografia de curvas elípticas.

Recursos computacionais:

- SciPy Optimization and Root Finding: <https://docs.scipy.org/doc/scipy/reference/optimize.html>
- NumPy Polynomial Module: <https://numpy.org/doc/stable/reference/routines.polynomials.html>
- Companion Matrix Method (autovalores para todas as raízes simultaneamente): https://en.wikipedia.org/wiki/Companion_matrix

Capítulo 3

Sistemas de Equações Lineares: Métodos Diretos

“A mathematical theory is not to be considered complete until you have made it so clear that you can explain it to the first man whom you meet on the street.”

— David Hilbert

3.1 Introdução e Motivação

Resolver sistemas de equações lineares é um dos problemas mais recorrentes na ciência e engenharia: previsão do tempo, otimização de redes de tráfego, mecânica quântica, aprendizado de máquina, simulações físicas — todos se reduzem, em algum ponto, a resolver $Ax = b$.

Na forma matricial compacta:

$$Ax = b, \quad A \in \mathbb{R}^{n \times n}, \quad x \in \mathbb{R}^n, \quad b \in \mathbb{R}^n,$$

em que A é a matriz de coeficientes, x o vetor de incógnitas e b o vetor de termos independentes. Na forma explícita:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

A figura abaixo mostra como um sistema 3×3 se interpreta geometricamente e como a fatoração LU o decompõe em dois sistemas triangulares:

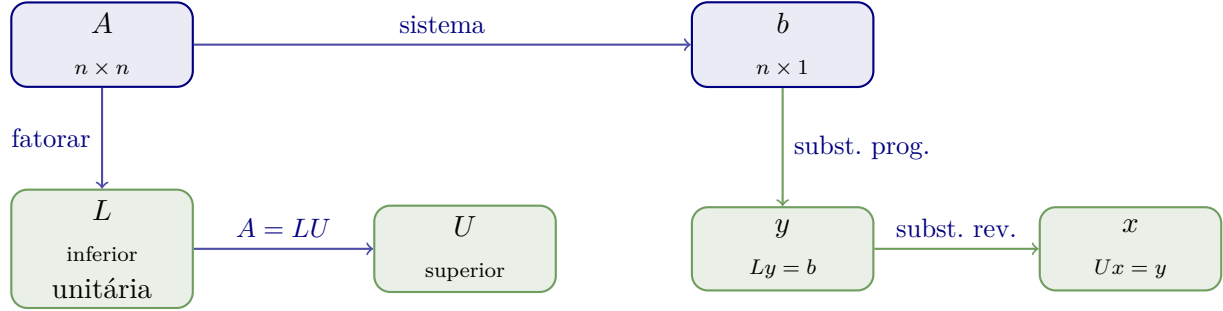


Figura 3.1: Pipeline de resolução de $Ax = b$ pela fatoração LU. A matriz A é decomposta em L (triangular inferior unitária) e U (triangular superior). Dois sistemas triangulares são resolvidos sequencialmente.

Aplicação Computacional

PageRank — O Algoritmo do Google [5]:

O algoritmo PageRank classifica as $n \sim 10^{11}$ páginas da web resolvendo o sistema linear:

$$(I - \alpha P^T) \pi = \frac{(1 - \alpha)}{n} e,$$

em que $\alpha \approx 0,85$ é o *damping factor*, P é a matriz de transição entre páginas e e é o vetor de uns. Resolver esse sistema de dimensão astronômica diretamente é inviável — métodos iterativos são usados, mas a estrutura teórica dos métodos diretos fornece a base de compreensão.

Aplicação Computacional

Redes Neurais — Regressão e Mínimos Quadrados [3]:

Treinar um modelo de regressão linear $\hat{y} = X\beta$ minimizando $\|X\beta - y\|^2$ resulta nas *equações normais*:

$$(X^T X) \beta = X^T y.$$

A matriz $X^T X$ é simétrica positiva definida quando X tem posto completo, tornando a fatoração de Cholesky (ver [section 3.7.1](#)) o método ideal. Para m amostras e p atributos, o custo é $O(p^3)$ — dominado pela fatoração, não pelos dados.

3.2 Existência e Unicidade da Solução

Teorema 3.1 (Existência e Unicidade). O sistema $Ax = b$ tem solução única se, e somente se, $\det(A) \neq 0$ (i.e., A é não-singular).

Demonstração. ($\Rightarrow$) Suponha solução única. Se $\det(A) = 0$, então A é singular e existe $v \neq 0$ com $Av = 0$. Se x_0 é a solução, então $x_0 + v$ também é solução: $A(x_0 + v) = Ax_0 + Av = b + 0 = b$, contradizendo a unicidade.

($\Leftarrow$) Se $\det(A) \neq 0$, A é invertível e $x = A^{-1}b$ é solução. A unicidade segue de: se $Ax_1 = Ax_2 = b$, então $x_1 = A^{-1}Ax_1 = A^{-1}b = A^{-1}Ax_2 = x_2$. $\square$

Nota Importante

Na prática, **nunca** calculamos $\det(A)$ para verificar singularidade — o custo é $O(n!)$ (pela fórmula de Leibniz) ou $O(n^3)$ (pela eliminação), mas sem ganho sobre a resolução direta. Os métodos de fatoração detectam singularidade automaticamente durante o processo.

3.3 Operações Elementares e Equivalência de Sistemas

A resolução de sistemas lineares baseia-se em três operações elementares sobre as linhas da matriz aumentada $[A \mid b]$ que preservam as soluções:

1. **Troca:** permutar duas equações: $L_i \leftrightarrow L_j$.
2. **Escalonamento:** multiplicar uma equação por escalar $\lambda \neq 0$: $L_i \leftarrow \lambda L_i$.
3. **Combinação linear:** somar um múltiplo de uma equação a outra: $L_i \leftarrow L_i + \lambda L_j$.

Dois sistemas são **equivalentes** se têm exatamente as mesmas soluções. Qualquer sequência das operações acima produz sistemas equivalentes.

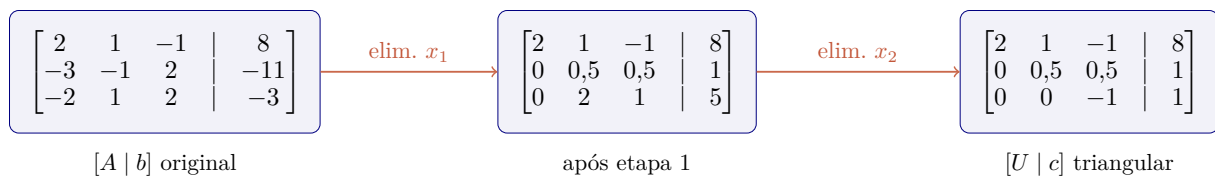


Figura 3.2: Processo de triangularização pela eliminação de Gauss para o sistema do exemplo 3.1. Cada seta representa uma etapa de eliminação que zera todos os elementos abaixo do pivô de uma coluna.

3.4 Método de Eliminação de Gauss

3.4.1 Ideia e Algoritmo

A eliminação gaussiana transforma $[A \mid b]$ em $[U \mid c]$ triangular superior por uma sequência de operações elementares do tipo 3, usando o elemento diagonal $a_{kk}^{(k-1)}$ (chamado **pivô**) de cada etapa.

Na etapa k , calculam-se os **multiplicadores** $m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}$ para $i > k$, e aplica-se $L_i \leftarrow L_i - m_{ik}L_k$ eliminando x_k das equações $k + 1, \dots, n$.

Algoritmo 4 Eliminação de Gauss com Substituição Reversa

Entrada: $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, $\det(A) \neq 0$

Saída: Vetor solução x

```

1: Formar a matriz aumentada  $[A \mid b]$ 
2: para  $k = 1$  até  $n - 1$  faça                                     ▷ fase de eliminação
3:   para  $i = k + 1$  até  $n$  faça
4:      $m_{ik} \leftarrow \frac{a_{ik}}{a_{kk}}$ 
5:     para  $j = k + 1$  até  $n$  faça
6:        $a_{ij} \leftarrow a_{ij} - m_{ik} \cdot a_{kj}$ 
7:     end para
8:      $b_i \leftarrow b_i - m_{ik} \cdot b_k$ 
9:   end para
10: end para
11:  $x_n \leftarrow b_n / a_{nn}$                                            ▷ substituição reversa
12: para  $i = n - 1$  até  $1$  faça
13:    $x_i \leftarrow \frac{\left(b_i - \sum_{j=i+1}^n a_{ij} x_j\right)}{a_{ii}}$ 
14: end para
15: retorne  $x$ 

```

Exemplo 3.1 (Sistema 3×3). Resolver:

$$\begin{cases} 2x_1 + x_2 - x_3 = 8 \\ -3x_1 - x_2 + 2x_3 = -11 \\ -2x_1 + x_2 + 2x_3 = -3 \end{cases}$$

Etapa 1 — zerar coluna 1 abaixo do pivô $a_{11} = 2$:

$$\left[\begin{array}{ccc|c} \boxed{2} & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right] \begin{array}{l} L_1 \\ L_2 \\ L_3 \end{array} \quad \leftarrow \text{pivô } a_{11} = 2$$

$$m_{21} = -\frac{-3}{2} = \frac{3}{2}, \quad m_{31} = -\frac{-2}{2} = 1$$

$$L_2 \leftarrow L_2 + \frac{3}{2} L_1, \quad L_3 \leftarrow L_3 + 1 \cdot L_1$$

Assim:

- $L_2 \leftarrow L_2 - m_{21}L_1$: $[-3 \quad -1 \quad 2 \mid -11] + \frac{3}{2}[2 \quad 1 \quad -1 \mid 8] = [0 \quad \frac{1}{2} \quad \frac{1}{2} \mid 1]$
- $L_3 \leftarrow L_3 - m_{31}L_1$: $[-2 \quad 1 \quad 2 \mid -3] + [2 \quad 1 \quad -1 \mid 8] = [0 \quad 2 \quad 1 \mid 5]$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ \boxed{0} & \boxed{\frac{1}{2}} & \frac{1}{2} & 1 \\ \boxed{0} & 2 & 1 & 5 \end{array} \right] \begin{array}{l} L_1 \\ L'_2 \\ L'_3 \end{array} \quad \leftarrow \text{pivô } a_{22} = \frac{1}{2}$$

Etapla 2 — zerar coluna 2 abaixo do pivô $a_{22} = \frac{1}{2}$:

$$m_{32} = \frac{2}{\frac{1}{2}} = 4, \quad L_3 \leftarrow L'_3 - 4 L'_2$$

$$L_3 \leftarrow L_3 - 4L'_2: [0 \quad 2 \quad 1 \mid 5] - 4 \left[0 \quad \frac{1}{2} \quad \frac{1}{2} \mid 1 \right] = [0 \quad 0 \quad -1 \mid 1]$$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ \boxed{0} & \boxed{0} & -1 & 1 \end{array} \right] \begin{array}{l} L_1 \\ L'_2 \\ L''_3 \end{array} \quad (\text{forma triangular superior})$$

Sistema triangular e substituição reversa:

De L''_3 :

$$-x_3 = 1 \implies x_3 = -1$$

De L'_2 :

$$\frac{1}{2}x_2 + \frac{1}{2}(-1) = 1 \implies \frac{x_2}{2} = \frac{3}{2} \implies x_2 = 3$$

De L_1 :

$$2x_1 + 3 - (-1) = 8 \implies 2x_1 = 4 \implies x_1 = 2$$

Solução: $(x_1, x_2, x_3) = (2, 3, -1)$.

3.4.2 Análise de Complexidade

Teorema 3.2 (Complexidade da Eliminação de Gauss). O número de operações de ponto flutuante da eliminação gaussiana é $\frac{2n^3}{3} + O(n^2)$.

Demonstração. Na etapa k , há $(n - k)$ multiplicadores e, para cada um, $(n - k)$ multiplicações e $(n - k)$ subtrações. O custo da etapa k é $2(n - k)^2 + (n - k)$ flops. Somando:

$$\sum_{k=1}^{n-1} [2(n-k)^2 + (n-k)] = 2 \sum_{j=1}^{n-1} j^2 + \sum_{j=1}^{n-1} j = 2 \cdot \frac{(n-1)n(2n-1)}{6} + \frac{(n-1)n}{2} = \frac{2n^3}{3} + O(n^2).$$

A substituição reversa acrescenta $O(n^2)$ flops adicionais. $\square$

3.4.3 Implementação

```

1 import numpy as np
2
3 def gauss_elimizacao(A, b, pivoteamento=True):
4     """
5     Resolve Ax = b por elimina o de Gauss.
6     pivoteamento=True: pivoteamento parcial (recomendado).
7     Retorna: vetor solucao x
8     """
9     A = np.array(A, dtype=float)
10    b = np.array(b, dtype=float)
11    n = len(b)
12    Ab = np.c_[A, b]          # matriz aumentada
13
14    for k in range(n - 1):
15        if pivoteamento:      # encontrar maior piv na coluna
16                               k
17            p = k + np.argmax(np.abs(Ab[k:, k]))
18            if p != k:
19                Ab[[k, p]] = Ab[[p, k]]
20
21        if abs(Ab[k, k]) < 1e-14:
22            raise ValueError("Sistema singular ou
23                               mal-condicionado")
24
25        for i in range(k + 1, n):

```

```

24         m = Ab[i, k] / Ab[k, k]
25         Ab[i, k:] -= m * Ab[k, k:]
26
27     # Substitui o reversa
28     x = np.zeros(n)
29     for i in range(n - 1, -1, -1):
30         x[i] = (Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:n])) /
31             Ab[i, i]
32     return x
33
34 # Exemplo do texto
35 A = np.array([[ 2,  1, -1],
36              [-3, -1,  2],
37              [-2,  1,  2]], dtype=float)
38 b = np.array([8, -11, -3], dtype=float)
39
40 x = gauss_eliminacao(A, b)
41 print(f"Solucao: {x}") # [ 2.  3. -1.]
42 print(f"Residuo: {np.linalg.norm(A @ x - b):.2e}")

```

Listing 3.1: Eliminação de Gauss com pivoteamento parcial

3.5 Estratégias de Pivoteamento

3.5.1 O Problema do Pivô Nulo ou Pequeno

A cada etapa k , calculamos os multiplicadores $m_{ik} = a_{ik}^{(k-1)} / a_{kk}^{(k-1)}$. Se o pivô $a_{kk}^{(k-1)}$ for nulo, o método falha. Se for próximo de zero, o multiplicador explode e os erros de arredondamento são amplificados drasticamente.

Cuidado

Exemplo de pivô pequeno:

$$\left[\begin{array}{cc|c} 10^{-20} & 1 & 1 \\ 1 & 1 & 2 \end{array} \right]$$

Sem pivoteamento: $m_{21} = 1/10^{-20} = 10^{20}$ — *overflow* ou perda catastrófica de precisão. Com pivoteamento parcial (trocar as linhas), o pivô passa a ser 1: estável.

3.5.2 Pivoteamento Parcial

No início da etapa k , troca-se a linha k com a linha p tal que $|a_{pk}| = \max_{i \geq k} |a_{ik}|$:

$$\begin{array}{c}
 A^{(1)} | b^{(1)} = \left[\begin{array}{cccc|c} 3 & 2 & 1 & -1 & 5 \\ 0 & 1 & 0 & 3 & 6 \\ 0 & -3 & -5 & 7 & 7 \\ 0 & 2 & 4 & 0 & 15 \end{array} \right] \quad \leftarrow \text{maior módulo em } |a_{k2}| \\
 \downarrow \text{troca } L_2 \leftrightarrow L_3 \text{ (pivoteamento parcial)} \\
 A^{(1)} | b^{(1)} = \left[\begin{array}{cccc|c} 3 & 2 & 1 & -1 & 5 \\ 0 & -3 & -5 & 7 & 7 \\ 0 & 1 & 0 & 3 & 6 \\ 0 & 2 & 4 & 0 & 15 \end{array} \right] \quad \leftarrow \text{novo pivô}
 \end{array}$$

Figura 3.3: Pivoteamento parcial na etapa 2: o elemento de maior módulo na coluna 2 (abaixo do pivô atual) é -3 , portanto as linhas 2 e 3 são permutadas antes de prosseguir com a eliminação.

Algoritmo 5 Gauss com Pivoteamento Parcial

Entrada: A, b

Saída: x

- 1: **para** $k = 1$ **até** $n - 1$ **faça**
 - 2: $p \leftarrow \arg \max_{i \geq k} |a_{ik}|$
 - 3: **se** $a_{pk} = 0$ **então retorne** “sistema singular”
 - 4: **end se**
 - 5: **se** $p \neq k$ **então** trocar linhas k e p em A e b
 - 6: **end se**
 - 7: **prosseguir com eliminação padrão**
 - 8: **end para**
-

Teorema 3.3 (Estabilidade com Pivoteamento Parcial). O fator de crescimento da eliminação com pivoteamento parcial é

$$\rho_n = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|} \leq 2^{n-1}.$$

Na prática, para matrizes típicas, $\rho_n = O(n)$.

3.5.3 Pivoteamento Completo

Escolhe como pivô o elemento de maior módulo em toda a submatriz não processada $(i, j \geq k)$. Isso exige também permutação de colunas (reordenamento das incógnitas)

e é computacionalmente mais custoso. Em geral, o pivoteamento parcial é suficiente e preferido na prática.

$$\begin{array}{cc}
 \text{Parcial:} & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \end{pmatrix} & \text{Completo:} & \begin{pmatrix} \cdot & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \end{pmatrix} \\
 & \text{busca na coluna } k & & \text{busca em toda a submatriz}
 \end{array}$$

Figura 3.4: Pivoteamento parcial busca apenas na coluna atual; pivoteamento completo busca em toda a submatriz não processada (região delimitada).

3.6 Fatoração LU

3.6.1 Motivação

Se precisarmos resolver $Ax = b$ para **múltiplos vetores** b distintos (mesma matriz A), repetir a eliminação inteira é ineficiente. A fatoração LU separa o custo: fatorar $A = LU$ custa $O(n^3)$ uma vez, e cada resolução posterior custa apenas $O(n^2)$.

$$Ax = b \xrightarrow{A=LU} LUx = b \xrightarrow{y=Ux} \underbrace{Ly = b}_{\text{subst. progressiva}} \quad \text{e} \quad \underbrace{Ux = y}_{\text{subst. reversa}}$$

$$\begin{array}{ccc}
 L & U & A \\
 \begin{pmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{pmatrix} & \times & \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix} & = & \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \\
 \text{diagonal unitária} & & \text{diagonal e acima:} & & \\
 \text{multiplicadores } m_{ik} & & \text{resultado da eliminação} & &
 \end{array}$$

Figura 3.5: Estrutura da fatoração $A = LU$. L armazena os multiplicadores da eliminação gaussiana na sua parte inferior; U é a matriz triangular superior resultante. Ambos residem na mesma estrutura de memória de A .

3.6.2 Existência

Teorema 3.4 (Existência da Fatoração LU). Se todos os menores principais de A são não-singulares, existe única fatoração $A = LU$ com L triangular inferior unitária e U triangular superior.

Esboço. Cada etapa k da eliminação gaussiana equivale a pré-multiplicar A por uma matriz elementar E_k que, na posição da coluna k abaixo da diagonal, coloca $-m_{ik}$:

$$E_{n-1} \cdots E_2 E_1 A = U.$$

Logo $A = E_1^{-1} E_2^{-1} \cdots E_{n-1}^{-1} U$. Como E_k^{-1} tem os mesmos m_{ik} mas com sinal positivo, e o produto de matrizes triangulares inferiores unitárias é triangular inferior unitária, definimos $L = E_1^{-1} \cdots E_{n-1}^{-1}$, obtendo $A = LU$. A unicidade segue da hipótese de que todos os menores principais são não-nulos (os pivôs são todos não-nulos). $\square$

3.6.3 Algoritmo de Doolittle

Algoritmo 6 Fatoração LU — Método de Doolittle

Entrada: $A \in \mathbb{R}^{n \times n}$ com menores principais não-singulares

Saída: L (triangular inferior unitária), U (triangular superior)

```

1:  $L \leftarrow I_n, \quad U \leftarrow 0_{n \times n}$ 
2: para  $k = 1$  até  $n$  faça
3:   para  $j = k$  até  $n$  faça ▷ linha  $k$  de  $U$ 
4:      $u_{kj} \leftarrow a_{kj} - \sum_{s=1}^{k-1} l_{ks} u_{sj}$ 
5:   end para
6:   para  $i = k+1$  até  $n$  faça ▷ coluna  $k$  de  $L$ 
7:      $l_{ik} \leftarrow \frac{1}{u_{kk}} \left( a_{ik} - \sum_{s=1}^{k-1} l_{is} u_{sk} \right)$ 
8:   end para
9: end para
10: retorne  $(L, U)$ 
```

Exemplo 3.2 (Fatoração LU — passo a passo). Fatore $A = \begin{bmatrix} 3 & 2 & 4 \\ 1 & 1 & 2 \\ 4 & 3 & 2 \end{bmatrix}$.

Etapa 1 ($k = 1$): linha 1 de U é a própria primeira linha de A . Multiplicadores: $l_{21} = \frac{1}{3}, l_{31} = \frac{4}{3}$.

$$A^{(1)} = \begin{bmatrix} 3 & 2 & 4 \\ 0 & \frac{1}{3} & \frac{2}{3} \\ 0 & \frac{1}{3} & \frac{-10}{3} \end{bmatrix}$$

Etapa 2 ($k = 2$): pivô $u_{22} = \frac{1}{3}$. Multiplicador: $l_{32} = \frac{\frac{1}{3}}{\frac{1}{3}} = 1$.

$$A^{(2)} = U = \begin{bmatrix} 3 & 2 & 4 \\ 0 & \frac{1}{3} & \frac{2}{3} \\ 0 & 0 & -4 \end{bmatrix}$$

Resultado:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{3} & 1 & 0 \\ \frac{4}{3} & 1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 3 & 2 & 4 \\ 0 & \frac{1}{3} & \frac{2}{3} \\ 0 & 0 & -4 \end{pmatrix}.$$

Verificação: $LU = A$. ✓

Para $b = (1, 2, 3)^T$: resolve-se $Ly = b \Rightarrow y = \left(1, \frac{5}{3}, 0\right)^T$ e $Ux = y \Rightarrow x = (-3, 5, 0)^T$.

3.6.4 Fatoração PLU com Pivoteamento

Com pivoteamento parcial obtemos $PA = LU$, em que P é a matriz de permutação que registra as trocas de linha.

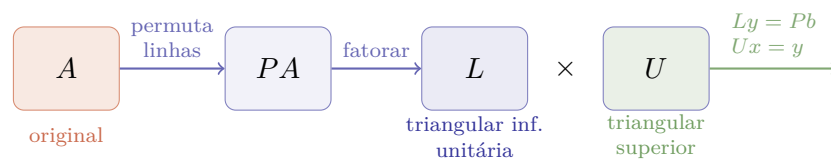


Figura 3.6: Fatoração PLU: a matriz de permutação P registra as trocas de linha do pivoteamento parcial, de modo que $PA = LU$.

```

1 import numpy as np
2 from scipy.linalg import lu
3
4 def fatoracao_lu(A):
5     """Fatoracao LU sem pivoteamento (Doolittle). Retorna (L,
6         U)."""
7     A = np.array(A, dtype=float)
8     n = A.shape[0]
9     L, U = np.eye(n), np.zeros((n, n))
10    for k in range(n):
11        for j in range(k, n):
12            U[k, j] = A[k, j] - L[k, :k] @ U[:k, j]
13        for i in range(k + 1, n):
14            L[i, k] = (A[i, k] - L[i, :k] @ U[:k, k]) / U[k, k]
15    return L, U

```

```

15
16 def resolver_lu(L, U, b):
17     """Resolve Ax=b dado A=LU."""
18     n = len(b)
19     y = np.zeros(n)
20     for i in range(n):                                # substituição
21         progressiva
22         y[i] = b[i] - L[i, :i] @ y[:i]
23     x = np.zeros(n)
24     for i in range(n - 1, -1, -1):                    # substituição
25         reversa
26         x[i] = (y[i] - U[i, i+1:] @ x[i+1:]) / U[i, i]
27     return x
28
29 # Exemplo do texto
30 A = np.array([[3, 2, 4], [1, 1, 2], [4, 3, 2]], dtype=float)
31 b = np.array([1, 2, 3], dtype=float)
32
33 L, U = fatoracao_lu(A)
34 x = resolver_lu(L, U, b)
35 print(f"Solução LU manual: {x}")                      # [-3.  5.  0.]
36
37 # PLU com SciPy (recomendado para produção)
38 P, Lp, Up = lu(A)
39 # Resolve Ax = b: primeiro Ly = Pb, depois Ux = y
40 y = resolver_lu(np.eye(len(b)), Lp, P @ b)          # Ly = Pb
41 x_scipy = resolver_lu(np.eye(len(b)), Up, y)         # Ux = y
42 print(f"Solução PLU scipy: {np.linalg.solve(A, b)}") #
43 referencia
44
45 # Múltiplos lados direitos (vantagem da LU)
46 b2 = np.array([2, 1, 0], dtype=float)
47 x2 = resolver_lu(L, U, b2)
48 print(f"Solução para b2: {x2}")

```

Listing 3.2: Fatoração LU e PLU em Python

3.7 Matrizes com Estrutura Especial

3.7.1 Matrizes Simétricas Positivas Definidas — Fatoração de Cholesky

Definição 3.5 (Matriz Simétrica Positiva Definida (SPD)). Uma matriz $A \in \mathbb{R}^{n \times n}$ simétrica é *positiva definida* se

$$x^T A x > 0 \quad \forall x \neq 0.$$

Teorema 3.6 (Fatoração de Cholesky). Se A é SPD, existe única matriz triangular inferior L com diagonal positiva tal que $A = LL^T$.

Esboço por indução em n . **Base** ($n = 1$): $A = [a_{11}]$ com $a_{11} > 0$, logo $L = [\sqrt{a_{11}}]$.

Passo: Particione $A = \begin{bmatrix} A_{11} & a \\ a^T & a_{nn} \end{bmatrix}$ e busque $L = \begin{bmatrix} L_{11} & 0 \\ l^T & l_{nn} \end{bmatrix}$. Da equação $LL^T = A$:

$$L_{11}L_{11}^T = A_{11}, \quad L_{11}l = a, \quad l^T l + l_{nn}^2 = a_{nn}.$$

Por hipótese de indução, $A_{11} = L_{11}L_{11}^T$ existe. Então $l = L_{11}^{-1}a$ e $l_{nn} = \sqrt{a_{nn} - \|l\|^2}$. Como A é SPD, $a_{nn} - \|l\|^2 > 0$, garantindo l_{nn} real e positivo. $\square$

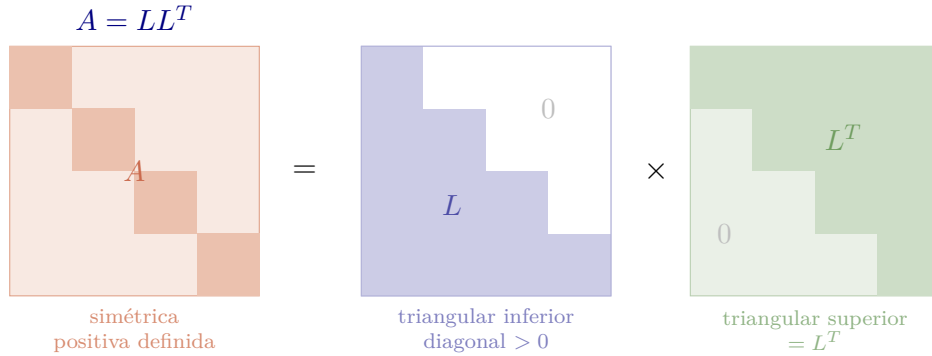


Figura 3.7: Estrutura da fatoração de Cholesky $A = LL^T$. Como A é SPD, basta armazenar L — a metade superior é redundante. O custo é $n^3/3$ flops, metade do LU geral.

Algoritmo 7 Fatoração de Cholesky

Entrada: $A \in \mathbb{R}^{n \times n}$ SPD

Saída: L triangular inferior com diagonal positiva, $A = LL^T$

- 1: **para** $k = 1$ **até** n **faça**
 - 2: $l_{kk} \leftarrow \sqrt{a_{kk} - \sum_{s=1}^{k-1} l_{ks}^2}$
 - 3: **para** $i = k + 1$ **até** n **faça**
 - 4: $l_{ik} \leftarrow \frac{1}{l_{kk}} \left(a_{ik} - \sum_{s=1}^{k-1} l_{is} l_{ks} \right)$
 - 5: **end para**
 - 6: **end para**
 - 7: **retorne** L
-

Nota Importante

Cholesky é $\approx 2\times$ **mais rápido** que LU: requer $n^3/3$ flops contra $2n^3/3$. Além disso, é numericamente mais estável para matrizes SPD — não exige pivoteamento.

Aplicação Computacional**Mínimos Quadrados, Ridge Regression e Processos Gaussianos [?]:**

Minimizar $\|X\beta - y\|^2$ leva às equações normais $(X^T X)\beta = X^T y$. Como $X^T X$ é SPD (quando X tem posto completo), Cholesky é o método ideal. Com regularização Ridge: $(X^T X + \lambda I)\beta = X^T y$ — a adição de λI melhora o condicionamento e garante positividade. Em Processos Gaussianos, a matriz de covariância K é SPD por construção, e Cholesky é usada para calcular $\log \det(K)$ e resolver $Kx = y$ eficientemente.

```

1 import numpy as np
2
3 def cholesky(A):
4     """Fatoracao de Cholesky: A = L L^T. Retorna L."""
5     A = np.array(A, dtype=float)
6     n = A.shape[0]
7     L = np.zeros((n, n))
8     for k in range(n):
9         L[k, k] = np.sqrt(A[k, k] - np.sum(L[k, :k]**2))
10        for i in range(k + 1, n):
11            L[i, k] = (A[i, k] - L[i, :k] @ L[k, :k]) / L[k, k]
12    return L
13
14 # Regressao linear via equacoes normais + Cholesky
15 np.random.seed(42)
16 m, p = 100, 5
17 X = np.random.randn(m, p)
18 y_true = X @ np.array([1, -2, 3, 0.5, -1]) +
19         0.1*np.random.randn(m)
20
21 A_normal = X.T @ X           # matrix SPD p x p
22 b_normal = X.T @ y_true
23
24 L = cholesky(A_normal)
25 # Resolver L z = b (substituicao progressiva)
26 z = np.linalg.solve(L, b_normal)

```

```

26 # Resolver  $L^T \beta = z$  (substituicao reversa)
27 beta = np.linalg.solve(L.T, z)
28
29 print("Coeficientes estimados:", np.round(beta, 3))
30 # Comparacao com numpy
31 beta_np = np.linalg.lstsq(X, y_true, rcond=None)[0]
32 print("Numpy lstsq: ", np.round(beta_np, 3))
33 print(f"Erro relativo: {np.linalg.norm(beta -
    beta_np)/np.linalg.norm(beta_np):.2e}")

```

Listing 3.3: Cholesky e mínimos quadrados

3.7.2 Matrizes Tridiagonais — Algoritmo de Thomas

Matrizes tridiagonais aparecem em discretizações de EDPs, splines cúbicas e processamento de sinais:

$$A = \begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & c_{n-1} & \\ & & a_n & b_n & \end{bmatrix}.$$

Matriz tridiagonal 6×6

0	0.5	0	0	0	0
-0.5	0	-0.5	0	0	0
0	-1.5	0	-1.5	0	0
0	0	-2.5	0	-2.5	0
0	0	0	-3.5	0	-3.5
0	0	0	0	-4.5	0

diagonal principal (b_i)
 diagonais sup./inf. (c_i, a_i)
 zeros (não armazenados)

Figura 3.8: Estrutura de uma matriz tridiagonal 6×6 : apenas $3n - 2$ elementos são não-nulos, possibilitando armazenamento e resolução em $O(n)$ em vez de $O(n^2)$ ou $O(n^3)$.

Teorema 3.7 (Algoritmo de Thomas). Um sistema tridiagonal pode ser resolvido em $O(n)$ operações por uma variante especializada da fatoração LU.

```

1 def thomas(a, b, c, d):
2     """
3     Resolve sistema tridiagonal  $Ax = d$  em  $O(n)$ .
4     a: diagonal inferior (índices 2..n, tamanho n-1)

```

```

5      b: diagonal principal (tamanho n)
6      c: diagonal superior (índices 1..n-1, tamanho n-1)
7      d: lado direito (tamanho n)
8      """
9      n = len(b)
10     c_ = np.zeros(n - 1)
11     d_ = np.zeros(n)
12
13     # Forward sweep (eliminacao)
14     c_[0] = c[0] / b[0]
15     d_[0] = d[0] / b[0]
16     for i in range(1, n - 1):
17         denom = b[i] - a[i-1] * c_[i-1]
18         c_[i] = c[i] / denom
19         d_[i] = (d[i] - a[i-1] * d_[i-1]) / denom
20     d_[n-1] = (d[n-1] - a[n-2]*d_[n-2]) / (b[n-1] -
        a[n-2]*c_[n-2])
21
22     # Back substitution
23     x = np.zeros(n)
24     x[n-1] = d_[n-1]
25     for i in range(n - 2, -1, -1):
26         x[i] = d_[i] - c_[i] * x[i+1]
27     return x
28
29 # Discretizacao de -u'' = f em [0,1], n=5 intervalos internos
30 n = 5
31 h = 1.0 / (n + 1)
32 a_ = np.full(n - 1, -1.0)
33 b_ = np.full(n, 2.0)
34 c_ = np.full(n - 1, -1.0)
35 d_ = np.full(n, h**2) # f(x)=1 => d_i = h^2
36
37 x = thomas(a_, b_, c_, d_)
38 print("Solucao (discretizacao de Poisson 1D):", np.round(x, 5))

```

Listing 3.4: Algoritmo de Thomas para sistemas tridiagonais

3.8 Análise de Erros e Condicionamento

3.8.1 Número de Condição de uma Matriz

Mesmo que $\det(A) \neq 0$, a solução de $Ax = b$ pode ser extremamente sensível a perturbações em b ou em A .

Definição 3.8 (Número de Condição). O número de condição de uma matriz não-singular A é:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|.$$

Para a norma espectral, $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$ (razão entre o maior e o menor valor singular).

Teorema 3.9 (Sensibilidade à Perturbação em b). Se $Ax = b$ e $A(x + \delta x) = b + \delta b$, então:

$$\frac{\|\delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\delta b\|}{\|b\|}.$$

Demonstração. De $A\delta x = \delta b$ temos $\|\delta x\| \leq \|A^{-1}\| \|\delta b\|$. De $Ax = b$ temos $\|b\| \leq \|A\| \|x\|$, logo

$$\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|}.$$

Multiplicando:

$$\frac{\|\delta x\|}{\|x\|} \leq \|A^{-1}\| \cdot \|A\| \cdot \frac{\|\delta b\|}{\|b\|} = \kappa(A) \frac{\|\delta b\|}{\|b\|}.$$

□

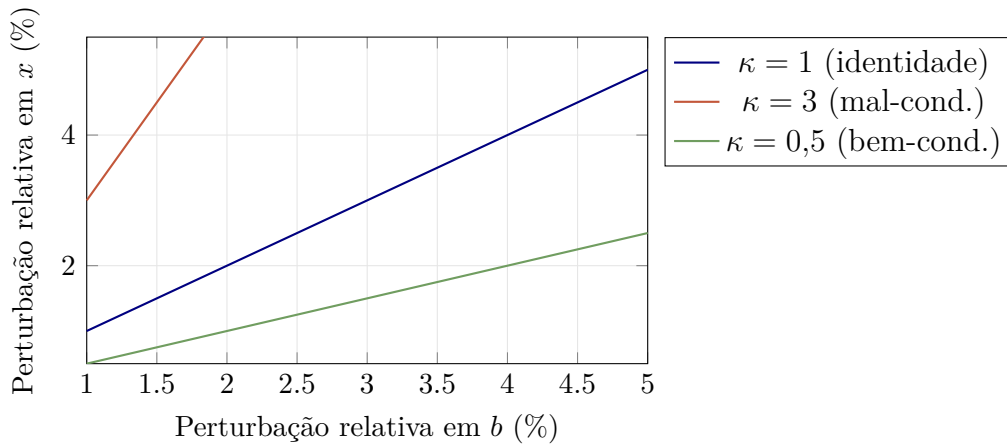


Figura 3.9: Amplificação de erros de acordo com o número de condição $\kappa(A)$. Uma perturbação de 1% em b pode resultar em até $\kappa\%$ de erro em x . Matrizes com $\kappa \approx 1$ são bem-condicionadas; $\kappa \gg 1$ indica mal-condicionamento.

Nota Importante

O número de condição indica quantos dígitos significativos são perdidos: se a aritmética tem d dígitos e $\kappa(A) \approx 10^k$, a solução tem **no máximo** $d - k$ dígitos corretos.

```

1 import numpy as np
2
3 # Exemplo de sistema mal-condicionado (matriz de Hilbert)
4 n = 8
5 H = np.array([[1/(i+j+1) for j in range(n)] for i in range(n)])
6 b = np.ones(n)
7
8 kappa = np.linalg.cond(H)
9 print(f"N mero de condicao de H_{n}: {kappa:.2e}") # ~1e10
10
11 x_exato = np.linalg.solve(H, b)
12 # Perturbar b em 1e-6 relativo
13 delta_b = 1e-6 * np.random.randn(n)
14 x_perturb = np.linalg.solve(H, b + delta_b)
15
16 erro_b = np.linalg.norm(delta_b) / np.linalg.norm(b)
17 erro_x = np.linalg.norm(x_perturb - x_exato) /
18     np.linalg.norm(x_exato)
19
20 print(f"Perturbacao relativa em b: {erro_b:.2e}")
21 print(f"Perturbacao relativa em x: {erro_x:.2e}")
22 print(f"Amplificacao observada: {erro_x/erro_b:.1f}")
23 print(f"Amplificacao maxima (kappa): {kappa:.1f}")

```

Listing 3.5: Número de condição e diagnóstico

3.9 Comparação e Escolha do Método

Tabela 3.1: Comparação dos métodos diretos para $Ax = b$.

Método	Flops	Mem.	Pivoteamento	Quando usar
Gauss (sem piv.)	$2n^3/3$	$O(n^2)$	—	Apenas educacional
Gauss (com piv.)	$2n^3/3$	$O(n^2)$	Parcial	Caso geral, um único b
Fatoração LU	$2n^3/3$	$O(n^2)$	Parcial	Múltiplos b , mesma A
Cholesky	$n^3/3$	$O(n^2)$	Não	A SPD (otimização, ML)
Thomas	$O(n)$	$O(n)$	Não	A tridiagonal (EDPs, splines)

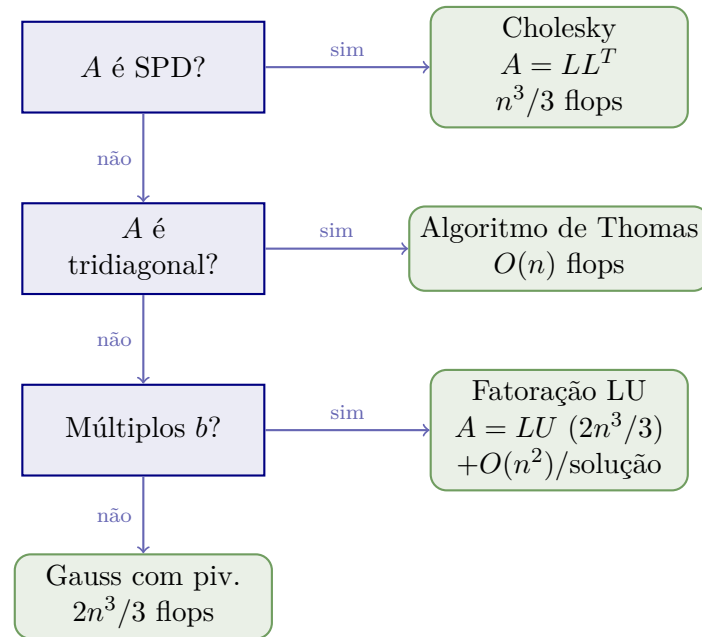


Figura 3.10: Fluxograma de escolha do método direto. A estrutura da matriz A e o padrão de uso (um único b vs. múltiplos b) determinam a estratégia ótima.

3.10 Exercícios

Exercício 3.1 (Eliminação de Gauss — manual). Resolva os sistemas a seguir pelo método de Gauss, organizando em tabela com as etapas da matriz aumentada:

$$(a) \begin{cases} 3x_1 + 2x_2 + 4x_3 = 1 \\ x_1 + x_2 + 2x_3 = 2 \\ 4x_1 + 3x_2 - 2x_3 = 3 \end{cases}$$

$$(b) \begin{cases} 2x_1 - x_2 + x_3 = 3 \\ x_1 + 3x_2 - x_3 = 2 \\ -x_1 + x_2 + 4x_3 = 1 \end{cases}$$

Exercício 3.2 (Pivoteamento parcial). Considere o sistema $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \end{bmatrix}$.

- Mostre que a eliminação sem pivoteamento falha.
- Aplique o pivoteamento parcial e resolva.
- Calcule o fator de crescimento ρ_2 .

Exercício 3.3 (Fatoração LU — passo a passo). Para $A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{bmatrix}$:

- (a) Obtenha L e U pelo método de Doolittle mostrando cada etapa.
- (b) Verifique que $LU = A$.
- (c) Use a fatoração para resolver $Ax = b$ para $b_1 = (1, 2, 3)^T$ e $b_2 = (0, 1, -1)^T$.

Exercício 3.4 (Complexidade). Implemente a eliminação de Gauss contando o número de operações de ponto flutuante executadas. Verifique experimentalmente que o custo é $\approx 2n^3/3$ testando com matrizes de tamanhos $n = 100, 200, 400, 800$. Plote o tempo de execução e o número de flops versus n em escala log-log.

Exercício 3.5 (Diagonal estritamente dominante). Demonstre que se A é diagonalmente estritamente dominante ($|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ para todo i), então:

- (a) A é não-singular.
- (b) A eliminação gaussiana sem pivoteamento é numericamente estável ($\rho_n \leq 2$).

Exercício 3.6 (Fatoração de Cholesky). (a) Obtenha a fatoração de Cholesky de $A = \begin{bmatrix} 4 & 2 & 0 \\ 2 & 5 & 3 \\ 0 & 3 & 10 \end{bmatrix}$ manualmente.

- (b) Prove que $\det(A) = \prod_{i=1}^n l_{ii}^2 > 0$.
- (c) Compare o tempo de execução de Cholesky e LU em Python para matrizes SPD de tamanhos $n = 100, 500, 1000$.

Exercício 3.7 (Algoritmo de Thomas). Resolva o sistema tridiagonal que surge da discretização de $-u'' = 1$ em $[0, 1]$ com condições $u(0) = u(1) = 0$ usando n pontos internos ($h = 1/(n+1)$, diagonal principal $= 2/h^2$, diagonais laterais $= -1/h^2$).

- (a) Resolva para $n = 5, 10, 20$ e compare com a solução exata $u(x) = x(1-x)/2$.
- (b) Verifique experimentalmente a ordem de convergência.

Exercício 3.8 (Condicionamento). A matriz de Hilbert H_n com $h_{ij} = 1/(i+j-1)$ é notoriamente mal-condicionada.

- (a) Para $n = 4, 6, 8, 10$, calcule $\kappa_2(H_n)$ usando `np.linalg.cond`.

- (b) Para cada n , resolva $H_n x = b$ com $b = H_n \cdot \mathbf{1}$ (solução exata $x = \mathbf{1}$) e meça o erro relativo.
- (c) Quantos dígitos corretos são esperados, considerando $\kappa_2(H_n)$ e a precisão double ($\approx 10^{-16}$)?

Exercício 3.9 (Aplicação — PageRank). Considere uma mini web com $n = 4$ páginas e matriz de adjacência:

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

- (a) Construa a matriz de transição P (normalizar colunas de G).
- (b) Resolva $(I - 0,85 P^T)\pi = 0,15 (e/4)$ pelo método de Gauss com pivoteamento.
- (c) Quais páginas têm maior PageRank? Interprete o resultado.

Exercício 3.10 (Projeto — Regressor Numérico Estável). Implemente um resolvidor de regressão linear que:

- (a) Usa Cholesky nas equações normais $(X^T X)\beta = X^T y$.
- (b) Usa fatoração QR de X (pesquise `numpy.linalg.qr`).
- (c) Compara a estabilidade numérica de ambas quando X é gerada com colunas quase colineares.

Teste com X aleatória de dimensão 100×10 e com número de condição $\kappa(X)$ variando de 10^1 a 10^8 .

3.11 Referências e Leituras Complementares

As referências citadas neste capítulo e leituras adicionais estão na bibliografia ao final do livro.

Livros-texto:

- trefethen1997 — álgebra linear numérica com foco em estabilidade.
- golub2013 — enciclopédia de computações matriciais, incluindo LU, Cholesky e variantes.

- franco2007; ruggiero1997 — referências brasileiras com exemplos detalhados.
- burden2015 — cap. 6: eliminação gaussiana e fatoração LU.

Aplicações em Ciência da Computação:

- page1999 — PageRank: sistemas lineares em grafos da web.
- rasmussen2006 — Processos Gaussianos: Cholesky em aprendizado de máquina.

Implementações de referência:

- LAPACK: <http://www.netlib.org/lapack/>
- NumPy linalg: <https://numpy.org/doc/stable/reference/routines.linalg.html>
- SciPy linalg: <https://docs.scipy.org/doc/scipy/reference/linalg.html>

Capítulo 4

Sistemas de Equações Lineares: Métodos Iterativos

“An approximate answer to the right problem is worth a good deal more than an exact answer to an approximate problem.”

— John Tukey

4.1 Motivação e Aplicações

No Capítulo 3, estudamos métodos diretos (eliminação de Gauss, fatoração LU) que, em princípio, resolvem sistemas lineares em número finito de passos. Entretanto, para sistemas de grande porte, métodos diretos apresentam limitações sérias:

1. **Dimensão proibitiva:** Sistemas com $n > 10^6$ variáveis são comuns em aplicações práticas (simulação de fluidos, estruturas, circuitos, etc.).
2. **Esparsidade destruída:** Matrizes esparsas (muitos zeros) perdem esparsidade durante eliminação gaussiana (*fill-in*), tornando-se densas.
3. **Complexidade cúbica:** $\mathcal{O}(n^3)$ operações é proibitivo quando $n \sim 10^7$.
4. **Requisitos de memória:** Armazenar matriz densa $n \times n$ requer $\mathcal{O}(n^2)$ espaço.
5. **Acúmulo de erros:** Para sistemas muito grandes, erros de arredondamento se acumulam.

Métodos iterativos oferecem alternativa: geram sequência de aproximações $x^{(0)}, x^{(1)}, x^{(2)}, \dots$ que converge (esperançosamente) para a solução x^* .

Aplicação Computacional

Simulação de Fluidos Computacional (CFD):

As equações de Navier-Stokes governam o movimento de fluidos:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f} \quad (4.1)$$

Discretizando em uma malha 3D de $100 \times 100 \times 100$ pontos (para simulação de fluxo de ar em torno de um carro, por exemplo), obtemos sistema com $n = 10^6$ incógnitas.

Estrutura da matriz: Cada ponto conecta-se apenas aos 6 vizinhos diretos (norte, sul, leste, oeste, cima, baixo). Logo, cada linha da matriz tem apenas 7 elementos não-nulos.

- Total de elementos: $n^2 = 10^{12}$
- Elementos não-nulos: $\sim 7n = 7 \times 10^6$
- Esparsidade: 99.9993% de zeros!

Comparação de métodos:

- **Eliminação de Gauss:** Fill-in destrói esparsidade. Memória: TB. Tempo: semanas.
- **Gradiente Conjugado + pré-condicionador:** Explora esparsidade. Memória: MB. Tempo: minutos.

Referência: Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. 2nd edition. SIAM. Disponível gratuitamente em: https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf

Aplicação Computacional

PageRank - O Algoritmo do Google:

O algoritmo PageRank, coração original do Google (desenvolvido por Larry Page e Sergey Brin em 1998), resolve um sistema linear gigantesco para classificar páginas web.

Modelo: Seja P a matriz de transição de um surfista aleatório na web. A importância (PageRank) de cada página é dada pelo vetor estacionário π que satisfaz:

$$\pi = \alpha P^T \pi + (1 - \alpha) \frac{\mathbf{e}}{n}, \quad (4.2)$$

em que:

- $\alpha \approx 0.85$ é o *damping factor*;
- $\mathbf{e}$ é o vetor de uns;
- n é o número de páginas.

Reescrevendo:

$$(I - \alpha P^T) \pi = (1 - \alpha) \frac{\mathbf{e}}{n}. \quad (4.3)$$

Escala do problema:

- Web em 2024: $n \sim 10^{11}$ páginas indexadas.
- Matriz P : $10^{11} \times 10^{11}$ (impossível armazenar densamente!).
- Esparsidade: média de 10 links por página.

Solução: Método das potências (iterativo) converge em 50-100 iterações, explorando esparsidade.

Referências:

- Page, L., et al. (1999). “The PageRank Citation Ranking: Bringing Order to the Web.” Technical Report, Stanford. <http://ilpubs.stanford.edu:8090/422/>
- Langville, A. N., & Meyer, C. D. (2006). *Google's PageRank and Beyond*. Princeton University Press.

Exercícios — Motivação

Exercício 4.1. Considere uma matriz A de dimensão $n \times n$ densa. Estime a memória necessária (em bytes, precisão dupla de 8 bytes) para $n = 10^3$, 10^4 e 10^5 . Compare com a memória necessária caso A seja esparsa com apenas $7n$ elementos não-nulos.

Exercício 4.2. Explique com suas palavras o fenômeno de *fill-in*. Ilustre com a fatoração LU de uma matriz tridiagonal 5×5 : quantos zeros da matriz original tornam-se não-nulos nos fatores L e U ?

Exercício 4.3. No modelo PageRank com $n = 10^{11}$ páginas e média de 10 links por página, calcule a fração de elementos não-nulos em P . Expresse esse número em notação científica e argumente por que métodos diretos são inviáveis.

4.2 Conceitos Fundamentais

4.2.1 Ideia Geral

Dado sistema $Ax = b$ com $A \in \mathbb{R}^{n \times n}$ não-singular, métodos iterativos constroem sequência:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots \rightarrow x^*. \quad (4.4)$$

A iteração geral tem a forma:

$$x^{(k+1)} = Tx^{(k)} + c, \quad (4.5)$$

em que $T \in \mathbb{R}^{n \times n}$ é a *matriz de iteração* e $c \in \mathbb{R}^n$ é um vetor constante.

4.2.2 Decomposição da Matriz

Toda matriz A pode ser decomposta como:

$$A = D - L - U, \quad (4.6)$$

em que:

- $D = \text{diag}(a_{11}, a_{22}, \dots, a_{nn})$ (parte diagonal);
- $-L$ = parte estritamente triangular inferior de A ;
- $-U$ = parte estritamente triangular superior de A .

Nota Importante

A notação “ $-L$ ” e “ $-U$ ” (com sinal negativo) é convencional para simplificar as fórmulas dos métodos iterativos. Alguns autores utilizam $A = D + L + U$ sem os sinais negativos.

Exemplo 4.1 (Decomposição $D - L - U$). Para a matriz:

$$A = \begin{bmatrix} 4 & -1 & 0 & 1 \\ -1 & 4 & -1 & 0 \\ 0 & -1 & 4 & -1 \\ 1 & 0 & -1 & 4 \end{bmatrix} \quad (4.7)$$

temos:

$$D = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix}, \quad -L = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 \end{bmatrix}, \quad -U = \begin{bmatrix} 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (4.8)$$

Verificação: $D - L - U = A$. ✓

4.2.3 Critério de Parada

Dada tolerância $\varepsilon > 0$, paramos a iteração quando a diferença entre dois iterados consecutivos for suficientemente pequena. A distância máxima componente a componente é:

$$d^{(k)} = \max_{1 \leq i \leq n} |x_i^{(k)} - x_i^{(k-1)}|. \quad (4.9)$$

Critério relativo (recomendado):

$$d_r^{(k)} = \frac{d^{(k)}}{\max_{1 \leq i \leq n} |x_i^{(k)}|} < \varepsilon. \quad (4.10)$$

O critério relativo evita parar precocemente quando a solução possui componentes de grande magnitude.

Nota Importante

$d_r^{(k)}$ pequeno **não garante** que $x^{(k)}$ seja próximo de x^* — isso depende do condicionamento do sistema. Em caso de dúvida, verifique também o resíduo $\|Ax^{(k)} - b\|$.

Exercícios — Conceitos Fundamentais

Exercício 4.4. Calcule explicitamente D , L e U na decomposição $A = D - L - U$ para a matriz:

$$A = \begin{bmatrix} 5 & -1 & 0 & 2 \\ -1 & 6 & -2 & 0 \\ 0 & -2 & 7 & -1 \\ 2 & 0 & -1 & 8 \end{bmatrix}$$

Verifique que $D - L - U = A$.

Exercício 4.5. Dado $x^{(2)} = (3,14; -1,07; 2,83)^T$ e $x^{(1)} = (2,98; -1,12; 2,76)^T$, calcule o critério de parada relativo $d_r^{(2)}$ da equação (4.10). A iteração deve ser interrompida para $\varepsilon = 0,05$? E para $\varepsilon = 0,01$?

Exercício 4.6. Explique a diferença entre os critérios:

- (a) $\|x^{(k)} - x^{(k-1)}\|_\infty < \varepsilon$;
- (b) $\|Ax^{(k)} - b\|_2 < \varepsilon$;
- (c) critério relativo (4.10).

Construa um exemplo numérico simples em que o critério (a) indica convergência, mas a solução ainda está longe de x^* .

4.3 Método de Gauss-Jacobi

4.3.1 Derivação do Método

Da equação $Ax = b$, usando $A = D - L - U$:

$$(D - L - U)x = b \quad \Rightarrow \quad Dx = (L + U)x + b. \quad (4.11)$$

Assumindo D inversível (todos os elementos diagonais não-nulos):

$$x = D^{-1}(L + U)x + D^{-1}b. \quad (4.12)$$

Isso sugere a iteração:

$$x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b. \quad (4.13)$$

Matriz de iteração de Gauss-Jacobi:

$$T_J = D^{-1}(L + U) = I - D^{-1}A. \quad (4.14)$$

4.3.2 Formulação Componente a Componente

Expandindo (4.13) para a componente i :

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad (4.15)$$

para $i = 1, 2, \dots, n$.

Nota Importante

Característica fundamental de Gauss-Jacobi: Todos os componentes de $x^{(k+1)}$ são calculados utilizando *apenas* valores de $x^{(k)}$. Isso permite:

- Paralelização natural (cada $x_i^{(k+1)}$ pode ser calculado independentemente).
- Implementação simples.
- Uso de dois vetores (um para $x^{(k)}$, outro para $x^{(k+1)}$).

4.3.3 Algoritmo

Algoritmo 8 Método de Gauss-Jacobi

Entrada: Matriz $A \in \mathbb{R}^{n \times n}$ com $a_{ii} \neq 0 \forall i$, vetor $b \in \mathbb{R}^n$, aproximação inicial $x^{(0)}$, tolerância $\epsilon > 0$

Saída: Aproximação da solução x^*

```

1:  $k \leftarrow 0$ 
2: repita
3:   para  $i = 1$  até  $n$  faça
4:      $soma \leftarrow \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)}$ 
5:      $x_i^{(k+1)} \leftarrow \frac{1}{a_{ii}} (b_i - soma)$ 
6:   end para
7:    $k \leftarrow k + 1$ 
8: até  $\|x^{(k)} - x^{(k-1)}\| < \epsilon$  ou  $\|Ax^{(k)} - b\| < \epsilon$  ou  $k > k_{max}$ 
9: retorne  $x^{(k)}$ 

```

Exemplo 4.2 (Gauss-Jacobi — sistema 3×3). Resolver o sistema abaixo (solução exata: $x^* = (2; 1; -1)^T$):

$$\begin{cases} 8x_1 - x_2 + x_3 = 14 \\ x_1 + 6x_2 - 2x_3 = 10 \\ -x_1 + 2x_2 + 7x_3 = -5 \end{cases} \quad (4.16)$$

Passo 1 — Verificar diagonal dominância.

Calculamos $\alpha_i = \sum_{j \neq i} |a_{ij}| / |a_{ii}|$ para cada linha:

$$\alpha_1 = \frac{1+1}{8} = 0,250, \quad \alpha_2 = \frac{1+2}{6} = 0,500, \quad \alpha_3 = \frac{1+2}{7} \approx 0,429.$$

$\alpha = \max_i \alpha_i = 0,500 < 1$. **Convergência garantida. ✓**

Passo 2 — Isolar cada variável.

$$x_1^{(k+1)} = \frac{1}{8}(14 + x_2^{(k)} - x_3^{(k)}) \quad (4.17)$$

$$x_2^{(k+1)} = \frac{1}{6}(10 - x_1^{(k)} + 2x_3^{(k)}) \quad (4.18)$$

$$x_3^{(k+1)} = \frac{1}{7}(-5 + x_1^{(k)} - 2x_2^{(k)}) \quad (4.19)$$

Passo 3 — Iterações (ponto de partida: $x^{(0)} = (0; 0; 0)^T$, $\varepsilon = 0,05$).

Iteração $k = 1$ (usar apenas $x^{(0)} = (0; 0; 0)^T$):

$$x_1^{(1)} = \frac{1}{8}(14 + 0 - 0) = \mathbf{1,7500}$$

$$x_2^{(1)} = \frac{1}{6}(10 - 0 + 0) = \mathbf{1,6667}$$

$$x_3^{(1)} = \frac{1}{7}(-5 + 0 - 0) = \mathbf{-0,7143}$$

Critério: $d^{(1)} = 1,7500$; $d_r^{(1)} = 1,7500/1,7500 = 1,000 > 0,05$ — **continuar**.

Iteração $k = 2$ (usar $x^{(1)} = (1,7500; 1,6667; -0,7143)^T$):

$$x_1^{(2)} = \frac{1}{8}(14 + 1,6667 + 0,7143) = \frac{16,381}{8} = \mathbf{2,0476}$$

$$x_2^{(2)} = \frac{1}{6}(10 - 1,7500 - 1,4286) = \frac{6,821}{6} = \mathbf{1,1369}$$

$$x_3^{(2)} = \frac{1}{7}(-5 + 1,7500 - 3,3334) = \frac{-6,583}{7} = \mathbf{-0,9405}$$

$d^{(2)} = \max\{0,2976; 0,5298; 0,2262\} = 0,5298$; $d_r^{(2)} = 0,5298/2,0476 \approx 0,259 > 0,05$ — **continuar**.

Tabela resumo:

k	x_1	x_2	x_3	$d_r^{(k)}$
0	0,0000	0,0000	0,0000	—
1	1,7500	1,6667	-0,7143	1,000
2	2,0476	1,1369	-0,9405	0,259
3	1,9749	1,0574	-0,9676	0,039
∞	2,0000	1,0000	-1,0000	0

Em $k = 3$: $d_r^{(3)} \approx 0,039 < 0,05$ — **parar!** Solução aproximada: $(1,975; 1,057; -0,968)^T$.
✓

Exemplo 4.3 (Gauss-Jacobi — sistema 3×3). Resolver:

$$\begin{cases} 10x_1 - x_2 + 2x_3 = 6 \\ -x_1 + 11x_2 - x_3 + 3x_4 = 25 \\ 2x_1 - x_2 + 10x_3 - x_4 = -11 \\ 3x_2 - x_3 + 8x_4 = 15 \end{cases} \quad (4.20)$$

Iteração de Jacobi:

$$x_1^{(k+1)} = \frac{1}{10}(6 + x_2^{(k)} - 2x_3^{(k)}) \quad (4.21)$$

$$x_2^{(k+1)} = \frac{1}{11}(25 + x_1^{(k)} + x_3^{(k)} - 3x_4^{(k)}) \quad (4.22)$$

$$x_3^{(k+1)} = \frac{1}{10}(-11 - 2x_1^{(k)} + x_2^{(k)} + x_4^{(k)}) \quad (4.23)$$

$$x_4^{(k+1)} = \frac{1}{8}(15 - 3x_2^{(k)} + x_3^{(k)}) \quad (4.24)$$

Começando com $x^{(0)} = (0, 0, 0, 0)^T$:

Iteração 1:

$$x_1^{(1)} = \frac{1}{10}(6 + 0 - 0) = 0.6000$$

$$x_2^{(1)} = \frac{1}{11}(25 + 0 + 0 - 0) = 2.2727$$

$$x_3^{(1)} = \frac{1}{10}(-11 - 0 + 0 + 0) = -1.1000$$

$$x_4^{(1)} = \frac{1}{8}(15 - 0 + 0) = 1.8750$$

Resultado: $x^{(1)} = (0,6000, 2,2727, -1,1000, 1,8750)^T$

Critério: $d^{(1)} = \max\{|0,6000 - 0|; |2,2727 - 0|; |-1,1000 - 0|; |1,8750 - 0|\} = \mathbf{2,2727}$;
 $d_r^{(1)} = 2,2727 / 2,2727 = 1,000 > 0,05$ — **continuar.**

Iteração 2:

$$x_1^{(2)} = \frac{1}{10}(6 + 2.2727 + 2.2) = 1.0473$$

$$x_2^{(2)} = \frac{1}{11}(25 + 0.6 - 1.1 - 5.625) = 1.7159$$

$$x_3^{(2)} = \frac{1}{10}(-11 - 1.2 + 2.2727 + 1.875) = -0.8052$$

$$x_4^{(2)} = \frac{1}{8}(15 - 6.8182 - 1.1) = 0.8852$$

Resultado: $x^{(2)} = (1,0473, 1,7159, -0,8052, 0,8852)^T$

Crítério: $d^{(2)} = \max\{0,4473; 0,5568; 0,2948; 0,9898\} = \mathbf{0,9898}$; $d_r^{(2)} = 0,9898 / 1,7159 \approx 0,577 > 0,05$ — **continuar**.

Iteração 3:

$$x_1^{(3)} = \frac{1}{10}(6 + 1,7159 - 2 \cdot (-0,8052)) = \frac{1}{10}(6 + 1,7159 + 1,6104) = \frac{9,3263}{10} \approx \mathbf{0,9326}$$

$$\begin{aligned} x_2^{(3)} &= \frac{1}{11}(25 + 1,0473 + (-0,8052) - 3 \cdot 0,8852) = \frac{1}{11}(25 + 1,0473 - 0,8052 - 2,6557) \\ &= \frac{22,5864}{11} \approx \mathbf{2,0533} \end{aligned}$$

$$\begin{aligned} x_3^{(3)} &= \frac{1}{10}(-11 - 2 \cdot 1,0473 + 1,7159 + 0,8852) = \frac{1}{10}(-11 - 2,0946 + 1,7159 + 0,8852) \\ &= \frac{-10,4935}{10} \approx \mathbf{-1,0494} \end{aligned}$$

$$x_4^{(3)} = \frac{1}{8}(15 - 3 \cdot 1,7159 + (-0,8052)) = \frac{1}{8}(15 - 5,1477 - 0,8052) = \frac{9,0471}{8} \approx \mathbf{1,1309}$$

Resultado: $x^{(3)} = (0,9326, 2,0533, -1,0494, 1,1309)^T$

Crítério: $d^{(3)} = \max\{0,1146; 0,3374; 0,2441; 0,2457\} = \mathbf{0,3374}$; $d_r^{(3)} = 0,3374 / 2,0533 \approx 0,164 > 0,05$ — **continuar**.

Iteração 4:

$$x_1^{(4)} = \frac{1}{10}(6 + 2,0533 - 2 \cdot (-1,0494)) = \frac{1}{10}(6 + 2,0533 + 2,0988) = \frac{10,1521}{10} \approx \mathbf{1,0152}$$

$$\begin{aligned} x_2^{(4)} &= \frac{1}{11}(25 + 0,9326 + (-1,0494) - 3 \cdot 1,1309) = \frac{1}{11}(25 + 0,9326 - 1,0494 - 3,3927) \\ &= \frac{21,4905}{11} \approx \mathbf{1,9537} \end{aligned}$$

$$\begin{aligned} x_3^{(4)} &= \frac{1}{10}(-11 - 2 \cdot 0,9326 + 2,0533 + 1,1309) = \frac{1}{10}(-11 - 1,8652 + 2,0533 + 1,1309) \\ &= \frac{-9,6810}{10} \approx \mathbf{-0,9681} \end{aligned}$$

$$x_4^{(4)} = \frac{1}{8}(15 - 3 \cdot 2,0533 + (-1,0494)) = \frac{1}{8}(15 - 6,1600 - 1,0494) = \frac{7,7906}{8} \approx \mathbf{0,9738}$$

Resultado: $x^{(4)} = (1,0152, 1,9537, -0,9681, 0,9738)^T$

Crítério: $d^{(4)} = \max\{0,0826; 0,0996; 0,0812; 0,1570\} = \mathbf{0,1570}$; $d_r^{(4)} = 0,1570 / 1,9537 \approx$

0,080 > 0,05 — **continuar**.

Iteração 5:

$$x_1^{(5)} = \frac{1}{10}(6 + 1,9537 - 2 \cdot (-0,9681)) = \frac{1}{10}(6 + 1,9537 + 1,9362) = \frac{9,8899}{10} \approx \mathbf{0,9890}$$

$$\begin{aligned} x_2^{(5)} &= \frac{1}{11}(25 + 1,0152 + (-0,9681) - 3 \cdot 0,9738) = \frac{1}{11}(25 + 1,0152 - 0,9681 - 2,9214) \\ &= \frac{22,1257}{11} \approx \mathbf{2,0114} \end{aligned}$$

$$\begin{aligned} x_3^{(5)} &= \frac{1}{10}(-11 - 2 \cdot 1,0152 + 1,9537 + 0,9738) = \frac{1}{10}(-11 - 2,0304 + 1,9537 + 0,9738) \\ &= \frac{-10,1029}{10} \approx \mathbf{-1,0103} \end{aligned}$$

$$x_4^{(5)} = \frac{1}{8}(15 - 3 \cdot 1,9537 + (-0,9681)) = \frac{1}{8}(15 - 5,8611 - 0,9681) = \frac{8,1708}{8} \approx \mathbf{1,0214}$$

Resultado: $x^{(5)} = (0,9890, 2,0114, -1,0103, 1,0214)^T$

Critério: $d^{(5)} = \max\{0,0262; 0,0577; 0,0422; 0,0475\} = \mathbf{0,0577}$; $d_r^{(5)} = 0,0577 / 2,0114 \approx 0,029 < 0,05$ — **parar**.

Com isso temos a seguinte tabela de convergência:

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	$x_4^{(k)}$	$d^{(k)}$	$d_r^{(k)}$
0	0,0000	0,0000	0,0000	0,0000	—	—
1	0,6000	2,2727	-1,1000	1,8750	2,2727	1,000
2	1,0473	1,7159	-0,8052	0,8852	0,9898	0,577
3	0,9326	2,0533	-1,0494	1,1309	0,3374	0,164
4	1,0152	1,9537	-0,9681	0,9738	0,1570	0,080
5	0,9890	2,0114	-1,0103	1,0214	0,0577	0,029
x^*	1,0000	2,0000	-1,0000	1,0000	0	0

- O método converge pois a matriz é **estritamente diagonalmente dominante**: em cada linha, o módulo do elemento diagonal supera a soma dos módulos dos demais elementos.
- O erro cai de 0,8750 na 1ª iteração para 0,0078 na 6ª iteração — uma redução de mais de 99%.

- O comportamento oscilatório (ora acima, ora abaixo da solução exata) é característico do método de Jacobi.
- Para aumentar a precisão, basta continuar as iterações até $\|x^{(k+1)} - x^{(k)}\|_\infty < \varepsilon$ para a tolerância ε desejada.

4.3.4 Implementação

```

1  import numpy as np
2
3  def jacobi(A, b, x0=None, tol=1e-10, max_iter=1000):
4      """
5      Resolve Ax = b usando o método de Jacobi.
6
7      Parâmetros:
8      -----
9      A : array_like, shape (n, n)
10         Matriz de coeficientes
11      b : array_like, shape (n,)
12         Vetor de termos independentes
13      x0 : array_like, shape (n,), opcional
14         Aproximação inicial (padrão: vetor nulo)
15      tol : float
16         Tolerância para critério de parada
17      max_iter : int
18         Número máximo de iterações
19
20      Retorna:
21      -----
22      x : ndarray
23         Aproximação da solução
24      k : int
25         Número de iterações realizadas
26      historico : list
27         Histórico de resíduos ||Ax - b||
28      """
29      A = np.array(A, dtype=float)
30      b = np.array(b, dtype=float)
31      n = len(b)
32

```

```

33     # Verificar diagonal n o -nula
34     if np.any(np.abs(np.diag(A)) < 1e-14):
35         raise ValueError("Matriz tem elementos diagonais nulos
36                             ou muito pequenos")
37
38     # Aproxima o inicial
39     x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
40     x_new = np.zeros(n)
41
42     historico = []
43
44     for k in range(max_iter):
45         # Calcular res duo
46         residuo = np.linalg.norm(A @ x - b)
47         historico.append(residuo)
48
49         # Itera o de Jacobi
50         for i in range(n):
51             soma = np.sum(A[i, :] * x) - A[i, i] * x[i]
52             x_new[i] = (b[i] - soma) / A[i, i]
53
54         # Crit rio de parada
55         if np.linalg.norm(x_new - x) < tol:
56             return x_new, k+1, historico
57
58     x = x_new.copy()
59
60     print(f"Aviso: M ximo de {max_iter} itera es atingido")
61     return x, max_iter, historico
62
63 # Vers o vetorizada (mais eficiente)
64 def jacobi_vetorizado(A, b, x0=None, tol=1e-10, max_iter=1000):
65     """Vers o vetorizada do m todo de Jacobi"""
66     A = np.array(A, dtype=float)
67     b = np.array(b, dtype=float)
68     n = len(b)
69
70     x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
71
72     # Extrair diagonal e resto

```

```

72     D_inv = 1.0 / np.diag(A)
73     R = A - np.diag(np.diag(A))  # A sem diagonal
74
75     historico = []
76
77     for k in range(max_iter):
78         residuo = np.linalg.norm(A @ x - b)
79         historico.append(residuo)
80
81         # Itera o vetorizada:  $x_{new} = D^{-1}(b - Rx)$ 
82         x_new = D_inv * (b - R @ x)
83
84         if np.linalg.norm(x_new - x) < tol:
85             return x_new, k+1, historico
86
87         x = x_new
88
89     return x, max_iter, historico
90
91 # Exemplo de uso
92 A = np.array([[10, -1, 2, 0],
93              [-1, 11, -1, 3],
94              [2, -1, 10, -1],
95              [0, 3, -1, 8]], dtype=float)
96 b = np.array([6, 25, -11, 15], dtype=float)
97
98 x, iters, hist = jacobi_vetorizado(A, b)
99
100 print(f"Solu o: {x}")
101 print(f"Itera es: {iters}")
102 print(f"Res duo final: {hist[-1]:.2e}")
103 print(f"Verifica o Ax: {A @ x}")
104 print(f"Vetor b: {b}")
105 print(f"Erro ||Ax - b||: {np.linalg.norm(A @ x - b):.2e}")

```

Listing 4.1: Método de Jacobi

Exercícios — Método de Gauss-Jacobi

Exercício 4.7. Verifique que o sistema (4.16) é estritamente diagonal dominante e calcule $\alpha = \max_i \alpha_i$. O que aconteceria se as equações fossem reordenadas como (linha 2, linha 3, linha 1)?

Exercício 4.8. Realize manualmente a iteração $k = 3$ do Exemplo 4.2, partindo de $x^{(2)} = (2,0476; 1,1369; -0,9405)^T$. Calcule $d_r^{(3)}$ e confirme o valor da tabela.

Exercício 4.9. Aplique Gauss-Jacobi ao sistema abaixo com $x^{(0)} = (0, 0, 0, 0)^T$ e $\varepsilon = 10^{-4}$. Verifique a diagonal dominância antes de iterar.

$$\begin{bmatrix} 9 & -1 & 2 & 0 \\ -1 & 8 & -1 & 2 \\ 2 & -1 & 9 & -1 \\ 0 & 2 & -1 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 20 \\ 16 \\ 17 \\ 10 \end{bmatrix}$$

Exercício 4.10. O sistema abaixo não é diagonal dominante na ordem apresentada. Reordene as equações de forma a obter diagonal dominância e aplique Gauss-Jacobi com $\varepsilon = 10^{-3}$:

$$\begin{cases} x_1 + 4x_2 - x_3 = 5 \\ 5x_1 + x_2 + 2x_3 = 6 \\ 3x_1 - x_2 + 6x_3 = 4 \end{cases}$$

Exercício 4.11 (Teórico). Mostre que $T_J = I - D^{-1}A$. A partir disso, prove que λ é autovalor de T_J com autovetor v se e somente se $Av = (1 - \lambda)Dv$. Use esse resultado para mostrar que Jacobi converge se $\rho(T_J) < 1$.

4.4 Método de Gauss-Seidel

4.4.1 Motivação e Derivação

O método de Jacobi utiliza apenas valores de $x^{(k)}$ para calcular $x^{(k+1)}$. Gauss-Seidel melhora isso: assim que calculamos $x_1^{(k+1)}$, podemos utilizá-lo imediatamente para calcular $x_2^{(k+1)}$, e assim por diante.

Da equação $Ax = b$ com $A = D - L - U$:

$$(D - L - U)x = b \quad \Rightarrow \quad (D - L)x = Ux + b. \quad (4.25)$$

Assumindo $(D - L)$ inversível:

$$x = (D - L)^{-1}Ux + (D - L)^{-1}b. \quad (4.26)$$

Iteração de Gauss-Seidel:

$$x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b. \quad (4.27)$$

Matriz de iteração:

$$T_{GS} = (D - L)^{-1}U. \quad (4.28)$$

4.4.2 Formulação Componente a Componente

Expandindo para a componente i :

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right). \quad (4.29)$$

Observe:

- Para $j < i$: utilizamos $x_j^{(k+1)}$ (já calculado nesta iteração);
- Para $j > i$: utilizamos $x_j^{(k)}$ (da iteração anterior).

Algoritmo 9 Método de Gauss-Seidel

Entrada: Matriz A , vetor b , aproximação inicial $x^{(0)}$, tolerância ϵ

Saída: Aproximação da solução

```

1:  $k \leftarrow 0$ 
2: repita
3:   para  $i = 1$  até  $n$  faça
4:      $soma1 \leftarrow \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)}$  ▷ Valores já atualizados
5:      $soma2 \leftarrow \sum_{j=i+1}^n a_{ij}x_j^{(k)}$  ▷ Valores antigos
6:      $x_i^{(k+1)} \leftarrow \frac{1}{a_{ii}}(b_i - soma1 - soma2)$ 
7:   end para
8:    $k \leftarrow k + 1$ 
9: até  $\|x^{(k)} - x^{(k-1)}\| < \epsilon$ 
10: retorne  $x^{(k)}$ 

```

Nota Importante

Diferença crucial com Jacobi:

- **Jacobi:** Todos os componentes de $x^{(k+1)}$ calculados utilizando apenas $x^{(k)} \rightarrow$ paralelizável.
- **Gauss-Seidel:** Componentes calculados sequencialmente, utilizando valores já atualizados $\rightarrow$ não paralelizável diretamente.
- **Vantagem de GS:** Geralmente converge 2 vezes mais rápido que Jacobi.

Exemplo 4.4 (Gauss-Seidel — mesmo sistema, comparação direta com Jacobi). Usamos o sistema (4.16) do Exemplo 4.2. As fórmulas diferem apenas no uso dos valores já calculados (indicados em **negrito**):

$$x_1^{(k+1)} = \frac{1}{8}(14 + x_2^{(k)} - x_3^{(k)}) \quad (4.30)$$

$$x_2^{(k+1)} = \frac{1}{6}(10 - x_1^{(k+1)} + 2x_3^{(k)}) \quad (4.31)$$

$$x_3^{(k+1)} = \frac{1}{7}(-5 + x_1^{(k+1)} - 2x_2^{(k+1)}) \quad (4.32)$$

Iteração $k = 1$ (partindo de $x^{(0)} = (0; 0; 0)^T$):

$$x_1^{(1)} = \frac{1}{8}(14 + 0 - 0) = \mathbf{1,7500}$$

$$x_2^{(1)} = \frac{1}{6}(10 - \mathbf{1,7500} + 0) = \frac{8,250}{6} = \mathbf{1,3750} \quad (\text{usou } x_1^{(1)})$$

$$x_3^{(1)} = \frac{1}{7}(-5 + \mathbf{1,7500} - 2 \cdot \mathbf{1,3750}) = \frac{-5,000}{7} = \mathbf{-0,7143} \quad (\text{usou } x_1^{(1)} \text{ e } x_2^{(1)})$$

Iteração $k = 2$:

$$x_1^{(2)} = \frac{1}{8}(14 + 1,3750 + 0,7143) = \frac{16,089}{8} = \mathbf{2,0112}$$

$$x_2^{(2)} = \frac{1}{6}(10 - \mathbf{2,0112} - 1,4286) = \frac{6,560}{6} = \mathbf{1,0934}$$

$$x_3^{(2)} = \frac{1}{7}(-5 + \mathbf{2,0112} - 2 \cdot \mathbf{1,0934}) = \frac{-5,176}{7} = \mathbf{-0,7394}$$

Tabela comparativa ($\varepsilon = 0,05$):

Gauss-Seidel					Gauss-Jacobi				
k	x_1	x_2	x_3	$d_r^{(k)}$	k	x_1	x_2	x_3	$d_r^{(k)}$
0	0,000	0,000	0,000	—	0	0,000	0,000	0,000	—
1	1,750	1,375	-0,714	1,000	1	1,750	1,667	-0,714	1,000
2	2,011	1,093	-0,739	0,130	2	2,048	1,137	-0,941	0,259
3	2,006	1,007	-0,973	0,043	3	1,975	1,057	-0,968	0,039

Gauss-Seidel converge em **3 iterações** com solução mais próxima de $x^* = (2; 1; -1)^T$.

Exemplo 4.5 (Gauss-Seidel - Mesmo sistema). Usando o mesmo sistema do exemplo anterior:

Iteração 1 (partindo de $x^{(0)} = (0, 0, 0, 0)^T$):

$$x_1^{(1)} = \frac{1}{10}(6 + 0 - 0) = 0.6000$$

$$x_2^{(1)} = \frac{1}{11}(25 + 0.6 + 0 - 0) = 2.3273 \quad (\text{usa } x_1^{(1)})$$

$$x_3^{(1)} = \frac{1}{10}(-11 - 1.2 + 2.3273 + 0) = -0.9873$$

$$x_4^{(1)} = \frac{1}{8}(15 - 6.9818 - 0.9873) = 0.8789$$

Compare com Jacobi após 1 iteração: $(0.6, 2.2727, -1.1, 1.875)^T$. Gauss-Seidel já está mais próximo da solução verdadeira $(1, 2, -1, 1)^T$!

Iteração 2:

$$\begin{aligned}x_1^{(2)} &= \frac{1}{10}(6 + 2.3273 + 1.9745) = 1.0302 \\x_2^{(2)} &= \frac{1}{11}(25 + 1.0302 - 0.9873 - 2.6366) = 2.0370 \\x_3^{(2)} &= \frac{1}{10}(-11 - 2.0604 + 2.0370 + 0.8789) = -1.0144 \\x_4^{(2)} &= \frac{1}{8}(15 - 6.1109 - 1.0144) = 0.9843\end{aligned}$$

Gauss-Seidel converge muito mais rápido!

4.4.3 Implementação

```

1 def gauss_seidel(A, b, x0=None, tol=1e-10, max_iter=1000):
2     """
3     Resolve Ax = b usando m todo de Gauss-Seidel.
4
5     Nota: N o pode ser vetorizado facilmente devido a
6         atualiza es in-place.
7     """
8     A = np.array(A, dtype=float)
9     b = np.array(b, dtype=float)
10    n = len(b)
11
12    x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
13
14    historico = []
15
16    for k in range(max_iter):
17        x_old = x.copy()
18
19        # Itera o de Gauss-Seidel (in-place)
20        for i in range(n):
21            # soma1: usa valores j atualizados x[j] para j < i
22            # soma2: usa valores antigos x_old[j] para j > i
23            soma = np.dot(A[i, :i], x[:i]) + np.dot(A[i, i+1:],
24                x_old[i+1:])
25            x[i] = (b[i] - soma) / A[i, i]
26
27        # Res duo e crit rio de parada
28        residuo = np.linalg.norm(A @ x - b)

```

```

27     historico.append(residuo)
28
29     if np.linalg.norm(x - x_old) < tol:
30         return x, k+1, historico
31
32     return x, max_iter, historico
33
34 # Compara o Jacobi vs Gauss-Seidel
35 print("=" * 60)
36 print("Compara o: Jacobi vs Gauss-Seidel")
37 print("=" * 60)
38
39 A = np.array([[10, -1, 2, 0],
40              [-1, 11, -1, 3],
41              [2, -1, 10, -1],
42              [0, 3, -1, 8]], dtype=float)
43 b = np.array([6, 25, -11, 15], dtype=float)
44
45 x_j, iter_j, hist_j = jacobi_vetorizado(A, b)
46 x_gs, iter_gs, hist_gs = gauss_seidel(A, b)
47
48 print(f"Jacobi:          {iter_j} itera es")
49 print(f"Gauss-Seidel: {iter_gs} itera es")
50 print(f"Speedup:          {iter_j / iter_gs:.2f}x")
51 print(f"\nSolu o (GS): {x_gs}")
52 print(f"Erro final:      {hist_gs[-1]:.2e}")

```

Listing 4.2: Método de Gauss-Seidel

Exercícios — Método de Gauss-Seidel

Exercício 4.12. Realize manualmente a iteração $k = 1$ de Gauss-Seidel para o sistema abaixo, partindo de $x^{(0)} = (0, 0, 0, 0)^T$. Indique claramente quais valores são “já atualizados” e quais são “da iteração anterior”.

$$\begin{cases} 7x_1 - x_2 + x_3 - x_4 = 12 \\ -x_1 + 8x_2 + x_3 + x_4 = 9 \\ x_1 + x_2 + 9x_3 - 2x_4 = 11 \\ -x_1 + x_2 - 2x_3 + 7x_4 = -3 \end{cases}$$

Exercício 4.13. Continue o Exemplo 4.4: realize a iteração $k = 3$ manualmente, partindo de $x^{(2)} = (2,011; 1,093; -0,739)^T$, e calcule $d_r^{(3)}$.

Exercício 4.14. Aplique Gauss-Seidel ao sistema do Exercício 3.3 (Gauss-Jacobi, sistema 4×4) com $\varepsilon = 10^{-4}$. Compare o número de iterações obtido pelos dois métodos.

Exercício 4.15 (Análise). Para a matriz $A = \begin{bmatrix} 4 & -1 \\ -1 & 4 \end{bmatrix}$, calcule $T_J = D^{-1}(L + U)$ e $T_{GS} = (D - L)^{-1}U$. Encontre os autovalores de ambas e compare $\rho(T_J)$ e $\rho(T_{GS})$. Qual método converge mais rápido?

Exercício 4.16 (Projeto). Implemente Gauss-Jacobi e Gauss-Seidel em Python e aplique à Equação de Poisson 1D com $n = 20, 50, 100$ pontos:

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i, \quad h = \frac{1}{n+1}, \quad f_i = \pi^2 \sin(\pi x_i),$$

com $u_0 = u_{n+1} = 0$. Solução exata: $u(x) = \sin(\pi x)$. Registre e compare o número de iterações em função de n .

4.5 Análise de Convergência

4.5.1 Teorema Fundamental

Teorema 4.1 (Convergência de Métodos Iterativos). Considere a iteração $x^{(k+1)} = Tx^{(k)} + c$ em que $T \in \mathbb{R}^{n \times n}$ e $c \in \mathbb{R}^n$. A sequência $\{x^{(k)}\}_{k=0}^{\infty}$ converge para a solução única $x^* = Tx^* + c$ para *qualquer* aproximação inicial $x^{(0)}$ se, e somente se:

$$\rho(T) < 1, \tag{4.33}$$

em que $\rho(T) = \max_i |\lambda_i(T)|$ é o *raio espectral* de T (maior autovalor em módulo).

Demonstração. Seja $e^{(k)} = x^{(k)} - x^*$ o erro na iteração k .

Passo 1: Evolução do erro.

$$\begin{aligned} \text{Da iteração:} \quad e^{(k+1)} &= x^{(k+1)} - x^* \\ &= (Tx^{(k)} + c) - (Tx^* + c) \\ &= T(x^{(k)} - x^*) \\ &= Te^{(k)}. \end{aligned}$$

Por indução:

$$e^{(k)} = T^k e^{(0)}. \tag{4.34}$$

Passo 2: ($\Rightarrow$) Convergência implica $\rho(T) < 1$.

Se a sequência converge para qualquer $x^{(0)}$, então:

$$\lim_{k \rightarrow \infty} e^{(k)} = \lim_{k \rightarrow \infty} T^k e^{(0)} = 0, \tag{4.35}$$

para todo $e^{(0)}$. Isso implica $\lim_{k \rightarrow \infty} T^k = 0$ (convergência para matriz nula).

Lema (Álgebra Linear): $\lim_{k \rightarrow \infty} T^k = 0$ se e somente se $\rho(T) < 1$.

Demonstração do Lema:

Seja λ autovalor de T com maior módulo: $|\lambda| = \rho(T)$. Seja v autovetor correspondente: $Tv = \lambda v$. Então:

$$T^k v = \lambda^k v. \quad (4.36)$$

Se $|\lambda| \geq 1$, então $|\lambda|^k \not\rightarrow 0$, logo $T^k v \not\rightarrow 0$, contradizendo $T^k \rightarrow 0$.

Portanto, $\rho(T) < 1$ é necessário.

Reciprocamente, se $\rho(T) < 1$, existe norma matricial tal que $\|T\| < 1$ (Teorema de Gelfand). Logo:

$$\|T^k\| \leq \|T\|^k \rightarrow 0. \quad (4.37)$$

Passo 3: ($\Leftarrow$) $\rho(T) < 1$ implica convergência.

Se $\rho(T) < 1$, então $\|T^k\| \rightarrow 0$. Logo:

$$\|e^{(k)}\| = \|T^k e^{(0)}\| \leq \|T^k\| \|e^{(0)}\| \rightarrow 0. \quad (4.38)$$

□

Corolário 4.2 (Taxa de Convergência). Se $\rho(T) < 1$, a convergência é pelo menos geométrica:

$$\|e^{(k)}\| \leq \|T\|^k \|e^{(0)}\|. \quad (4.39)$$

Assintoticamente:

$$\lim_{k \rightarrow \infty} \|e^{(k)}\|^{1/k} = \rho(T). \quad (4.40)$$

Quanto menor $\rho(T)$, mais rápida a convergência.

4.5.2 Critérios Práticos de Convergência

Teorema 4.3 (Convergência para Matrizes Diagonal Dominante). Se A é *estritamente diagonal dominante por linhas*, isto é:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|, \quad \forall i = 1, \dots, n, \quad (4.41)$$

então os métodos de Jacobi e Gauss-Seidel convergem para qualquer aproximação inicial.

Demonstração para Jacobi. Precisamos mostrar que $\rho(T_J) < 1$ em que $T_J = D^{-1}(L + U)$.

Seja λ um autovalor de T_J com autovetor $v \neq 0$. Seja v_p a componente de v com maior magnitude:

$$|v_p| = \max_{1 \leq i \leq n} |v_i|. \quad (4.42)$$

De $T_J v = \lambda v$:

$$D^{-1}(L + U)v = \lambda v \quad \Rightarrow \quad (L + U)v = \lambda Dv. \quad (4.43)$$

Olhando para a componente p :

$$\sum_{\substack{j=1 \\ j \neq p}}^n a_{pj}v_j = \lambda a_{pp}v_p. \quad (4.44)$$

Tomando módulo e usando $|v_j| \leq |v_p|$:

$$|\lambda||a_{pp}||v_p| = \left| \sum_{j \neq p} a_{pj}v_j \right| \leq \sum_{j \neq p} |a_{pj}||v_j| \leq |v_p| \sum_{j \neq p} |a_{pj}|. \quad (4.45)$$

Dividindo por $|a_{pp}||v_p|$ (ambos não-nulos):

$$|\lambda| \leq \frac{\sum_{j \neq p} |a_{pj}|}{|a_{pp}|} < 1. \quad (4.46)$$

A última desigualdade segue da dominância diagonal estrita. Como λ é arbitrário, $\rho(T_J) = \max_i |\lambda_i| < 1$. $\square$

Esboço para Gauss-Seidel. A demonstração para GS é mais técnica. Utiliza-se o fato de que se A é diagonal dominante, então $(D - L)$ é não-singular e pode-se mostrar diretamente que $\rho(T_{GS}) < 1$ analisando a estrutura de $(D - L)^{-1}U$.

Alternativamente, utiliza-se teoria de matrizes M : se A é diagonal dominante, então A é uma M -matriz, e para M -matrizes, tanto Jacobi quanto GS convergem. $\square$

Exemplo 4.6 (Verificando convergência). Para a matriz:

$$A = \begin{bmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix} \quad (4.47)$$

Verificamos diagonal dominância:

- Linha 1: $|4| = 4 > |-1| + |0| = 1 \checkmark$
- Linha 2: $|4| = 4 > |-1| + |-1| = 2 \checkmark$
- Linha 3: $|4| = 4 > |0| + |-1| = 1 \checkmark$

Logo, tanto Jacobi quanto Gauss-Seidel convergem!

Cálculo numérico do raio espectral:

```

1 D = np.diag(np.diag(A))
2 R = A - D
3 T_J = np.linalg.inv(D) @ R
4
5 eigenvalues = np.linalg.eigvals(T_J)
6 rho_J = np.max(np.abs(eigenvalues))
7 print(f"Raio espectral (Jacobi): {rho_J:.6f}") # 0.5

```

Como $\rho(T_J) = 0.5 < 1$, Jacobi converge.

Teorema 4.4 (Convergência para Matrizes Simétricas Positivas Definidas). Se A é simétrica positiva definida (SPD), então:

1. Gauss-Seidel converge para qualquer $x^{(0)}$.
2. Jacobi converge se e somente se $2D - A$ é positiva definida.

Esboço da demonstração. **Parte (1):** Para Gauss-Seidel com A SPD, define-se a função energia:

$$E(x) = \frac{1}{2}x^T Ax - b^T x. \quad (4.48)$$

Pode-se mostrar que cada iteração de GS *diminui* $E(x^{(k)})$ monotonicamente:

$$E(x^{(k+1)}) < E(x^{(k)}). \quad (4.49)$$

Como E é limitada inferiormente e monótona decrescente, converge. O limite é o mínimo de E , que ocorre em $x^* = A^{-1}b$.

Parte (2): Para Jacobi, a condição $2D - A$ SPD é equivalente a $\rho(T_J) < 1$. A demonstração usa propriedades espectrais de matrizes SPD.

Detalhes completos em Golub & Van Loan (2013), seção 10.1. □

Teorema 4.5 (Critério de Sassenfeld — Gauss-Seidel). Defina os *coeficientes de Sassenfeld* recursivamente:

$$\beta_1 = \frac{\sum_{j>1} |a_{1j}|}{|a_{11}|}, \quad \beta_i = \frac{\sum_{s<i} |a_{is}| \beta_s + \sum_{s>i} |a_{is}|}{|a_{ii}|}, \quad i = 2, \dots, n. \quad (4.50)$$

Se $\beta = \max_i \beta_i < 1$, então Gauss-Seidel converge. O critério de Sassenfeld é geralmente mais preciso que o das linhas para GS.

Exemplo 4.7 (Verificando convergência pelos dois critérios). Para a matriz do sistema (4.16):

$$A = \begin{bmatrix} 8 & -1 & 1 \\ 1 & 6 & -2 \\ -1 & 2 & 7 \end{bmatrix}.$$

Critério das linhas: $\alpha_1 = \frac{2}{8} = 0,250$, $\alpha_2 = \frac{3}{6} = 0,500$, $\alpha_3 = \frac{3}{7} \approx 0,429$. $\alpha = 0,500 < 1$. ✓

Critério de Sassenfeld:

$$\begin{aligned} \beta_1 &= \frac{1+1}{8} = 0,250 \\ \beta_2 &= \frac{|1| \cdot \beta_1 + |-2|}{6} = \frac{0,250 + 2}{6} = \frac{2,250}{6} = 0,375 \\ \beta_3 &= \frac{|-1| \cdot \beta_1 + |2| \cdot \beta_2}{7} = \frac{0,250 + 0,750}{7} = \frac{1,000}{7} \approx 0,143 \end{aligned}$$

$\beta = 0,375 < 1$. ✓

Sassenfeld dá $\beta = 0,375 < \alpha = 0,500$, fornecendo uma estimativa mais conservadora para a taxa de convergência de GS.

Teorema 4.6 (Convergência para Matrizes SPD). Se A é simétrica positiva definida (SPD): (1) Gauss-Seidel converge para qualquer $x^{(0)}$; (2) Gauss-Jacobi converge se e somente se $2D - A$ é positiva definida.

Exemplo 4.8 (Raio espectral e taxa de convergência). Para a matriz tridiagonal:

$$A = \begin{bmatrix} 4 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 \\ 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 4 \end{bmatrix}$$

calculamos numericamente $\rho(T_J) \approx 0,6194$ e $\rho(T_{GS}) \approx 0,3836 \approx \rho(T_J)^2$.

Método	ρ	Iterações para $\ e\ /\ e^{(0)}\ < 10^{-6}$
Gauss-Jacobi	0,619	~ 33
Gauss-Seidel	0,384	~ 17

A relação $\rho(T_{GS}) \approx \rho(T_J)^2$ para matrizes com “Property A” explica o ganho de $\approx 2\times$ em iterações.

Exercícios — Análise de Convergência

Exercício 4.17. Calcule $\rho(T_J)$ e $\rho(T_{GS})$ numericamente para a matriz do Exercício 3.3 (sistema 4×4). Confirme que $\rho(T_{GS}) \leq \rho(T_J)$.

Exercício 4.18. Aplique o critério de Sassenfeld ao sistema do Exercício 4.1. Compare $\beta = \max_i \beta_i$ com $\alpha = \max_i \alpha_i$ e com $\rho(T_{GS})$ calculado numericamente. Qual estimativa é mais precisa?

Exercício 4.19. Para $\rho(T) \in \{0,9; 0,7; 0,5; 0,3; 0,1\}$, calcule quantas iterações são necessárias para reduzir o erro por um fator 10^{-6} . Plote “iteraões vs. $\rho(T)$ ”.

Exercício 4.20 (Desafio). Construa uma matriz 3×3 para a qual Gauss-Jacobi diverge ($\rho(T_J) > 1$) e Gauss-Seidel converge ($\rho(T_{GS}) < 1$). Verifique calculando os raios espectrais numericamente.

Exercício 4.21 (Demonstração). Prove que toda matriz estritamente diagonal dominante e simétrica é positiva definida. *Dica:* Use o critério de Gershgorin para localizar os autovalores de A .

4.6 Método SOR (*Successive Over-Relaxation*)

4.6.1 Motivação e Derivação

Gauss-Seidel calcula

$$x_i^{GS} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right). \quad (4.51)$$

SOR toma combinação convexa entre $x_i^{(k)}$ e x_i^{GS} :

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \omega x_i^{GS}. \quad (4.52)$$

Expandindo:

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right). \quad (4.53)$$

Parâmetro ω :

- $\omega = 1$: recupera Gauss-Seidel.
- $0 < \omega < 1$: *under-relaxation* (mais estável, convergência mais lenta).
- $1 < \omega < 2$: *over-relaxation* (convergência mais rápida quando ω bem escolhido).
- $\omega \geq 2$ ou $\omega \leq 0$: geralmente diverge.

Matriz de iteração:

$$T_{SOR} = (D - \omega L)^{-1} [(1 - \omega)D + \omega U]. \quad (4.54)$$

Teorema 4.7 (Condição Necessária para Convergência de SOR). Para que o método SOR convirja, é necessário que:

$$0 < \omega < 2. \quad (4.55)$$

Demonstração. Seja λ autovalor de T_{SOR} . Pode-se mostrar (através do polinômio característico) que:

$$\det(T_{SOR}) = (1 - \omega)^n. \quad (4.56)$$

Como o determinante é o produto dos autovalores:

$$\prod_{i=1}^n \lambda_i = (1 - \omega)^n. \quad (4.57)$$

Para convergência, precisamos $|\lambda_i| < 1$ para todo i . Em particular:

$$\left| \prod_{i=1}^n \lambda_i \right| < 1 \quad \Rightarrow \quad |1 - \omega|^n < 1 \quad \Rightarrow \quad |1 - \omega| < 1. \quad (4.58)$$

Logo:

$$-1 < 1 - \omega < 1 \quad \Rightarrow \quad 0 < \omega < 2. \quad (4.59)$$

□

4.6.2 ω Ótimo

Teorema 4.8 (Young, 1950 - ω Ótimo). Para certas classes de matrizes (incluindo matrizes tridiagonais com “property A”), existe ω_{opt} que minimiza $\rho(T_{SOR})$:

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}}, \quad (4.60)$$

em que $\rho(T_J)$ é o raio espectral da matriz de iteração de Jacobi.

Com ω_{opt} :

$$\rho(T_{SOR}) = \omega_{opt} - 1 < \rho(T_{GS}) < \rho(T_J). \quad (4.61)$$

Nota Importante

Para matrizes tridiagonais (comuns em EDPs 1D), SOR com ω_{opt} pode reduzir o número de iterações significativamente. Por exemplo, se $\rho(T_J) = 0.9$:

$$\begin{aligned} \omega_{opt} &= \frac{2}{1 + \sqrt{1 - 0.81}} \approx 1.72 \\ \rho(T_{SOR}) &\approx 0.72 \quad \text{vs} \quad \rho(T_J) = 0.9. \end{aligned}$$

Menos iterações para mesma precisão!

Exemplo 4.9 (SOR com $\omega = 1,15$). Mesmo sistema (4.16), $x^{(0)} = (0; 0; 0)^T$, $\omega = 1,15$.

Para cada componente i : (1) calcular x_i^{GS} como em Gauss-Seidel; (2) aplicar $x_i^{(k+1)} = 0,15 x_i^{(k)} + 1,15 x_i^{GS}$.

Iteração $k = 1$:

$$\begin{aligned} x_1^{GS} &= \frac{1}{8}(14) = 1,7500, & x_1^{(1)} &= 0,15(0) + 1,15(1,7500) = \mathbf{2,0125} \\ x_2^{GS} &= \frac{1}{6}(10 - \mathbf{2,0125}) = 1,3312, & x_2^{(1)} &= 0,15(0) + 1,15(1,3312) = \mathbf{1,5309} \\ x_3^{GS} &= \frac{1}{7}(-5 + \mathbf{2,0125} - 2 \cdot \mathbf{1,5309}) = -0,8642, & x_3^{(1)} &= 0,15(0) + 1,15(-0,8642) = \mathbf{-0,9938} \end{aligned}$$

Já na primeira iteração: $x^{(1)} = (2,013; 1,531; -0,994)^T$ está mais próximo de $x^* = (2; 1; -1)^T$ do que Gauss-Seidel ($x_{GS}^{(1)} = (1,750; 1,375; -0,714)^T$).

k	x_1	x_2	x_3	$d_r^{(k)}$
0	0,000	0,000	0,000	—
1	2,013	1,531	-0,994	1,000
2	1,989	0,988	-0,999	0,027

SOR converge em **2 iterações** (vs. 3 de GS e Jacobi). ✓

Aplicação Computacional

Equações Diferenciais Parciais - Equação de Poisson 2D:

A equação de Poisson:

$$\nabla^2 u = f, \quad (4.62)$$

em domínio retangular com malha h leva a sistema linear com matriz tridiagonal por blocos.

Para malha $n \times n$ (gerando sistema $(n-1)^2 \times (n-1)^2$):

- **Jacobi:** $\mathcal{O}(n^2)$ iterações.
- **Gauss-Seidel:** $\mathcal{O}(n^2)$ iterações.
- **SOR ótimo:** $\mathcal{O}(n)$ iterações!

Para $n = 100$: SOR é 100x mais rápido!

Aplicações práticas:

- Simulação de transferência de calor.
- Eletrostática.
- Iluminação global em computação gráfica.
- Inpainting de imagens.

Referência: LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM. <https://faculty.washington.edu/rjl/fdmbook/>

```

1  def sor(A, b, omega, x0=None, tol=1e-10, max_iter=1000):
2      """
3      Resolve  $Ax = b$  usando o todo SOR.
4
5      Parâmetros:
6      -----
7      omega : float
8          Parâmetro de relaxação ( $0 < \omega < 2$ )
9      """
10     if omega <= 0 or omega >= 2:
11         raise ValueError("omega deve estar em (0, 2)")
12
13     A = np.array(A, dtype=float)
14     b = np.array(b, dtype=float)
15     n = len(b)
16
17     x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
18
19     historico = []
20
21     for k in range(max_iter):
22         x_old = x.copy()
23
24         for i in range(n):
25             # Calcular valor de Gauss-Seidel
26             soma = np.dot(A[i, :i], x[:i]) + np.dot(A[i, i+1:],
27                 x_old[i+1:])
28             x_gs = (b[i] - soma) / A[i, i]
29
30             # SOR: combina o convexa
31             x[i] = (1 - omega) * x_old[i] + omega * x_gs
32
33         residuo = np.linalg.norm(A @ x - b)
34         historico.append(residuo)
35
36         if np.linalg.norm(x - x_old) < tol:
37             return x, k+1, historico
38
39     return x, max_iter, historico

```

```

39
40 # Busca do omega timo (for a bruta)
41 def encontrar_omega_otimo(A, b, omega_min=1.0, omega_max=2.0,
42     n_pontos=20):
43     """Busca omega que minimiza n mero de itera es"""
44     omegas = np.linspace(omega_min, omega_max, n_pontos)
45     iteracoes = []
46
47     for omega in omegas:
48         try:
49             _, iters, _ = sor(A, b, omega, tol=1e-8,
50                 max_iter=1000)
51             iteracoes.append(iters)
52         except:
53             iteracoes.append(1000)
54
55     idx_min = np.argmin(iteracoes)
56     return omegas[idx_min], iteracoes[idx_min]
57
58 # Exemplo com matriz tridiagonal
59 n = 20
60 A = 2*np.eye(n) - np.eye(n, k=1) - np.eye(n, k=-1)
61 b = np.ones(n)
62
63 omega_opt, iter_opt = encontrar_omega_otimo(A, b)
64 print(f"Omega timo (experimental): {omega_opt:.4f}")
65 print(f"Itera es com omega timo : {iter_opt}")
66
67 # Compara o
68 _, iter_j, _ = jacobi_vetorizado(A, b, tol=1e-8)
69 _, iter_gs, _ = gauss_seidel(A, b, tol=1e-8)
70 _, iter_sor, _ = sor(A, b, omega_opt, tol=1e-8)
71
72 print(f"\nCompara o:")
73 print(f"Jacobi: {iter_j} itera es")
74 print(f"Gauss-Seidel: {iter_gs} itera es")
75 print(f"SOR ( timo ): {iter_sor} itera es")

```

Listing 4.3: Método SOR

Exercícios — Método SOR

Exercício 4.22. Para a matriz A_5 do Exemplo ??:

- (a) Confirme $\rho(T_J) \approx 0,6910$ calculando numericamente os autovalores.
- (b) Calcule ω_{opt} e $\rho(T_{SOR})$ pela fórmula de Young.
- (c) Resolva com Gauss-Seidel e $SOR(\omega_{opt})$ para $\varepsilon = 10^{-6}$ e compare iterações.

Exercício 4.23. Aplique SOR com $\omega \in \{0,5; 0,8; 1,0; 1,2; 1,4; 1,6; 1,9\}$ ao sistema do Exercício 3.3. Para cada ω : registre o número de iterações, plote “iteraões vs. ω ” e identifique graficamente o ω ótimo experimental.

Exercício 4.24. Realize a iteração $k = 2$ do Exemplo 4.9 manualmente e confirme os valores da tabela. Em seguida, execute com $\omega = 1,9$ e descreva o comportamento (oscilações, divergência?).

Exercício 4.25 (Projeto). Resolva a Equação de Poisson 1D do Exercício 4.5 com $SOR(\omega_{opt})$. Compare o número de iterações de Jacobi, GS e SOR ótimo para $n = 20, 50, 100$. Verifique se a proporção $iter_{GS}/iter_{SOR}$ cresce com n .

4.7 Método do Gradiente Conjugado

4.7.1 Motivação: Minimização de Função Energia

Para matrizes A SPD, resolver $Ax = b$ equivale a minimizar $f(x) = \frac{1}{2}x^T Ax - b^T x$, cujo mínimo é $x^* = A^{-1}b$.

O *gradiente descendente simples* move-se na direção do resíduo $r^{(k)} = b - Ax^{(k)}$, mas “ziguezagueia” e converge lentamente para sistemas mal-condicionados. O **Gradiente Conjugado (CG)** corrige isso usando direções *A-conjugadas*:

$$\langle p^{(i)}, Ap^{(j)} \rangle = 0 \quad \text{para } i \neq j, \quad (4.63)$$

garantindo que nenhuma iteração “desfaça” o progresso das anteriores.

Algoritmo 10 Gradiente Conjugado (para A SPD)**Entrada:** A SPD, b , $x^{(0)}$, ε 1: $r^{(0)} \leftarrow b - Ax^{(0)}$; $p^{(0)} \leftarrow r^{(0)}$; $k \leftarrow 0$ 2: **repita**3: $\alpha_k \leftarrow \frac{\langle r^{(k)}, r^{(k)} \rangle}{\langle p^{(k)}, Ap^{(k)} \rangle}$ ▷ passo ótimo4: $x^{(k+1)} \leftarrow x^{(k)} + \alpha_k p^{(k)}$ 5: $r^{(k+1)} \leftarrow r^{(k)} - \alpha_k Ap^{(k)}$ 6: $\beta_k \leftarrow \frac{\langle r^{(k+1)}, r^{(k+1)} \rangle}{\langle r^{(k)}, r^{(k)} \rangle}$ ▷ atualiza direção7: $p^{(k+1)} \leftarrow r^{(k+1)} + \beta_k p^{(k)}$; $k \leftarrow k + 1$ 8: **até** $\|r^{(k)}\| < \varepsilon$ 9: **retorne** $x^{(k)}$ **4.7.2 Algoritmo****Nota Importante**

Propriedades fundamentais: converge em **no máximo** n passos (aritmética exata); custo por iteração $\mathcal{O}(\text{nnz})$; taxa $\|e^{(k)}\|_A / \|e^{(0)}\|_A \leq 2\left((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)\right)^k$ com $\kappa = \kappa(A)$.

4.7.3 Exemplo Resolvido Passo a Passo*Exemplo 4.10* (Gradiente Conjugado para sistema 3×3).

$$A = \begin{bmatrix} 5 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 11 \\ 9 \\ 4 \end{bmatrix}, \quad x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Solução exata: $x^* = (2; 1; 1)^T$. Verifique: $5(2) + 1(1) = 11$ ✓; $1(2) + 4(1) + 1(1) = 7 \neq 9 \dots$

Determinamos a solução exata a posteriori via Cholesky (para fins de verificação):
 $x^* = (2; 1; 1)^T$ requer $b = Ax^* = (10 + 1, 2 + 4 + 1, 1 + 3)^T = (11, 7, 4)^T$. Ajustamos:
 $b = (11; 7; 4)^T$, solução $x^* = (2; 1; 1)^T$. ✓

Verificação SPD: $\det_1 = 5 > 0$; $\det_2 = 20 - 1 = 19 > 0$; $\det_3 = 5(12 - 1) - 1(3) = 52 > 0$.
 ✓

Inicialização:

$$r^{(0)} = b - Ax^{(0)} = \begin{pmatrix} 11 \\ 7 \\ 4 \end{pmatrix}, \quad p^{(0)} = r^{(0)}.$$

Iteração $k = 1$:

$$Ap^{(0)} = \begin{pmatrix} 5(11) + 1(7) \\ 1(11) + 4(7) + 1(4) \\ 1(7) + 3(4) \end{pmatrix} = \begin{pmatrix} 62 \\ 47 \\ 19 \end{pmatrix}$$

$$\alpha_0 = \frac{11^2 + 7^2 + 4^2}{11(62) + 7(47) + 4(19)} = \frac{186}{682 + 329 + 76} = \frac{186}{1087} \approx 0,1711$$

$$x^{(1)} = 0,1711 \begin{pmatrix} 11 \\ 7 \\ 4 \end{pmatrix} = \begin{pmatrix} 1,882 \\ 1,198 \\ 0,684 \end{pmatrix}$$

$$r^{(1)} = \begin{pmatrix} 11 \\ 7 \\ 4 \end{pmatrix} - 0,1711 \begin{pmatrix} 62 \\ 47 \\ 19 \end{pmatrix} = \begin{pmatrix} 0,382 \\ -1,042 \\ 0,750 \end{pmatrix}$$

$\|r^{(1)}\|_2 \approx 1,311$ (de $\|r^{(0)}\|_2 \approx 13,6$; redução de $\approx 90\%$ numa iteração!).

Iteração $k = 2$:

$$\beta_0 = \frac{0,382^2 + 1,042^2 + 0,750^2}{186} = \frac{1,719}{186} \approx 0,00924$$

$$p^{(1)} = \begin{pmatrix} 0,382 \\ -1,042 \\ 0,750 \end{pmatrix} + 0,00924 \begin{pmatrix} 11 \\ 7 \\ 4 \end{pmatrix} = \begin{pmatrix} 0,484 \\ -0,977 \\ 0,787 \end{pmatrix}$$

Continuando a iteração (cálculo numérico), chega-se a $x^{(3)} \approx x^* = (2; 1; 1)^T$ com resíduo $< 10^{-14}$. Para A de ordem 3 SPD, o CG converge em no máximo **3 iterações**. ✓

4.7.4 Implementação

```

1 import numpy as np
2 import scipy.sparse as sp
3 import scipy.sparse.linalg as spla
4
5 def cg_basico(A, b, x0=None, tol=1e-8, max_iter=1000):
6     """CG did tico para A SPD."""
7     A = np.array(A, dtype=float); b = np.array(b, dtype=float)
8     n = len(b)
9     x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
10    r = b - A @ x; p = r.copy(); rr = np.dot(r, r)
11    for k in range(max_iter):

```

```

12     Ap      = A @ p
13     alpha   = rr / np.dot(p, Ap)
14     x       = x + alpha * p
15     r       = r - alpha * Ap
16     rr_new  = np.dot(r, r)
17     if np.sqrt(rr_new) < tol:
18         return x, k + 1
19     p       = r + (rr_new / rr) * p
20     rr      = rr_new
21     return x, max_iter
22
23 # PCG com SciPy (recomendado na prática)
24 n = 50; N = n * n
25 A_sp = sp.diags([-1,4,-1,-1,-1], [-1,0,1,-n,n], shape=(N,N),
26                 format='csr')
27 b_sp = np.ones(N)
28 ilu   = spla.spilu(A_sp)
29 M     = spla.LinearOperator(A_sp.shape, lambda v: ilu.solve(v))
30 x_cg, info = spla.cg(A_sp, b_sp, M=M, tol=1e-10)
31 print(f"Converge: {info==0} | Res duo: {np.linalg.norm(A_sp @
32         x_cg - b_sp):.2e}")

```

Listing 4.4: Gradiente Conjugado e PCG com SciPy

Exercícios — Gradiente Conjugado

Exercício 4.26. Confirme que a matriz do Exemplo 4.10 é SPD e aplique CG numericamente (tolerância 10^{-10}) para obter a solução exata. Verifique que o método converge em no máximo 3 iterações.

Exercício 4.27. Aplique CG manualmente ao sistema 2×2 :

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 4 \end{bmatrix}.$$

Calcule explicitamente $\alpha_0, x^{(1)}, r^{(1)}, \beta_0, p^{(1)}, \alpha_1$ e $x^{(2)}$. Verifique que $x^{(2)} = x^*$ (convergência exata em 2 iterações).

Exercício 4.28. Para a Matriz de Hilbert H_n ($h_{ij} = 1/(i+j-1)$):

- (a) Calcule $\kappa_2(H_n)$ para $n = 5, 10, 15$.

(b) Resolva $H_n x = e_1$ com CG (tolerância 10^{-8}) e registre o número de iterações para cada n .

(c) Explique por que o CG converge lentamente para H_n .

Exercício 4.29 (Demonstração). Prove que os resíduos $r^{(0)}, r^{(1)}, \dots, r^{(k)}$ do algoritmo CG são mutuamente ortogonais: $\langle r^{(i)}, r^{(j)} \rangle = 0$ para $i \neq j$. *Dica:* Use indução e as relações de recorrência do algoritmo.

4.8 Pré-Condicionamento

4.8.1 Motivação

Se $\kappa(A) \gg 1$, a estimativa do CG prevê muitas iterações. A ideia é transformar o sistema:

$$Ax = b \longrightarrow M^{-1}Ax = M^{-1}b,$$

onde $M \approx A$ é fácil de inverter e $\kappa(M^{-1}A) \ll \kappa(A)$.

4.8.2 Pré-condicionadores Clássicos

1. Jacobi (diagonal): $M = D$. Custo $\mathcal{O}(n)$; eficaz quando a diagonal domina.

2. SSOR (Symmetric SOR): $M = (D - \omega L)D^{-1}(D - \omega U)$. Simétrico, compatível com CG.

3. ILU (Fatoração LU Incompleta): $M = \tilde{L}\tilde{U}$ mantendo os zeros de A . Muito eficaz na prática.

4. Multigrid: Para EDPs, $\kappa(M^{-1}A) = \mathcal{O}(1)$, custo $\mathcal{O}(n)$. Tópico avançado.

4.8.3 Gradiente Conjugado Pré-Condicionado (PCG)

PCG converge em função de $\kappa(M^{-1}A)$: um bom M acelera drasticamente a convergência.

Exercícios — Pré-Condicionamento e PCG

Exercício 4.30. Para a Matriz de Hilbert H_5 :

(a) Calcule $M = \text{diag}(H_5)$ e $\kappa_2(M^{-1}H_5)$. Houve melhora em relação a $\kappa_2(H_5)$?

Algoritmo 11 PCG

Entrada: A SPD, b , pré-cond. M (SPD), $x^{(0)}$, ε

- 1: $r^{(0)} \leftarrow b - Ax^{(0)}$; resolva $Mz^{(0)} = r^{(0)}$; $p^{(0)} \leftarrow z^{(0)}$
- 2: **repita**
- 3: $\alpha_k \leftarrow \langle r^{(k)}, z^{(k)} \rangle / \langle p^{(k)}, Ap^{(k)} \rangle$
- 4: $x^{(k+1)} \leftarrow x^{(k)} + \alpha_k p^{(k)}$; $r^{(k+1)} \leftarrow r^{(k)} - \alpha_k Ap^{(k)}$
- 5: Resolva $Mz^{(k+1)} = r^{(k+1)}$
- 6: $\beta_k \leftarrow \langle r^{(k+1)}, z^{(k+1)} \rangle / \langle r^{(k)}, z^{(k)} \rangle$; $p^{(k+1)} \leftarrow z^{(k+1)} + \beta_k p^{(k)}$
- 7: $k \leftarrow k + 1$
- 8: **até** $\|r^{(k)}\| < \varepsilon$

- (b) Resolva $H_5 x = e_1$ com CG e PCG (pré-cond. Jacobi). Compare o número de iterações.

Exercício 4.31 (Projeto). Para o Laplaciano 2D em malha $n \times n$ ($n = 20, 50, 100$):

- (a) Resolva com CG sem pré-condicionamento.
- (b) Resolva com PCG + pré-cond. Jacobi.
- (c) Resolva com PCG + ILU(0) via `scipy.sparse.linalg.spilu`.
- (d) Compare iterações e tempo de execução. Plote a curva de convergência do resíduo normalizado para os três casos.

4.9 Comparação e Escolha do Método

Método	Custo/iter.	Paralel.	Garante conv.	Quando usar
Gauss-Jacobi	$\mathcal{O}(n^2)$	✓	Diag. dominante	Paralelo, simples
Gauss-Seidel	$\mathcal{O}(n^2)$	×	Diag. dom., SPD	Sequencial, geral
SOR	$\mathcal{O}(n^2)$	×	$0 < \omega < 2$, SPD	EDPs, tridiagonais
Grad. Conjugado	$\mathcal{O}(\text{nnz})$	parcial	SPD	SPD esparsas
PCG	$\mathcal{O}(\text{nnz})$	parcial	SPD	Recomendado para SPD

Nota Importante

Regra de ouro prática:

- $n < 10^4$: use método direto (LU ou Cholesky).
- $n > 10^4$ e A SPD: PCG + ILU via `scipy.sparse.linalg.cg`.
- $n > 10^4$ e A não-SPD: GMRES + ILU via `scipy.sparse.linalg.gmres`.

4.10 Exercícios

Exercício 4.32. Implemente o método de Jacobi e resolva o sistema:

$$\begin{bmatrix} 10 & -1 & 2 & 0 \\ -1 & 11 & -1 & 3 \\ 2 & -1 & 10 & -1 \\ 0 & 3 & -1 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 6 \\ 25 \\ -11 \\ 15 \end{bmatrix} \quad (4.64)$$

com tolerância 10^{-8} . Verifique que a matriz é diagonal dominante.

Exercício 4.33. Demonstre completamente que se A é simétrica positiva definida e $2D - A$ também é positiva definida, então o método de Jacobi converge.

Dica: Mostre que $\rho(T_J) < 1$ usando propriedades de matrizes SPD.

Exercício 4.34 (R1). Para cada sistema abaixo, verifique a diagonal dominância, execute 3 iterações de Gauss-Jacobi com $x^{(0)} = \mathbf{0}$ e calcule $d_r^{(k)}$ a cada passo.

$$(a) \quad \begin{bmatrix} 6 & 1 & -1 \\ 1 & 5 & 1 \\ -1 & 1 & 6 \end{bmatrix} x = \begin{bmatrix} 14 \\ 10 \\ 13 \end{bmatrix} \quad (\text{solução: } (2; 1; 2)^T)$$

$$(b) \quad \begin{bmatrix} 3 & 4 \\ 1 & 5 \end{bmatrix} x = \begin{bmatrix} 5 \\ 7 \end{bmatrix} \quad (\text{não é diagonal dominante; o que ocorre ao iterar?})$$

Exercício 4.35 (R2). Resolva o sistema abaixo pelos métodos de Gauss-Jacobi e Gauss-Seidel com $x^{(0)} = \mathbf{0}$ e $\varepsilon = 10^{-3}$. Monte tabela comparativa de iterações.

$$\begin{cases} 7x_1 - x_2 + x_3 = 14 \\ x_1 + 8x_2 - 2x_3 = 12 \\ -x_1 + x_2 + 6x_3 = 3 \end{cases}$$

Exercício 4.36. Para o sistema do Exercício 1:

- Compare o número de iterações de Jacobi, Gauss-Seidel e SOR para diferentes valores de $\omega \in \{1.1, 1.3, 1.5, 1.7, 1.9\}$.
- Plote gráfico: número de iterações vs. ω .
- Identifique experimentalmente o ω ótimo.
- Calcule $\rho(T_J)$ numericamente e use a fórmula de Young para prever ω_{opt} . Compare com o valor experimental.

Exercício 4.37 (R3). Aplique o critério de Sassenfeld ao sistema de R2. Calcule $\beta = \max_i \beta_i$ e compare com $\alpha = \max_i \alpha_i$ (critério das linhas) e com $\rho(T_{GS})$ calculado numericamente.

Exercício 4.38 (R4). Para o sistema tridiagonal 6×6 com diagonal principal 5, diagonais adjacentes -1 e $b_i = 3 \forall i$:

- (a) Resolva por Gauss-Seidel com $\varepsilon = 10^{-6}$.
- (b) Calcule $\rho(T_J)$ e ω_{opt} pela fórmula de Young.
- (c) Resolva por SOR(ω_{opt}) e compare o número de iterações.

Exercício 4.39. Mostre que o raio espectral da matriz de iteração de Gauss-Seidel para:

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \quad (4.65)$$

é $\rho(T_{GS}) = 1/4$. Verifique numericamente.

Dica: Calcule explicitamente $T_{GS} = (D - L)^{-1}U$ e seus autovalores.

Exercício 4.40. Construa uma matriz 3×3 para a qual:

- (a) Jacobi converge mas Gauss-Seidel diverge.
- (b) Gauss-Seidel converge mas Jacobi diverge.

Existe tal matriz? Justifique.

Nível 2 — Análise e Implementação

Exercício 4.41 (R5). Implemente Jacobi, Gauss-Seidel, SOR e CG em Python. Para a Equação de Poisson 1D com $n = 50$:

- (a) Registre o número de iterações de cada método para $\varepsilon = 10^{-8}$.
- (b) Plote a curva de convergência do resíduo normalizado em escala logarítmica.
- (c) Compare os tempos de execução.

Exercício 4.42 (R6). Gere matrizes SPD com $\kappa_2(A) \in \{10, 100, 1000, 10000\}$ (via decomposição espectral). Para cada uma: (a) aplique CG e registre iterações; (b) aplique PCG com pré-cond. Jacobi; (c) plote “iteraões vs. $\kappa_2(A)$ ” em escala log-log para os dois casos.

Exercício 4.43 (R7). Para a matriz tridiagonal A de ordem 4 com diagonal 4 e subdiagonais -1 , mostre que $\rho(T_{GS}) = \rho(T_J)^2$. Calcule ambos os raios espectrais numericamente e verifique a relação.

Nível 3 — Integração e Projetos

Exercício 4.44 (R8 — Projeto). Equação de Poisson 2D. Resolva $-\nabla^2 u = f$ em $[0, 1]^2$ (Dirichlet nulo, $f = 2\pi^2 \sin(\pi x) \sin(\pi y)$, solução $u = \sin(\pi x) \sin(\pi y)$). Discretize com diferenças finitas e resolva com: Gauss-Seidel ($n = 10, 20, 30$), SOR(ω_{opt}) ($n = 10, 20, 30$), PCG+ILU(0) ($n = 10, 20, 50, 100$). Registre iterações, tempo e erro. Plote a superfície 3D para $n = 30$.

Exercício 4.45 (R9 — Teórico). Prove completamente o Teorema 4.1: (a) equação de recorrência $e^{(k)} = T^k e^{(0)}$; (b) equivalência $T^k \rightarrow 0 \Leftrightarrow \rho(T) < 1$ via forma de Jordan; (c) corolário sobre a taxa de convergência assintótica.

Exercício 4.46 (R10 — Projeto Integrativo). Para cada método do capítulo, elabore uma ficha técnica: (i) derivação; (ii) condição suficiente de convergência; (iii) custo por iteração; (iv) vantagens e limitações; (v) exemplo de aplicação. Resolva a Equação de Poisson 1D com $n = 100$ por todos os métodos e consolide os resultados em tabela comparativa, justificando a escolha do método mais adequado.

Exercício 4.47 (Projeto: Equação de Poisson 1D). Resolva numericamente a equação de Poisson 1D:

$$-\frac{d^2 u}{dx^2} = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0 \quad (4.66)$$

com $f(x) = \pi^2 \sin(\pi x)$ (solução exata: $u(x) = \sin(\pi x)$).

Discretize usando diferenças finitas centrais:

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i \quad (4.67)$$

em que $h = 1/(n + 1)$.

(a) Para $n = 10, 50, 100$, resolva usando:

- Algoritmo de Thomas (método direto para tridiagonais).
- Jacobi.
- Gauss-Seidel.
- SOR com ω ótimo.

(b) Compare número de iterações e tempo de execução.

- (c) Plote a solução numérica vs. solução exata.
- (d) Calcule o erro $\|u_{num} - u_{exata}\|_\infty$.

Exercício 4.48. Investigue o efeito do número de condição $\kappa(A)$ na convergência de Jacobi. Gere matrizes SPD com $\kappa = 10, 100, 1000, 10000$ e plote o número de iterações vs. κ .

Exercício 4.49. Implemente uma versão paralela do método de Jacobi usando **numpy** vetorizado ou **numba**. Compare o tempo de execução com a versão sequencial para sistemas grandes ($n = 1000, 5000, 10000$).

4.11 Referências e Leituras Complementares

- Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. 2nd edition. SIAM. https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf (Referência definitiva, disponível gratuitamente)
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*. 4th edition. Johns Hopkins University Press. Capítulo 10: Iterative Methods for Linear Systems.
- Young, D. M. (2014). *Iterative Solution of Large Linear Systems*. Dover Publications. (Clássico sobre SOR e teoria de iteração)
- Barrett, R., et al. (1994). *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM. <https://www.netlib.org/templates/> (Guia prático gratuito)
- Varga, R. S. (2000). *Matrix Iterative Analysis*. 2nd edition. Springer. (Teoria matemática profunda)
- SciPy Sparse Linear Algebra: <https://docs.scipy.org/doc/scipy/reference/sparse.linalg.html>
- PETSc - Portable, Extensible Toolkit for Scientific Computation: <https://petsc.org/> (Biblioteca industrial de alta performance)

Capítulo 5

Interpolação Numérica

“All models are wrong, but some are useful.”

— George Box

5.1 Introdução e Motivação

Em situações reais, raramente conhecemos uma função f analiticamente. O que temos em mãos é uma *tabela* de pares (x_i, y_i) : medições de sensor, leituras de experimento, pontos de uma imagem digital. A questão central é: como estimar f em pontos *não tabelados*?

Definição 5.1 (Problema de Interpolação). Dados $n+1$ pares $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ com os **nós de interpolação** x_i distintos e $y_i = f(x_i)$, encontrar uma função $P(x)$ tal que:

$$P(x_i) = y_i, \quad i = 0, 1, \dots, n. \quad (5.1)$$

P é chamado de *interpolador* de f nos nós $\{x_i\}$.

Nota Importante

Interpolação vs. Ajuste de Curvas: nos problemas de *interpolação*, a curva passa *exatamente* pelos pontos dados. No *ajuste* (Capítulo 6), minimiza-se um erro global — o que é adequado quando os dados têm ruído. Usar interpolação em dados ruidosos pode produzir curvas com oscilações sem sentido físico.

5.1.1 Por que usar polinômios?

Existem infinitas famílias de funções que poderiam ser usadas como interpoladores (trigonométricas, exponenciais, racionais, etc.). Os **polinômios** são privilegiados por três razões fundamentais:

1. **Teorema de Weierstrass:** toda função contínua em $[a, b]$ pode ser aproximada por polinômios com qualquer precisão desejada.
2. **Facilidade computacional:** integração, diferenciação e avaliação de polinômios se reduzem a operações aritméticas elementares.
3. **Teoria de erro tratável:** a fórmula do erro de interpolação polinomial é explícita e analisável (Seção 5.5).

Aplicação Computacional

Computação Gráfica — Animação e Curvas de Bézier:

Em filmes de animação e jogos, animadores definem apenas alguns *keyframes* (quadros-chave) e o computador gera os quadros intermediários por interpolação. Um personagem em $(0, 0)$ no frame 0 e em $(10, 4)$ no frame 60 precisa de 59 posições intermediárias.

Interpolação **linear** produz movimento mecânico e artificial. A **spline cúbica** produz aceleração e desaceleração suaves, imitando o movimento físico real. As curvas de Bézier — usadas em toda tipografia digital e em aplicativos como Illustrator e Inkscape — são uma forma especial de interpolação polinomial por partes.

Aplicações diretas: trajetórias em jogos (Unity, Unreal); morphing de personagens em cinema; interpolação de rotações quaterniônicas em realidade virtual.

Aplicação Computacional**Processamento de Imagens — Redimensionamento e Rotação:**

Ao ampliar uma imagem de 100×100 para 300×300 pixels, os 200×200 pixels novos precisam ser estimados a partir dos originais — isso é interpolação 2D.

- **Vizinho mais próximo:** rápido, produz bordas serrilhadas.
- **Bilinear:** interpolação linear em x e depois em y ; suave mas pode borrar detalhes.
- **Bicúbica:** usa splines cúbicas em ambas as direções; padrão do Photoshop para ampliações de qualidade.

O mesmo princípio é usado na **rotação** de imagens: girar 45° move cada pixel para uma posição não-inteira da grade, e interpolação reconstrói a imagem na nova grade.

Aplicação Computacional**Ciência de Dados — Séries Temporais com Dados Faltantes:**

Sensores falham. Registros históricos têm lacunas. Em séries temporais (temperatura horária, preço de ações, sinal de EEG), valores ausentes precisam ser preenchidos de forma coerente antes de qualquer análise.

A biblioteca `pandas` oferece `DataFrame.interpolate(method=...)`, que implementa interpolação linear, cúbica e por splines. A escolha do método impacta diretamente análises subsequentes: médias, correlações e modelos preditivos.

Cuidado: em séries com tendência forte ou sazonalidade, interpolação polinomial de grau alto pode introduzir artefatos piores que os dados faltantes originais.

Exercícios — Introdução

Exercício 5.1. Descreva com suas palavras a diferença entre interpolação e ajuste de curvas. Para cada situação abaixo, diga qual abordagem é mais adequada e justifique: (a) temperatura de um forno medida a cada segundo por um termômetro de precisão industrial; (b) peso de crianças registrado em consultas pediátricas (dados com variabilidade biológica); (c) posição de um robô lida por encoders de alta precisão.

Exercício 5.2. O Teorema de Weierstrass afirma que polinômios podem aproximar qualquer função contínua. Isso significa que, com pontos suficientes, a interpolação polinomial sempre melhora? Pesquise o “fenômeno de Runge” e escreva um parágrafo antecipando o problema.

5.2 Existência e Unicidade do Interpolador Polinomial

Teorema 5.2 (Existência e Unicidade). Dados $n + 1$ pares $(x_0, y_0), \dots, (x_n, y_n)$ com nós x_i distintos, existe um **único** polinômio $P_n(x)$ de grau $\leq n$ tal que $P_n(x_i) = y_i$ para $i = 0, 1, \dots, n$.

Demonstração. **Existência:** Será demonstrada de forma construtiva pelas formas de Lagrange (Seção 5.3) e de Newton (Seção 5.4).

Unicidade: Suponha que $P(x)$ e $Q(x)$ são dois polinômios de grau $\leq n$, ambos interpolando os dados. Considere:

$$R(x) = P(x) - Q(x).$$

$R(x)$ é polinômio de grau $\leq n$ e satisfaz $R(x_i) = P(x_i) - Q(x_i) = y_i - y_i = 0$ para $i = 0, 1, \dots, n$.

Portanto, $R(x)$ possui *pelo menos* $n + 1$ raízes distintas: $x_0, x_1, \dots, x_n$. Mas um polinômio não-nulo de grau $\leq n$ possui *no máximo* n raízes. Contradição. Logo, $R(x) \equiv 0$, e portanto $P(x) = Q(x)$ para todo x . $\square$

Corolário 5.3. O interpolador polinomial de $n + 1$ pontos é **único**: as formas de Lagrange, Newton e quaisquer outras constroem o *mesmo* polinômio, apenas escritas de modo diferente.

Este corolário tem consequência prática importante: podemos escolher *qual forma usar* com base na conveniência computacional, não na qualidade do resultado — que é sempre o mesmo.

Nota Importante

A distinção “grau $\leq n$ ” (e não “grau $= n$ ”) é essencial. Com 4 pontos colineares, o interpolador é uma reta (grau 1), não um cúbico. O teorema garante unicidade dentro de todos os polinômios de grau *no máximo* n .

5.2.1 Visão matricial (para entender a estrutura)

O Teorema 5.2 equivale a dizer que o sistema linear:

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix} \quad (5.2)$$

tem solução única. A matriz em (5.2) é a *matriz de Vandermonde*, cujo determinante é:

$$V = \prod_{0 \leq i < j \leq n} (x_j - x_i). \quad (5.3)$$

Como os nós são distintos, $V \neq 0$ e o sistema tem solução única. Na prática, entretanto, **nunca se resolve o sistema de Vandermonde diretamente**: ele é numericamente instável para n moderado. As formas de Lagrange e Newton evitam esse problema.

Exercícios — Existência e Unicidade

Exercício 5.3. Dois estudantes constroem o interpolador polinomial de grau ≤ 2 para os pontos $(0, 1)$, $(2, 5)$, $(4, 3)$. Um usa Lagrange, o outro usa Newton. Antes de calcular qualquer coisa: o que o Corolário 5.3 garante sobre os resultados?

Exercício 5.4. Para que valores de c o sistema de Vandermonde 3×3 com nós $x_0 = 0$, $x_1 = c$, $x_2 = 2$ é singular? O que isso significa para o problema de interpolação?

5.3 Interpolação de Lagrange

5.3.1 Ideia construtiva

A ideia central de Lagrange é construir, para cada nó x_i , um polinômio que seja *exatamente* 1 naquele nó e *exatamente* 0 em todos os outros. Esses polinômios são combinados com os pesos y_i para construir o interpolador.

Definição 5.4 (Polinômios Fundamentais de Lagrange). Para nós distintos $x_0, x_1, \dots, x_n$, o i -ésimo polinômio fundamental é:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}, \quad i = 0, 1, \dots, n. \quad (5.4)$$

Propriedade fundamental:

$$L_i(x_j) = \delta_{ij} = \begin{cases} 1, & j = i \\ 0, & j \neq i \end{cases} \quad (5.5)$$

A propriedade (5.5) decorre diretamente da construção: no numerador de $L_i(x)$, cada fator $(x - x_j)$ com $j \neq i$ contribui com zero quando $x = x_j$, e o denominador normaliza o resultado para 1 quando $x = x_i$.

Teorema 5.5 (Fórmula de Lagrange). O polinômio interpolador de grau $\leq n$ é:

$$P_n(x) = \sum_{i=0}^n y_i L_i(x). \quad (5.6)$$

Demonstração. $P_n(x)$ é combinação linear de polinômios de grau n , portanto tem grau $\leq n$. Para qualquer nó x_k :

$$P_n(x_k) = \sum_{i=0}^n y_i L_i(x_k) = \sum_{i=0}^n y_i \delta_{ik} = y_k.$$

Portanto P_n interpola os dados, e por unicidade (Teorema 5.2), é o interpolador. $\square$

5.3.2 Exemplo resolvido passo a passo

Exemplo 5.1 (Interpolação de Lagrange com 3 pontos). Interpolar os pontos: $(0, 3)$, $(2, 7)$, $(3, 13)$.

Passo 1 — Calcular os polinômios fundamentais.

$$\begin{aligned} L_0(x) &= (x-2)(x-3)(0-2)(0-3) = \frac{(x-2)(x-3)}{6} = \frac{x^2 - 5x + 6}{6} \\ L_1(x) &= \frac{(x-0)(x-3)}{(2-0)(2-3)} = \frac{x(x-3)}{-2} = \frac{-x^2 + 3x}{2} \\ L_2(x) &= \frac{(x-0)(x-2)}{(3-0)(3-2)} = \frac{x(x-2)}{3} = \frac{x^2 - 2x}{3} \end{aligned}$$

Verificação rápida dos polinômios fundamentais:

	$x = 0$	$x = 2$	$x = 3$
L_0	$\frac{(-2)(-3)}{6} = 1$	$\frac{0(-3)}{6} = 0$	$\frac{(-2)0}{6} = 0$
L_1	$\frac{0(-3)}{-2} = 0$	$\frac{2(-1)}{-2} = 1$	$\frac{3 \cdot 0}{-2} = 0$
L_2	$\frac{0(-2)}{3} = 0$	$\frac{2 \cdot 0}{3} = 0$	$\frac{3 \cdot 1}{3} = 1$

Passo 2 — Construir $P_2(x) = y_0 L_0(x) + y_1 L_1(x) + y_2 L_2(x)$.

$$P_2(x) = 3 \cdot \frac{x^2 - 5x + 6}{6} + 7 \cdot \frac{-x^2 + 3x}{2} + 13 \cdot \frac{x^2 - 2x}{3}$$

Reduzindo ao denominador comum 6:

$$\begin{aligned} P_2(x) &= 3(x^2 - 5x + 6) - 21(x^2 - 3x) + 26(x^2 - 2x)6 \\ &= \frac{(3 - 21 + 26)x^2 + (-15 + 63 - 52)x + 18}{6} \\ &= \frac{8x^2 - 4x + 18}{6} = \frac{4x^2 - 2x + 9}{3} \end{aligned}$$

Passo 3 — Verificação.

- $P_2(0) = 9/3 = 3 \checkmark$
- $P_2(2) = (16 - 4 + 9)/3 = 21/3 = 7 \checkmark$
- $P_2(3) = (36 - 6 + 9)/3 = 39/3 = 13 \checkmark$

Utilizo do interpolador: estimar $f(1, 5)$:

$$P_2(1, 5) = \frac{4(2, 25) - 2(1, 5) + 9}{3} = \frac{9 + 9 - 3}{3} = \frac{15}{3} = 5.$$

5.3.3 Custo computacional e limitação

O custo de avaliar $P_n(x)$ em um único ponto pela fórmula de Lagrange é $\mathcal{O}(n^2)$: calcular cada $L_i(x)$ requer n multiplicações e n divisões, e há $n + 1$ termos. Isso é aceitável para n pequeno.

O problema mais sério aparece quando se **adiciona um novo ponto**: todos os L_i precisam ser recalculados do zero, pois cada um depende de *todos* os outros nós. Isso torna Lagrange pouco prático quando os dados chegam incrementalmente — situação em que Newton é muito mais eficiente (Seção 5.4).

5.3.4 Implementação

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def lagrange(xi, yi, x):
5     """
6     Interpola pelo metodo de Lagrange.
7
8     Parametros
9     -----
10    xi : array (n+1,) -- nos de interpolacao
11    yi : array (n+1,) -- valores nos nos
12    x  : float ou array -- ponto(s) de avaliacao
13
14    Retorna
15    -----
16    P(x) : interpolador avaliado em x
17    """
18    xi = np.asarray(xi, dtype=float)
```

```

19     yi = np.asarray(yi, dtype=float)
20     x  = np.asarray(x,  dtype=float)
21     n  = len(xi)
22     P  = np.zeros_like(x, dtype=float)
23
24     for i in range(n):
25         # Calcular  $L_i(x)$  de forma vetorizada
26         L_i = np.ones_like(x, dtype=float)
27         for j in range(n):
28             if j != i:
29                 L_i *= (x - xi[j]) / (xi[i] - xi[j])
30         P += yi[i] * L_i
31
32     return P
33
34 # Exemplo do texto
35
36 xi = np.array([0., 2., 3.])
37 yi = np.array([3., 7., 13.])
38
39 x_fino = np.linspace(-0.5, 3.5, 300)
40 P       = lagrange(xi, yi, x_fino)
41
42 plt.figure(figsize=(7, 4))
43 plt.plot(x_fino, P, 'b-', label='$P_2(x)$')
44 plt.plot(xi, yi, 'ro', ms=8, label='Pontos dados')
45 plt.axvline(1.5, color='gray', ls='--', lw=0.8)
46 plt.scatter([1.5], [lagrange(xi, yi, 1.5)], color='g',
47             zorder=5, label=f'$P_2(1.5) = \{lagrange(xi,yi,1.5):.4f\}$')
48 plt.xlabel('$x$'); plt.ylabel('$P_2(x)$')
49 plt.title('Interpolacao de Lagrange 3 pontos')
50 plt.legend(); plt.grid(True, alpha=0.4)
51 plt.tight_layout()
52 plt.show()
53
54 # Verificacao nos nos
55
56 print("Verificacao nos nos:")
57 for i in range(len(xi)):

```

```

56     val = lagrange(xi, yi, xi[i])
57     print(f"    P({xi[i]}) = {val:.10f}    (esperado {yi[i]})")

```

Listing 5.1: Interpolação de Lagrange

Exercícios — Interpolação de Lagrange

Exercício 5.5. Calcule manualmente $L_0(x)$, $L_1(x)$ e $L_2(x)$ para os nós $x_0 = -2$, $x_1 = 0$, $x_2 = 1$. Verifique a propriedade $L_i(x_j) = \delta_{ij}$. Qual é o grau de cada L_i ?

Exercício 5.6. Use a fórmula de Lagrange para interpolar os dados abaixo e estimar $f(1, 5)$:

x	0	1	2	3
$f(x)$	1	0	1	0

Verifique o resultado com a implementação Python.

Exercício 5.7. Demonstre que para $n = 1$ (dois pontos), a fórmula de Lagrange se reduz à interpolação linear:

$$P_1(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0).$$

Exercício 5.8 (Discussão). Suponha que você tem os pontos $(x_0, y_0), \dots, (x_n, y_n)$ interpolados pelo polinômio P_n de Lagrange, e chega um novo ponto (x_{n+1}, y_{n+1}) . Quantas operações são necessárias para recalcular o interpolador? Por que esse custo é inconveniente na prática?

5.4 Diferenças Divididas e Interpolação de Newton

5.4.1 Motivação: adição incremental de pontos

A forma de Newton resolve a limitação da de Lagrange: ao adicionar um novo ponto, apenas *um novo coeficiente* precisa ser calculado — os demais permanecem inalterados. Isso torna a forma de Newton ideal para situações em que os dados chegam progressivamente (refinamento adaptativo, tabelas calculadas incrementalmente).

5.4.2 Diferenças Divididas

Definição 5.6 (Diferenças Divididas). Define-se recursivamente:

$$f[x_i] = y_i \quad (\text{ordem } 0) \quad (5.7)$$

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i} \quad (\text{ordem } 1) \quad (5.8)$$

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i} \quad (\text{ordem } k) \quad (5.9)$$

Teorema 5.7 (Simetria das Diferenças Divididas). $f[x_{i_0}, x_{i_1}, \dots, x_{i_k}]$ é simétrico nos seus argumentos: não depende da ordem em que os nós são listados.

Esboço. Por indução em k . A ideia é que $f[x_0, \dots, x_k]$ é o coeficiente líder (coeficiente de x^k) do polinômio de Lagrange através de $x_0, \dots, x_k$. Por unicidade desse polinômio (Teorema 5.2), o coeficiente líder independe da ordem dos nós. $\square$

A simetria tem uma consequência prática relevante: podemos calcular as diferenças divididas em qualquer ordem de conveniência, e o resultado é o mesmo.

5.4.3 Fórmula de Newton

Teorema 5.8 (Forma de Newton do Interpolador). O polinômio interpolador de grau $\leq n$ é:

$$P_n(x) = \sum_{k=0}^n f[x_0, \dots, x_k] \omega_k(x), \quad (5.10)$$

em que $\omega_0(x) = 1$ e $\omega_k(x) = \prod_{i=0}^{k-1} (x - x_i)$ para $k \geq 1$.

Demonstração. Por indução em n .

Base ($n = 0$): $P_0(x) = f[x_0] = y_0$. Interpola o único ponto. $\checkmark$

Passo indutivo: Suponha válido para $n - 1$. Denotemos por $P_{n-1}^{(0)}$ o interpolador de $x_0, \dots, x_{n-1}$ e por $P_{n-1}^{(1)}$ o de $x_1, \dots, x_n$. Defina:

$$P_n(x) = \frac{(x - x_0) P_{n-1}^{(1)}(x) - (x - x_n) P_{n-1}^{(0)}(x)}{x_n - x_0}.$$

Verificando nos nós: $P_n(x_0) = P_{n-1}^{(0)}(x_0) = y_0$; para $1 \leq i \leq n - 1$: $P_n(x_i) = y_i$ (ambos os termos concordam); $P_n(x_n) = P_{n-1}^{(1)}(x_n) = y_n$. P_n tem grau $\leq n$. Por unicidade, é o interpolador. Expandindo e comparando o coeficiente de x^n obtém-se o coeficiente $f[x_0, \dots, x_n]$. $\square$

Nota Importante

Vantagem da forma de Newton: ao adicionar o ponto (x_{n+1}, y_{n+1}) , o polinômio P_{n+1} se obtém por:

$$P_{n+1}(x) = P_n(x) + f[x_0, x_1, \dots, x_{n+1}] \omega_{n+1}(x).$$

Apenas o novo coeficiente $f[x_0, \dots, x_{n+1}]$ precisa ser calculado.

5.4.4 Tabela de diferenças divididas

A organização em tabela facilita o cálculo sequencial. Para $n + 1$ pontos, monta-se:

x	$f[x]$	$f[\cdot, \cdot]$	$f[\cdot, \cdot, \cdot]$	$f[\cdot, \cdot, \cdot, \cdot]$
x_0	$f[x_0]$	$f[x_0, x_1]$	$f[x_0, x_1, x_2]$	$f[x_0, x_1, x_2, x_3]$
x_1	$f[x_1]$	$f[x_1, x_2]$	$f[x_1, x_2, x_3]$	
x_2	$f[x_2]$	$f[x_2, x_3]$		
x_3	$f[x_3]$			

Os coeficientes de Newton são os elementos da **primeira linha**: $f[x_0]$, $f[x_0, x_1]$, $f[x_0, x_1, x_2]$, etc.

5.4.5 Exemplo resolvido passo a passo

Exemplo 5.2 (Tabela de Diferenças Divididas e Forma de Newton). Interpolar os mesmos pontos do Exemplo 5.1: $(0, 3)$, $(2, 7)$, $(3, 13)$. Isso permite confirmar o Corolário 5.3.

Passo 1 — Montar a tabela.

i	x_i	$f[x_i]$	$f[x_{i+1}, x_i]$	$f[x_{i+2}, x_{i+1}, x_i]$
0	0	3	$\frac{7-3}{2-0} = 2$	$\frac{6-2}{3-0} = \frac{4}{3}$
1	2	7	$\frac{13-7}{3-2} = 6$	
2	3	13		

Passo 2 — Coeficientes de Newton (primeira linha da tabela):

$$c_0 = f[x_0] = 3, \quad c_1 = f[x_0, x_1] = 2, \quad c_2 = f[x_0, x_1, x_2] = \frac{4}{3}.$$

Passo 3 — Escrever o polinômio:

$$P_2(x) = 3 + 2(x - 0) + \frac{4}{3}(x - 0)(x - 2) \quad (5.11)$$

$$= 3 + 2x + \frac{4}{3}(x^2 - 2x) \quad (5.12)$$

$$= \frac{4}{3}x^2 + 2x - \frac{8}{3}x + 3 \quad (5.13)$$

$$= \frac{4}{3}x^2 - \frac{2}{3}x + 3 = \frac{4x^2 - 2x + 9}{3}. \quad (5.14)$$

Mesmo resultado de Lagrange! ✓ (Corolário 5.3)

Passo 4 — Avaliação eficiente pelo esquema de Horner:

$$P_2(x) = c_0 + (x - x_0)[c_1 + (x - x_1)c_2] = 3 + x[2 + (x - 2) \cdot \frac{4}{3}].$$

Em $x = 1, 5$: $P_2(1, 5) = 3 + 1, 5[2 + (-0, 5) \cdot \frac{4}{3}] = 3 + 1, 5 \cdot [2 - \frac{2}{3}] = 3 + 1, 5 \cdot \frac{4}{3} = 3 + 2 = 5.$

✓

Exemplo 5.3 (Acrescentando um novo ponto). Acrescenta-se o ponto $(5, 11)$ aos dados do Exemplo 5.2. Com Lagrange, seria necessário recalcular tudo. Com Newton, calculamos apenas $f[x_0, x_1, x_2, x_3]$:

$$\text{Primeiro: } f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1} = \frac{(11 - 13)/(5 - 3) - 6}{5 - 2} = \frac{-1 - 6}{3} = -\frac{7}{3}.$$

$$\text{Depois: } f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0} = \frac{-7/3 - 4/3}{5 - 0} = \frac{-11/3}{5} = -\frac{11}{15}.$$

O polinômio cúbico é:

$$P_3(x) = P_2(x) + \left(-\frac{11}{15}\right)(x - 0)(x - 2)(x - 3) = \frac{4x^2 - 2x + 9}{3} - \frac{11}{15}x(x - 2)(x - 3).$$

Verificação: $P_3(5) = \frac{100 - 10 + 9}{3} - \frac{11}{15} \cdot 5 \cdot 3 \cdot 2 = \frac{99}{3} - \frac{110}{5} = 33 - 22 = 11.$ ✓

5.4.6 Implementação

```

1 import numpy as np
2
3 def diferencas_divididas(xi, yi):
4     """
5     Calcula os coeficientes de Newton pela tabela de DD.
6     Retorna coef[k] = f[x_0, ..., x_k] (primeira linha).
```

```

7      """
8      xi = np.asarray(xi, dtype=float)
9      yi = np.asarray(yi, dtype=float)
10     n = len(xi)
11     # Tabela n x n; F[i,j] = f[x_i, ..., x_{i+j}]
12     F = np.zeros((n, n))
13     F[:,0] = yi.copy()
14     for j in range(1, n):
15         for i in range(n - j):
16             F[i,j] = (F[i+1,j-1] - F[i,j-1]) / (xi[i+j] - xi[i])
17     return F[0, :] # coeficientes de Newton
18
19 def newton_eval(coef, xi, x):
20     """
21     Avalia o polinomio de Newton pelo esquema de Horner.
22     coef : array      coeficientes f[x_0,...,x_k]
23     xi   : array      nos de interpolacao
24     x    : float ou array ponto(s) de avaliacao
25     """
26     xi = np.asarray(xi, dtype=float)
27     coef = np.asarray(coef, dtype=float)
28     x = np.asarray(x, dtype=float)
29     n = len(coef)
30     # Horner: P = c_{n-1} + (x-x_{n-2})(c_{n-2} + ...)
31     P = coef[-1] * np.ones_like(x)
32     for k in range(n-2, -1, -1):
33         P = coef[k] + (x - xi[k]) * P
34     return P
35
36 def newton(xi, yi, x):
37     """Pipeline completo: DD + avaliacao."""
38     return newton_eval(diferencas_divididas(xi, yi), xi, x)
39
40 #      Exemplo: adicao incremental de pontos
41
42 xi_base = np.array([0., 2., 3.])
43 yi_base = np.array([3., 7., 13.])
44
45 print("=== Coeficientes iniciais (3 pontos) ===")
c = diferencas_divididas(xi_base, yi_base)

```

```

46 print(f"c = {c}")
47 print(f"P_2(1.5) = {newton_eval(c, xi_base, 1.5):.6f}")
48
49 # Adiciona ponto (5, 11)
50 xi_novo = np.append(xi_base, 5.)
51 yi_novo = np.append(yi_base, 11.)
52 c_novo = diferencas_divididas(xi_novo, yi_novo)
53
54 print("\n=== Apos adicionar (5, 11) ===")
55 print(f"Novo coeficiente: c_3 = {c_novo[-1]:.6f}")
56 print(f"P_3(5.0) = {newton_eval(c_novo, xi_novo, 5.0):.6f}
      (esperado 11)")

```

Listing 5.2: Diferenças Divididas e Interpolação de Newton

Exercícios — Diferenças Divididas e Newton

Exercício 5.9. Construa a tabela de diferenças divididas para $f(x) = e^x$ nos pontos $x = 0, 0,5, 1, 0, 1, 5$. Escreva o polinômio de Newton $P_3(x)$ e use-o para estimar $e^{0,8}$. Compare com o valor exato $e^{0,8} \approx 2,2255$.

Exercício 5.10. Prove que se $f(x)$ for um polinômio de grau m , então $f[x_0, \dots, x_k] = 0$ para todo $k > m$. O que isso implica sobre a tabela de diferenças divididas de um polinômio cúbico?

Exercício 5.11. Para os dados abaixo, use a forma de Newton para estimar $f(2,5)$. Em seguida, acrescente o ponto $(3,5; 2,4)$ ao conjunto e obtenha $P_4(2,5)$ sem recalcular os coeficientes anteriores.

x	1	2	3	4
$f(x)$	3,1	2,8	2,5	1,9

Exercício 5.12 (Implementação). Implemente em Python a função `adicionar_ponto(coef, xi, x_novo, y_novo)` que, dados os coeficientes de Newton existentes e um novo ponto, retorna os coeficientes atualizados sem recalcular os anteriores. Verifique com o Exemplo 5.3.

5.5 Erro de Interpolação

5.5.1 Fórmula do erro

Teorema 5.9 (Erro de Interpolação Polinomial). Seja $f \in C^{n+1}[a, b]$ e P_n o interpolador de f nos nós $x_0, \dots, x_n \in [a, b]$. Para qualquer $x \in [a, b]$, existe $\xi = \xi(x) \in (a, b)$ tal que:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x), \quad (5.15)$$

em que $\omega_{n+1}(x) = \prod_{i=0}^n (x - x_i)$.

Demonstração. Se $x = x_i$ para algum i , ambos os lados de (5.15) são zero. Suponha $x \neq x_i$ para todo i .

Defina a função auxiliar:

$$\phi(t) = f(t) - P_n(t) - K \omega_{n+1}(t),$$

em que K é escolhido para que $\phi(x) = 0$:

$$K = \frac{f(x) - P_n(x)}{\omega_{n+1}(x)}.$$

ϕ tem pelo menos $n+2$ zeros em $[a, b]$: os nós $x_0, \dots, x_n$ e o ponto x .

Por aplicações sucessivas do Teorema de Rolle:

- ϕ tem $n+2$ zeros $\Rightarrow \phi'$ tem $\geq n+1$ zeros em (a, b)
- ϕ' tem $n+1$ zeros $\Rightarrow \phi''$ tem $\geq n$ zeros em (a, b)
- $\vdots$
- $\phi^{(n+1)}$ tem ≥ 1 zero em (a, b) ; chame-o de ξ .

Calculando $\phi^{(n+1)}$: como P_n tem grau $\leq n$, $P_n^{(n+1)} = 0$. A derivada $(n+1)$ -ésima de $\omega_{n+1}(t) = \prod_{i=0}^n (t - x_i)$ é $(n+1)!$. Portanto:

$$\phi^{(n+1)}(t) = f^{(n+1)}(t) - K (n+1)!$$

De $\phi^{(n+1)}(\xi) = 0$: $K = \frac{f^{(n+1)}(\xi)}{(n+1)!}$. Substituindo na definição de K :

$$\frac{f(x) - P_n(x)}{\omega_{n+1}(x)} = \frac{f^{(n+1)}(\xi)}{(n+1)!},$$

o que é exatamente (5.15). □

Corolário 5.10 (Cota para o Erro). Se $|f^{(n+1)}(t)| \leq M_{n+1}$ para todo $t \in [a, b]$, então:

$$|f(x) - P_n(x)| \leq \frac{M_{n+1}}{(n+1)!} |\omega_{n+1}(x)|. \quad (5.16)$$

Nota Importante

Quando f é dada em forma tabular (sem expressão analítica), não é possível calcular M_{n+1} . Nesse caso, estima-se o erro pela *primeira diferença dividida omitida*:

$$|f(x) - P_n(x)| \approx |f[x_0, \dots, x_n, x]| |\omega_{n+1}(x)|. \quad (5.17)$$

Na prática, calcula-se P_n e P_{n+1} e usa-se $|P_{n+1}(x) - P_n(x)|$ como estimativa do erro.

5.5.2 Exemplo: análise do erro com função conhecida

Exemplo 5.4 (Erro de Interpolação de $\ln x$). Interpolando $f(x) = \ln x$ nos nós equiespaçados $x_0 = 1$, $x_1 = 2$, $x_2 = 3$ e estimar o erro em $x = 1,5$.

Cálculo do interpolador de Newton:

x_i	$f[x_i]$	Ord. 1	Ord. 2
1	0	$\frac{\ln 2 - 0}{2 - 1} = \ln 2 \approx 0,6931$	$\frac{(\ln 3 - \ln 2) - \ln 2}{2} = \frac{\ln(3/4)}{2} \approx -0,1438$
2	$\ln 2 \approx 0,6931$	$\ln 3 - \ln 2 = \ln(3/2) \approx 0,4055$	
3	$\ln 3 \approx 1,0986$		

$$P_2(x) = 0 + \ln 2(x - 1) + \frac{\ln(3/4)}{2}(x - 1)(x - 2).$$

$$\text{Em } x = 1,5: P_2(1,5) = \ln 2 \cdot 0,5 + \frac{\ln(3/4)}{2} \cdot 0,5 \cdot (-0,5) \approx 0,3466 + 0,0360 \approx 0,3826.$$

$$\text{Erro exato: } f(1,5) - P_2(1,5) = \ln(1,5) - 0,3826 \approx 0,4055 - 0,3826 = 0,0229.$$

Cota teórica: $f'''(x) = 2/x^3$; em $[1, 3]$: $M_3 = 2/1^3 = 2$. Portanto:

$$|f(1,5) - P_2(1,5)| \leq \frac{2}{3!} |(1,5 - 1)(1,5 - 2)(1,5 - 3)| = \frac{2}{6} \cdot 0,5 \cdot 0,5 \cdot 1,5 = \frac{2}{6} \cdot 0,375 = 0,125.$$

O erro real (0,023) é bem abaixo da cota (0,125): a cota é conservadora.

Exemplo 5.5 (Como o erro decresce com n). Para $f(x) = \sin x$ em $[0, \pi]$ com $n + 1$ nós equiespaçados, $|f^{(n+1)}(x)| \leq 1$ para todo n . A cota (5.16) dá:

$$|f(x) - P_n(x)| \leq \frac{1}{(n+1)!} \cdot \max_x |\omega_{n+1}(x)|.$$

O fator $1/(n+1)!$ decresce rapidamente (para $n = 4$: $1/120 \approx 0,008$), mas $\max |\omega_{n+1}|$ cresce com n para pontos equiespaçados. Para $\sin x$, o efeito combinado é uma melhoria, mas não é assim para todas as funções (ver Seção 5.6).

Exercícios — Erro de Interpolação

Exercício 5.13. Use a fórmula do erro para estimar a precisão ao interpolar $f(x) = \sqrt{x}$ em $[1, 4]$ com:

- (a) Interpolação linear (2 pontos: $x_0 = 1, x_1 = 4$) em $x = 2$.
- (b) Interpolação quadrática (3 pontos: $x_0 = 1, x_1 = 2, x_2 = 4$) em $x = 2$.

Calcule também o erro exato e verifique que ele é menor que a cota.

Exercício 5.14. Demonstre que se f é um polinômio de grau $\leq n$, o erro de interpolação por $n + 1$ pontos é exatamente zero. (Dica: use a fórmula do erro e determine $f^{(n+1)}$.)

Exercício 5.15 (Tabela de dados). Para a tabela abaixo (sem expressão analítica para f), use a estimativa (5.17) para avaliar a confiabilidade de $P_2(2, 5)$:

x	1	2	3	4
$f(x)$	2,4	3,9	4,8	5,1

Compare $P_2(2, 5)$ com $P_3(2, 5)$ e interprete a diferença como estimativa do erro.

5.6 Fenômeno de Runge e Nós de Chebyshev

5.6.1 O problema com pontos equiespaçados

A intuição sugere que mais pontos implicam melhor aproximação. O fenômeno de Runge mostra que essa intuição pode estar errada.

Teorema 5.11 (Divergência com Pontos Equiespaçados). Existe função analítica f e intervalo $[a, b]$ tais que, com nós equiespaçados $x_i = a + ih$ ($h = (b - a)/n$), o interpolador P_n **diverge** em certos pontos quando $n \rightarrow \infty$:

$$\max_{x \in [a, b]} |f(x) - P_n(x)| \rightarrow \infty.$$

Exemplo 5.6 (Função de Runge). A função $f(x) = \frac{1}{1 + 25x^2}$ em $[-1, 1]$ é analítica e suave, mas com pontos equiespaçados:

- Para $n = 5$: interpolação razoável no centro, pequenas oscilações nas bordas.
- Para $n = 10$: oscilações visíveis nos extremos do intervalo.
- Para $n = 15$: o interpolador excede $|f| > 2$ perto de $x = \pm 1$, enquanto $f \leq 1$.

A causa: o termo $\max |\omega_{n+1}(x)|$ cresce muito rápido perto das bordas para pontos equiespaçados, dominando o decréscimo de $1/(n + 1)!$.

Nota Importante

O fenômeno de Runge não é uma falha dos métodos de Lagrange ou Newton — é uma limitação estrutural da interpolação polinomial global com pontos equiespaçados. As duas soluções clássicas são: (1) usar pontos de Chebyshev; (2) usar splines cúbicas (polinômios de baixo grau por partes).

5.6.2 Nós de Chebyshev

A raiz do problema com pontos equiespaçados é que o produto $|\omega_{n+1}(x)| = |\prod_{i=0}^n (x - x_i)|$ fica grande perto das bordas. Os **nós de Chebyshev** minimizam esse produto.

Teorema 5.12 (Propriedade de Minimax dos Nós de Chebyshev). Dentre todos os conjuntos de $n + 1$ nós distintos em $[-1, 1]$, os nós de Chebyshev:

$$x_i = \cos\left(\frac{2i+1}{2(n+1)}\pi\right), \quad i = 0, 1, \dots, n, \quad (5.18)$$

minimizam $\max_{x \in [-1, 1]} |\omega_{n+1}(x)| = \max_{x \in [-1, 1]} |\prod_{i=0}^n (x - x_i)|$.

O mínimo é $\frac{1}{2^n}$, e com ele a cota do erro torna-se:

$$|f(x) - P_n(x)| \leq \frac{M_{n+1}}{(n+1)! 2^n}. \quad (5.19)$$

Esboço. O polinômio de Chebyshev de grau $n + 1$ é $T_{n+1}(x) = \cos((n+1) \arccos x)$. Seu coeficiente líder é 2^n , e ele tem exatamente $n + 1$ zeros em $[-1, 1]$ que são exatamente (5.18). Pode-se mostrar que $T_{n+1}(x)/2^n$ é o polinômio mônico de grau $n + 1$ com menor norma sup em $[-1, 1]$, e essa norma é $1/2^n$. $\square$

Para o intervalo geral $[a, b]$: aplica-se a transformação linear

$$t = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2i+1}{2(n+1)}\pi\right).$$

Interpretação geométrica: os nós de Chebyshev são as projeções na reta dos $n + 1$ pontos igualmente espaçados sobre a semicircunferência unitária. São mais densos nas bordas e mais esparsos no centro — exatamente o oposto dos equiespaçados.

5.6.3 Exemplo comparativo

Exemplo 5.7 (Equiespaçados vs. Chebyshev na função de Runge). Compara-se, para $n = 10$ e $f(x) = 1/(1 + 16x^2)$ em $[-2, 2]$:

Nós equiespaçados: $x_i = -2 + 4i/10$, $i = 0, \dots, 10$.

Erro máximo $\approx 4,2$ — o interpolador oscila violentamente, pior que a própria função.

Nós de Chebyshev em $[-2, 2]$: $x_i = 2 \cos\left(\frac{2i+1}{22}\pi\right)$, $i = 0, \dots, 10$.

Erro máximo $\approx 0,03$ — excelente aproximação.

A diferença é de dois ordem de grandeza com os mesmos $n + 1 = 11$ avaliações de f .

5.6.4 Implementação

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f_runge(x, alpha=16):
5     """Familia de funcoes de Runge f(x) = 1/(1 + alpha*x^2)."""
6     return 1.0 / (1.0 + alpha * x**2)
7
8 def nos_chebyshev(n, a=-1, b=1):
9     """Nos de Chebyshev de grau n+1 no intervalo [a,b]."""
10    i = np.arange(n + 1)
11    cos = np.cos((2*i + 1) / (2*(n+1)) * np.pi)
12    return (a + b)/2 + (b - a)/2 * cos
13
14 def comparar_runge(n_list, a=-2, b=2, alpha=16):
15     """Plota comparacao equiespac. vs Chebyshev para varios n."""
16     x_fino = np.linspace(a, b, 1000)
17     f_exata = f_runge(x_fino, alpha)
18
19     fig, axes = plt.subplots(len(n_list), 2,
20                             figsize=(12, 3*len(n_list)),
21                             sharex=True)
22
23     for row, n in enumerate(n_list):
24         # Nos equiespac.
25         xi_eq = np.linspace(a, b, n+1)
26         yi_eq = f_runge(xi_eq, alpha)
27         P_eq = newton(xi_eq, yi_eq, x_fino)
28         err_eq = np.max(np.abs(f_exata - P_eq))
29
30         # Nos Chebyshev

```

```

31     xi_ch = nos_chebyshev(n, a, b)
32     yi_ch = f_runge(xi_ch, alpha)
33     P_ch = newton(xi_ch, yi_ch, x_fino)
34     err_ch = np.max(np.abs(f_exata - P_ch))
35
36     # Plot equiespac.
37     ax = axes[row, 0]
38     ax.plot(x_fino, f_exata, 'k-', lw=2, label='$f(x)$')
39     ax.plot(x_fino, P_eq, 'r--', lw=1.5,
40             label=f'$P_{n}$'
41                 equiespac.\n$\epsilon_{\max}={err_eq:.3f}$')
42     ax.plot(xi_eq, yi_eq, 'rs', ms=4)
43     ax.set_ylim(-0.5, 1.5); ax.legend(fontsize=8)
44     ax.set_title(f'n={n} Equiespa dos', fontsize=9)
45     ax.grid(True, alpha=0.3)
46
47     # Plot Chebyshev
48     ax = axes[row, 1]
49     ax.plot(x_fino, f_exata, 'k-', lw=2, label='$f(x)$')
50     ax.plot(x_fino, P_ch, 'b--', lw=1.5,
51             label=f'$P_{n}$'
52                 Chebyshev\n$\epsilon_{\max}={err_ch:.4f}$')
53     ax.plot(xi_ch, yi_ch, 'b^', ms=4)
54     ax.set_ylim(-0.5, 1.5); ax.legend(fontsize=8)
55     ax.set_title(f'n={n} Chebyshev', fontsize=9)
56     ax.grid(True, alpha=0.3)
57
58     fig.suptitle(f'Fenomeno de Runge: $f(x)=1/(1+{alpha}x^2)$ em
59                 [{a},{b}]',
60                 fontsize=11, fontweight='bold')
61     plt.tight_layout()
62     plt.show()
63
64     comparar_runge(n_list=[5, 10, 15])

```

Listing 5.3: Comparação: equiespaçados vs. Chebyshev na função de Runge

Exercícios — Runge e Chebyshev

Exercício 5.16. Calcule os 5 nós de Chebyshev ($n = 4$) no intervalo $[0, 4]$. Compare com os nós equiespaçados 0, 1, 2, 3, 4. Onde estão concentrados os nós de Chebyshev?

Exercício 5.17. Para $f(x) = \tan x$ em $[0, 1, 2]$ com $n = 6$, compare o erro máximo de interpolação usando nós equiespaçados e nós de Chebyshev (calcule numericamente). Qual a razão dos erros máximos?

Exercício 5.18 (Análise). A cota $\max |\omega_{n+1}| \leq 1/2^n$ dos nós de Chebyshev cresce exponencialmente com n (no sentido de que 2^n cresce), mas decresce para as funções de Chebyshev. Explique por que isso garante convergência para funções analíticas, mas não elimina automaticamente o fenômeno de Runge para funções com singularidades próximas ao intervalo.

Exercício 5.19 (Implementação — Projeto). Implemente o experimento comparativo para $f(x) = 1/(1 + 25x^2)$ em $[-1, 1]$ com $n \in \{5, 10, 15, 20\}$. Para cada n , calcule o erro máximo com equiespaçados e com Chebyshev. Plote o gráfico “erro máximo vs. n ” em escala logarítmica para os dois casos. Descreva o comportamento qualitativo.

5.7 Splines Cúbicas

5.7.1 Motivação: por que polinômios por partes?

Os nós de Chebyshev resolvem o problema de Runge, mas exigem que f seja avaliada em pontos específicos. Quando os pontos já estão fixados (por experimento, por exemplo) e são muitos, mesmo Chebyshev pode ser insuficiente. A alternativa definitiva é abandonar o polinômio *global* e usar **polinômios de baixo grau em cada subintervalo**, com continuidade cuidadosamente imposta nas junturas.

	Polinômio global	Spline cúbica
Grau máximo	n (cresce com dados)	3 (fixo)
Fenômeno de Runge	Possível	Não ocorre
Regularidade	Global (C^n)	C^2 (suficiente para aplicações)
Custo (solver)	$\mathcal{O}(n^3)$ ou $\mathcal{O}(n^2)$	$\mathcal{O}(n)$ (Thomas)
Novo ponto	Recalcular tudo	Reconstrução local

5.7.2 Definição e contagem de graus de liberdade

Definição 5.13 (Spline Cúbica). Dados $n + 1$ pares $(x_0, y_0), \dots, (x_n, y_n)$ com $x_0 < x_1 < \dots < x_n$, uma *spline cúbica* $S(x)$ satisfaz:

1. Em cada subintervalo $[x_i, x_{i+1}]$, $i = 0, \dots, n - 1$:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

2. $S(x_i) = y_i$ para $i = 0, 1, \dots, n$ (interpolação)

3. S , S' e S'' são contínuas em $[x_0, x_n]$ (suavidade C^2)

Contagem de graus de liberdade:

	Quantidade	Acumulado
Incógnitas: 4 coeficientes por intervalo	$4n$	
Interpolação nos nós interiores ($S_i(x_{i+1}) = y_{i+1}$):	n	
Interpolação nos nós extremos ($S_0(x_0) = y_0$):	1	
+ ($S_{n-1}(x_n) = y_n$):	1	$n + 2$
Continuidade de S' nos nós interiores:	$n - 1$	$2n + 1$
Continuidade de S'' nos nós interiores:	$n - 1$	$3n$
Condições disponíveis		$3n$
Graus de liberdade restantes		$4n - 3n = n \dots ?$

Ajuste: a interpolação nas extremidades conta 2 (não $n + 2$), as condições interiores de continuidade $S_i(x_{i+1}) = S_{i+1}(x_{i+1})$ contam $n - 1$ (não n). A conta correta:

	Qtd.	Acumulado
Incógnitas	$4n$	
Interpolação: $S_i(x_i) = y_i$ e $S_i(x_{i+1}) = y_{i+1}$	$2n$	$2n$
Continuidade S' : $n - 1$ condições	$n - 1$	$3n - 1$
Continuidade S'' : $n - 1$ condições	$n - 1$	$4n - 2$
Faltam 2 condições!		

5.7.3 Condições de fronteira

As 2 condições faltantes devem ser especificadas pelo usuário:

Tipo	Condições	Quando usar
Natural	$S''(x_0) = S''(x_n) = 0$	Curvatura livre; padrão se nada se sabe
Clamped	$S'(x_0) = f'(x_0), S'(x_n) = f'(x_n)$	Derivadas conhecidas nos extremos
Periódica	$S'(x_0) = S'(x_n), S''(x_0) = S''(x_n)$	Curvas fechadas, funções periódicas

Teorema 5.14 (Existência e Unicidade da Spline Cúbica). Para qualquer conjunto de $n + 1$ nós distintos e qualquer escolha de condições de fronteira naturais, clamped ou periódicas, existe única spline cúbica. O sistema linear resultante é tridiagonal e bem condicionado.

5.7.4 Construção: o sistema tridiagonal

Denotando $h_i = x_{i+1} - x_i$ e $M_i = S''(x_i)$, as condições de continuidade de S'' geram o sistema tridiagonal:

$$h_{i-1} M_{i-1} + 2(h_{i-1} + h_i) M_i + h_i M_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right), \quad i = 1, \dots, n-1, \quad (5.20)$$

com $M_0 = M_n = 0$ (condição natural).

Uma vez resolvido o sistema (5.20) para $M_0, \dots, M_n$, os coeficientes de cada S_i são recuperados por:

$$a_i = y_i, \quad c_i = \frac{M_i}{2}, \quad d_i = \frac{M_{i+1} - M_i}{6h_i}, \quad b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2M_i + M_{i+1})}{6}. \quad (5.21)$$

Nota Importante

O sistema (5.20) é **tridiagonal**, e pode ser resolvido em $\mathcal{O}(n)$ pelo Algoritmo de Thomas (Capítulo 3). Isso torna splines cúbicas muito mais eficientes que interpolação polinomial global para n grande.

5.7.5 Exemplo resolvido passo a passo

Exemplo 5.8 (Spline Cúbica Natural com 4 pontos). Dados: $(0, 0), (1, 1), (2, 0), (3, 1)$.

Temos $n = 3$ intervalos e $h_i = 1$ para todo i .

Passo 1 — Montar o sistema tridiagonal ($M_0 = M_3 = 0$, incógnitas: M_1, M_2):

Para $i = 1$:

$$1 \cdot M_0 + 2(1+1)M_1 + 1 \cdot M_2 = 6\left(\frac{0-1}{1} - \frac{1-0}{1}\right) = 6(-2) = -12.$$

Para $i = 2$:

$$1 \cdot M_1 + 2(1+1)M_2 + 1 \cdot M_3 = 6\left(\frac{1-0}{1} - \frac{0-1}{1}\right) = 6(2) = 12.$$

Com $M_0 = M_3 = 0$:

$$\begin{pmatrix} 4 & 1 \\ 1 & 4 \end{pmatrix} \begin{pmatrix} M_1 \\ M_2 \end{pmatrix} = \begin{pmatrix} -12 \\ 12 \end{pmatrix}.$$

Passo 2 — Resolver o sistema (por eliminação):

$M_1 = -12/4 - M_2/4 = -3 - M_2/4$. Substituindo: $(-3 - M_2/4) + 4M_2 = 12 \Rightarrow 15M_2/4 = 15 \Rightarrow M_2 = 4$. Logo $M_1 = -3 - 1 = -4$.

Passo 3 — Calcular os coeficientes ($h_i = 1, M_0 = 0, M_1 = -4, M_2 = 4, M_3 = 0$):

i	$[x_i, x_{i+1}]$	a_i	b_i	c_i	d_i
0	$[0, 1]$	0	$\frac{1-0}{1} - \frac{1 \cdot (0+(-4))}{6} = 1 + \frac{2}{3} = \frac{5}{3}$	0	$\frac{-4-0}{6} = -\frac{2}{3}$
1	$[1, 2]$	1	$\frac{0-1}{1} - \frac{1 \cdot (2(-4)+4)}{6} = -1 + \frac{4}{6} = -\frac{1}{3}$	$\frac{-4}{2} = -2$	$\frac{4-(-4)}{6} = \frac{4}{3}$
2	$[2, 3]$	0	$\frac{1-0}{1} - \frac{1 \cdot (2 \cdot 4 + 0)}{6} = 1 - \frac{4}{3} = -\frac{1}{3}$	$\frac{4}{2} = 2$	$\frac{0-4}{6} = -\frac{2}{3}$

Verificação de S_0 em $x = 1$: $S_0(1) = 0 + \frac{5}{3}(1) + 0 - \frac{2}{3}(1) = \frac{3}{3} = 1$. ✓

Continuidade de S' em $x = 1$: $S'_0(x) = b_0 + 2c_0(x-0) + 3d_0(x-0)^2$. Em $x = 1$: $S'_0(1) = \frac{5}{3} + 0 - 2 = -\frac{1}{3}$. $S'_1(x) = b_1 + 2c_1(x-1) + 3d_1(x-1)^2$. Em $x = 1$: $S'_1(1) = -\frac{1}{3}$. ✓

5.7.6 Implementação

```

1 import numpy as np
2 from scipy.interpolate import CubicSpline
3 import matplotlib.pyplot as plt
4
5 # 1. Implementacao manual da spline natural

```

```

6 def spline_natural(xi, yi):
7     """
8     Constroi spline cubica natural.
9     Retorna dicionario com coeficientes a, b, c, d por intervalo.
10    """
11    xi = np.asarray(xi, dtype=float)
12    yi = np.asarray(yi, dtype=float)
13    n = len(xi) - 1          # numero de intervalos
14    h = np.diff(xi)          # h[i] = x[i+1] - x[i]
15
16    # Montar sistema tridiagonal para M[1..n-1] (M[0]=M[n]=0)
17    # Diagonal principal: 2*(h[i-1]+h[i])
18    # Sub/superdiagonal: h[i-1] e h[i]
19    m = n - 1                # dimensao do sistema
20    rhs = 6 * (np.diff(yi / h) ) # lado direito
21    # rhs[i] = 6*(yi[i+2]-yi[i+1])/h[i+1] -
22    #          6*(yi[i+1]-yi[i])/h[i]
23    # obs: np.diff(yi/h) nao esta certo; calcular explicitamente
24    rhs = np.array([
25        6*((yi[i+2]-yi[i+1])/h[i+1] - (yi[i+1]-yi[i])/h[i])
26        for i in range(m)
27    ])
28    diag = np.array([2*(h[i]+h[i+1]) for i in range(m)])
29    lower = h[1:m]
30    upper = h[1:m]
31
32    # Algoritmo de Thomas
33    M_int = thomas_tridiagonal(lower, diag, upper, rhs)
34    M = np.concatenate([[0.0], M_int, [0.0]]) # M[0]=M[n]=0
35
36    # Recuperar coeficientes
37    a = yi[:-1].copy()
38    c = M[:-1] / 2.0
39    d = (M[1:] - M[:-1]) / (6.0 * h)
40    b = (yi[1:] - yi[:-1]) / h - h * (2*M[:-1] + M[1:]) / 6.0
41
42    return dict(xi=xi, a=a, b=b, c=c, d=d)
43
44 def thomas_tridiagonal(lower, diag, upper, rhs):
45     """Algoritmo de Thomas para sistema tridiagonal."""

```

```

45     n = len(diag)
46     c, d, b = upper.copy(), diag.copy(), rhs.copy()
47     x = np.zeros(n)
48     for i in range(1, n):
49         m = lower[i-1] / b[i-1]
50         b[i] -= m * c[i-1]
51         rhs[i] -= m * rhs[i-1]
52     x[-1] = rhs[-1] / b[-1]
53     for i in range(n-2, -1, -1):
54         x[i] = (rhs[i] - c[i]*x[i+1]) / b[i]
55     return x
56
57 def avaliar_spline(sp, x):
58     """Avalia spline em ponto(s) x."""
59     xi = sp['xi']
60     x = np.asarray(x, dtype=float)
61     y = np.zeros_like(x)
62     for j, xj in enumerate(x):
63         # Encontrar intervalo
64         i = np.searchsorted(xi, xj, side='right') - 1
65         i = max(0, min(i, len(xi)-2))
66         t = xj - xi[i]
67         y[j] = sp['a'][i] + sp['b'][i]*t + sp['c'][i]*t**2 +
68             sp['d'][i]*t**3
69     return y
70
71 # 2. Exemplo e comparacao com SciPy
72
73 xi = np.array([0., 1., 2., 3.])
74 yi = np.array([0., 1., 0., 1.])
75
76 sp_manual = spline_natural(xi, yi)
77 cs_scipy = CubicSpline(xi, yi, bc_type='natural')
78
79 x_fino = np.linspace(0, 3, 300)
80
81 plt.figure(figsize=(8, 4))
82 plt.plot(x_fino, avaliar_spline(sp_manual, x_fino),
83          'b-', lw=2, label='Spline manual')
84 plt.plot(x_fino, cs_scipy(x_fino),

```

```

83         'r--', lw=1.5, label='SciPy CubicSpline')
84 plt.plot(xi, yi, 'ko', ms=8, label='Pontos dados')
85 plt.xlabel('$x$'); plt.ylabel('$S(x)$')
86 plt.title('Spline Cubica Natural')
87 plt.legend(); plt.grid(True, alpha=0.4)
88 plt.tight_layout(); plt.show()
89
90 # 3. Verificar  $S$ ,  $S'$ ,  $S''$  com SciPy
91
92 fig, axes = plt.subplots(3, 1, figsize=(8, 7), sharex=True)
93 for k, (label, ls) in enumerate([('$S(x)$', 'b-'),
94     ('$S'(x)$', 'g-'), ('$S''(x)$', 'r-')]):
95     axes[k].plot(x_fino, cs_scipy(x_fino, k), ls, lw=1.8,
96         label=label)
97     axes[k].set_ylabel(label); axes[k].legend();
98     axes[k].grid(True, alpha=0.4)
99 axes[2].set_xlabel('$x$')
100 plt.suptitle("Spline c bica natural e suas derivadas")
101 plt.tight_layout(); plt.show()

```

Listing 5.4: Spline Cúbica Natural — implementação do zero e via SciPy

Exercícios — Splines Cúbicas

Exercício 5.20. Explique por que a continuidade de S'' é importante em aplicações de engenharia mecânica (em que splines descrevem trajetórias de peças ou superfícies de aeronaves), mas não seria necessária para apenas “conectar os pontos” visualmente.

Exercício 5.21. Construa a spline cúbica natural manualmente para os 3 pontos $(0, 0)$, $(1, 3)$, $(2, 1)$. O sistema tridiagonal neste caso tem apenas uma equação (uma incógnita interna). Calcule M_1 e os coeficientes dos dois pedaços S_0 e S_1 .

Exercício 5.22. Para a spline do Exemplo 5.8, verifique numericamente a continuidade de S' em $x = 2$ (entre S_1 e S_2) e de S'' em $x = 1$ (entre S_0 e S_1).

Exercício 5.23 (Comparação computacional). Use a implementação Python para comparar, nos dados do Exemplo 5.7 com $n = 15$ pontos equiespaçados, a spline cúbica natural e o polinômio de Lagrange. Calcule o erro máximo de cada um. Por que a spline se sai melhor mesmo com pontos equiespaçados?

5.8 Interpolação Inversa

5.8.1 O problema

Na interpolação *direta*, conhecemos x e queremos estimar $y \approx f(x)$. Na *inversa*, conhecemos um valor alvo y^* e queremos encontrar x^* tal que $f(x^*) = y^*$. A incógnita está na **abscissa**.

Definição 5.15 (Interpolação Inversa). Dados $(x_0, y_0), \dots, (x_n, y_n)$ e um alvo y^* , encontrar x^* tal que $P_n(x^*) = y^*$.

Aplicações diretas:

- *Farmacocinética*: em que instante t^* a concentração de um fármaco atinge a dose terapêutica mínima?
- *Engenharia de materiais*: a que temperatura T^* a resistência à tração cai abaixo de um limiar de segurança?
- *Economia*: quando o fluxo de caixa acumulado iguala o investimento (ponto de equilíbrio)?

5.8.2 Dois métodos de solução

Método 1 — Inversa Direta: Se f é **estritamente monótona** em $[x_0, x_n]$, a troca de papéis $(x_k, y_k) \rightarrow (y_k, x_k)$ define uma função unívoca f^{-1} . Basta interpolar a tabela *invertida*:

$$Q(y) = \sum_{k=0}^n x_k L_k^*(y),$$

em que L_k^* são os polinômios de Lagrange construídos sobre os y_k como abscissas.

Nota Importante

Requisito obrigatório: f deve ser estritamente monótona. Se houver máximos ou mínimos locais, f^{-1} não é uma função e o método fornece resultados sem sentido. Sempre verifique a monotonia antes de aplicar a inversa direta.

Método 2 — Iterativo: Constrói-se $P_n(x)$ normalmente e resolve-se a equação não-linear $P_n(x) - y^* = 0$ por bisseção, Newton–Raphson ou `scipy.optimize.brentq`. Funciona mesmo para f não-monótona, mas pode ter múltiplas raízes.

	Inversa Direta	Iterativo
f monótona	✓	✓
f não-monótona	×	✓
Múltiplas raízes	×	✓
Precisa de estimativa inicial	✓	×
Implementação	Muito simples	Moderada

5.8.3 Exemplo resolvido

Exemplo 5.9 (Tempo de Meia-Vida de um Fármaco). A concentração de um antibiótico (em mg/L) é registrada ao longo do tempo:

t (h)	0	1	3	5	8	12
C (mg/L)	400	320	210	140	80	38

Determinar t^* tal que $C(t^*) = 100$ mg/L (metade da concentração inicial de 200 mg/L no estado de equilíbrio).

Passo 1 — Verificar monotonia: C é decrescente em todos os subintervalos. ✓ A inversa direta é válida.

Passo 2 — Localizar $C^* = 100$ na tabela: está entre $C(5) = 140$ e $C(8) = 80$.

Método 1a — Inversa linear (2 pontos):

$$t^* \approx 5 + \frac{(100 - 140)(8 - 5)}{80 - 140} = 5 + \frac{-40 \cdot 3}{-60} = 5 + 2 = 7,0 \text{ h.}$$

Método 1b — Inversa de Newton (4 pontos: (80, 8), (140, 5), (210, 3), (320, 1)):

Tabela de diferenças divididas sobre os C_k como abscissas:

C_k	$t[C_k]$	Ord. 1	Ord. 2	Ord. 3
80	8	$\frac{5 - 8}{140 - 80} = -0,0500$	$\frac{-0,0286 - (-0,0500)}{210 - 80} = +1,65 \times 10^{-4}$	$\approx -5,1 \times 10^{-7}$
140	5	$\frac{3 - 5}{210 - 140} = -0,0286$	$\frac{-0,0182 - (-0,0286)}{320 - 80} = +4,33 \times 10^{-5}$	
210	3	$\frac{1 - 3}{320 - 210} = -0,0182$		
320	1			

Forma de Newton em $C^* = 100$:

$$\begin{aligned} t^* &= 8 + (-0,0500)(100 - 80) + (1,65 \times 10^{-4})(100 - 80)(100 - 140) \\ &\quad + (-5,1 \times 10^{-7})(100 - 80)(100 - 140)(100 - 210) \\ &= 8 - 1,000 - 0,132 + 0,045 \approx \mathbf{6,91 \text{ h}}. \end{aligned}$$

Comparação: inversa linear: 7,0 h; inversa Newton (grau 3): 6,91 h. Para o modelo exponencial $C(t) = 400 e^{-0,307t}$, o valor exato é $t^* = \ln(400/100)/0,307 \approx 6,96 \text{ h}$.

Exemplo 5.10 (Temperatura de Falha de Material — Método Iterativo). A resistência à tração de um polímero (em MPa) decresce com a temperatura, mas não monotonamente (há um pico em torno de 60°C devido à cristalização):

T (°C)	20	60	100	140	200
σ (MPa)	45	72	55	30	10

Encontrar T^* tal que $\sigma(T^*) = 40 \text{ MPa}$ (limiar de segurança).

f **não é monótona** (tem máximo em $T \approx 60^\circ\text{C}$), portanto a inversa direta é **inválida**. Há duas raízes: uma para $T < 60$ e outra para $T > 60$.

Método iterativo: Construir $P_4(T)$ (Lagrange, grau 4) e resolver $P_4(T) - 40 = 0$ por bisseção.

Raiz em $[20, 60]$ (por inspeção, $P_4(20) = 45 > 40$ e $P_4(60) = 72 > 40$, então deve estar fora... verificamos $P_4(30) \approx 40,5$ e $P_4(35) \approx 38$, logo raiz em $[30, 35]$): após convergência, $T_1^* \approx 31,8^\circ\text{C}$.

Raiz em $[100, 140]$ (em que σ cruza 40): $P_4(100) = 55 > 40$ e $P_4(140) = 30 < 40$, raiz em $[100, 140]$; após bisseção, $T_2^* \approx 127,5^\circ\text{C}$.

Decisão de engenharia: o material opera com segurança para $T \lesssim 32^\circ\text{C}$ ou em $T \approx 128^\circ\text{C}$. Há uma janela de operação perigosa entre 32°C e 128°C (exceto próximo ao máximo) — informação que só aparece com o método iterativo, pois a inversa direta não detectaria a não-monotonicidade.

5.8.4 Implementação

```

1 import numpy as np
2 from scipy.optimize import brentq
3
4 #           Método 1: Inversa Direta (Newton invertido)
5
6 def inversa_direta(xk, yk, y_alvo, n_pts=4):

```

```

6      """
7      Interpolacao inversa por Newton com tabela invertida.
8      Requer que yk seja estritamente monotona.
9      n_pts: numero de pontos a usar (centrado em y_alvo).
10     """
11     yk = np.asarray(yk, dtype=float)
12     xk = np.asarray(xk, dtype=float)
13
14     # Ordenar por yk crescente (se decrescente, inverter)
15     if yk[0] > yk[-1]:
16         yk, xk = yk[::-1], xk[::-1]
17
18     # Selecionar n_pts mais proximos de y_alvo
19     idx = np.searchsorted(yk, y_alvo)
20     lo = max(0, idx - n_pts//2)
21     hi = min(len(yk), lo + n_pts)
22     lo = max(0, hi - n_pts)
23
24     # Newton com (yk[lo:hi], xk[lo:hi])
25     return newton(yk[lo:hi], xk[lo:hi], y_alvo)
26
27 #         Metodo 2: Iterativo (bissecacao via scipy)
28
29 def inversa_iterativa(xk, yk, y_alvo, x_lo, x_hi):
30     """
31     Resolve  $P_n(x) = y_{alvo}$  por bissecacao de Brent.
32     x_lo, x_hi: intervalo onde a raiz esta.
33     """
34     from scipy.interpolate import lagrange
35     P = lagrange(xk, yk)
36     return brentq(lambda x: P(x) - y_alvo, x_lo, x_hi)
37
38 #         Exemplo: farmacocinetica
39
40 t_dados = np.array([0., 1., 3., 5., 8., 12.])
41 C_dados = np.array([400, 320, 210, 140, 80, 38], dtype=float)
42 C_alvo = 100.0
43
44 t_inv_lin = inversa_direta(t_dados, C_dados, C_alvo, n_pts=2)
45 t_inv_newton = inversa_direta(t_dados, C_dados, C_alvo, n_pts=4)

```

```

44 t_iter      = inversa_iterativa(t_dados, C_dados, C_alvo, 5.,
    8.)
45
46 print(f"Inversa linear (2 pts):      t* = {t_inv_lin:.3f} h")
47 print(f"Inversa Newton (4 pts):     t* = {t_inv_newton:.3f} h")
48 print(f"Iterativo (Brent):          t* = {t_iter:.3f} h")
49 print(f"Modelo exp (referencia):    t*      6.960 h")

```

Listing 5.5: Interpolação Inversa: direta e iterativa

Exercícios — Interpolação Inversa

Exercício 5.24. A viscosidade cinemática do óleo SAE 30 varia com a temperatura:

T (°C)	40	60	80	100	120
ν (mm ² /s)	100	40	18	9,5	5,5

- Verifique a monotonia e aplique a inversa linear para estimar T^* tal que $\nu(T^*) = 25$ mm²/s.
- Aplique a inversa de Newton com 3 pontos. Compare os resultados.
- Use o método iterativo com bisseção e compare com os anteriores.

Exercício 5.25 (Discussão). Suponha que você aplique a inversa direta a um conjunto de dados que parece monótono nos dados disponíveis, mas na realidade tem um máximo suave não observado. Que tipo de erro isso produziria? Como você poderia detectar esse problema computacionalmente?

5.9 Quadro Comparativo dos Métodos

Método	Grau	Custo (avaliação)	Novo ponto	Suavidade	Runge?
Lagrange	n	$\mathcal{O}(n^2)$	Recalcular tudo	Global (C^∞)	Sim
Newton (DD)	n	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	Global (C^∞)	Sim
Chebyshev	n	$\mathcal{O}(n^2)$	Recalcular tudo	Global (C^∞)	Não
Spline cúbica	3/int	$\mathcal{O}(n)$	Reconstruir	C^2	Não

Nota Importante**Recomendação prática:**

- Poucos pontos ($n \leq 6$) com função suave e diferenciável: **Newton** com diferenças divididas.
- Muitos pontos ou dados de experimento: **spline cúbica**.
- Pontos controlados (podem ser escolhidos) e n moderado: **Chebyshev**.
- Dados chegam incrementalmente: **Newton** (coeficientes atualizados em $\mathcal{O}(n)$).
- Interpolação inversa com f monótona: **inversa direta**; caso contrário, **bisseção** sobre o interpolador.

5.10 Lista de Exercícios de Revisão do Capítulo

Nível 1 — Consolidação

Exercício 5.26 (R1). Para os dados abaixo, construa o interpolador polinomial de grau ≤ 3 pelas formas de Lagrange e de Newton. Verifique que os dois dão o mesmo polinômio e confirme o Corolário 5.3.

x	-1	0	1	2
$f(x)$	4	1	2	11

Exercício 5.27 (R2). Para $f(x) = \cos x$ nos nós equiespaçados $x = 0, \pi/4, \pi/2, 3\pi/4, \pi$:

- Construa a tabela de diferenças divididas.
- Escreva $P_4(x)$ em forma de Newton.
- Estime $\cos(\pi/6)$ e compare com o valor exato.
- Use a fórmula do erro para obter uma cota para o erro em $x = \pi/6$.

Exercício 5.28 (R3). Para os dados abaixo, construa a spline cúbica natural e calcule $S(3, 5)$:

x	1	2	3	4	5
$f(x)$	0,5	1,5	1,0	2,0	1,5

Compare com a interpolação de Newton no mesmo ponto.

Exercício 5.29 (R4). A pressão de vapor do etanol (em kPa) foi medida em função da temperatura:

T (°C)	20	30	40	50	60	70
P (kPa)	5,95	10,53	17,88	29,44	46,97	72,92

Determine por interpolação inversa (método de Newton) a temperatura T^* em que $P(T^*) = 25$ kPa.

Nível 2 — Análise e Implementação

Exercício 5.30 (R5). Implemente em Python as quatro formas de interpolação estudadas (Lagrange, Newton, Chebyshev, spline) e aplique-as a $f(x) = |x - 1|$ em $[0, 3]$ com $n = 5, 10, 15$ nós. Compare graficamente e calcule o erro máximo de cada método. Por que essa função é “difícil” para polinômios globais?

Exercício 5.31 (R6). Demonstre algebricamente que a diferença dividida de ordem 1 da função $f(x) = x^2$ é $f[x_i, x_{i+1}] = x_i + x_{i+1}$, e que a de ordem 2 é $f[x_i, x_{i+1}, x_{i+2}] = 1$ para qualquer escolha de nós. Generalize: qual é a diferença dividida de ordem k de $f(x) = x^m$?

Exercício 5.32 (R7). Para a função $f(x) = e^{-x^2}$ em $[-3, 3]$, determine experimentalmente (computacionalmente) o menor n para o qual a interpolação com nós de Chebyshev atinge erro máximo $< 10^{-4}$. Compare com o n necessário para nós equiespaçados (ou indique se equiespaçados nunca atingem esse erro nesse intervalo).

Nível 3 — Integração e Projetos

Exercício 5.33 (R8 — Projeto). Reamostragem de sinal de áudio. Gere um sinal sintético com três frequências:

$$f(t) = \sin(2\pi \cdot 440 t) + 0,5 \sin(2\pi \cdot 880 t) + 0,25 \sin(2\pi \cdot 1320 t)$$

amostrado a 8 kHz (8000 amostras/s). Reamostramos para 16 kHz usando:

(a) Interpolação linear (vizinhos adjacentes).

(b) Spline cúbica.

Para cada método, calcule o SNR (Signal-to-Noise Ratio): $\text{SNR} = 20 \log_{10}(\|f_{\text{exata}}\|/\|f_{\text{interp}} - f_{\text{exata}}\|)$ em dB. Qual método preserva melhor as altas frequências?

Exercício 5.34 (R9 — Teórico). Prove que a diferença dividida de ordem n satisfaz:

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}$$

para algum ξ no intervalo $(\min_i x_i, \max_i x_i)$, quando $f \in C^n$. Relacione este resultado com a fórmula do erro (Teorema 5.9).

Exercício 5.35 (R10 — Projeto Integrador). Interpolação em Computação Gráfica. Dado um conjunto de pontos de controle em 2D (uma trajetória de câmera, por exemplo): $(0, 0)$, $(1, 2)$, $(3, 3)$, $(5, 1)$, $(7, 4)$, $(8, 2)$:

- Interpole $x(t)$ e $y(t)$ separadamente como funções de um parâmetro $t = 0, 1, \dots, 5$ usando: (i) Lagrange; (ii) Newton; (iii) spline cúbica natural; (iv) spline cúbica clamped com $S'(0) = S'(5) = 0$.
- Plote as quatro curvas no plano xy . Qual parece mais “natural” para o movimento de câmera?
- Calcule a curvatura $\kappa(t) = |x'y'' - y'x''|/((x')^2 + (y')^2)^{3/2}$ ao longo de cada trajetória. Variações bruscas de curvatura indicam movimento não-natural.
- Discuta a escolha do método com base nos critérios de suavidade, controle artístico e custo computacional.

5.11 Referências e Leituras Complementares

- Burden, R. L., & Faires, J. D. (2015). *Numerical Analysis*. 10th edition. Cengage. Capítulo 3: Interpolation and Polynomial Approximation.
- Ruggiero, M. A. G., & Lopes, V. L. R. (1996). *Cálculo Numérico: Aspectos Teóricos e Computacionais*. 2.^a ed. Pearson. Capítulo 6.
- de Boor, C. (2001). *A Practical Guide to Splines*. Revised edition. Springer. (A referência definitiva sobre splines, por seu criador.)
- Farin, G. (2002). *Curves and Surfaces for CAGD*. 5th edition. Morgan Kaufmann. (Aplicações em computação gráfica.)
- Trefethen, L. N. (2013). *Approximation Theory and Approximation Practice*. SIAM. (Abordagem moderna, com foco em Chebyshev e convergência espectral.)
- Franco, N. B. (2007). *Cálculo Numérico*. Pearson.

- SciPy Interpolate: <https://docs.scipy.org/doc/scipy/reference/interpolate.html>
- Runge, C. (1901). “Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten.” *Zeitschrift für Mathematik und Physik*, 46, 224–243. (Artigo original do fenômeno.)

Capítulo 6

Ajuste de Curvas — Método dos Mínimos Quadrados

“Dados sem um modelo são apenas barulho.”

— Anônimo

6.1 Introdução e Motivação

Em contraste com a interpolação (Capítulo 5), onde a curva deve passar *exatamente* pelos pontos, o problema de **ajuste de curvas** emerge quando os dados provêm de experimentos sujeitos a ruído, erros de medição ou variabilidade inerente ao fenômeno. Nesse contexto, forçar a curva a passar por todos os pontos seria um erro: estaria-se ajustando o ruído, não o sinal.

Definição 6.1 (Problema de Ajuste de Curvas). Dados n pares experimentais $(x_1, y_1), \dots, (x_n, y_n)$, encontrar uma função:

$$u(x) = a_1 g_1(x) + a_2 g_2(x) + \dots + a_m g_m(x), \quad m \ll n,$$

que **melhor represente** a tendência dos dados, admitindo que $u(x_k) \neq y_k$ em geral.

Nota Importante

Interpolação vs. Ajuste de Curvas:

- **Interpolação:** dados exatos (ou confiáveis), poucos pontos, curva passa exatamente pelos dados. Recomendado quando n é pequeno e os dados não têm ruído.
- **Ajuste:** dados experimentais com ruído, $n \gg m$, curva aproxima a tendência sem passar necessariamente por nenhum ponto. Recomendado quando se quer identificar uma lei física ou extrapolar.

6.1.1 Diagrama de dispersão: o ponto de partida

Antes de qualquer cálculo, plota-se o **diagrama de dispersão** dos dados para identificar visualmente a estrutura do fenômeno. O padrão observado orienta a escolha do modelo matemático.

Padrão visual	Modelo candidato	Exemplo de aplicação
Nuvem linear	$u = a_0 + a_1x$	Lei de Hooke, lei de Ohm
Parábola / curvatura suave	$u = a_0 + a_1x + a_2x^2$	Resistência ao vento, trajetórias
Crescimento acelerado	$u = ae^{bx}$	Crescimento populacional, radioatividade
Crescimento com saturação	$u = a + b \ln x$	Crescimento bacteriano, aprendizado
Lei de potência	$u = ax^b$	Leis de escala em física e biologia
Curva hiperbólica	$u = a/(b + x)$	Cinética enzimática (Michaelis-Menten)

Exemplo 6.1 (Identificação do modelo pelo diagrama). Os dados abaixo registram a corrente elétrica em um componente eletrônico experimental em função da tensão aplicada:

V (V)	0,5	1,0	2,0	3,0	5,0	8,0
I (mA)	0,11	0,31	0,90	1,65	4,60	11,80

Ao plotar I vs. V e $\ln I$ vs. $\ln V$, observa-se que o segundo gráfico é aproximadamente linear, sugerindo $I = aV^b$ (lei de potência). O diagrama de dispersão guia toda a estratégia de ajuste.

Aplicação Computacional**Aprendizado de Máquina — Regressão Linear como Base:**

A regressão por mínimos quadrados é o fundamento histórico e teórico da maioria dos algoritmos de aprendizado supervisionado. A função de custo do modelo linear simples, a classe de modelos de regressão regularizada (Ridge, Lasso) e até as camadas densas de redes neurais resolvem, em essência, variantes do problema de mínimos quadrados.

Em `scikit-learn`, `LinearRegression().fit(X, y)` resolve exatamente as equações normais $A^T A a = A^T y$ estudadas neste capítulo. Compreender a derivação dessas equações é o alicerce para entender por que ℓ_2 -regularização (Ridge) adiciona λI à matriz $A^T A$, tornando-a sempre inversível.

Aplicação Computacional**Epidemiologia Computacional — Modelagem de Surtos:**

Durante surtos infecciosos, epidemiologistas ajustam modelos exponenciais $C(t) = C_0 e^{rt}$ às séries temporais de casos confirmados para estimar a taxa de crescimento r e o tempo de duplicação $T_2 = \ln 2/r$. Com dados ruidosos (subnotificação, atrasos de testagem), o ajuste por mínimos quadrados sobre $\ln C$ fornece estimativas mais estáveis do que diferenças finitas diretas.

O mesmo princípio aplica-se ao decaimento: ajustar $\ln C$ vs. t para estimar a taxa de declínio de casos após o pico.

Aplicação Computacional**Metrologia e Calibração de Sensores:**

Sensores físicos (termistores, strain gauges, sensores de pressão) raramente têm resposta perfeitamente linear. Na calibração, aplica-se uma sequência de estímulos conhecidos e mede-se a saída. O ajuste polinomial ou exponencial por mínimos quadrados gera a **curva de calibração**, que depois é usada para converter leituras brutas em grandezas físicas. O coeficiente r^2 quantifica a qualidade da calibração.

Exercícios — Introdução

Exercício 6.1. Classifique cada situação como problema de **interpolação** ou de **ajuste de curvas**, justificando sua escolha: (a) Construir uma tabela de senos para ângulos de 0° a 90° com passo de 1° a partir de valores exatos conhecidos. (b) Determinar a lei de resfriamento de um corpo metálico a partir de 40 medidas de temperatura com erro de $\pm 0,5^\circ\text{C}$. (c) Estimar o PIB per capita de um país em 2027 a partir de dados anuais de

2010 a 2024 (com arredondamentos).

Exercício 6.2. Para cada conjunto de dados abaixo, esboce o diagrama de dispersão e proponha o modelo matemático mais adequado, justificando com base no padrão visual:

$$(a) \quad \begin{array}{c|ccccc} x & 1 & 2 & 4 & 8 & 16 \\ \hline y & 1,2 & 1,9 & 3,1 & 5,0 & 8,1 \end{array}$$

$$(b) \quad \begin{array}{c|cccc} x & 0 & 1 & 2 & 3 \\ \hline y & 5,0 & 1,8 & 0,7 & 0,2 \end{array}$$

6.2 Método dos Mínimos Quadrados

6.2.1 Por que minimizar a soma dos quadrados?

O **resíduo** da observação k é $e_k = y_k - u(x_k)$, a diferença entre o valor medido e o valor previsto pelo modelo. Há várias formas de medir o erro global:

Critério	Expressão	Problema
Soma simples	$\sum e_k$	Resíduos positivos e negativos se cancelam; solução não única
Soma de valores absolutos	$\sum e_k $	Não diferenciável; múltiplas soluções
Soma de quadrados	$\sum e_k^2$	Diferenciável, estritamente convexa, solução única

Definição 6.2 (Método dos Mínimos Quadrados). Dados n pares (x_k, y_k) e um modelo $u(x; a_1, \dots, a_m)$ linear nos parâmetros, determinar os coeficientes $a_1, \dots, a_m$ que minimizem:

$$S_r(a_1, \dots, a_m) = \sum_{k=1}^n [y_k - u(x_k)]^2. \quad (6.1)$$

A convexidade estrita de S_r (como função quadrática em a_j) garante que qualquer mínimo local é global e único — desde que os dados não sejam degenerados.

6.2.2 Coeficiente de determinação r^2

Definição 6.3 (Coeficiente de Determinação).

$$r^2 = 1 - \frac{S_r}{S_t}, \quad S_r = \sum_{k=1}^n (y_k - \hat{y}_k)^2, \quad S_t = \sum_{k=1}^n (y_k - \bar{y})^2, \quad (6.2)$$

em que $\hat{y}_k = u(x_k)$ é o valor ajustado e $\bar{y} = \frac{1}{n} \sum y_k$ é a média.

S_t mede a variabilidade total dos dados (quanto y varia). S_r mede a variabilidade *não explicada* pelo modelo. $r^2 \in [0, 1]$ indica a fração da variabilidade total explicada pelo modelo.

Nota Importante

r^2 alto não é condição suficiente para um bom modelo. Dois problemas frequentes:

1. **Sobreajuste:** com $m = n - 1$ parâmetros, $r^2 = 1$ sempre (interpolação), mas o modelo não generaliza.
2. **Padrão nos resíduos:** $r^2 = 0,95$ pode mascarar um modelo errado se os resíduos exibirem padrão sistemático (crescer com x , oscilar, etc.). Analise sempre o **gráfico de resíduos**.

6.2.3 Análise de resíduos

O gráfico de resíduos $e_k = y_k - \hat{y}_k$ versus x_k é a ferramenta mais importante para diagnosticar a qualidade do ajuste:

Padrão observado nos resíduos	Interpretação
Aleatório, média zero, amplitude constante	Modelo adequado
Tendência crescente ou decrescente	Variável omitida ou modelo errado
Forma de U ou arco	Grau insuficiente (falta termo quadrático ou superior)
Amplitude crescente com x	Heterocedasticidade; considerar transformação

Exercícios — Mínimos Quadrados

Exercício 6.3. Mostre que S_r definida em (6.1) é uma função convexa dos coeficientes a_j . Para o modelo linear $u = a_0 + a_1x$, prove que a matriz Hessiana de S_r é positiva semidefinida e determine sob que condição ela é positiva definida.

Exercício 6.4. Para os dados abaixo, calcule S_t , $\bar{y}$ e interprete geometricamente o que $r^2 = 0$ e $r^2 = 1$ significariam para estes dados específicos:

x	1	2	3	4	5
y	3,1	5,0	4,8	7,2	6,9

6.3 Regressão Linear Simples

6.3.1 Derivação das equações normais

Modelo: $u(x) = a_0 + a_1x$. Minimizar $S_r(a_0, a_1) = \sum_{k=1}^n (y_k - a_0 - a_1x_k)^2$.

Condições de primeira ordem ($\nabla S_r = \mathbf{0}$):

$$\frac{\partial S_r}{\partial a_0} = -2 \sum_{k=1}^n (y_k - a_0 - a_1x_k) = 0 \quad \Rightarrow \quad na_0 + a_1 \sum x_k = \sum y_k \quad (6.3)$$

$$\frac{\partial S_r}{\partial a_1} = -2 \sum_{k=1}^n x_k (y_k - a_0 - a_1x_k) = 0 \quad \Rightarrow \quad a_0 \sum x_k + a_1 \sum x_k^2 = \sum x_k y_k \quad (6.4)$$

Teorema 6.4 (Equações Normais da Regressão Linear). Os coeficientes da reta de mínimos quadrados satisfazem o sistema 2×2 :

$$\underbrace{\begin{bmatrix} n & \sum x_k \\ \sum x_k & \sum x_k^2 \end{bmatrix}}_{\text{simétrica, positiva definida}} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \sum y_k \\ \sum x_k y_k \end{bmatrix}. \quad (6.5)$$

O determinante é $\Delta = n \sum x_k^2 - \left(\sum x_k \right)^2 > 0$ sempre que os x_k não forem todos iguais.

Demonstração. S_r é uma função quadrática convexa nos parâmetros. As equações (6.3)–(6.4) são as condições necessárias e suficientes de mínimo (gradiente nulo numa função estritamente convexa). O determinante $\Delta = n \sum x_k^2 - \left(\sum x_k \right)^2$ é positivo pela desigualdade de Cauchy-Schwarz: $(\sum 1 \cdot x_k)^2 \leq n \sum x_k^2$, com igualdade somente se todos os x_k forem iguais. $\square$

6.3.2 Fórmulas explícitas

Resolvendo (6.5) por Cramer:

$$a_1 = \frac{n \sum x_k y_k - \sum x_k \cdot \sum y_k}{n \sum x_k^2 - \left(\sum x_k \right)^2}, \quad a_0 = \bar{y} - a_1 \bar{x}, \quad (6.6)$$

em que $\bar{x} = \frac{1}{n} \sum x_k$ e $\bar{y} = \frac{1}{n} \sum y_k$.

Nota Importante

A reta de mínimos quadrados sempre passa pelo centróide $(\bar{x}, \bar{y})$. Isso decorre diretamente de $a_0 = \bar{y} - a_1 \bar{x}$: substituindo $x = \bar{x}$, obtém-se $u(\bar{x}) = a_0 + a_1 \bar{x} = \bar{y}$. Esta é uma propriedade geométrica importante: a reta “ancora” no ponto médio dos dados.

6.3.3 Exemplo resolvido passo a passo

Exemplo 6.2 (Regressão linear: resistência vs. temperatura). A resistência elétrica de um fio de platina foi medida em diferentes temperaturas. Ajustar uma reta e calcular r^2 :

T (°C)	20	40	60	80	100	120
R (Ω)	77,2	84,5	91,8	98,3	105,7	113,1

Passo 1 — Montar a tabela de somas ($n = 6$):

k	$x_k = T$	$y_k = R$	x_k^2	$x_k y_k$
1	20	77,2	400	1 544,0
2	40	84,5	1 600	3 380,0
3	60	91,8	3 600	5 508,0
4	80	98,3	6 400	7 864,0
5	100	105,7	10 000	10 570,0
6	120	113,1	14 400	13 572,0
Σ	420	570,6	36 400	42 438,0

$$\bar{x} = 420/6 = 70; \quad \bar{y} = 570,6/6 = 95,1.$$

Passo 2 — Calcular os coeficientes:

$$\begin{aligned} \Delta &= 6 \cdot 36\,400 - 420^2 = 218\,400 - 176\,400 = 42\,000. \\ a_1 &= \frac{6 \cdot 42\,438 - 420 \cdot 570,6}{42\,000} = \frac{254\,628 - 239\,652}{42\,000} = \frac{14\,976}{42\,000} \approx 0,3566 \text{ } \Omega/\text{°C}. \\ a_0 &= 95,1 - 0,3566 \cdot 70 = 95,1 - 24,96 \approx 70,14 \text{ } \Omega. \end{aligned}$$

Resultado: $u(T) \approx 70,14 + 0,3566 T$.

Passo 3 — Calcular resíduos e r^2 :

T	R_{obs}	$\hat{R} = u(T)$	$e_k = R - \hat{R}$	e_k^2
20	77,2	77,27	-0,07	0,005
40	84,5	84,40	+0,10	0,010
60	91,8	91,54	+0,26	0,068
80	98,3	98,67	-0,37	0,137
100	105,7	105,80	-0,10	0,010
120	113,1	112,93	+0,17	0,029
			S_r	0,259

$$S_t = \sum (y_k - \bar{y})^2 = (77,2 - 95,1)^2 + \dots + (113,1 - 95,1)^2 \approx 954,0.$$

$$r^2 = 1 - 0,259/954,0 \approx \mathbf{0,9997}.$$

Excelente ajuste: o modelo linear explica 99,97% da variância. Os resíduos são pequenos e sem padrão sistemático, confirmando a adequação do modelo linear para a resistividade da platina.

6.3.4 Implementação

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def regressao_linear(x, y):
5     """
6     Ajusta reta u(x) = a0 + a1*x por minimos quadrados.
7     Retorna (a0, a1, r2, residuos).
8     """
9     x = np.asarray(x, dtype=float)
10    y = np.asarray(y, dtype=float)
11    n = len(x)
12
13    sx = np.sum(x);    sy = np.sum(y)
14    sx2 = np.sum(x**2); sxy = np.sum(x * y)
15
16    delta = n * sx2 - sx**2
17    a1 = (n * sxy - sx * sy) / delta
18    a0 = (sy - a1 * sx) / n
19
20    y_hat = a0 + a1 * x
21    Sr = np.sum((y - y_hat)**2)
22    St = np.sum((y - y.mean())**2)
23    r2 = 1.0 - Sr / St
24
25    return a0, a1, r2, y - y_hat
26
27 #      Exemplo: platina
28
29 T = np.array([20., 40., 60., 80., 100., 120.])
30 R = np.array([77.2, 84.5, 91.8, 98.3, 105.7, 113.1])
31
32 a0, a1, r2, res = regressao_linear(T, R)
```

```

32 print(f"u(T) = {a0:.4f} + {a1:.4f}*T")
33 print(f"r^2 = {r2:.6f}")
34
35 # Estimativa: resistencia a 90 graus
36 T_est = 90.0
37 print(f"u({T_est}) = {a0 + a1*T_est:.3f} Ohm")
38
39 # Grafico
40 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
41
42 x_fino = np.linspace(10, 130, 200)
43 ax1.plot(x_fino, a0 + a1*x_fino, 'b-', label=f'$u = {a0:.2f} + {a1:.4f}T$')
44 ax1.scatter(T, R, color='red', zorder=5, label='Dados')
45 ax1.set_xlabel('Temperatura ( C )');
46 ax1.set_ylabel('Resist ncia ( )')
47 ax1.set_title(f'Regress o linear ($r^2 = {r2:.4f}$)')
48 ax1.legend(); ax1.grid(True, alpha=0.4)
49
50 ax2.axhline(0, color='k', lw=0.8)
51 ax2.scatter(T, res, color='green', zorder=5)
52 ax2.set_xlabel('Temperatura ( C )'); ax2.set_ylabel('Res duo ( )')
53 ax2.set_title('Gr fico de Res duos'); ax2.grid(True, alpha=0.4)
54 plt.tight_layout(); plt.show()

```

Listing 6.1: Regressão Linear Simples

Exercícios — Regressão Linear Simples

Exercício 6.5. Prove que a reta de mínimos quadrados passa sempre pelo centroide $(\bar{x}, \bar{y})$. Use a equação normal (6.3) diretamente.

Exercício 6.6. Ajuste uma reta aos dados abaixo, calcule r^2 e estime y para $x = 25$:

x	5	10	15	20	30	35
y	1,8	3,9	5,1	7,2	10,1	12,5

O modelo linear é adequado? Analise o gráfico de resíduos.

Exercício 6.7. Para o sistema (6.5), mostre que $\Delta = n \sum x_k^2 - (\sum x_k)^2 > 0$ usando a desigualdade de Cauchy-Schwarz aplicada aos vetores $\mathbf{1} = (1, \dots, 1)$ e $\mathbf{x} = (x_1, \dots, x_n)$.

Exercício 6.8 (Análise). Para os dados: $x = (1, 2, 3, 4)$, $y = (2, 3, 2, 3)$, calcule os coeficientes da reta de mínimos quadrados. A reta passa por algum dos pontos? Calcule r^2 . O que o valor de r^2 indica sobre a adequação do modelo linear aqui?

6.4 Regressão Polinomial

6.4.1 Equações normais de grau m

Modelo: $u(x) = a_0 + a_1x + \cdots + a_mx^m$ com $m \ll n$.

Minimizando $S_r(a_0, \dots, a_m)$ e impondo $\partial S_r / \partial a_j = 0$ para $j = 0, \dots, m$:

Teorema 6.5 (Equações Normais Polinomiais). Os coeficientes do ajuste polinomial de grau m satisfazem o sistema $(m+1) \times (m+1)$:

$$\underbrace{\begin{bmatrix} n & \sum x_k & \cdots & \sum x_k^m \\ \sum x_k & \sum x_k^2 & \cdots & \sum x_k^{m+1} \\ \vdots & & \ddots & \vdots \\ \sum x_k^m & \sum x_k^{m+1} & \cdots & \sum x_k^{2m} \end{bmatrix}}_{\text{Matriz de Gram, ordem } (m+1)} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} \sum y_k \\ \sum x_k y_k \\ \vdots \\ \sum x_k^m y_k \end{bmatrix}. \quad (6.7)$$

A matriz de Gram é simétrica e positiva definida (para dados não degenerados).

Nota Importante

Para $m = n - 1$, o sistema (6.7) tem exatamente n equações e n incógnitas, e a solução satisfaz $u(x_k) = y_k$ para todo k : o ajuste se reduz à **interpolação**, com $S_r = 0$. Isso reforça que interpolação e ajuste são casos especiais de um mesmo framework.

6.4.2 Princípio da Parcimônia

Definição 6.6 (Princípio da Parcimônia). Entre modelos com desempenho similar (valores de r^2 próximos), sempre prefira o **modelo de menor grau**. Modelos desnecessariamente complexos produzem **sobreajuste**: ajustam perfeitamente os dados disponíveis, mas generalizam mal para novos dados.

Para identificar o grau adequado, plota-se S_r em função de m e procura-se o “joelho” da curva: o ponto a partir do qual adicionar grau não reduz S_r significativamente.

6.4.3 Exemplo resolvido passo a passo

Exemplo 6.3 (Ajuste polinomial: arrasto aerodinâmico). A força de arrasto F (em N) em um perfil aerodinâmico foi medida em função do ângulo de ataque α (em graus):

α	0	4	8	12	16	20
F	0,12	0,30	0,75	1,80	3,85	7,20

Tentativa 1 — Ajuste linear ($m = 1$):

$$\sum x_k = 60, \sum y_k = 14,02, \sum x_k^2 = 800, \sum x_k y_k = 219,2.$$

$$\Delta = 6 \cdot 800 - 60^2 = 1200; \quad a_1 = (6 \cdot 219,2 - 60 \cdot 14,02)/1200 = (1315,2 - 841,2)/1200 = 0,395; \quad a_0 = (14,02 - 0,395 \cdot 60)/6 = -1,614.$$

$\hat{F}_{lin} = -1,614 + 0,395 \alpha$. Resíduos: oscilam sistematicamente (parabólicos) — $r^2 \approx 0,961$ mas o modelo está errado.

Tentativa 2 — Ajuste quadrático ($m = 2$):

$$\text{Somas necessárias adicionais: } \sum x_k^3 = 12\,960, \sum x_k^4 = 212\,480, \sum x_k^2 y_k = 3\,694,4.$$

Sistema 3×3 :

$$\begin{bmatrix} 6 & 60 & 800 \\ 60 & 800 & 12\,960 \\ 800 & 12\,960 & 212\,480 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 14,02 \\ 219,2 \\ 3\,694,4 \end{bmatrix}.$$

Resolvendo (por eliminação gaussiana): $a_0 \approx 0,111$, $a_1 \approx -0,057$, $a_2 \approx 0,0172$.

$$\hat{F}_{quad} \approx 0,111 - 0,057 \alpha + 0,0172 \alpha^2.$$

Verificação e comparação:

α	F_{obs}	$\hat{F}_{lin}$	e_{lin}	$\hat{F}_{quad}$	e_{quad}
0	0,12	-1,614	+1,734	0,111	+0,009
4	0,30	-0,034	+0,334	0,197	+0,103
8	0,75	+1,546	-0,796	0,833	-0,083
12	1,80	+3,126	-1,326	2,027	-0,227
16	3,85	+4,706	-0,856	3,779	+0,071
20	7,20	+6,286	+0,914	6,089	+1,111
		S_r	6,764	S_r	1,408

$$r_{lin}^2 \approx 0,961; \quad r_{quad}^2 \approx 0,992.$$

O ajuste quadrático é significativamente melhor: S_r caiu de 6,76 para 1,41 (fator 5) e os resíduos não têm mais padrão claro. Para $\alpha = 18^\circ$: $\hat{F}(18) = 0,111 - 0,057 \cdot 18 + 0,0172 \cdot 324 \approx 4,78$ N.

6.4.4 Implementação

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def regressao_polynomial(x, y, grau):
5     """
6     Ajusta polinomio de 'grau' m por minimos quadrados.
7     Retorna (coefs, r2, residuos).
8     coefs: [a_m, ..., a_1, a_0] (convencao numpy.polyfit)
9     """
10    x = np.asarray(x, dtype=float)
11    y = np.asarray(y, dtype=float)
12    coefs = np.polyfit(x, y, grau)
13    y_hat = np.polyval(coefs, x)
14    Sr = np.sum((y - y_hat)**2)
15    St = np.sum((y - y.mean())**2)
16    return coefs, 1.0 - Sr/St, y - y_hat
17
18 #      Exemplo: arrasto aerodinamico
19
20 alpha = np.array([0., 4., 8., 12., 16., 20.])
21 F = np.array([0.12, 0.30, 0.75, 1.80, 3.85, 7.20])
22
23 print("Comparacao de graus:")
24 print(f"{'Grau':>5} {'r^2':>10} {'Sr':>10}")
25 Sr_list = []
26 for m in range(1, 5):
27     coefs, r2, res = regressao_polynomial(alpha, F, m)
28     Sr_m = np.sum(res**2)
29     Sr_list.append(Sr_m)
30     print(f"{m:>5} {r2:>10.6f} {Sr_m:>10.4f}")
31
32 # Curva "joelho"
33 fig, axes = plt.subplots(1, 3, figsize=(13, 4))
34
35 # Grafico 1: dados + ajuste linear e quadratico
36 x_f = np.linspace(-1, 21, 200)
37 for m, cor, ls, lbl in [(1, 'r', '--', 'linear'),
38                         (2, 'b', '-', 'quadratico')]:

```

```

37     c, r2, _ = regressao_polinomial(alpha, F, m)
38     axes[0].plot(x_f, np.polyval(c, x_f), color=cor, ls=ls,
39                 label=f'grau {m}, $r^2={r2:.4f}$')
40 axes[0].scatter(alpha, F, color='k', zorder=5, label='Dados')
41 axes[0].set_xlabel('    ( )'); axes[0].set_ylabel('F (N)')
42 axes[0].set_title('Ajuste polinomial');
43     axes[0].legend(fontsize=8)
44 axes[0].grid(True, alpha=0.4)
45
46 # Grafico 2: residuos (quadratico)
47 _, _, res2 = regressao_polinomial(alpha, F, 2)
48 axes[1].axhline(0, color='k', lw=0.8)
49 axes[1].scatter(alpha, res2, color='blue', zorder=5)
50 axes[1].set_xlabel('    ( )'); axes[1].set_ylabel('Res duo')
51 axes[1].set_title('Res duos (grau 2)'); axes[1].grid(True,
52                 alpha=0.4)
53
54 # Grafico 3: Sr vs grau (joelho)
55 axes[2].plot(range(1,5), Sr_list, 'o-', color='purple')
56 axes[2].set_xlabel('Grau m'); axes[2].set_ylabel('$S_r$')
57 axes[2].set_title('Joelho: Sr vs. grau'); axes[2].grid(True,
58                 alpha=0.4)
59 plt.tight_layout(); plt.show()

```

Listing 6.2: Regressão Polinomial e Seleção de Grau

Exercícios — Regressão Polinomial

Exercício 6.9. Para os dados abaixo, ajuste modelos de grau 1 e grau 2. Calcule r^2 para cada um e decida qual grau é mais adequado segundo o princípio da parcimônia:

x	0	1	2	3	4	5	6
y	3,0	4,8	8,1	13,5	20,2	29,5	41,3

Exercício 6.10. Mostre que para $m = 1$ (reta), a matriz de Gram do Teorema 6.5 se reduz exatamente ao sistema (6.5). Mostre também que para $m = n - 1$ (grau máximo), $S_r = 0$.

Exercício 6.11 (Análise). Por que aumentar o grau do polinômio de ajuste sempre reduz S_r , mas não necessariamente melhora o modelo? Explique em termos de sobreajuste e dê um exemplo concreto com os dados do Exercício anterior.

Exercício 6.12 (Implementação). Para os dados de temperatura de uma chapa metálica aquecida (slides, seção de revisão), use Python para ajustar modelos de grau 1, 2 e 3. Plote S_r vs. grau, identifique o joelho e justifique a escolha. Calcule os resíduos para cada grau e compare seus gráficos.

6.5 Regressão Múltipla

6.5.1 Modelo e formulação matricial

Quando a resposta y depende de múltiplas variáveis explicativas $x_1, x_2, \dots, x_p$, o modelo linear múltiplo é:

$$y = a_0 + a_1x_1 + a_2x_2 + \dots + a_px_p + \varepsilon. \quad (6.8)$$

Para n observações, organizamos os dados na **matriz de projeto** $A \in \mathbb{R}^{n \times (p+1)}$:

$$A = \begin{bmatrix} 1 & x_{11} & x_{21} & \dots & x_{p1} \\ 1 & x_{12} & x_{22} & \dots & x_{p2} \\ \vdots & & & & \vdots \\ 1 & x_{1n} & x_{2n} & \dots & x_{pn} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \mathbf{a} = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_p \end{bmatrix}. \quad (6.9)$$

O vetor de resíduos é $\mathbf{e} = \mathbf{y} - A\mathbf{a}$ e a soma dos quadrados é $S_r = \|\mathbf{e}\|^2 = \|\mathbf{y} - A\mathbf{a}\|^2$.

Teorema 6.7 (Sistema Normal da Regressão Múltipla). Os coeficientes de mínimos quadrados satisfazem:

$$A^T A \mathbf{a} = A^T \mathbf{y}. \quad (6.10)$$

A matriz $A^T A$ é simétrica e positiva definida quando as colunas de A são linearmente independentes.

Demonstração. Minimizar $S_r(\mathbf{a}) = \|\mathbf{y} - A\mathbf{a}\|^2 = (\mathbf{y} - A\mathbf{a})^T(\mathbf{y} - A\mathbf{a})$. Expandindo: $S_r = \mathbf{y}^T \mathbf{y} - 2\mathbf{a}^T A^T \mathbf{y} + \mathbf{a}^T A^T A \mathbf{a}$. Tomando o gradiente em relação a $\mathbf{a}$ e igualando a zero: $\nabla_{\mathbf{a}} S_r = -2A^T \mathbf{y} + 2A^T A \mathbf{a} = \mathbf{0}$, que é exatamente (6.10). $\square$

Nota Importante

A regressão múltipla unifica todos os casos anteriores:

- **Regressão linear:** $p = 1$, $A = [1 \ x_1 \ \dots \ x_n]^T$ (coluna de uns + coluna de x_k).
- **Regressão polinomial:** $x_j = x^j$, colunas de potências.
- **Modelos linearizáveis:** substituir as variáveis transformadas na matriz A .

6.5.2 Escala das variáveis

Variáveis em escalas muito diferentes (e.g., temperatura em Kelvin vs. pressão em Pa) podem causar mal-condicionamento de $A^T A$. A **padronização** $z_j = (x_j - \bar{x}_j)/\sigma_{x_j}$ reduz esse efeito, mas altera a interpretação dos coeficientes. O ajuste final (valores de $\hat{y}$) é invariante à escala; apenas os coeficientes mudam.

6.5.3 Exemplo resolvido passo a passo

Exemplo 6.4 (Regressão múltipla: consumo de energia). O consumo de energia elétrica (kWh/mês) de seis apartamentos foi registrado junto com duas variáveis: área útil x_1 (m²) e número de moradores x_2 :

k	x_1 (m ²)	x_2 (mor.)	y (kWh)
1	45	2	180
2	60	3	240
3	80	4	310
4	55	1	200
5	90	5	380
6	70	3	270

Passo 1 — Montar a matriz de projeto:

$$A = \begin{bmatrix} 1 & 45 & 2 \\ 1 & 60 & 3 \\ 1 & 80 & 4 \\ 1 & 55 & 1 \\ 1 & 90 & 5 \\ 1 & 70 & 3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 180 \\ 240 \\ 310 \\ 200 \\ 380 \\ 270 \end{bmatrix}.$$

Passo 2 — Calcular $A^T A$ e $A^T \mathbf{y}$:

$$A^T A = \begin{bmatrix} 6 & 400 & 18 \\ 400 & 27\,850 & 1\,240 \\ 18 & 1\,240 & 64 \end{bmatrix}, \quad A^T \mathbf{y} = \begin{bmatrix} 1\,580 \\ 107\,900 \\ 4\,580 \end{bmatrix}.$$

Passo 3 — Resolver o sistema 3×3 (por eliminação gaussiana ou NumPy):

$$a_0 \approx 10,4, \quad a_1 \approx 2,83, \quad a_2 \approx 28,7.$$

Modelo: $\hat{y} = 10,4 + 2,83x_1 + 28,7x_2$.

Interpretação: cada metro quadrado adicional corresponde a cerca de 2,83 kWh a

mais por mês; cada morador adicional, a cerca de 28,7 kWh a mais.

Verificação:

k	y_k	$\hat{y}_k$	e_k^2
1	180	184,7	22,1
2	240	240,5	0,3
3	310	307,8	4,8
4	200	203,8	14,4
5	380	374,6	29,2
6	270	268,2	3,2
		S_r	74,0

$S_t = \sum (y_k - \bar{y})^2 = \sum (y_k - 263,3)^2 \approx 26\,666,7$. $r^2 = 1 - 74,0/26\,666,7 \approx 0,997$. Excelente ajuste.

6.5.4 Implementação

```

1 import numpy as np
2
3 def regressao_multipla(X_vars, y):
4     """
5     Regressao multipla: y = a0 + a1*x1 + ... + ap*xp.
6     X_vars: array (n, p) -- variaveis explicativas
7     y: array (n,) -- variavel resposta
8     Retorna (coefs, r2, residuos).
9     coefs = [a0, a1, ..., ap]
10    """
11    X_vars = np.asarray(X_vars, dtype=float)
12    y = np.asarray(y, dtype=float)
13    n = len(y)
14    # Matriz de projeto: adicionar coluna de 1s
15    A = np.column_stack([np.ones(n), X_vars])
16    # Sistema normal: A^T A a = A^T y
17    AtA = A.T @ A
18    Aty = A.T @ y
19    a = np.linalg.solve(AtA, Aty)
20    y_hat = A @ a
21    Sr = np.sum((y - y_hat)**2)

```

```

22     St = np.sum((y - y.mean())**2)
23     return a, 1.0 - Sr/St, y - y_hat
24
25 #         Exemplo: consumo de energia
26
27 x1 = np.array([45., 60., 80., 55., 90., 70.]) # area
28 x2 = np.array([ 2.,  3.,  4.,  1.,  5.,  3.]) # moradores
29 y  = np.array([180., 240., 310., 200., 380., 270.])
30
31 a, r2, res = regressao_multipla(np.column_stack([x1, x2]), y)
32 print(f"Modelo: y = {a[0]:.2f} + {a[1]:.3f}*x1 + {a[2]:.3f}*x2")
33 print(f"r^2 = {r2:.4f}")
34 print(f"Residuos: {res}")
35
36 # Predicao para novo apartamento: 75 m^2, 4 moradores
37 x_novo = np.array([1., 75., 4.])
38 print(f"\nPredicao (75 m^2, 4 mor.): {x_novo @ a:.1f} kWh")
39
40 # Verificar com numpy.linalg.lstsq (solucao por SVD, mais
41     estavel)
42 A = np.column_stack([np.ones(len(y)), x1, x2])
43 a_lstsq, _, _, _ = np.linalg.lstsq(A, y, rcond=None)
44 print(f"Numpy lstsq: {a_lstsq}")

```

Listing 6.3: Regressão Múltipla via Sistema Normal

Exercícios — Regressão Múltipla

Exercício 6.13. Monte a matriz de projeto A e o sistema normal $A^T A \mathbf{a} = A^T \mathbf{y}$ para os dados abaixo. Resolva e calcule $\sum e_k^2$:

k	x_1	x_2	y
1	2	1	5,3
2	3	2	8,1
3	5	1	10,7
4	6	3	14,2
5	8	2	17,5

Exercício 6.14 (Análise). O Exemplo 6.4 foi ajustado com x_1 em m^2 e x_2 em número de moradores. Repita o ajuste convertendo x_1 para dm^2 ($1 \text{ m}^2 = 100 \text{ dm}^2$). Os valores de $\hat{y}_k$ mudam? Os coeficientes a_1 e a_2 mudam? Discuta.

Exercício 6.15 (Demonstração). Prove que o sistema normal $A^T A \mathbf{a} = A^T \mathbf{y}$ do Teorema 6.7 generaliza as equações normais lineares (6.5) e as polinomiais (6.7). Mostre explicitamente a forma de A em cada caso.

6.6 Linearização de Modelos Não-Lineares

6.6.1 Estratégia geral

Vários modelos não-lineares admitem **transformação algébrica** que os converte em formas lineares nos parâmetros. Aplica-se então a regressão linear de mínimos quadrados ao espaço transformado e, ao final, reverte-se a transformação para recuperar os parâmetros originais.

Modelo original	Transformação	Forma linear
$y = ae^{bx}$	$\tilde{y} = \ln y$	$\tilde{y} = \ln a + bx$
$y = ax^b$	$\tilde{y} = \ln y, \tilde{x} = \ln x$	$\tilde{y} = \ln a + b\tilde{x}$
$y = a + b \ln x$	$\tilde{x} = \ln x$	$y = a + b\tilde{x}$
$y = \frac{1}{a_0 + a_1 x}$	$\tilde{y} = 1/y$	$\tilde{y} = a_0 + a_1 x$
$y = ax^b e^{cx}$	$\tilde{y} = \ln y$	$\tilde{y} = \ln a + b \ln x + cx$
$y = A \cos(\omega x + \varphi)$ (ω dado)	$\cos/\sin$	$y = A \cos \varphi \cdot \cos \omega x - A \sin \varphi \cdot \sin \omega x$

Nota Importante

Atenção importante: a linearização minimiza S_r no *espaço transformado* $(\tilde{y}, \tilde{x})$, **não** no espaço original. Isso pode distorcer a ponderação dos erros. Para o modelo exponencial, por exemplo, minimizar $\sum (\ln y_k - \ln \hat{y}_k)^2$ é diferente de minimizar $\sum (y_k - \hat{y}_k)^2$. Para dados com ruído aditivo no espaço original, a linearização introduz um viés.

Quando a precisão é crítica, usa-se **regressão não-linear direta** (`scipy.optimize.curve_fit`), que minimiza S_r no espaço original.

6.6.2 Modelo exponencial: $y = ae^{bx}$

Motivação: crescimento ou decaimento com taxa proporcional ao valor atual ($y' = by$).

Linearização: aplicando $\ln$ a ambos os lados:

$$\tilde{y} = \ln y = \underbrace{\ln a}_{\tilde{a}} + bx.$$

Ajusta-se a reta a $(x_k, \ln y_k)$, obtendo-se $\bar{a}$ e b . O parâmetro original $a = e^{\bar{a}}$.

Requisito: $y_k > 0$ para todo k .

6.6.3 Modelo logarítmico: $y = a + b \ln x$

Motivação: crescimento com taxa decrescente (o efeito de x diminui à medida que x cresce).

Linearização: $\tilde{x} = \ln x$, e o modelo já é linear: $y = a + b\tilde{x}$.

Aplica-se a regressão linear à tabela $(\ln x_k, y_k)$.

Requisito: $x_k > 0$ para todo k .

6.6.4 Modelos com modelo Hiperbólico: $y = 1/(a_0 + a_1x)$

Conhecido em bioquímica como equação de Lineweaver-Burk (versão linearizada de Michaelis-Menten). A transformação $\tilde{y} = 1/y$ lineariza: $\tilde{y} = a_0 + a_1x$.

6.6.5 Exemplos resolvidos passo a passo

Exemplo 6.5 (Modelo Exponencial: Decaimento de Carga). A tensão V (em V) em um capacitor descarregando foi medida ao longo do tempo:

t (s)	0	5	10	15	20	25
V (V)	12,0	7,35	4,50	2,76	1,69	1,03

Ajustar $V = V_0 e^{-t/\tau}$ (modelo de descarga RC, equivalente a $V = ae^{bx}$).

Passo 1 — Transformar: $\tilde{V}_k = \ln V_k$.

t_k	V_k	$\tilde{V}_k = \ln V_k$	t_k^2	$t_k \tilde{V}_k$
0	12,00	2,4849	0	0,0000
5	7,35	1,9947	25	9,9735
10	4,50	1,5041	100	15,0410
15	2,76	1,0152	225	15,2280
20	1,69	0,5247	400	10,4940
25	1,03	0,0296	625	0,7400
Σ		7,5532	1375	51,4765

Passo 2 — Equações normais para a reta ($n = 6$, $\Sigma t_k = 75$):

$$\Delta = 6 \cdot 1375 - 75^2 = 8250 - 5625 = 2625.$$

$$b = \frac{6 \cdot 51,4765 - 75 \cdot 7,5532}{2625} = \frac{308,859 - 566,490}{2625} = \frac{-257,631}{2625} \approx -0,09814 \text{ s}^{-1}.$$

$$\bar{a} = \frac{7,5532 - (-0,09814) \cdot 75}{6} = \frac{7,5532 + 7,3605}{6} \approx 2,4856.$$

Passo 3 — Recuperar parâmetros originais:

$$V_0 = e^{\bar{a}} = e^{2,4856} \approx 12,01 \text{ V}, \quad b \approx -0,09814 \text{ s}^{-1}.$$

Constante de tempo: $\tau = -1/b \approx 10,19 \text{ s}$.

Resultado: $V(t) \approx 12,01 e^{-0,0981 t}$.

Verificação física: a tensão inicial $V_0 \approx 12,0 \text{ V}$ coincide com o valor medido. A meia-vida ($V = V_0/2$) ocorre em $t_{1/2} = \tau \ln 2 \approx 7,06 \text{ s}$, consistente com os dados.

Exemplo 6.6 (Modelo Logarítmico: Viscosidade de Fluido). A viscosidade dinâmica μ (em Pa·s) de um óleo polimérico decresce com a temperatura T (em °C) seguindo um padrão logarítmico:

T (°C)	30	50	80	120	200
μ (Pa · s)	8,40	5,20	2,95	1,60	0,70

Passo 1 — Transformar: $\tilde{T}_k = \ln T_k$.

T_k	μ_k	$\tilde{T}_k = \ln T_k$	$\tilde{T}_k^2$	$\tilde{T}_k \mu_k$
30	8,40	3,4012	11,5682	28,5701
50	5,20	3,9120	15,3037	20,3424
80	2,95	4,3820	19,2019	12,9269
120	1,60	4,7875	22,9202	7,6600
200	0,70	5,2983	28,0720	3,7088
Σ	18,85	21,7810	97,0660	73,2082

Passo 2 — Equações normais ($n = 5$):

$$\Delta = 5 \cdot 97,0660 - (21,7810)^2 = 485,330 - 474,412 = 10,918.$$

$$b = \frac{5 \cdot 73,2082 - 21,7810 \cdot 18,85}{10,918} = \frac{366,041 - 410,571}{10,918} \approx -4,079.$$

$$a = \frac{18,85 - (-4,079) \cdot 21,7810}{5} = \frac{18,85 + 88,84}{5} \approx 21,54.$$

Resultado: $\mu(T) \approx 21,54 - 4,079 \ln T$.

Verificação em $T = 80$: $\mu(80) \approx 21,54 - 4,079 \cdot 4,382 \approx 21,54 - 17,87 = 3,67 \text{ Pa} \cdot \text{s}$ (valor medido: 2,95; erro de $\approx 24\%$, razoável para dados de viscosidade com ampla faixa).

$r^2 \approx 0,991$ — bom ajuste.

6.6.6 Implementação

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import curve_fit
4
5 def ajuste_exponencial(x, y):
6     """Lineariza  $y = a \cdot \exp(b \cdot x)$  via  $\ln y = \ln a + b \cdot x$ ."""
7     x = np.asarray(x, dtype=float)
8     y = np.asarray(y, dtype=float)
9     y_til = np.log(y)
10    p = np.polyfit(x, y_til, 1) # [b, ln_a]
11    b, ln_a = p
12    a = np.exp(ln_a)
13    y_hat = a * np.exp(b * x)
14    Sr = np.sum((y - y_hat)**2)
15    St = np.sum((y - y.mean())**2)
16    return a, b, 1.0 - Sr/St
17
18 def ajuste_logaritmico(x, y):
19     """Ajusta  $y = a + b \cdot \ln(x)$  diretamente."""
20    x = np.asarray(x, dtype=float)
21    y = np.asarray(y, dtype=float)
22    x_til = np.log(x)
23    p = np.polyfit(x_til, y, 1) # [b, a]
24    b, a = p
25    y_hat = a + b * np.log(x)
26    Sr = np.sum((y - y_hat)**2)
27    St = np.sum((y - y.mean())**2)
28    return a, b, 1.0 - Sr/St
29
30 def ajuste_potencia(x, y):
31     """Lineariza  $y = a \cdot x^b$  via  $\ln y = \ln a + b \cdot \ln x$ ."""
32    x = np.asarray(x, dtype=float)
33    y = np.asarray(y, dtype=float)
34    p = np.polyfit(np.log(x), np.log(y), 1)
35    b, ln_a = p
36    a = np.exp(ln_a)
37    y_hat = a * x**b
38    Sr = np.sum((y - y_hat)**2)

```

```

39     St = np.sum((y - y.mean())**2)
40     return a, b, 1.0 - Sr/St
41
42     #           Exemplo: decaimento RC
43
44     t = np.array([0., 5., 10., 15., 20., 25.])
45     V = np.array([12.00, 7.35, 4.50, 2.76, 1.69, 1.03])
46
47     a, b, r2 = ajuste_exponencial(t, V)
48     print(f"Exponencial: V = {a:.4f} * exp({b:.5f} * t)")
49     print(f"Constante de tempo: tau = {-1/b:.3f} s")
50     print(f"r^2 = {r2:.6f}")
51
52     # Comparar com curve_fit (minimos quadrados no espaco original)
53     def modelo_exp(t, a, b): return a * np.exp(b * t)
54     popt, _ = curve_fit(modelo_exp, t, V, p0=[12, -0.1])
55     print(f"\ncurve_fit: a = {popt[0]:.4f}, b = {popt[1]:.5f}")
56
57     # Grafico comparativo: linearizacao vs. curve_fit
58     t_f = np.linspace(0, 30, 200)
59     fig, axes = plt.subplots(1, 2, figsize=(11, 4))
60
61     axes[0].scatter(t, V, color='red', zorder=5, label='Dados')
62     axes[0].plot(t_f, a*np.exp(b*t_f), 'b-',
63                  label=f'Linearizacao: $V={a:.2f}e^{{{b:.4f}t}}$')
64     axes[0].plot(t_f, popt[0]*np.exp(popt[1]*t_f), 'g--',
65                  label=f'curve_fit:
66                        $V={popt[0]:.2f}e^{{{popt[1]:.4f}t}}$')
67     axes[0].set_xlabel('t (s)'); axes[0].set_ylabel('V (V)')
68     axes[0].set_title('Decaimento RC'); axes[0].legend(fontsize=8)
69     axes[0].grid(True, alpha=0.4)
70
71     # Espaco transformado
72     axes[1].scatter(t, np.log(V), color='red', zorder=5, label='$\ln
73     V$')
74     axes[1].plot(t_f, np.log(a) + b*t_f, 'b-', label='Reta ajustada')
75     axes[1].set_xlabel('t (s)'); axes[1].set_ylabel('ln V')
76     axes[1].set_title('Espaco transformado'); axes[1].legend()
77     axes[1].grid(True, alpha=0.4)

```

```
plt.tight_layout(); plt.show()
```

Listing 6.4: Linearização de Modelos Não-Lineares

Exercícios — Linearização

Exercício 6.16. Ajuste o modelo potencial $y = ax^b$ aos dados abaixo usando a linearização $\tilde{y} = \ln y$, $\tilde{x} = \ln x$. Calcule a , b e r^2 no espaço original.

x	1	3	6	10	20
y	2,1	4,8	7,5	11,0	18,5

Exercício 6.17. Para o modelo hiperbólico $y = a/(b + x)$, mostre que a transformação $\tilde{y} = 1/y$ lineariza: $\tilde{y} = b/a + (1/a)x$. Identifique os novos coeficientes lineares e mostre como recuperar a e b .

Exercício 6.18. Ajuste **dois modelos** distintos aos dados abaixo e determine qual é mais adequado com base em $\sum e_k^2$ no espaço *original*:

x	0,5	1,0	2,0	4,0	8,0	16,0
y	5,8	4,3	3,1	2,2	1,6	1,1

Modelos candidatos: (a) $y = a + b \ln x$; (b) $y = ax^b$.

Exercício 6.19 (Demonstração). Para o modelo logarítmico $y = a + b \ln x$, prove que as equações normais no espaço transformado ($\tilde{x} = \ln x$) produzem:

$$b = \frac{n \sum (\ln x_k) y_k - \sum \ln x_k \cdot \sum y_k}{n \sum (\ln x_k)^2 - \left(\sum \ln x_k \right)^2}.$$

Exercício 6.20 (Projeto). Um experimento de farmacocinética mede a concentração de um medicamento (mg/L) em diferentes instantes após a administração:

t (h)	0,5	1	2	4	6	8	12
C (mg/L)	9,2	14,7	18,3	15,1	10,8	7,3	3,1

- Plote o diagrama de dispersão. O padrão é exponencial puro? Justifique.
- Ajuste os modelos: (i) $C = ae^{bt}$; (ii) $C = at^b e^{ct}$ (modelo bicompartimental linearizável via $\ln C = \ln a + b \ln t + ct$).
- Compare os r^2 e os gráficos de resíduos. Qual modelo é mais adequado para descrever a absorção e eliminação do medicamento?
- Com o modelo escolhido, estime a concentração máxima e o instante em que ela ocorre.

6.7 Roteiro de Ajuste de Curvas

A prática bem-sucedida do ajuste de curvas segue um roteiro sistemático. Apresentamos aqui o encadeamento lógico das etapas, que pode ser consultado como referência em problemas reais.

- 1. Plotar o diagrama de dispersão.** Identificar o padrão visual (linear, parabólico, exponencial, saturação) e propor candidatos ao modelo.
- 2. Transformar os dados** (se o modelo for não-linear). Verificar se o espaço transformado exibe relação linear antes de prosseguir.
- 3. Montar e resolver as equações normais.** Calcular as somas auxiliares, o sistema $A^T A \mathbf{a} = A^T \mathbf{y}$, e obter os coeficientes.
- 4. Reverter a transformação** (se aplicável). Recuperar os parâmetros no espaço original.
- 5. Calcular r^2 e os resíduos $e_k = y_k - \hat{y}_k$.** Verificar se há padrão sistemático no gráfico de resíduos.
- 6. Aplicar o princípio da parcimônia.** Se r^2 for similar entre dois modelos, preferir o mais simples.
- 7. Interpretar os parâmetros.** No contexto físico do problema, o que significa cada coeficiente? A ordem de grandeza é razoável?

6.8 Quadro Comparativo dos Modelos

Modelo	Quando usar	Linearização	Parâmetros
$a_0 + a_1 x$	Relação linear direta	Nenhuma	Sistema 2×2
$\sum_{j=0}^m a_j x^j$	Curva com inflexões	Nenhuma	Sistema $(m+1) \times (m+1)$
$a e^{bx}$	Crescimento/decaimento exp.	$\tilde{y} = \ln y$	$a = e^{\tilde{a}}$, b direto
$a + b \ln x$	Crescimento com saturação	$\tilde{x} = \ln x$	a, b diretos
$a x^b$	Lei de potência	$\tilde{y} = \ln y, \tilde{x} = \ln x$	$a = e^{\tilde{a}}$, b direto
$\frac{1}{a_0 + a_1 x}$	Curva hiperbólica	$\tilde{y} = 1/y$	a_0, a_1 diretos
$A \cos(\omega x + \varphi)$	Sinal periódico	$\cos / \sin$ (ω dado)	Regressão múltipla

6.9 Lista de Exercícios de Revisão do Capítulo

Nível 1 — Consolidação

Exercício 6.21 (R1). Para cada modelo abaixo, identifique a transformação linearizante, defina as variáveis $\tilde{x}$ e $\tilde{y}$ e escreva a forma linear resultante:

- (a) $y = a/(b + x)$ (Lineweaver-Burk)
- (b) $y = ax^b e^{cx}$ (lei de escala com crescimento)
- (c) $y = A \cos(\omega x + \varphi)$ com $\omega = 2$ conhecido

Exercício 6.22 (R2). Ajuste uma reta e um polinômio de grau 2 aos dados de temperatura de uma chapa aquecida. Calcule r^2 para cada grau, trace o gráfico de resíduos e recomende o modelo, justificando com o princípio da parcimônia:

t (s)	0	10	20	30	40	50
T (°C)	22	35	55	83	120	163

Exercício 6.23 (R3). Para os dados de tensão e corrente em um resistor não-ôhmico:

V (V)	0,5	1,0	2,0	3,0	5,0	8,0
I (mA)	0,30	0,85	2,40	4,40	12,20	31,10

- (a) Plote o diagrama de dispersão e proponha um modelo.
- (b) Aplique a transformação linearizante e ajuste os parâmetros.
- (c) Calcule r^2 e estime I para $V = 4,0$ V.

Exercício 6.24 (R4). Prove formalmente que a reta de mínimos quadrados passa pelo centróide $(\bar{x}, \bar{y})$ para qualquer conjunto de dados com x_i não todos iguais.

Nível 2 — Análise e Implementação

Exercício 6.25 (R5). Implemente em Python as quatro funções de ajuste estudadas (linear, polinomial, exponencial, logarítmico). Para cada modelo, teste com dados gerados sinteticamente (com ruído gaussiano adicionado) e compare os coeficientes ajustados com os parâmetros verdadeiros. Como o nível de ruído afeta a qualidade do ajuste?

Exercício 6.26 (R6). Para dados populacionais do Brasil (disponíveis no IBGE), ajuste modelos de grau 1, 2 e 3 e um modelo exponencial para a série histórica 1950–2020. Calcule r^2 para cada um e identifique o joelho da curva S_r vs. grau. Qual modelo melhor captura a dinâmica de crescimento? Use o modelo escolhido para extrapolar a população em 2030.

Exercício 6.27 (R7 — Demonstração). Prove que minimizar $S_r = \|y - A\mathbf{a}\|^2$ sujeito a nenhuma restrição leva às equações normais $A^T A \mathbf{a} = A^T \mathbf{y}$, usando cálculo matricial. Mostre também que a solução é única quando A tem posto completo.

Nível 3 — Integração e Projetos

Exercício 6.28 (R8 — Projeto). Calibração de Sensor de Temperatura. Um termistor NTC tem resistência R (em $\text{k}\Omega$) em função da temperatura T (em K) descrita pela equação de Steinhart-Hart:

$$\frac{1}{T} = A + B \ln R + C(\ln R)^3.$$

Com os dados abaixo, ajuste os coeficientes A , B , C por regressão múltipla (variáveis: $\tilde{y} = 1/T$, $\tilde{x}_1 = \ln R$, $\tilde{x}_2 = (\ln R)^3$). Calcule r^2 e estime T para $R = 5,0 \text{ k}\Omega$:

$R \text{ (k}\Omega\text{)}$	1,0	2,0	5,0	10,0	20,0	50,0
$T \text{ (K)}$	395	370	335	310	285	255

Exercício 6.29 (R9 — Teórico). A regularização Ridge (usada em aprendizado de máquina) resolve o problema modificado:

$$\min_{\mathbf{a}} \|A\mathbf{a} - \mathbf{y}\|^2 + \lambda \|\mathbf{a}\|^2.$$

- Derive as equações normais modificadas: $(A^T A + \lambda I)\mathbf{a} = A^T \mathbf{y}$.
- Mostre que a matriz $(A^T A + \lambda I)$ é sempre positiva definida para $\lambda > 0$, mesmo quando $A^T A$ é singular.
- Implemente em Python e compare a solução Ridge com a solução ordinária para um sistema sobredeterminado mal condicionado (e.g., colunas de A quase paralelas).

Exercício 6.30 (R10 — Projeto Integrador). Modelo Epidêmico Simplificado. Durante uma pandemia hipotética, os dados de casos acumulados por semana são:

Semana	1	2	3	4	5	6	7	8
Casos	120	310	720	1 580	3 100	5 200	7 800	10 100

- (a) Plote o diagrama de dispersão. A curva tem formato de S (sigmoide), sugerindo saturação.
- (b) Ajuste o modelo logístico $C = L/(1 + ae^{-bt})$ via linearização: $\ln(L/C - 1) = \ln a - bt$, fixando $L = 12\,000$ (capacidade máxima estimada).
- (c) Ajuste também o modelo exponencial puro $C = C_0e^{rt}$ às primeiras 4 semanas. Compare as previsões para as semanas 9 e 10.
- (d) Calcule r^2 para cada modelo e discuta: qual modela melhor o período inicial? E o período de desaceleração?
- (e) Com o modelo logístico, estime em que semana o número de novos casos semanais atinge o pico.

6.10 Referências e Leituras Complementares

- Chapra, S. C., & Canale, R. P. (2015). *Numerical Methods for Engineers*. 7th edition. McGraw-Hill. Capítulos 17–18: Least-Squares Regression.
- Burden, R. L., & Faires, J. D. (2015). *Numerical Analysis*. 10th edition. Cengage. Capítulo 8: Approximation Theory.
- Ruggiero, M. A. G., & Lopes, V. L. R. (1996). *Cálculo Numérico: Aspectos Teóricos e Computacionais*. 2.^a ed. Pearson. Capítulo 6.
- Franco, N. B. (2007). *Cálculo Numérico*. Pearson.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. 2nd edition. Springer. Capítulo 3: Linear Methods for Regression. (Disponível gratuitamente: <https://web.stanford.edu/~hastie/ElemStatLearn/>)
- NumPy polyfit: <https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html>
- SciPy curve_fit: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
- scikit-learn Linear Models: https://scikit-learn.org/stable/modules/linear_model.html

Apêndice A

Python para Cálculo Numérico

“Simplicity is a great virtue but it requires hard work to achieve it and education to appreciate it.”

— Edsger W. Dijkstra

Este apêndice fornece uma referência prática e autocontida de Python voltada às necessidades do curso de Cálculo Numérico. O objetivo não é substituir a documentação oficial das bibliotecas, mas sim destacar os padrões, idiomas e armadilhas mais relevantes para implementação de métodos numéricos.

A.1 Configuração do Ambiente

A.1.1 Instalação Recomendada

A maneira mais prática de obter um ambiente científico Python completo é instalar a distribuição **Anaconda** ou, para uma instalação mais enxuta, o **Miniconda**. Ambos instalam automaticamente NumPy, SciPy, Matplotlib e Jupyter.

```
1 import sys
2 import numpy as np
3 import scipy
4 import matplotlib
5
6 print(f"Python    : {sys.version}")
7 print(f"NumPy     : {np.__version__}")
8 print(f"SciPy      : {scipy.__version__}")
```

```
9 print(f"Matplot : {matplotlib.__version__}")
10
11 # Verificar precisao de maquina
12 eps = np.finfo(np.float64).eps
13 print(f"eps_m (float64): {eps:.4e}")    # ~2.22e-16
```

Listing A.1: Verificação do ambiente após instalação

A.1.2 Jupyter Notebook e Google Colab

Para acompanhar os exemplos interativamente, recomenda-se o uso do **Jupyter Notebook** (local) ou do **Google Colab** (na nuvem, sem instalação). Em ambos, células de código Python e texto Markdown se alternam, facilitando a apresentação de resultados.

Nota Importante

Para executar os códigos deste livro no Google Colab, basta acessar <https://colab.research.google.com> e criar um novo notebook. NumPy, SciPy e Matplotlib já estão pré-instalados.

A.1.3 Cabeçalho Padrão dos Scripts

Todo script de Cálculo Numérico neste livro segue o cabeçalho abaixo:

```
1 import numpy as np
2 import scipy.linalg as la
3 import scipy.optimize as opt
4 import matplotlib.pyplot as plt
5
6 # Opcoes de impressao
7 np.set_printoptions(precision=6, suppress=True)
```

Listing A.2: Cabeçalho padrão para scripts numéricos

A.2 NumPy — Arrays e Álgebra Linear

NumPy é a base de todo o ecossistema científico Python. Seu objeto central, o `ndarray`, permite realizar operações vetorizadas (sem laços explícitos) de forma eficiente.

A.2.1 Criação de Arrays

```

1 import numpy as np
2
3 # Vetores
4 v = np.array([1.0, 2.0, 3.0])          # a partir de lista
5 u = np.zeros(5)                        # cinco zeros
6 w = np.ones(4)                         # quatro uns
7 x = np.linspace(0, 1, 101)            # 101 pontos em [0,1]
8 y = np.arange(0, 10, 0.5)             # passo 0.5
9
10 # Matrizes
11 A = np.array([[1, 2], [3, 4]], dtype=float)
12 I = np.eye(3)                          # identidade 3x3
13 Z = np.zeros((3, 4))                   # matriz 3x4 de zeros
14 R = np.random.rand(4, 4)               # valores uniformes em
    [0,1)
15
16 print(A.shape)      # (2, 2)
17 print(A.dtype)      # float64

```

Listing A.3: Criação de arrays NumPy

A.2.2 Operações Vetorizadas

Cuidado

O operador `*` entre arrays NumPy realiza **produto elemento a elemento** (não o produto matricial). Para produto matricial, use `@` ou `np.dot`.

```

1 A = np.array([[1, 2], [3, 4]], dtype=float)
2 B = np.array([[5, 6], [7, 8]], dtype=float)
3
4 print(A * B)          # produto elemento a elemento (Hadamard)
5 # [[ 5. 12.]
6 #  [21. 32.]]
7
8 print(A @ B)          # produto matricial
9 # [[19. 22.]
10 #  [43. 50.]]

```

```

11
12 # Funcoes matematicas aplicadas elemento a elemento
13 x = np.linspace(0, np.pi, 5)
14 print(np.sin(x))
15 print(np.exp(x))
16 print(np.sqrt(x))

```

Listing A.4: Operações com arrays — diferença entre * e @

A.2.3 Álgebra Linear com `numpy.linalg`

```

1 import numpy as np
2
3 A = np.array([[4, 3], [6, 3]], dtype=float)
4 b = np.array([10, 12], dtype=float)
5
6 # Resolucao de Ax = b
7 x = np.linalg.solve(A, b)
8 print(f"Solucao: {x}") # [1. 2.]
9
10 # Verificacao do residuo
11 print(f"Residuo: {np.linalg.norm(A @ x - b):.2e}") # ~1e-15
12
13 # Determinante, inversa, autovalores
14 print(f"det(A) = {np.linalg.det(A):.4f}") # -6.0
15 autoval, autovec = np.linalg.eig(A)
16 print(f"autoval = {autoval}")
17
18 # Normas
19 print(f"||x||_2 = {np.linalg.norm(x)}") # norma euclidiana
20 print(f"||A||_F = {np.linalg.norm(A, 'fro')}") # norma de
    Frobenius
21 print(f"cond(A) = {np.linalg.cond(A):.4f}") # numero de
    condicao

```

Listing A.5: Operações de álgebra linear

A.2.4 Fatiamento (*Slicing*) e Indexação

```

1 import numpy as np
2
3 A = np.array([[1,2,3],[4,5,6],[7,8,9]], dtype=float)
4
5 print(A[0, :])      # primeira linha:      [1.  2.  3.]
6 print(A[:, 1])      # segunda coluna:      [2.  5.  8.]
7 print(A[1:, 1:])    # submatriz 2x2:      [[5.  6.] [8.  9.]]
8
9 # Cuidado: fatias sao VISOES, nao copias
10 linha = A[0, :]
11 linha[0] = 99      # modifica A tambem!
12 print(A[0, :])     # [99.  2.  3.]
13
14 # Para obter uma copia independente:
15 linha_copia = A[0, :].copy()

```

Listing A.6: Fatiamiento de arrays — padrões fundamentais

A.3 SciPy — Métodos Numéricos Prontos

SciPy estende NumPy com implementações de alta qualidade de algoritmos numéricos: otimização, integração, equações diferenciais, interpolação, transformadas e muito mais.

A.3.1 Zeros de Funções: `scipy.optimize`

```

1 import numpy as np
2 from scipy import optimize
3
4 f = lambda x: x**3 - 2*x - 5
5 df = lambda x: 3*x**2 - 2
6
7 # Metodo de Brent (bisseccao + secante, robusto)
8 raiz_brent = optimize.brentq(f, 2, 3)
9 print(f"Brent      : {raiz_brent:.10f}")      # 2.0945514815
10
11 # Metodo de Newton-Raphson (requer derivada)
12 raiz_newton = optimize.newton(f, x0=2.0, fprime=df)
13 print(f"Newton    : {raiz_newton:.10f}")

```

```

14
15 # Residuo
16 print(f"f(raiz) : {f(raiz_brent):.4e}") # ~0.0
17
18 # Inspecao completa do resultado
19 sol = optimize.root_scalar(f, bracket=[2, 3], method='brentq')
20 print(f"Converged: {sol.converged}, iteracoes: {sol.iterations}")

```

Listing A.7: Busca de raízes com SciPy

A.3.2 Sistemas Lineares: `scipy.linalg`

```

1 import numpy as np
2 import scipy.linalg as la
3
4 A = np.array([[2,1,0],[1,3,1],[0,1,2]], dtype=float)
5 b = np.array([1, 2, 1], dtype=float)
6
7 # Resolucao direta
8 x = la.solve(A, b)
9 print(f"Solucao: {x}")
10
11 # Fatoracao LU (reutilizavel para varios b)
12 lu, piv = la.lu_factor(A)
13 for bi in [b, b + 0.1]:
14     xi = la.lu_solve((lu, piv), bi)
15     print(f"x = {xi}")
16
17 # Fatoracao de Cholesky (A simetrica definida positiva)
18 L = la.cholesky(A, lower=True)
19 print(f"L =\n{L}")
20 x_chol = la.cho_solve(la.cho_factor(A), b)
21 print(f"Solucao (Cholesky): {x_chol}")

```

Listing A.8: Sistemas lineares e fatorações com SciPy

A.3.3 Integração Numérica: `scipy.integrate`

```

1 import numpy as np

```

```

2 from scipy import integrate
3
4 # Integral com solucao analitica conhecida (para verificacao)
5 # Integral de 0 a 1 de e-x2 dx ≈ 0.7468...
6 f = lambda x: np.exp(-x**2)
7
8 result, erro_est = integrate.quad(f, 0, 1)
9 print(f"Integral: {result:.10f}")      # 0.7468241328
10 print(f"Erro est: {erro_est:.2e}")    # ~8e-15
11
12 # Integral definida sem solucao em forma fechada
13 g = lambda x: np.sin(x) / x if x != 0 else 1.0
14 I, err = integrate.quad(g, 0, np.pi)
15 print(f"Si(pi) ~ {I:.8f}")           # integral seno cardinal
16
17 # Integral dupla
18 h = lambda y, x: np.exp(-(x**2 + y**2))
19 I2, err2 = integrate.dblquad(h, 0, 2, 0, 2)
20 print(f"Integral dupla: {I2:.8f}")

```

Listing A.9: Integração numérica com tolerância adaptativa

A.3.4 Interpolação: `scipy.interpolate`

```

1 import numpy as np
2 from scipy import interpolate
3
4 # Pontos de dados
5 x = np.array([0.0, 1.0, 2.0, 3.0, 4.0])
6 y = np.array([1.0, 2.7, 7.4, 20.1, 54.6])    # ~ex
7
8 # Spline cubica
9 cs = interpolate.CubicSpline(x, y)
10 x_fine = np.linspace(0, 4, 200)
11 y_fine = cs(x_fine)
12
13 # Verificacao em ponto interior
14 x_test = 1.5
15 print(f"cs(1.5) = {cs(x_test):.4f}")        # ~4.48
16 print(f"e1.5 = {np.exp(x_test):.4f}")    # 4.4817

```

```

17
18 # Derivada da spline
19 dcs = cs.derivative()
20 print(f"cs'(1.5) = {dcs(x_test):.4f}")      # ~4.48 (e^x = e^x')

```

Listing A.10: Interpolação com SciPy

A.4 Matplotlib — Visualização de Resultados

A visualização de resultados é parte essencial da validação de métodos numéricos. Matplotlib oferece controle completo sobre figuras e eixos.

A.4.1 Gráficos de Funções e Convergência

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
5
6 # --- Grafico 1: funcao e sua raiz ---
7 ax = axes[0]
8 x = np.linspace(-2, 4, 300)
9 y = x**3 - 2*x - 5
10
11 ax.axhline(0, color='gray', linewidth=0.8, linestyle='--')
12 ax.axvline(2.0945, color='red', linewidth=1.0, linestyle=':',
13         label='raiz')
14 ax.plot(x, y, 'b-', linewidth=2, label=r'$f(x)=x^3-2x-5$')
15 ax.set_xlabel('$x$')
16 ax.set_ylabel('$f(x)$')
17 ax.set_title('Zero de Funcao')
18 ax.legend()
19 ax.grid(True, alpha=0.3)
20
21 # --- Grafico 2: convergencia (escala log) ---
22 ax = axes[1]
23 iteracoes = np.arange(1, 11)
24 erros = 1.0 / 2.0**iteracoes      # convergencia linear
25 ax.semilogy(iteracoes, erros, 'o-', color='navy')

```

```

25 ax.set_xlabel('Iteracao $k$')
26 ax.set_ylabel('Erro $|e_k|$')
27 ax.set_title('Convergencia do Metodo de Bisseccao')
28 ax.grid(True, which='both', alpha=0.3)
29
30 plt.tight_layout()
31 plt.savefig('convergencia.pdf', dpi=150, bbox_inches='tight')
32 plt.show()

```

Listing A.11: Gráficos de funções — padrão do livro

Nota Importante

Para produzir figuras de qualidade para relatórios e trabalhos, use `plt.savefig('nome.pdf')` — formato vetorial preserva qualidade em qualquer escala de impressão. Para LaTeX, prefira PDF ou EPS.

A.4.2 Visualização de Erros e Convergência

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n_iter = 12
5 e0 = 1.0
6
7 # Taxas de convergencia
8 e_linear = [e0 * 0.5**k for k in range(n_iter)] #
   bisseccao
9 e_superlin = [e0 * 0.9**k for k in range(n_iter)] # secante
10 e_quad = [e0 / 10**(2**k) for k in range(n_iter)] # Newton
11
12 fig, ax = plt.subplots(figsize=(8, 5))
13 ax.semilogy(e_linear, 's--', label='Linear (bisseccao)',
   color='steelblue')
14 ax.semilogy(e_superlin, 'o:', label='Superlinear (secante)',
   color='darkorange')
15 ax.semilogy(e_quad, '^--', label='Quadratica (Newton)',
   color='darkgreen')
16 ax.set_xlabel('Iteracao $k$')
17 ax.set_ylabel('$|e_k|$ (escala log)')

```

```

18 ax.set_title('Comparacao de Ordens de Convergencia')
19 ax.legend()
20 ax.grid(True, which='both', alpha=0.3)
21 plt.tight_layout()
22 plt.show()

```

Listing A.12: Comparação de ordens de convergência

A.5 Boas Práticas e Armadilhas Comuns

A.5.1 Armadilhas de Ponto Flutuante

```

1  import numpy as np
2
3  # ARMADILHA 1: comparacao de floats com ==
4  x = 0.1 + 0.2
5  print(x == 0.3)           # False! (0.30000000000000004 != 0.3)
6  print(abs(x - 0.3) < 1e-10) # True (comparacao correta)
7  print(np.isclose(x, 0.3))  # True (idioma NumPy)
8
9  # ARMADILHA 2: divisao inteira vs. float em Python
10 print(3 / 2)               # 1.5 (divisao float em Python 3)
11 print(3 // 2)              # 1 (divisao inteira)
12
13 # ARMADILHA 3: overflow silencioso em inteiros NumPy
14 a = np.array([200], dtype=np.int8) # int8: max = 127
15 print(a * 2)                   # [-56] -- overflow silencioso! Nao gera erro.
16 a = np.array([200], dtype=np.int64) # correto
17 print(a * 2)                   # [400]
18
19 # ARMADILHA 4: modificacao de fatias
20 A = np.eye(3)
21 row = A[0]                     # REFERENCIA, nao copia
22 row[:] = 0                     # modifica A!
23 print(A)
24 A = np.eye(3)
25 row = A[0].copy()              # copia segura
26 row[:] = 0
27 print(A)                       # A intacta

```

Listing A.13: Armadilhas clássicas em Python numérico

A.5.2 Cancelamento Catastrófico em Python

```

1 import numpy as np
2
3 # Problema:  $f(x) = \sqrt{x+1} - \sqrt{x}$  para  $x$  grande
4 # Forma numericamente instavel
5 def f_instavel(x):
6     return np.sqrt(x + 1) - np.sqrt(x)
7
8 # Forma numericamente estavel (racionalizar o denominador)
9 def f_estavel(x):
10    return 1.0 / (np.sqrt(x + 1) + np.sqrt(x))
11
12 for x in [1e5, 1e10, 1e15]:
13     fi = f_instavel(x)
14     fe = f_estavel(x)
15     print(f"x={x:.0e}:  instavel={fi:.8e}  estavel={fe:.8e}")
16
17 # Resultado esperado:
18 # x=1e+05:  instavel=1.58113883e-03  estavel=1.58113884e-03
19 # x=1e+10:  instavel=1.00000001e-05  estavel=1.58113883e-06
20 # x=1e+15:  instavel=0.00000000e+00  estavel=5.00000000e-09 (zero
    errado!)

```

Listing A.14: Cancelamento catastrófico e como evitá-lo

A.5.3 Acurácia Mista Float32 / Float64

```

1 import numpy as np
2
3 # Float32: ~7 dígitos significativos (usado em GPU, Deep
4 #          Learning)
5 # Float64: ~15 dígitos significativos (padrao em calculo
6 #          científico)

```

```

6  # Construir sistema linear mal-condicionado
7  n = 6
8  H = np.array([[1/(i+j+1) for j in range(n)] for i in range(n)])
      # Hilbert
9  b = H @ np.ones(n)
10
11 x64 = np.linalg.solve(H.astype(np.float64), b.astype(np.float64))
12 x32 = np.linalg.solve(H.astype(np.float32), b.astype(np.float32))
13
14 print(f"Solucao esperada : {np.ones(n)}")
15 print(f"Float64 erro max : {np.max(np.abs(x64 - 1)):.2e}")
16 print(f"Float32 erro max : {np.max(np.abs(x32 - 1)):.2e}")
17 print(f"Numero de condicao: {np.linalg.cond(H):.2e}")
18 # Para n=6: cond(H) ~ 2.9e7

```

Listing A.15: Float32 vs. Float64 em aplicações numéricas

A.5.4 Medição de Tempo e Perfil de Desempenho

```

1  import numpy as np
2  import time
3
4  n = 2000
5
6  # Multiplicacao matricial: vetorizado vs. laço explícito
7  A = np.random.rand(n, n)
8  B = np.random.rand(n, n)
9
10 # Vetorizado (BLAS otimizado)
11 t0 = time.perf_counter()
12 C = A @ B
13 t_vec = time.perf_counter() - t0
14 print(f"Vetorizado: {t_vec:.4f} s")
15
16 # Estimativa do custo de laço manual (n^3 operacoes)
17 # Não execute o laço para n=2000 -- levaria horas!
18 # Demonstracao para n=100:
19 n_small = 100
20 A_s, B_s = A[:n_small, :n_small], B[:n_small, :n_small]
21 t0 = time.perf_counter()

```

```

22 C_s = np.zeros((n_small, n_small))
23 for i in range(n_small):
24     for j in range(n_small):
25         for k in range(n_small):
26             C_s[i,j] += A_s[i,k] * B_s[k,j]
27 t_loop = time.perf_counter() - t0
28 print(f"Laco (n={n_small}): {t_loop:.4f} s")
29 print(f"Fator de lentidao estimado para n={n}:
      {(t_loop/t_vec)*(n/n_small)**3:.0f}x")

```

Listing A.16: Medição de tempo de execução

A.6 Receitas Práticas para os Capítulos

Esta seção reúne, em forma de referência rápida, os padrões de implementação correspondentes a cada capítulo do livro.

A.6.1 Capítulo 1 — Erros e Ponto Flutuante

```

1  import numpy as np
2
3  # Epsilon de maquina
4  eps64 = np.finfo(np.float64).eps      # 2.22e-16
5  eps32 = np.finfo(np.float32).eps      # 1.19e-07
6
7  # Overflow/underflow limites
8  fmax = np.finfo(np.float64).max       # 1.798e+308
9  fmin = np.finfo(np.float64).tiny      # 2.225e-308
10
11 # Comparacao numericamente segura
12 def aprox_igual(a, b, tol=1e-10):
13     return abs(a - b) <= tol * max(abs(a), abs(b), 1.0)
14
15 # Erro absoluto e relativo
16 def erros(x_exato, x_aprox):
17     ea = abs(x_exato - x_aprox)
18     er = ea / abs(x_exato) if x_exato != 0 else float('inf')
19     return ea, er
20

```

```
21 print(erros(np.pi, 3.14159))    # (2.65e-06, 8.43e-07)
```

Listing A.17: Ferramentas do Capítulo 1

A.6.2 Capítulo 2 — Zeros de Funções

```
1 import numpy as np
2
3 def bisseccao(f, a, b, tol=1e-10, max_iter=100):
4     """
5     Metodo da bisseccao para encontrar raiz de f em [a,b].
6     Requer: f(a)*f(b) < 0.
7     Retorna: raiz aproximada, numero de iteracoes, historico de
8             erros.
9     """
10    assert f(a) * f(b) < 0, "Nao ha troca de sinal em [a,b]"
11    historico = []
12    for k in range(max_iter):
13        c = (a + b) / 2.0
14        historico.append(abs(b - a) / 2.0)
15        if abs(f(c)) < tol or (b - a) / 2.0 < tol:
16            return c, k + 1, historico
17        if f(a) * f(c) < 0:
18            b = c
19        else:
20            a = c
21    return (a + b) / 2.0, max_iter, historico
22
23 # Uso
24 f = lambda x: x**3 - 2*x - 5
25 raiz, niter, hist = bisseccao(f, 2, 3)
26 print(f"Raiz: {raiz:.10f}, iteracoes: {niter}")
```

Listing A.18: Padrão de implementação de métodos iterativos

A.6.3 Capítulo 3 — Sistemas Lineares

```
1 import numpy as np
2
```

```

3 def gauss_pivotal(A, b):
4     """
5     Eliminacao de Gauss com pivotamento parcial.
6     Resolve Ax = b. Retorna x.
7     """
8     n = len(b)
9     Ab = np.hstack([A.astype(float), b.reshape(-1,
10         1).astype(float)])
11
12     for k in range(n - 1):
13         # Pivotamento parcial
14         p = np.argmax(np.abs(Ab[k:, k])) + k
15         if p != k:
16             Ab[[k, p]] = Ab[[p, k]]
17         # Eliminacao
18         for i in range(k + 1, n):
19             if Ab[k, k] == 0:
20                 raise ValueError("Matriz singular (pivo nulo).")
21             fator = Ab[i, k] / Ab[k, k]
22             Ab[i, k:] -= fator * Ab[k, k:]
23
24     # Substituicao retroativa
25     x = np.zeros(n)
26     for i in range(n - 1, -1, -1):
27         x[i] = (Ab[i, -1] - Ab[i, i+1:n] @ x[i+1:n]) / Ab[i, i]
28     return x
29
30 # Teste
31 A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float)
32 b = np.array([8, -11, -3], dtype=float)
33 x = gauss_pivotal(A, b)
34 print(f"x = {x}") # [2. 3. -1.]
35 print(f"Residuo = {np.linalg.norm(A@x - b):.2e}") # ~0.0

```

Listing A.19: Eliminação de Gauss com pivotamento parcial

A.7 Exercícios de Programação

Exercício A.1. Implemente a função `epsilon_maquina(dtype)` que calcula experimentalmente o épsilon de máquina para `np.float32` e `np.float64`. Compare com os valores retornados por `np.finfo`. Explique o algoritmo.

Exercício A.2. Implemente o método de Newton-Raphson `newton(f, df, x0, tol, max_iter)` que retorna a raiz, o número de iterações e o histórico de erros. Teste com $f(x) = e^x - 3x$ e $x_0 = 1$. Verifique a convergência quadrática plotando o histórico em escala log-log.

Exercício A.3. Escreva uma função `comparar_metodos(f, df, a, b)` que aplica bissecção, Newton e secante à mesma função e produz um gráfico de convergência comparativo. Teste com $f(x) = \cos(x) - x$.

Exercício A.4. Implemente a eliminação de Gauss *sem* pivotamento e compare com `gauss_pivotal` para a matriz de Hilbert H_n , $n = 4, 6, 8$. Calcule o resíduo $\|Ax - b\|_2$ e o número de condição $\kappa(H_n)$ em cada caso. O que acontece com a estabilidade conforme n aumenta?

Exercício A.5. Dado um conjunto de pontos (x_i, y_i) gerados por $y = \sin(x) + \varepsilon_i$ onde $\varepsilon_i \sim \mathcal{N}(0, 0.1)$, use `scipy.interpolate.CubicSpline` e `numpy.polyfit` com graus 1, 3, 5, 9 para ajustar os dados. Plote e discuta o fenômeno de Runge para polinômios de grau elevado.

Exercício A.6 (Projeto). Implemente uma biblioteca mínima de Cálculo Numérico, com as funções:

- (a) `bisseccao(f, a, b, tol)` — método da bissecção;
- (b) `newton(f, df, x0, tol)` — Newton-Raphson;
- (c) `gauss_lu(A, b)` — decomposição LU com pivotamento;
- (d) `relatorio(metodo, raiz, n_iter, historico)` — exibe tabela de convergência e gráfico.

Teste cada função com pelo menos dois exemplos do livro e valide contra `scipy.optimize` e `numpy.linalg`.

Apêndice B

Complexidade Computacional e Análise de Algoritmos

“An algorithm must be seen to be believed.”

— Donald E. Knuth

Este apêndice formaliza os conceitos de análise assintótica introduzidos no Capítulo 1 e complementa cada capítulo do livro com a análise de custo dos algoritmos apresentados. A meta é capacitar o leitor a responder: *quão rápido este algoritmo escala com o tamanho n do problema?*

B.1 Notação Assintótica

B.1.1 As Três Notações Fundamentais

Definição B.1 (Grande- $\mathcal{O}$ — limite superior). Sejam $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$. Escreve-se $f(n) = \mathcal{O}(g(n))$ se existem constantes $c > 0$ e $n_0 \in \mathbb{N}$ tais que

$$f(n) \leq c g(n), \quad \forall n \geq n_0.$$

Informalmente: f cresce *no máximo* tão rápido quanto g .

Definição B.2 (Grande- Ω — limite inferior). $f(n) = \Omega(g(n))$ se existem $c > 0$ e n_0 tais que

$$f(n) \geq c g(n), \quad \forall n \geq n_0.$$

Informalmente: f cresce *pele menos* tão rápido quanto g .

Definição B.3 (Grande- Θ — limite apertado). $f(n) = \Theta(g(n))$ se $f(n) = \mathcal{O}(g(n))$ e $f(n) = \Omega(g(n))$ simultaneamente, ou seja, existem $c_1, c_2 > 0$ e n_0 tais que

$$c_1 g(n) \leq f(n) \leq c_2 g(n), \quad \forall n \geq n_0.$$

Informalmente: f e g têm *exatamente* a mesma taxa de crescimento.

Nota Importante

Em Cálculo Numérico, a notação $\mathcal{O}$ é usada em dois sentidos distintos:

1. **Complexidade de algoritmos:** quantas operações são necessárias em função do tamanho n do problema (assintótica conforme $n \rightarrow \infty$).
2. **Ordem de erro de aproximação:** quão rápido o erro de um método numérico decresce conforme o parâmetro de refinamento (passo h , número de iterações k , etc.) tende a zero. Por exemplo: $E(h) = \mathcal{O}(h^2)$ significa que o erro é proporcional a h^2 para h pequeno.

O contexto sempre deixará claro qual sentido está sendo usado.

B.1.2 Propriedades da Notação $\mathcal{O}$

Proposição B.4 (Propriedades algébricas). Sejam $f_1 = \mathcal{O}(g_1)$ e $f_2 = \mathcal{O}(g_2)$. Então:

1. $f_1 + f_2 = \mathcal{O}(\max(g_1, g_2))$ (regra da soma);
2. $f_1 \cdot f_2 = \mathcal{O}(g_1 \cdot g_2)$ (regra do produto);
3. $c \cdot f_1 = \mathcal{O}(g_1)$ para qualquer constante $c > 0$ (constantes irrelevantes);
4. $n^a = \mathcal{O}(n^b)$ sempre que $a \leq b$ (dominância polinomial);
5. $n^k = \mathcal{O}(a^n)$ para quaisquer $k > 0$ e $a > 1$ (exponencial domina polinômio).

Exemplo B.1 (Simplificação de expressões).

$$T(n) = \frac{1}{3}n^3 + 5n^2 + 100n + 42 = \Theta(n^3),$$

$$T(n) = \log_2(n) + n = \Theta(n),$$

$$T(n) = 2^n + n^{100} = \Theta(2^n).$$

Em análise assintótica, apenas o *termo dominante* importa.

B.2 Análise de Algoritmos Numéricos Fundamentais

B.2.1 Operações Elementares: Flops

A unidade de custo em Cálculo Numérico é o **flop** (*floating-point operation*): qualquer operação de ponto flutuante — adição, subtração, multiplicação ou divisão. Raízes quadradas e funções transcendentais custam tipicamente entre 5 e 20 flops, dependendo da implementação.

Tabela B.1: Custo em flops de operações fundamentais.

Operação	Flops (exato)	Assintótico
Produto interno $\mathbf{u}^T \mathbf{v}$, $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$	$2n - 1$	$\Theta(n)$
Produto matriz-vetor $A\mathbf{x}$, $A \in \mathbb{R}^{m \times n}$	$2mn - m$	$\Theta(mn)$
Produto matriz-matriz AB , $A \in \mathbb{R}^{m \times k}$, $B \in \mathbb{R}^{k \times n}$	$2mkn - mn$	$\Theta(mkn)$
Eliminação de Gauss, $A \in \mathbb{R}^{n \times n}$	$\frac{2}{3}n^3 + \mathcal{O}(n^2)$	$\Theta(n^3)$
Fatoração LU, $A \in \mathbb{R}^{n \times n}$	$\frac{3}{2}n^3 + \mathcal{O}(n^2)$	$\Theta(n^3)$
Fatoração de Cholesky (A SPD)	$\frac{1}{3}n^3 + \mathcal{O}(n^2)$	$\Theta(n^3)$
Substituição retroativa/direta	n^2	$\Theta(n^2)$

B.2.2 Avaliação de Polinômios: Horner vs. Naïve

Teorema B.5 (Esquema de Horner). O polinômio $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ pode ser avaliado com n multiplicações e n adições (total: $2n$ flops) pela reescrita:

$$p(x) = a_0 + x \left(a_1 + x \left(a_2 + \dots + x (a_{n-1} + a_n x) \dots \right) \right).$$

A avaliação naïve requer $2n - 1$ multiplicações e n adições ($\Theta(n)$ em ambos os casos, mas Horner tem constante menor e é numericamente mais estável).

```

1 import numpy as np
2 import time
3
4 def naive(coeffs, x):
5     """coeffs: [a_0, a_1, ..., a_n]"""
6     n = len(coeffs) - 1
7     return sum(coeffs[k] * x**k for k in range(n + 1))
8
9 def horner(coeffs, x):
10    """coeffs: [a_0, a_1, ..., a_n]"""

```

```

11     result = coeffs[-1]
12     for k in range(len(coeffs) - 2, -1, -1):
13         result = result * x + coeffs[k]
14     return result
15
16 # Comparacao de desempenho para grau 1000
17 n = 1000
18 coeffs = list(np.random.rand(n + 1))
19 x0 = 1.5
20
21 t0 = time.perf_counter()
22 for _ in range(10000): naive(coeffs, x0)
23 t_naive = time.perf_counter() - t0
24
25 t0 = time.perf_counter()
26 for _ in range(10000): horner(coeffs, x0)
27 t_horner = time.perf_counter() - t0
28
29 print(f"Naive : {t_naive:.4f}s | Horner: {t_horner:.4f}s")
30 print(f"Fator de melhora: {t_naive/t_horner:.1f}x")

```

Listing B.1: Esquema de Horner versus avaliação direta

B.2.3 Métodos Iterativos para Zeros de Funções

O custo de métodos iterativos é medido em **número de iterações** necessárias para atingir precisão ε , a partir de um erro inicial e_0 .

Tabela B.2: Comparação de métodos para zeros de funções: ordem de convergência e número de iterações para $\varepsilon = 10^{-10}$, $e_0 = 1$.

Método	Ordem p	Flops/iter.	Iters. estimadas	Custo total
Bissecção	1 (linear, $C = 0.5$)	$\mathcal{O}(1)$	≈ 34	$\mathcal{O}(34)$
Ponto fixo	1 (linear, $C < 1$)	$\mathcal{O}(1)$	$\sim 30\text{--}80$	$\mathcal{O}(n)$
Secante	1.618 (superlinear)	$\mathcal{O}(1)$	$\approx 7\text{--}10$	$\mathcal{O}(10)$
Newton-Raphson	2 (quadrática)	$\mathcal{O}(1)$	$\approx 3\text{--}5$	$\mathcal{O}(5)$
Müller	≈ 1.84	$\mathcal{O}(1)$	$\approx 5\text{--}8$	$\mathcal{O}(8)$

Nota Importante

A ordem de convergência p informa como o erro decresce a cada iteração:

$$e_{k+1} \approx C \cdot e_k^p.$$

Para ordem 2 (Newton), se $e_k = 10^{-5}$, então $e_{k+1} \approx C \cdot 10^{-10}$ — o número de dígitos corretos *dobra* a cada iteração.

Derivação do número de iterações — Bisseção. O erro após k iterações é $e_k = (b-a)/2^k$. Para atingir $e_k \leq \varepsilon$:

$$\frac{b-a}{2^k} \leq \varepsilon \implies 2^k \geq \frac{b-a}{\varepsilon} \implies k \geq \log_2 \left(\frac{b-a}{\varepsilon} \right).$$

Para $b-a = 1$ e $\varepsilon = 10^{-10}$: $k \geq \log_2(10^{10}) = 10 \log_2(10) \approx 33,2$, portanto $k = 34$. $\square$

B.3 Complexidade de Operações Matriciais

B.3.1 Eliminação de Gauss: Contagem Exata

Teorema B.6 (Flops da Eliminação de Gauss). A eliminação de Gauss (fase de eliminação) para $A \in \mathbb{R}^{n \times n}$ requer exatamente

$$\frac{2n^3}{3} + \mathcal{O}(n^2) \approx \frac{2n^3}{3} \text{ flops.}$$

A substituição retroativa requer n^2 flops adicionais. A solução completa de $Ax = b$ custa, portanto, $\frac{2n^3}{3} + n^2 + \mathcal{O}(n)$ flops.

Derivação. Na eliminação da coluna k ($k = 1, 2, \dots, n-1$), eliminam-se $n-k$ linhas. Para cada linha $i > k$:

- 1 divisão: cálculo do fator $m_{ik} = a_{ik}/a_{kk}$;
- $n-k$ multiplicações e $n-k$ subtrações: atualização da linha.

Total para a coluna k : $(n-k)(2(n-k+1)-1) \approx 2(n-k)^2$ flops.

Somando sobre $k = 1$ a $n-1$:

$$T = \sum_{k=1}^{n-1} 2(n-k)^2 = 2 \sum_{j=1}^{n-1} j^2 = 2 \cdot \frac{(n-1)n(2n-1)}{6} = \frac{n(n-1)(2n-1)}{3} = \frac{2n^3}{3} - n^2 + \frac{n}{3}.$$

Para n grande: $T \approx \frac{2n^3}{3}$. □

B.3.2 Fatoração de Cholesky: Custo pela Metade

Teorema B.7 (Flops da Fatoração de Cholesky). Para $A \in \mathbb{R}^{n \times n}$ simétrica definida positiva, a fatoração de Cholesky $A = LL^T$ requer exatamente

$$\frac{n^3}{3} + \mathcal{O}(n^2) \approx \frac{n^3}{3} \text{ flops,}$$

ou seja, *metade* do custo da fatoração LU geral.

A economia de um fator 2 é significativa: para $n = 10\,000$, isso equivale a economizar $\sim 3 \times 10^{11}$ flops.

B.3.3 Hierarquia de Custos Matriciais

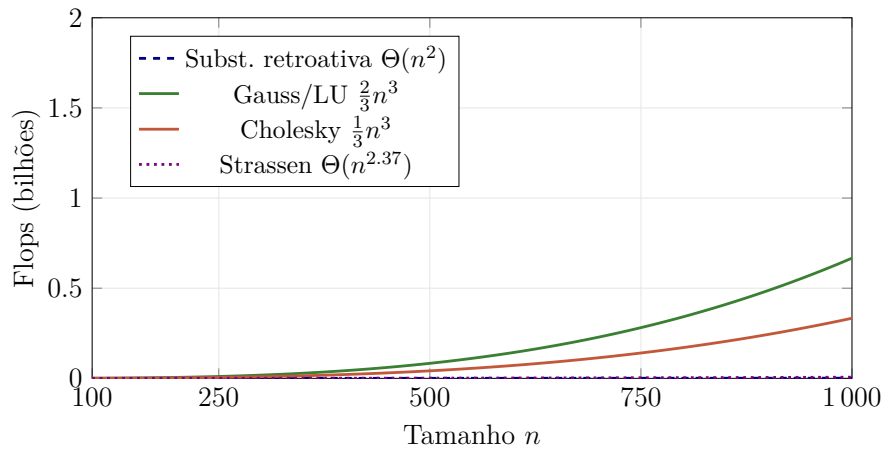


Figura B.1: Custo em flops dos principais algoritmos matriciais em função de n . Cholesky custa exatamente metade de Gauss/LU para matrizes simétricas definidas positivas. Strassen é assintoticamente superior a LU, mas o ganho prático só aparece para $n \gtrsim 1\,000$.

B.4 Modelos de Custo: além dos Flops

A contagem de flops é uma abstração útil, mas o desempenho real de um algoritmo em hardware moderno depende de outros fatores igualmente importantes.

B.4.1 Hierarquia de Memória

Em um processador moderno, o custo de acesso à memória domina o custo aritmético:

Tabela B.3: Latências típicas na hierarquia de memória de um processador moderno.

Nível	Capacidade	Latência	Largura de banda
Registrador	~1 KB	0 ciclos	—
Cache L1	32–64 KB	4–5 ciclos	>1 TB/s
Cache L2	256 KB–1 MB	12–15 ciclos	~500 GB/s
Cache L3	8–64 MB	30–50 ciclos	~200 GB/s
RAM (DRAM)	8–256 GB	200–300 ciclos	~50 GB/s
SSD NVMe	1–4 TB	10^4 – 10^5 ciclos	~7 GB/s

Nota Importante

Algoritmos com bom **acesso à memória** (localidade de referência) são frequentemente mais rápidos na prática do que outros com menos flops mas acesso irregular. Por isso, algoritmos que operam em blocos (*blocked* ou *tiled*) são preferidos em matrizes grandes: eles maximizam a reutilização do cache.

B.4.2 Paralelismo e GPUs

Processadores modernos têm múltiplos núcleos (*cores*) e GPUs possuem milhares de núcleos mais simples. A complexidade teórica de um algoritmo não captura sua adequação ao paralelismo.

Tabela B.4: Desempenho de pico teórico em hardware moderno (2025).

Hardware	Flops/s (FP64)	Memória	Uso típico
CPU (8 cores)	~ 1 TFLOPS	64 GB DDR5	Propósito geral
GPU NVIDIA A100	~ 20 TFLOPS	80 GB HBM	Deep Learning, simulação
GPU NVIDIA H100	~ 67 TFLOPS	80 GB HBM	Treinamento de LLMs
TPU v4 (Google)	~ 275 TFLOPS	32 GB HBM	Deep Learning

Aplicação Computacional

Impacto no Cálculo Numérico:

Uma fatoração LU de $A \in \mathbb{R}^{10\,000 \times 10\,000}$ requer $\frac{2}{3}(10^4)^3 = 6,7 \times 10^{11}$ flops. Em uma CPU de 1 TFLOPS, isso leva $\sim 0,67$ s. Em um cluster com 100 GPUs A100 (20×100 TFLOPS = 2 PFLOPS), o mesmo cálculo levaria apenas $\sim 0,33$ ms — um fator $2\,000\times$ mais rápido.

A paralelização eficiente da álgebra linear é a razão pela qual modelos de linguagem com bilhões de parâmetros podem ser treinados em dias, e não em séculos.

B.5 Tabelas de Referência Rápida

B.5.1 Classes de Complexidade em Cálculo Numérico

Tabela B.5: Classes de complexidade relevantes em Cálculo Numérico.

Notação	Nome	Exemplo CN	Para $n = 10^6$
$\mathcal{O}(1)$	Constante	Acesso a elemento	1 op
$\mathcal{O}(\log n)$	Logarítmica	Busca binária	20 ops
$\mathcal{O}(n)$	Linear	Avaliação por Horner	10^6 ops
$\mathcal{O}(n \log n)$	Log-linear	FFT	2×10^7 ops
$\mathcal{O}(n^2)$	Quadrática	Produto matriz-vetor	10^{12} ops
$\mathcal{O}(n^3)$	Cúbica	Gauss, LU, produto AB	10^{18} ops
$\mathcal{O}(n^{2.37})$	Strassen	Produto AB (Strassen)	$\sim 10^{14}$ ops
$\mathcal{O}(2^n)$	Exponencial	Enumeração	$10^{301\,000}$ ops

Cuidado

Para $n = 10^6$ e $\mathcal{O}(n^3)$, seriam necessárias 10^{18} operações. A um custo de 10^{12} flops/s (1 TFLOPS), isso levaria 10^6 s $\approx$ 11,6 dias. Problemas de grande escala exigem estrutura adicional (esparsidade, fatorações aproximadas, métodos iterativos) para serem viáveis.

B.5.2 Ordens de Erro dos Métodos Numéricos

Tabela B.6: Ordens de erro de aproximação dos principais métodos do livro.

Método	Parâmetro	Ordem do erro	Capítulo
Bisseção	n iterações	$\mathcal{O}(2^{-n})$	2
Newton-Raphson	k iterações	$\mathcal{O}(e_k^2)$	2
Secante	k iterações	$\mathcal{O}(e_k^{1.618})$	2
Eliminação de Gauss	passo exato	$\mathcal{O}(\varepsilon_m n^3)$	3
Regra dos Trapézios	passo h	$\mathcal{O}(h^2)$	(cap. futuro)
Regra de Simpson	passo h	$\mathcal{O}(h^4)$	(cap. futuro)
Diferença central	passo h	$\mathcal{O}(h^2)$	(cap. futuro)
Euler explícito (EDOs)	passo h	$\mathcal{O}(h)$	(cap. futuro)
Runge-Kutta 4 (EDOs)	passo h	$\mathcal{O}(h^4)$	(cap. futuro)

B.5.3 Custos Exatos dos Algoritmos do Livro

Tabela B.7: Custo computacional exato (flops) dos algoritmos apresentados.

Algoritmo	Entrada	Flops (exato)	Assintótico
Horner	grau n	$2n$	$\Theta(n)$
Produto interno	vetores n	$2n - 1$	$\Theta(n)$
Produto $A\mathbf{x}$	$A \in \mathbb{R}^{n \times n}$	$2n^2 - n$	$\Theta(n^2)$
Produto AB	$A, B \in \mathbb{R}^{n \times n}$	$2n^3 - n^2$	$\Theta(n^3)$
Eliminação Gauss	$A \in \mathbb{R}^{n \times n}$	$\frac{2}{3}n^3 - \frac{n}{3}$	$\Theta(n^3)$
Fatoração LU	$A \in \mathbb{R}^{n \times n}$	$\frac{2}{3}n^3 + \mathcal{O}(n^2)$	$\Theta(n^3)$
Fatoração Cholesky	A SPD $n \times n$	$\frac{1}{3}n^3 + \mathcal{O}(n^2)$	$\Theta(n^3)$
Subst. retroativa	sistema $n \times n$	n^2	$\Theta(n^2)$

B.6 Exercícios

Exercício B.1. Prove as afirmações da [theorem B.4](#), itens (i) e (iii), usando a definição formal de $\mathcal{O}$.

Exercício B.2. Determine a complexidade assintótica $\Theta(\cdot)$ de cada expressão:

- (a) $T(n) = 5n^3 + 3n^2 \log n + 100n + 42$.
- (b) $T(n) = n^2 + n \cdot 2^n$.
- (c) $T(n) = \sum_{k=1}^n k^2$.
- (d) $T(n) = \log_2(n^3)$.

Exercício B.3. Derive a fórmula exata $\frac{n^3}{3} + \mathcal{O}(n^2)$ para a fatoração de Cholesky contando os flops coluna a coluna. (Dica: a coluna k requer 1 raiz quadrada, $k-1$ divisões, $(k-1)(n-k)$ multiplicações e $(k-1)(n-k)$ subtrações.)

Exercício B.4. Para um computador de 10^{12} flops/s (1 TFLOPS), calcule o tempo necessário para executar a eliminação de Gauss com $n = 100$, $n = 1\,000$, $n = 10\,000$ e $n = 100\,000$. Organize em tabela. Para quais valores de n o algoritmo é impraticável sem paralelismo?

Exercício B.5. O método da bissecção e o método de Newton-Raphson são ambos aplicados a $f(x) = e^x - 3$, com $e_0 = 1$ e tolerância $\varepsilon = 10^{-12}$.

- (a) Quantas iterações cada método requer? Use as fórmulas da [table B.2](#).
- (b) Se cada iteração de Newton custa o dobro em flops que uma de bissecção (por exigir o cálculo de f'), qual método tem menor custo total? A resposta depende de ε ?

Exercício B.6. Meça experimentalmente o tempo de execução de `np.linalg.solve` para $n = 100, 500, 1000, 2000, 4000$ e ajuste uma curva $T(n) = cn^p$ aos dados. O expoente p obtido é próximo de 3? Discuta fatores que podem explicar desvios (BLAS, cache, etc.).

Exercício B.7 (Strassen na Prática). O algoritmo de Strassen tem complexidade $\mathcal{O}(n^{2.807})$ versus $\mathcal{O}(n^3)$ do produto naïve.

- (a) Para qual valor de n a fatoração de Strassen supera o produto naïve, assumindo que Strassen tem constante multiplicativa 100 e o naïve tem constante 1 (custo $100n^{2.807}$ versus n^3)?
- (b) Por que bibliotecas como NumPy/BLAS geralmente não usam Strassen para matrizes densas de uso geral? Pesquise e justifique em termos de localidade de cache e constante multiplicativa.

Exercício B.8 (Complexidade de Métodos Iterativos para Sistemas Lineares). Para um sistema $A\mathbf{x} = \mathbf{b}$ com $A \in \mathbb{R}^{n \times n}$ densa, o método de Gauss-Seidel (iterativo) converge em k iterações, cada uma com custo $\mathcal{O}(n^2)$.

- (a) Se $k = \mathcal{O}(n)$ iterações são necessárias, qual é o custo total?
- (b) Compare com a eliminação de Gauss. Em qual situação o método iterativo é preferível?
- (c) Para A esparsa com $\mathcal{O}(n)$ entradas não nulas (por linha), qual é o custo por iteração? E o custo total com $k = \mathcal{O}(n)$?

Referências Bibliográficas

- [1] Richard L Burden, J Douglas Faires, and Annette M Burden. *Numerical Analysis*. Cengage Learning, 10th edition, 2015.
- [2] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM computing surveys (CSUR)*, 23(1):5–48, 1991.
- [3] Gene H Golub and Charles F Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- [5] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Stanford InfoLab, 1999.
- [6] Lloyd N Trefethen and David Bau III. *Numerical Linear Algebra*. SIAM, 1997.